Multi-Sensor Fusion & Trajectory Planning on nuScenes

Technical Handover & Developer Documentation

Camera BEV · LiDAR · Radar · Learned Feature Selection · Fusion · 3-D Detection · Tracking · Planning · Streamlit Demo

Item	Value
Project location	D:\researcpaper\multi_sensor_fusion
Dataset location	D:\researcpaper (nuScenes v1.0-mini)
Python environment	C:\Users\shash\.venvs\msf_fusion
Document generated	2026-10-07 13:12
Code base analysed	103 Python modules · 8,601 lines · 47 classes · 406 functions/methods · 138 project files
Generator	docs/handover/build_handover.py (regenerate after code changes)

Table of Contents

About This Document 6

Executive Summary 7

What the system does 7

Status at hand-over 7

Key results (nuScenes-mini, 20 epochs, final weights, 81 validation frames, one seed) 7

Most important things to know 8

Getting Started in Ten Minutes 9

1. Environment 9

2. Dataset 9

3. First commands 9

4. Where things are written 10

Project File Structure & Responsibilities 11

File inventory 13

Package dependency graph 23

System Architecture 24

Module map 25

End-to-End Data Flow 26

Step by step 27

Training variant 27

Program Execution Flow 28

Dashboard (python app/streamlit_app.py) 28

Training (python main.py train ...) 29

Evaluation, ablation and scripts 29

Tensor Dimension Reference 31

Symbols 31

Module-by-module shapes (proposed model F, GTX 1650 profile) 31

Training targets 32

Dynamic shapes 32

Coordinate Systems & Transformations 33

Frames 33

Homogeneous transformations 34

Sensor → reference frame (time synchronisation) 34

Ground truth 34

Radar velocity . . . 34
Camera projection and lifting . . . 34
BEV grid . . . 35
Where each transformation happens . . . 35

Camera → BEV Pipeline . . . 36

LiDAR Pipeline . . . 38

Radar Pipeline . . . 39
nuScenes radar fields . . . 39
Processing chain . . . 39
How velocity is represented . . . 39

Deep-Learning Feature Selection . . . 41
Problem it addresses . . . 41
Where it sits . . . 41
Mathematical definition . . . 42
Training objective . . . 42
Statistics and explainability . . . 43
What the trained network learned (measured) . . . 43
Effect on performance . . . 44
Configuration knobs . . . 44

Sensor Fusion & BEV Backbone . . . 45
Fusion methods . . . 45
Deep supervision (L_fusion) . . . 45
BEV backbone . . . 45
Adding a fusion method . . . 45

3D Object Detection . . . 46
Classes . . . 46
Targets (models/detection/targets.py) . . . 46
Head and loss (models/detection/center_head.py) . . . 46
Decoding (models/detection/decoder.py) . . . 46
Attributes . . . 46

Trajectory Planning & Tracking . . . 47
Inputs . . . 47
Baseline planner (planning/trajectory_planner.py: BaselinePlanner) . . . 48
Learned and hybrid planners . . . 48
Tracking (planning/tracking.py) . . . 48
Measured planning performance (mini-val, 69 frames with a full 3 s future) . . . 48

Loss Functions . . . 49
Trajectory loss details . . . 49
Depth loss . . . 49
Which stage uses which loss . . . 49

Training System . . . 50
Training curriculum . . . 50
Trainer internals (training/trainer.py) . . . 51
Hardware and measured cost (GTX 1650, 4 GB, fp32) . . . 51
Training commands . . . 51

Evaluation & Experiments . . . 52
Detection metrics and their verification . . . 52
Planning metrics . . . 52
Ablation study . . . 52

Visualization & Streamlit Application . . . 54
Visualisation modules . . . 54
Dashboard structure . . . 54
Modes and honesty rules . . . 55
Launching . . . 55

Configuration Reference . . . 56
Hardware profiles . . . 61
Experiments . . . 62

Command-Line Reference . . . 63

Line-by-Line Code Walkthroughs . . . 66
Utilities and data layer . . . 66
Neural-network modules . . . 92
Planning, training, evaluation, inference and application . . . 119

API Reference . . . 154
Package app . . . 154
Package datasets . . . 158
Package evaluation . . . 167
Package inference . . . 171
Package (root) . . . 173
Package models . . . 173
Package planning . . . 187
Package scripts . . . 192
Package tests . . . 197
Package training . . . 200

Package utils . . . 203
Package visualization . . . 212

Testing . . . 219

Debugging & Troubleshooting Guide . . . 220
Useful inspection points . . . 220

Extending the Project . . . 221

Artifacts: Checkpoints & Outputs . . . 222
Checkpoints . . . 222
Generated outputs . . . 222

Known Limitations, Open Issues & Next Steps . . . 223
Limitations . . . 223
Recommended next steps . . . 223

Appendix A – Glossary . . . 224

Appendix B – Not Identifiable from the Current Codebase . . . 225

Appendix C – Software Environment . . . 226

About This Document

This document is the technical handover of the project located in `D:\researcpaper\multi_sensor_fusion` (nuScenes data in `D:\researcpaper`). It is written for the next developer or researcher, so that the project can be run, understood, debugged, modified and extended without asking the original author.

How it was produced. The reference parts — directory tree, file inventory, every class and function signature, parameter types, return types, docstrings, raised exceptions, file side effects, import graph and call graph, configuration values, command-line arguments, test list and checkpoint metadata — are **generated from the source code and the files on disk** by `docs/handover/build_handover.py` (static analysis with Python's `ast` module). The code listings in the walkthrough chapter are read from the actual files at build time and each explanation is attached to an exact line of code; the build fails if code and explanation do not match. Narrative chapters were written after inspecting the complete code base. Measured numbers (validation, timings, metrics) come from runs performed on the development machine (GTX 1650, 4 GB) and are labelled as such.

Regenerating the document after code changes:

```
C:\Users\shash\.venvs\msf_fusion\Scripts\python.exe docs/handover/build_handover.py
```

Conventions used in this document

- `monospace` = code identifiers, file paths, configuration keys (`dataset.root` means key `root` in section `dataset`).
- Tensor shapes are written `(B, C, H, W)`; the symbols are defined in the *Tensor Dimension Reference* chapter.
- Default numbers refer to the configuration actually used on the development machine (`configs/*.yaml` + `configs/hardware/gtx1650.yaml` + `configs/local.yaml`).
- "Not identifiable from the current codebase." marks information that cannot be determined from the code or files.

Scope note. The request that defined this document was received truncated after its section 14 ("Deep-Learning Feature Selection"). Sections 1–14 of the request are covered in full. The remaining chapters (fusion, detection, planning, losses, training, evaluation, application, configuration, testing, debugging, extension guide, artifacts, limitations) follow standard technical-handover practice; adjust them if the missing part of the request asked for something different.

Executive Summary

What the system does

The project is a research prototype for **multi-sensor perception and planning for autonomous driving** on the **nuScenes** dataset. For every nuScenes key-frame (2 Hz) it:

1. loads the six surround cameras, the LIDAR_TOP point cloud (multiple sweeps) and the five radars (multiple sweeps), time-synchronised and calibrated into one vehicle frame;
2. converts the cameras into a **Bird's-Eye-View (BEV)** feature map with a Lift-Splat-Shoot network;
3. encodes LiDAR and radar into BEV feature maps on the same 200 × 200 grid (0.5 m cells, ±50 m);
4. applies a **learned, camera-guided feature-selection** network that gates LiDAR and radar features per channel and per BEV cell (the research contribution);
5. fuses the (selected) BEV maps, detects 3-D objects of 10 classes with velocity (CenterPoint-style head);
6. tracks the objects and plans a 3-second ego trajectory (classical A* + speed profile + iLQR MPC, or a learned planner);
7. visualises everything in a Streamlit dashboard built for a faculty demonstration (scene replay, explainability).

Status at hand-over

Area	Status
Data pipeline, calibration, synchronisation	Implemented and validated on nuScenes-mini (LiDAR-in-box agreement r = 0.9997)
Camera BEV, LiDAR, radar encoders, feature selection, fusion, detection, learned planner	Implemented, trained on nuScenes-mini (6 experiments × 20 epochs)
Classical planner, tracker	Implemented; planner beats constant velocity with GT perception (ADE 1.65 vs 1.77 m, collisions 1.4 % vs 14.5 %)
Metrics	nuScenes NDS/mAP port matches the official devkit to $3 \cdot 10^{-4}$; planning metrics with a calibrated collision check
7-stage training curriculum	Implemented and mechanically verified (all stages run, weights hand over); not trained to convergence
Dashboard	Three modes, replay, explainability, method comparison — tested headlessly and in the browser
Full nuScenes (trainval)	Supported by the code; **never run** (needs a larger GPU and the full dataset)
Tests	29 automated tests, all passing

Key results (nuScenes-mini, 20 epochs, final weights, 81 validation frames, one seed)

Experiment	NDS	mAP	velocity error (m/s)	planning ADE (m)
A LiDAR only	0.060	0.032	1.10	1.37
B Radar only	0.053	0.007	1.48	1.26
C LiDAR + radar, concatenation	**0.116**	0.035	**0.77**	1.36
D LiDAR + radar, attention	0.070	0.021	1.42	1.28
E LiDAR + radar, feature selection	0.055	0.027	1.03	1.51
F Camera + LiDAR + radar, feature selection (proposed)	0.094	**0.044**	0.91	1.53

Radar improves LiDAR (C vs A: +0.056 NDS, −0.33 m/s velocity error) and the camera BEV improves LiDAR+radar (F vs E: +0.039 NDS). **Learned feature selection did not beat plain concatenation on nuScenes-mini** (E vs C: −0.061 NDS); the hypothesis remains untested at a meaningful data scale. Absolute numbers are low because nuScenes-mini contains only 323 training frames.

Most important things to know

- **Environment**: Python 3.11 virtual environment at `C:\Users\shash\.venvs\msf_fusion` (outside the project folder).
- **Data**: `configs/local.yaml` sets `dataset.root: D:/researcpaper` and `runtime.hardware_profile: gtx1650`.
- **fp16 is disabled on the GTX 1650** because cuDNN 9.10 returns NaN for fp16 3×3 convolutions with 256 input channels on that GPU (`utils/device.py` tests this at start-up for every GPU).
- **One command per task**: `python main.py <command>` (see *Command-Line Reference*).
- **Every checkpoint embeds its full configuration**; evaluation and the dashboard rebuild the exact model from it.

Getting Started in Ten Minutes

1. Environment

The tested environment is Windows 11, Python 3.11, NVIDIA driver 566 (CUDA 12.7), GTX 1650. The virtual environment already exists on the development machine:

```
C:\Users\shash\.venvs\msf_fusion\Scripts\python.exe -c "import torch; print(torch.__version__, torch.cuda.is_availab
↪    le())"
```

On a new machine:

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
pip install nuscenes-devkit --no-deps
```

`nuscenes-devkit` is optional (official evaluation and the full train/val scene lists) and is installed without dependencies because it pins a matplotlib version without a Python 3.11 wheel; its real runtime dependencies are listed in `requirements.txt`.

2. Dataset

The code expects the standard nuScenes layout under `dataset.root`:

```
<root>/samples/   <root>/sweeps/   <root>/maps/   <root>/v1.0-mini/   [<root>/v1.0-trainval/]   [<root>/can_bus/]
```

Set the root without touching code — `configs/local.yaml` (already set to `D:/researcpaper`), the environment variable `NUSCENES_ROOT`, or `--set dataset.root=...` on any command. Then build the caches:

```
python main.py preprocess --maps
```

3. First commands

Goal	Command	Output
Check calibration / sync	`python main.py validate`	`outputs/validation/report.json` + images
Check every model config	`python main.py smoke --experiments all --quiet`	shapes, losses, memory, latency
Start the dashboard	`python app/streamlit_app.py`	browser at http://localhost:8501
Train the proposed model	`python main.py train --stage joint --experiment F_cam_lidar_radar_fs --epochs 20`	`checkpoints/F_cam_lidar_radar_fs/joint/`
Evaluate a checkpoint	`python main.py evaluate --checkpoint checkpoints/F_cam_lidar_radar_fs/joint/last.pt`	`outputs/evaluation/.../report.json`
Run the ablation	`python main.py ablation --train --epochs 20`	`outputs/ablation/`
Render all BEVs of a frame	`python main.py bev --scene scene-0916 --frame 20`	`outputs/bev/*.jpg`
Export a replay video	`python main.py infer --scene scene-0103 --video`	`outputs/inference/...`
Run the tests	`python main.py test`	29 tests

All commands are run from `D:\researcpaper\multi_sensor_fusion` with the virtual environment's Python (`C:\Users\shash\.venvs\msf_fusion\Scripts\python.exe`).

4. Where things are written

Folder	Content
`outputs/cache/`	per-sample info caches (`infos_<version>_<split>_<hash>.pkl`), reduced map rasters
`outputs/logs/<experiment>/<stage>/`	`run.log`, `history.jsonl`
`checkpoints/<experiment>/<stage>/`	`best.pt`, `last.pt` (shared single-sensor stages under `_shared/`)
`outputs/ablation/`	results table, CSV/JSON, plot
`outputs/validation/`, `outputs/bev/`, `outputs/inference/`	images and videos

Project File Structure & Responsibilities

The tree below is generated from the files on disk (generated data folders such as `outputs/` and `checkpoints/` are summarised; their content is listed in the *Artifacts* chapter). The following table lists **every file** with its purpose, inputs, outputs and dependencies (dependencies of Python files are taken from their import statements: project modules first, then third-party packages).

```
multi_sensor_fusion/
├── .streamlit/
│   └── config.toml
├── app/
│   ├── ui/
│   │   ├── __init__.py
│   │   ├── panels.py
│   │   ├── resources.py
│   │   └── styles.py
│   ├── __init__.py
│   └── streamlit_app.py
├── checkpoints/
│   └── ... 12 generated files (see Artifacts chapter)
├── configs/
│   ├── hardware/
│   │   ├── gtx1650.yaml
│   │   ├── jetson_orin_nano.yaml
│   │   ├── rtx3060.yaml
│   │   ├── rtx4060.yaml
│   │   └── rtx4070.yaml
│   ├── ablation.yaml
│   ├── dataset.yaml
│   ├── fusion.yaml
│   ├── local.example.yaml
│   ├── local.yaml
│   ├── model.yaml
│   ├── planning.yaml
│   └── training.yaml
├── datasets/
│   ├── __init__.py
│   ├── camera_loader.py
│   ├── info_builder.py
│   ├── lidar_loader.py
│   ├── map_prior.py
│   ├── nuscenes_dataset.py
│   ├── nuscenes_db.py
│   ├── radar_loader.py
│   └── splits.py
├── docs/
│   ├── handover/
│   │   ├── content/
│   │   │   ├── brief.py
│   │   │   ├── chapters_a.py
│   │   │   ├── chapters_b.py
│   │   │   ├── file_notes.py
│   │   │   ├── function_notes.py
│   │   │   ├── walk_data.py
│   │   │   ├── walk_models.py
│   │   │   └── walk_system.py
│   │   ├── analysis.py
│   │   ├── build_brief.py
│   │   ├── build_handover.py
│   │   ├── diagrams.py
│   │   └── render.py
│   ├── ARCHITECTURE.md
│   ├── EXPERIMENTS.md
│   └── FACULTY_DEMO.md
├── evaluation/
│   ├── __init__.py
│   ├── ablation.py
│   ├── detection_metrics.py
│   ├── evaluator.py
│   ├── official_eval.py
│   └── trajectory_metrics.py
├── inference/
│   ├── __init__.py
│   └── pipeline.py
├── models/
│   ├── camera_bev/
│   │   ├── __init__.py
│   │   ├── image_backbone.py
│   │   └── lss.py
│   ├── common/
│   │   ├── __init__.py
```

```
│   │   ├── blocks.py
│   │   └── pillars.py
│   ├── detection/
│   │   ├── __init__.py
│   │   ├── center_head.py
│   │   ├── decoder.py
│   │   └── targets.py
│   ├── feature_selection/
│   │   ├── __init__.py
│   │   ├── losses.py
│   │   └── selection.py
│   ├── fusion/
│   │   ├── __init__.py
│   │   ├── bev_backbone.py
│   │   ├── fusion_methods.py
│   │   └── registry.py
│   ├── lidar_encoder/
│   │   ├── __init__.py
│   │   └── pillar_encoder.py
│   ├── radar_encoder/
│   │   ├── __init__.py
│   │   └── radar_encoder.py
│   ├── trajectory/
│   │   ├── __init__.py
│   │   ├── losses.py
│   │   └── planning_head.py
│   ├── __init__.py
│   └── fusion_model.py
├── outputs/
│   └── ... 167 generated files (see Artifacts chapter)
├── planning/
│   ├── __init__.py
│   ├── astar.py
│   ├── mpc.py
│   ├── occupancy_grid.py
│   ├── tracking.py
│   └── trajectory_planner.py
├── scripts/
│   ├── __init__.py
│   ├── benchmark.py
│   ├── check_eval_vs_devkit.py
│   ├── download_dataset.py
│   ├── eval_planner.py
│   ├── evaluate.py
│   ├── inference.py
│   ├── preprocess.py
│   ├── render_bev.py
│   ├── smoke_test.py
│   └── validate_sensors.py
├── tests/
│   ├── __init__.py
│   ├── conftest.py
│   ├── test_app.py
│   ├── test_dataset.py
│   ├── test_geometry.py
│   ├── test_metrics_planning.py
│   └── test_models.py
├── training/
│   ├── __init__.py
│   ├── cli.py
│   ├── train_bev.py
│   ├── train_detection.py
│   ├── train_end_to_end.py
│   ├── train_fusion.py
│   ├── train_joint.py
│   ├── train_lidar.py
│   ├── train_radar.py
│   ├── train_trajectory.py
│   └── trainer.py
├── utils/
│   ├── __init__.py
│   ├── bev_grid.py
│   ├── checkpoint.py
│   ├── config.py
│   ├── device.py
│   ├── geometry.py
│   ├── logging_utils.py
│   ├── raster.py
│   ├── seed.py
│   └── timer.py
├── visualization/
│   ├── __init__.py
│   ├── bev_view.py
│   ├── camera_view.py
│   ├── common.py
```

```
│   ├── feature_selection.py
│   ├── lidar_view.py
│   ├── radar_view.py
│   └── trajectory_view.py
├── .gitignore
├── environment.yml
├── main.py
├── pytest.ini
├── README.md
└── requirements.txt
```

File inventory

File	Location	Purpose	Inputs	Outputs	Dependencies
.gitignore	(root)	Git ignore rules: caches, outputs, checkpoints, local config, videos. *Why:* Keeps large/generated files out of version control.	–	–	–
config.toml	.streamlit	Streamlit theme (dark, accent #3987e5) and server options. *Why:* Consistent dashboard look when launched from the project root.	–	–	–
__init__.py	app	Package marker (makes the folder an importable Python package). *Why:* Required for `from <package>.<module> import ...`.	–	–	–
streamlit_app.py	app	Streamlit dashboard / faculty demo: three modes, scene replay, all panels, explainability, method comparison, ablation and training curves. *Why:* The interactive application.	User interaction, configs, checkpoints, nuScenes data	Web UI (localhost:8501)	app.ui, app.ui.resources, app.ui.styles, evaluation.detection_metrics, evaluation.trajectory_metrics, inference.pipeline, utils.config, altair, collections, numpy, pandas, pathlib, streamlit, sys, time
__init__.py	app/ui	Package marker (makes the folder an importable Python package). *Why:* Required for `from <package>.<module> import ...`.	–	–	–
panels.py	app/ui	Panel renderers: camera, fusion tiles, driving BEV, detection table, GT comparison, ego status, trajectory close-up, feature-selection explanation. *Why:* Keeps the app script readable.	`FrameResult`, images, options	RGB images, DataFrames, dicts	inference.pipeline, utils.geometry, visualization.bev_view, visualization.camera_view, visualization.common, visualization.feature_selection, visualization.lidar_view, visualization.radar_view, visualization.trajectory_view, numpy, pandas, pathlib
resources.py	app/ui	Checkpoint discovery, dataset-version detection, base config, ablation results and training history loading. *Why:* I/O helpers for the app.	Config, filesystem	Paths, dicts, lists	utils.config, json, pathlib
styles.py	app/ui	Dashboard CSS and HTML snippets (pipeline strip, banners, KPI tiles, chips). *Why:* Presentation layer.	Stage timings, texts	HTML strings	html
ablation.yaml	configs	Experiment table (A–I) as config overrides, default run list, dashboard method mapping. *Why:* Defines the research experiments.	–	Read by `load_config`, ablation runner, dashboard	–
dataset.yaml	configs	Dataset location/version/splits, sensors, sweep counts, camera augmentation, radar filters, BEV augmentation, GT filter, the BEV grid and the class list. *Why:* Single source of data settings.	–	Read by `utils.config.load_config`	–

File	Location	Purpose	Inputs	Outputs	Dependencies
fusion.yaml	configs	Fusion method, modality switches, feature-selection settings and sparsity targets. *Why:* The research knobs in one place.	–	Read by `load_config`	–
gtx1650.yaml	configs/hardware	GTX 1650 profile: 128×352 images, 5 sweeps, batch 1 × accum 4, fp32, checkpointing. *Why:* Fits 4 GB and avoids the fp16 cuDNN bug.	–	Merged after base configs	–
jetson_orin_nano.yaml	configs/hardware	Jetson Orin Nano profile (inference/demo only). *Why:* Embedded deployment.	–	–	–
rtx3060.yaml	configs/hardware	RTX 3060 profile: 256×704 images, 10 LiDAR sweeps, batch 4, AMP. *Why:* Bigger GPUs.	–	–	–
rtx4060.yaml	configs/hardware	RTX 4060 profile: 256×704, batch 2 × accum 2, AMP. *Why:* 8 GB GPUs.	–	–	–
rtx4070.yaml	configs/hardware	RTX 4070 profile: 256×704, ResNet34, batch 4, AMP. *Why:* 12 GB GPUs.	–	–	–
local.example.yaml	configs	Template for local.yaml on a new machine. *Why:* Onboarding.	–	–	–
local.yaml	configs	Machine-specific settings: dataset root `D:/researcpaper`, hardware profile `gtx1650`. *Why:* Keeps machine paths out of the committed configs.	–	Merged last before CLI overrides	–
model.yaml	configs	Architecture hyper-parameters of all sub-networks (camera BEV, LiDAR, radar, SECOND-FPN, CenterHead, planning head). *Why:* Architecture changes without code changes.	–	Read by `load_config`	–
planning.yaml	configs	Planner type, horizon, ego geometry and limits, collision classes, planning grid, occupancy, A*, speed and MPC parameters. *Why:* Planner tuning.	–	Read by `load_config`	–
training.yaml	configs	Runtime (seed, device, profile, folders), optimisation, evaluation, loss weights (λ) and the 7-stage curriculum (+ joint stage). *Why:* Training behaviour.	–	Read by `load_config`	–
__init__.py	datasets	Package marker (makes the folder an importable Python package). *Why:* Required for `from <package>.<module> import ...`.	–	–	–
camera_loader.py	datasets	Image loading, augmentation and calibration matrices; LiDAR depth targets. *Why:* Inputs of the camera BEV encoder.	Info dict, image files	Images (6,3,H,W) + K, post_rot, post_trans, cam2ref, cam_valid; depth maps	utils.geometry, utils.logging_utils, PIL, dataclasses, logging, numpy, pathlib
info_builder.py	datasets	Builds and caches per-sample infos: calibration, poses, sweeps, GT in the reference frame, ego state, ego future/past, agent futures, command. *Why:* Parse tables once; portable cache.	`NuScenesDB`, config	`outputs/cache/infos_*.pkl`, list of info dicts	datasets.nuscenes_db, datasets.splits, utils.config, utils.geometry, hashlib, json, logging, numpy, pathlib, pickle
lidar_loader.py	datasets	Multi-sweep LiDAR accumulation in the reference frame. *Why:* LiDAR encoder input.	Info dict, `.pcd.bin` files	Points (P, 5)	utils.bev_grid, utils.geometry, utils.logging_utils, logging, numpy, pathlib
map_prior.py	datasets	nuScenes raster map prior: decode once, cache at 0.5 m/px, crop around the ego. *Why:* Road background and optional planner cost.	Map PNG, ego pose	Map patch (display layout)	PIL, cv2, logging, numpy, pathlib

File	Location	Purpose	Inputs	Outputs	Dependencies
nuscenes_dataset.py	datasets	PyTorch Dataset (sensors + targets + augmentation) and `collate_batch`. *Why:* Feeds training, evaluation and inference.	Infos, config	Item dicts / batches	datasets.camera_loader, datasets.info_builder, datasets.lidar_loader, datasets.radar_loader, models.detection.targets, utils.bev_grid, utils.config, utils.geometry, utils.raster, logging, numpy, pathlib, torch
nuscenes_db.py	datasets	Loads the 13 nuScenes JSON tables and builds reverse indexes. *Why:* Devkit-free table access.	`<root>/<version>/*.json`	`NuScenesDB` with token lookup	json, logging, pathlib, time
radar_loader.py	datasets	Radar PCD parsing, filtering, multi-sweep accumulation, velocity rotation. *Why:* Radar encoder input.	Info dict, `.pcd` files	Points (Q, 10)	utils.bev_grid, utils.logging_utils, logging, numpy, pathlib
splits.py	datasets	Official scene splits (mini embedded; train/val from the devkit). *Why:* Correct benchmark splits.	Split name, available scenes	Scene name list	logging, nuscenes
ARCHITECTURE.md	docs	Architecture diagrams, tensor-flow table, model choices, per-module details, coordinate frames, memory engineering. *Why:* Design documentation.	–	–	–
EXPERIMENTS.md	docs	Hypothesis, experiment design, results, analysis and next experiments. *Why:* Research record.	–	–	–
FACULTY_DEMO.md	docs	Step-by-step presentation script and honest-demo notes. *Why:* Presentation support.	–	–	–
analysis.py	docs/handover	AST analysis of the codebase used to generate this document. *Why:* Documentation tooling.	Project `*.py` files	Module/class/function metadata	–
build_brief.py	docs/handover	Builds the plain-language brief study guide (content in content/brief.py). *Why:* Documentation tooling.	content/brief.py, diagrams, result images	Project_Explained_Brief_Guide.pdf	–
build_handover.py	docs/handover	Builds this PDF. *Why:* Documentation tooling.	Code, configs, content	The handover PDF	–
brief.py	docs/handover/content	Hand-written documentation content used by build_handover.py. *Why:* Documentation tooling.	–	–	–
chapters_a.py	docs/handover/content	Hand-written documentation content used by build_handover.py. *Why:* Documentation tooling.	–	–	–
chapters_b.py	docs/handover/content	Hand-written documentation content used by build_handover.py. *Why:* Documentation tooling.	–	–	–
file_notes.py	docs/handover/content	Hand-written documentation content used by build_handover.py. *Why:* Documentation tooling.	–	–	–
function_notes.py	docs/handover/content	Hand-written documentation content used by build_handover.py. *Why:* Documentation tooling.	–	–	–
walk_data.py	docs/handover/content	Hand-written documentation content used by build_handover.py. *Why:* Documentation tooling.	–	–	–
walk_models.py	docs/handover/content	Hand-written documentation content used by build_handover.py. *Why:* Documentation tooling.	–	–	–
walk_system.py	docs/handover/content	Hand-written documentation content used by build_handover.py. *Why:* Documentation tooling.	–	–	–
diagrams.py	docs/handover	Matplotlib diagrams of this document. *Why:* Documentation tooling.	–	PNG files in build/	–

File	Location	Purpose	Inputs	Outputs	Dependencies
render.py	docs/handover	Markdown-subset → ReportLab renderer, styles, document template. *Why:* Documentation tooling.	Markdown text, code	ReportLab flowables	–
environment.yml	(root)	Conda environment (Python 3.11 + pip requirements). *Why:* Alternative to venv.	–	–	–
__init__.py	evaluation	Package marker (makes the folder an importable Python package). *Why:* Required for `from <package>.<module> import ...`.	–	–	–
ablation.py	evaluation	Ablation runner: train (optional) + evaluate experiments, tables, hypothesis deltas, plot. *Why:* Research experiment automation.	Experiment names, checkpoints	`outputs/ablation/*`	datasets.nuscenes_dataset, evaluation.evaluator, models.fusion_model, training.trainer, utils.checkpoint, utils.config, utils.device, utils.logging_utils, argparse, csv, json, logging, matplotlib, matplotlib.pyplot, pathlib, sys, time, torch
detection_metrics.py	evaluation	nuScenes detection metrics (mAP, TP errors, NDS) — in-memory port. *Why:* Fast metrics during training and in the app.	Predictions + GT per sample	Metrics dict incl. per-class	utils.geometry, collections, numpy
evaluator.py	evaluation	Runs a model over a loader and computes all metric families. *Why:* Shared by trainer, ablation and scripts.	Model, DataLoader, config	Result dict	evaluation.detection_metrics, evaluation.trajectory_metrics, models.fusion_model, planning.trajectory_planner, utils.config, utils.device, utils.timer, collections, logging, numpy, torch
official_eval.py	evaluation	Export to the nuScenes submission format and run the devkit `DetectionEval`. *Why:* Official cross-check.	Detections, infos	results JSON, devkit metrics	evaluation.detection_metrics, utils.geometry, json, numpy, nuscenes, pathlib
trajectory_metrics.py	evaluation	ADE, FDE, L2@t, collision, lateral/longitudinal error, smoothness. *Why:* Planning evaluation.	Predicted & logged trajectories, future agents	Metrics dicts	utils.geometry, numpy
__init__.py	inference	Package marker (makes the folder an importable Python package). *Why:* Required for `from <package>.<module> import ...`.	–	–	–

File	Location	Purpose	Inputs	Outputs	Dependencies
pipeline.py	inference	Frame-by-frame processing (data → model → tracking → planning → metrics). *Why:* Shared by the app and scripts.	Config, checkpoint, frame index	`FrameResult`	datasets.info_builder, datasets.map_prior, datasets.nuscenes_dataset, datasets.splits, evaluation.detection_metrics, evaluation.trajectory_metrics, models.fusion_model, planning.tracking, planning.trajectory_planner, utils.bev_grid, utils.checkpoint, utils.config, utils.device, utils.timer, dataclasses, logging, numpy, nuscenes, pathlib, time, torch
main.py	(root)	Single CLI entry point dispatching sub-commands to the scripts. *Why:* One memorable command for every task.	`sys.argv` (sub-command + its arguments)	Runs the selected script / Streamlit / pytest	pathlib, runpy, subprocess, sys
__init__.py	models	Package marker (makes the folder an importable Python package). *Why:* Required for `from <package>.<module> import ...`.	–	–	–
__init__.py	models/camera_bev	Package marker (makes the folder an importable Python package). *Why:* Required for `from <package>.<module> import ...`.	–	–	–
image_backbone.py	models/camera_bev	ResNet + FPN-lite image backbone (stride 16). *Why:* Image features.	(B·N, 3, H, W)	(B·N, 256, H/16, W/16)	models.common.blocks, logging, torch, torch.nn.functional, torchvision
lss.py	models/camera_bev	Lift-Splat-Shoot camera BEV encoder with camera-aware depth and depth loss. *Why:* Camera → BEV.	Images + calibration	(B, 64, 200, 200), depth	models.camera_bev.image_backbone, models.common.blocks, utils.bev_grid, math, torch, torch.nn.functional
__init__.py	models/common	Package marker (makes the folder an importable Python package). *Why:* Required for `from <package>.<module> import ...`.	–	–	–
blocks.py	models/common	ConvBNReLU, ResBlock, SE, spatial attention, CBAM. *Why:* Reusable layers.	(B, C, H, W)	(B, C', H', W')	torch
pillars.py	models/common	Dynamic pillar VFE, scatter ops, BEV raster statistics. *Why:* Point → BEV encoding.	Batched points (P, 1+F)	(B, C, ny, nx)	utils.bev_grid, torch
__init__.py	models/detection	Package marker (makes the folder an importable Python package). *Why:* Required for `from <package>.<module> import ...`.	–	–	–
center_head.py	models/detection	CenterPoint head + detection losses. *Why:* 3-D detection.	(B, 256, 100, 100)	Head maps, losses	models.common.blocks, torch, torch.nn.functional
decoder.py	models/detection	Peak extraction, box decoding, circle NMS. *Why:* Detections from head maps.	Head maps	Boxes (K, 9), scores, labels	utils.bev_grid, numpy, torch, torch.nn.functional
targets.py	models/detection	CenterPoint heatmap/regression targets and fg heatmap (NumPy). *Why:* Supervision.	GT boxes	Target arrays	utils.bev_grid, numpy

File	Location	Purpose	Inputs	Outputs	Dependencies
__init__.py	models/feature_selection	Package marker (makes the folder an importable Python package). *Why:* Required for `from <package>.<module> import ...`.	–	–	–
losses.py	models/feature_selection	Coverage / sparsity / binarisation losses for the gates. *Why:* Train the selection.	Gates, fg heatmap	Loss dict	torch
selection.py	models/feature_selection	Camera-guided cross-modal feature selection (channel + spatial gates). *Why:* Research contribution.	LiDAR/radar BEV (+ camera guide)	Selected features + gates	models.common.blocks, torch
__init__.py	models/fusion	Package marker (makes the folder an importable Python package). *Why:* Required for `from <package>.<module> import ...`.	–	–	–
bev_backbone.py	models/fusion	SECOND-FPN BEV backbone. *Why:* Multi-scale BEV features.	(B, 128, 200, 200)	(B, 256, 100, 100)	models.common.blocks, torch
fusion_methods.py	models/fusion	ConcatFusion and AttentionGatedFusion. *Why:* Fusion.	Dict of BEV maps	(B, 128, 200, 200) + info	models.common.blocks, torch
registry.py	models/fusion	Fusion registry and factory. *Why:* Pluggable fusion methods.	Method name	Module	models.fusion.fusion_methods, torch
fusion_model.py	models	`MultiSensorFusionNet`: wires encoders, feature selection, fusion, backbone and heads; total loss; decoding. *Why:* The model.	Batch dict	Output dict, losses, detections	models.camera_bev.lss, models.detection.center_head, models.detection.decoder, models.feature_selection.losses, models.feature_selection.selection, models.fusion.bev_backbone, models.fusion.registry, models.lidar_encoder.pillar_encoder, models.radar_encoder.radar_encoder, models.trajectory.losses, models.trajectory.planning_head, utils.bev_grid, utils.config, utils.timer, torch
__init__.py	models/lidar_encoder	Package marker (makes the folder an importable Python package). *Why:* Required for `from <package>.<module> import ...`.	–	–	–
pillar_encoder.py	models/lidar_encoder	LiDAR BEV encoder. *Why:* LiDAR features.	LiDAR points	(B, 64, 200, 200)	models.common.blocks, models.common.pillars, utils.bev_grid, torch
__init__.py	models/radar_encoder	Package marker (makes the folder an importable Python package). *Why:* Required for `from <package>.<module> import ...`.	–	–	–
radar_encoder.py	models/radar_encoder	Radar BEV encoder with velocity raster. *Why:* Radar features.	Radar points	(B, 64, 200, 200)	models.common.blocks, models.common.pillars, utils.bev_grid, torch
__init__.py	models/trajectory	Package marker (makes the folder an importable Python package). *Why:* Required for `from <package>.<module> import ...`.	–	–	–
losses.py	models/trajectory	Planning losses (L1, collision, smoothness, goal). *Why:* Train the planner.	Waypoints, batch	Loss dict	utils.bev_grid, torch, torch.nn.functional
planning_head.py	models/trajectory	Learned planner head. *Why:* Learned trajectory.	BEV features, ego state, command	Waypoints (B, 6, 2)	models.common.blocks, torch, torch.nn.functional

File	Location	Purpose	Inputs	Outputs	Dependencies
__init__.py	planning	Package marker (makes the folder an importable Python package). *Why:* Required for `from <package>.<module> import ...`.	–	–	–
astar.py	planning	Grid A* and nearest-free-cell search. *Why:* Geometric path.	Cost grid, start, goal	Cell path	heapq, math, numpy
mpc.py	planning	iLQR MPC on a kinematic bicycle model. *Why:* Feasible smooth trajectory.	Initial state, reference	States, controls	utils.geometry, dataclasses, numpy
occupancy_grid.py	planning	Occupancy and cost layers for planning. *Why:* Planner scene model.	Boxes, LiDAR points	Occupancy / cost arrays	utils.bev_grid, utils.raster, cv2, numpy
tracking.py	planning	Greedy multi-object tracker. *Why:* Track IDs, motion trails.	Detections + ego pose	Track ids, histories	utils.geometry, dataclasses, numpy
trajectory_planner.py	planning	Baseline / learned / hybrid planners. *Why:* Ego trajectory.	Detections, ego state, command, LiDAR	`PlanResult`	planning.astar, planning.mpc, planning.occupancy_grid, cv2, dataclasses, numpy, time
pytest.ini	(root)	pytest configuration: test path, `pythonpath = .`, no cache, ignore deprecation warnings. *Why:* Lets `python -m pytest` find the project packages.	–	–	–
README.md	(root)	User guide: architecture summary, validation results, install, dataset, training, evaluation, ablation results, demo, hardware notes, limitations, command cheat-sheet. *Why:* Entry point for humans.	–	–	–
requirements.txt	(root)	pip requirements (PyTorch cu126 wheels + runtime deps; devkit installed separately with --no-deps). *Why:* Reproducible environment.	–	–	–
__init__.py	scripts	Package marker (makes the folder an importable Python package). *Why:* Required for `from <package>.<module> import ...`.	–	–	–
benchmark.py	scripts	Latency / memory benchmark per experiment. *Why:* Hardware sizing.	Config	Console table	datasets.nuscenes_dataset, models.fusion_model, utils.config, utils.device, utils.logging_utils, utils.timer, argparse, json, numpy, pathlib, sys, torch
check_eval_vs_devkit.py	scripts	Cross-check of the metric port against the official devkit. *Why:* Metric correctness.	mini_val infos	Comparison table	datasets.info_builder, evaluation.detection_metrics, evaluation.official_eval, utils.config, utils.logging_utils, json, numpy, pathlib, sys
download_dataset.py	scripts	Dataset verification / optional mini download after accepting the terms. *Why:* Onboarding.	Config / flags	Dataset folder	utils.config, argparse, pathlib, sys, tarfile, urllib.request
eval_planner.py	scripts	Planner evaluation with GT perception / constant velocity / logged trajectory. *Why:* Planner baseline.	Infos	Console JSON	datasets.info_builder, datasets.lidar_loader, evaluation.trajectory_metrics, planning.trajectory_planner, utils.bev_grid, utils.config, utils.logging_utils, argparse, json, numpy, pathlib, sys

File	Location	Purpose	Inputs	Outputs	Dependencies
evaluate.py	scripts	Evaluate a checkpoint (+ optional official devkit). *Why:* Reporting.	Checkpoint	`outputs/evaluation/<dir>/report.json`	datasets.nuscenes_dataset, evaluation.evaluator, evaluation.official_eval, models.fusion_model, utils.checkpoint, utils.config, utils.device, utils.logging_utils, argparse, json, numpy, pathlib, sys, torch
inference.py	scripts	Scene replay export: composite frames, MP4, summary and global-frame detections. *Why:* Offline demo / backup video.	Scene, checkpoint	`outputs/inference/<scene>_<mode>/`	app.ui, app.ui.resources, evaluation.detection_metrics, evaluation.official_eval, evaluation.trajectory_metrics, inference.pipeline, utils.config, utils.logging_utils, visualization.common, argparse, cv2, json, numpy, pathlib, sys
preprocess.py	scripts	Build info caches (+ map caches). *Why:* Faster start-up.	Dataset	`outputs/cache/`	datasets.info_builder, datasets.map_prior, datasets.nuscenes_db, utils.config, utils.logging_utils, argparse, pathlib, sys, time
render_bev.py	scripts	Render all BEV representations of one frame into one image. *Why:* BEV inspection.	Scene, frame, checkpoint	`outputs/bev/*.jpg`	app.ui, app.ui.resources, inference.pipeline, utils.config, utils.logging_utils, visualization.camera_view, visualization.common, visualization.feature_selection, visualization.lidar_view, visualization.radar_view, argparse, cv2, numpy, pathlib, sys
smoke_test.py	scripts	One training step + inference per experiment with shapes, memory and latency. *Why:* Quick health check.	Config	Console report	datasets.nuscenes_dataset, models.fusion_model, utils.config, utils.device, utils.logging_utils, utils.seed, utils.timer, argparse, json, pathlib, sys, time, torch
validate_sensors.py	scripts	Sync / calibration validation and verification images. *Why:* Phase-2 validation.	Infos, sensor files	`outputs/validation/report.json` + images	datasets.camera_loader, datasets.info_builder, datasets.lidar_loader, datasets.radar_loader, utils.bev_grid, utils.config, utils.geometry, utils.logging_utils, visualization.camera_view, visualization.common, argparse, cv2, json, numpy, pathlib, sys

File	Location	Purpose	Inputs	Outputs	Dependencies
__init__.py	tests	Package marker (makes the folder an importable Python package). *Why:* Required for `from <package>.<module> import ...`.	–	–	–
conftest.py	tests	pytest fixtures (`cfg`, `val_infos`) and the `requires_dataset` skip marker. *Why:* Shared test setup.	–	Fixtures	datasets.info_builder, utils.config, pathlib, pytest, sys
test_app.py	tests	Headless dashboard tests (all modes, explain, next frame). *Why:* UI regression tests.	–	–	tests.conftest, pathlib, pytest, streamlit
test_dataset.py	tests	Real-data tests: calibration agreement, item shapes, LSS geometry round trip. *Why:* Data correctness.	–	–	datasets.lidar_loader, datasets.nuscenes_dataset, models.camera_bev.lss, tests.conftest, utils.bev_grid, utils.geometry, numpy, pathlib, torch
test_geometry.py	tests	Geometry and rasterisation unit tests. *Why:* Convention checks.	–	–	utils.bev_grid, utils.geometry, utils.raster, numpy
test_metrics_planning.py	tests	Metric and planner unit tests. *Why:* Metric / planner correctness.	–	–	evaluation.detection_metrics, evaluation.trajectory_metrics, planning.astar, planning.mpc, planning.trajectory_planner, utils.config, numpy
test_models.py	tests	Forward/backward for every experiment; gate properties. *Why:* Model regression tests.	–	–	models.feature_selection.selection, models.fusion_model, utils.config, pytest, torch
__init__.py	training	Package marker (makes the folder an importable Python package). *Why:* Required for `from <package>.<module> import ...`.	–	–	–
cli.py	training	Training CLI (single stage or full curriculum). *Why:* Shared entry for train_*.py.	argv	Runs Trainer.fit	training.trainer, argparse, json, pathlib, sys
train_bev.py	training	Stage 1 entry (camera BEV). *Why:* Convenience.	argv	–	training.cli, pathlib, sys
train_detection.py	training	Stage 5 entry (detection fine-tuning). *Why:* Convenience.	argv	–	training.cli, pathlib, sys
train_end_to_end.py	training	Stage 7 entry (end-to-end). *Why:* Convenience.	argv	–	training.cli, pathlib, sys
train_fusion.py	training	Stage 4 entry (feature selection + fusion). *Why:* Convenience.	argv	–	training.cli, pathlib, sys
train_joint.py	training	Single-stage joint training entry (ablation recipe). *Why:* Convenience.	argv	–	training.cli, pathlib, sys
train_lidar.py	training	Stage 2 entry (LiDAR). *Why:* Convenience.	argv	–	training.cli, pathlib, sys
train_radar.py	training	Stage 3 entry (radar). *Why:* Convenience.	argv	–	training.cli, pathlib, sys
train_trajectory.py	training	Stage 6 entry (learned planner). *Why:* Convenience.	argv	–	training.cli, pathlib, sys

File	Location	Purpose	Inputs	Outputs	Dependencies
trainer.py	training	Stage-aware trainer. *Why:* Training loop.	Experiment, stage, overrides	Checkpoints, logs, history	datasets.nuscenes_dataset, evaluation.evaluator, models.fusion_model, utils.checkpoint, utils.config, utils.device, utils.logging_utils, utils.seed, json, logging, math, pathlib, time, torch
__init__.py	utils	Package marker (makes the folder an importable Python package). *Why:* Required for `from <package>.<module> import ...`.	–	–	–
bev_grid.py	utils	BEV grid definition. *Why:* Shared spatial layout.	`bev` config	`BEVGrid`	dataclasses, numpy
checkpoint.py	utils	Atomic checkpoint save/load, partial weight loading, discovery. *Why:* Persistence.	Model, config	`.pt` files	logging, pathlib, torch
config.py	utils	Layered YAML configuration. *Why:* Configuration system.	YAML files, env vars, CLI	`Config`	copy, os, pathlib, re, yaml
device.py	utils	Device choice, fp16 self-test, memory report, recursive `.to()`. *Why:* Hardware robustness.	Preference, tensors	Device, flags, dicts	logging, torch
geometry.py	utils	Rigid transforms, quaternions, boxes, projection, BEV augmentation. *Why:* Geometry conventions.	Arrays	Arrays	numpy
logging_utils.py	utils	Logging setup and warn-once helper. *Why:* Consistent logs.	Log dir	Log files	logging, pathlib, sys
raster.py	utils	Box / point rasterisation. *Why:* Occupancy and targets.	Boxes, points, grid	Arrays	utils.bev_grid, utils.geometry, cv2, numpy
seed.py	utils	Seeding and DataLoader worker seeding. *Why:* Reproducibility.	Seed	–	numpy, os, random, torch
timer.py	utils	CUDA-synchronised stage timer with callbacks. *Why:* Latency measurement / live pipeline strip.	Stage names	Timings	collections, contextlib, time, torch
__init__.py	visualization	Package marker (makes the folder an importable Python package). *Why:* Required for `from <package>.<module> import ...`.	–	–	–
bev_view.py	visualization	Autonomous-driving BEV scene renderer. *Why:* Main BEV panel.	`SceneLayers`	RGB image	visualization.common, cv2, dataclasses, numpy
camera_view.py	visualization	Projection of points/boxes into images, camera mosaic, geometric IPM camera BEV. *Why:* Camera panel.	Images, calibration, boxes	RGB images	utils.geometry, visualization.common, PIL, cv2, numpy, pathlib
common.py	visualization	Colours, colormaps, BEV canvas, display conversion, labels, tiling. *Why:* Drawing primitives.	Arrays	RGB uint8 images	utils.bev_grid, utils.geometry, cv2, functools, matplotlib, numpy
feature_selection.py	visualization	Explainability visualisations for the gates. *Why:* Explain feature selection.	Maps, gates, boxes	RGB images, tables	utils.bev_grid, utils.raster, visualization.common, cv2, numpy
lidar_view.py	visualization	LiDAR BEV rendering, feature energy. *Why:* LiDAR tile.	Points	RGB image	visualization.common, numpy
radar_view.py	visualization	Radar BEV rendering with velocity arrows. *Why:* Radar tile.	Radar points	RGB image	visualization.common, numpy
trajectory_view.py	visualization	Trajectory close-up renderer. *Why:* Trajectory panel.	Trajectories, boxes	RGB image	visualization.common, cv2, numpy

Package dependency graph

Which project packages import which (derived from the import statements):

Package	Imports project packages
app	evaluation, inference, utils, visualization
datasets	models, utils
evaluation	datasets, models, planning, training, utils
inference	datasets, evaluation, models, planning, utils
main	–
models	utils
planning	utils
scripts	app, datasets, evaluation, inference, models, planning, utils, visualization
tests	datasets, evaluation, models, planning, utils
training	datasets, evaluation, models, utils
utils	–
visualization	utils

System Architecture

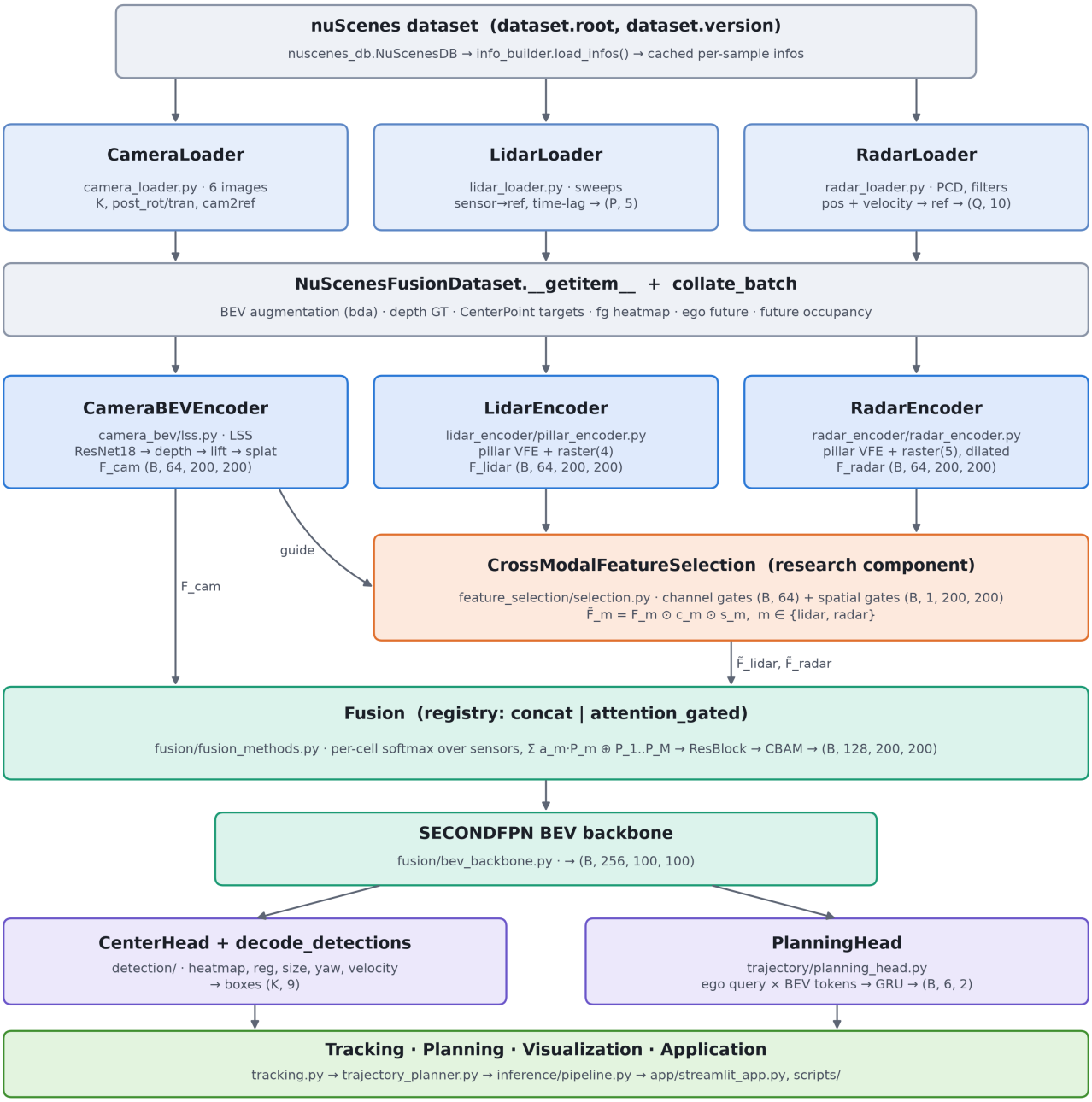


Figure 1 – System architecture as implemented (class / file names and default tensor shapes).

The architecture has five layers.

1. **Data layer** (`datasets/`). `NuScenesDB` reads the nuScenes tables; `info_builder` turns them into cached, self-contained per-sample records ("infos"); three loaders read the sensor files; `NuScenesFusionDataset` assembles training/inference items and `collate_batch` builds batches.
2. **Sensor encoders** (`models/camera_bev`, `models/lidar_encoder`, `models/radar_encoder`). Each produces a `(B, 64, 200, 200)` BEV map on the *same* grid in the *same* reference frame (ego vehicle at the LiDAR key-frame time). LiDAR and radar outputs end with explicit, named physical channels.
3. **Feature selection and fusion** (`models/feature_selection`, `models/fusion`). The selection network gates LiDAR and radar features, using the camera BEV as a guide; the fusion module combines camera, LiDAR and radar maps (`concat` or `attention_gated`, chosen by configuration).
4. **Heads** (`models/fusion/bev_backbone.py`, `models/detection`, `models/trajectory`). A SECOND-FPN backbone feeds a CenterPoint detection head and a learned planning head. `models/fusion_model.py` wires everything and computes the total loss.

5. **Downstream and application** (`planning/`, `inference/`, `visualization/`, `app/`, `scripts/`). Tracking and the classical/learned/hybrid planners, the frame pipeline, rendering, the Streamlit dashboard and the CLI tools. Training and evaluation live in `training/` and `evaluation/`.

Module map

Component	File	Main class / function	Output
Table access	`datasets/nuscenes_db.py`	`NuScenesDB`	indexed tables
Sample infos	`datasets/info_builder.py`	`load_infos`, `build_sample_info`	list of info dicts (cached)
Camera input	`datasets/camera_loader.py`	`CameraLoader`, `lidar_depth_maps`	images + calibration, depth targets
LiDAR input	`datasets/lidar_loader.py`	`LidarLoader`	`(P, 5)` points
Radar input	`datasets/radar_loader.py`	`RadarLoader`, `read_radar_pcd`	`(Q, 10)` points
Dataset	`datasets/nuscenes_dataset.py`	`NuScenesFusionDataset`, `collate_batch`	item / batch dicts
Camera BEV	`models/camera_bev/lss.py`	`CameraBEVEncoder`	`(B, 64, 200, 200)`
LiDAR BEV	`models/lidar_encoder/pillar_encoder.py`	`LidarEncoder`	`(B, 64, 200, 200)`
Radar BEV	`models/radar_encoder/radar_encoder.py`	`RadarEncoder`	`(B, 64, 200, 200)`
Feature selection	`models/feature_selection/selection.py`	`CrossModalFeatureSelection`	selected maps + gates
Fusion	`models/fusion/fusion_methods.py`	`AttentionGatedFusion`, `ConcatFusion`	`(B, 128, 200, 200)`
BEV backbone	`models/fusion/bev_backbone.py`	`SECONDFPN`	`(B, 256, 100, 100)`
Detection	`models/detection/*.py`	`CenterHead`, `decode_detections`	boxes `(K, 9)`
Learned planner	`models/trajectory/planning_head.py`	`PlanningHead`	waypoints `(B, 6, 2)`
Whole network	`models/fusion_model.py`	`MultiSensorFusionNet`	output dict, losses
Tracking	`planning/tracking.py`	`MultiObjectTracker`	track ids
Planning	`planning/trajectory_planner.py`	`BaselinePlanner`, `LearnedPlanner`, `HybridPlanner`	`PlanResult`
Frame pipeline	`inference/pipeline.py`	`FusionPipeline`	`FrameResult`
Application	`app/streamlit_app.py`	(script)	web UI

End-to-End Data Flow

This chapter follows **one nuScenes sample** through the system as implemented in `FusionPipeline.run()` (dashboard and scripts). Training differs only in that augmentation is random, targets are built and losses are computed (described at the end).

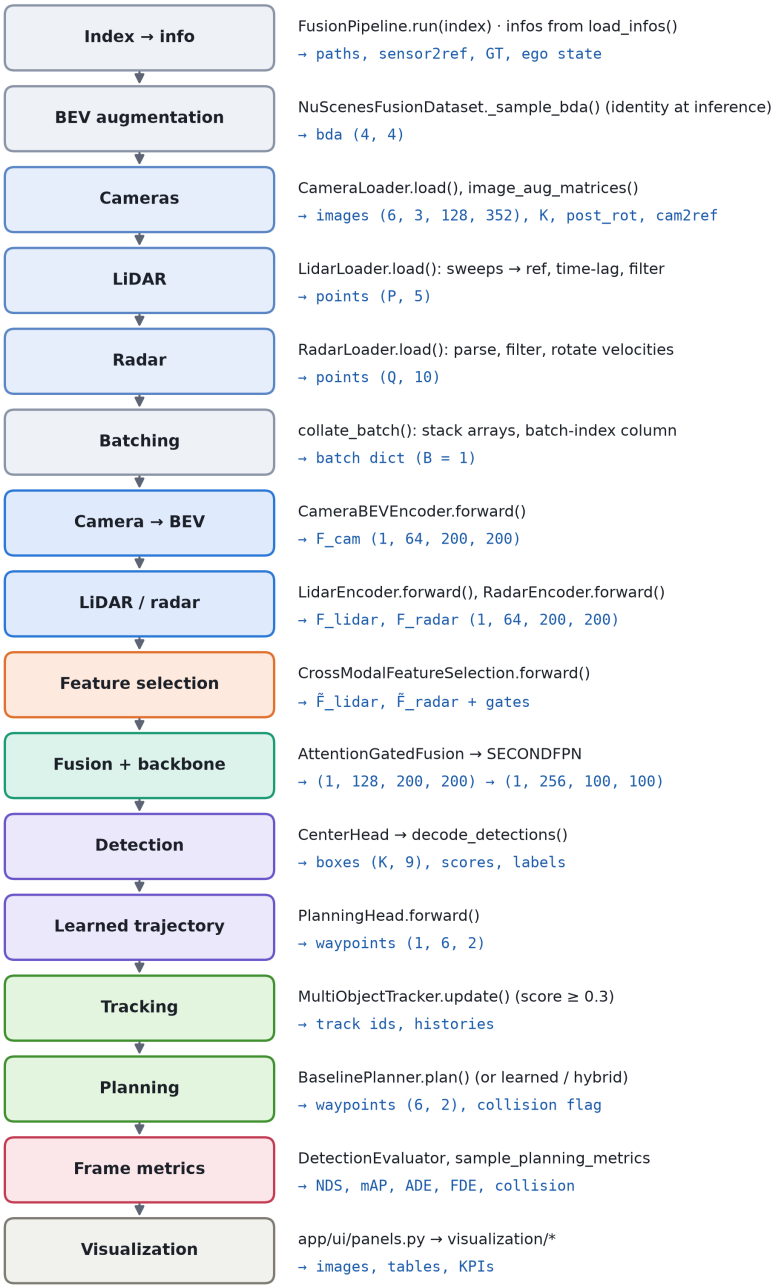


Figure 2 – Data flow of one sample with the functions that perform each step and the resulting data.

Step by step

1. **Scene → sample → info.** The dashboard lists scenes from the cached infos (`FusionPipeline.scenes()`); the selected key-frame is an index into the info list. The info contains relative file paths of all sensor files, every calibration matrix, GT boxes already in the reference frame, ego state, ego future and future agents.
2. **Dataset item.** `NuScenesFusionDataset.__getitem__(index)`:
   - `_sample_bda()` → identity at inference (random flip/rotate/scale in training);
   - `CameraLoader.load()` → six images resized/cropped to 128 × 352 (GTX 1650 profile), normalised, plus `K`, `post_rot`, `post_trans`, `cam2ref`, `cam_valid`;
   - `LidarLoader.load()` → up to 5 sweeps (key-frame + 4 previous) transformed to the reference frame with a time-lag column, range-filtered, ≤ 150 000 points → `(P, 5)`;
   - `RadarLoader.load()` → 5 radars × 5 sweeps, filtered, positions and velocities in the reference frame → `(Q, 10)`;
   - planning inputs: ego state `[speed, yaw rate, accel]`, command, logged future.
3. **Batch.** `collate_batch([item])` stacks arrays (`B = 1`) and concatenates point clouds with a batch-index column.
4. **Network.** `MultiSensorFusionNet.forward(batch)` runs camera BEV → LiDAR encoder → radar encoder → feature selection → fusion → SECOND-FPN → detection head → planning head, timing each stage.
5. **Decoding.** `decode()` → boxes `[x, y, z, l, w, h, yaw, vx, vy]`, scores, labels (score ≥ 0.1, circle NMS).
6. **Tracking.** Detections with score ≥ 0.3 are associated with tracks in the global frame → track ids and trails.
7. **Planning.** `BaselinePlanner.plan()` (default) builds occupancy from the detections and the LiDAR obstacles, finds an A* path, selects a speed profile and smooths with iLQR → six waypoints at 0.5 s steps and a collision flag.
8. **Metrics.** Single-frame NDS/mAP against the GT of the frame and ADE/FDE/collision against the logged future.
9. **Visualisation.** `app/ui/panels.py` renders camera images with boxes, the BEV tiles, the driving BEV, the trajectory close-up and tables; the dashboard displays them.

Training variant

During training (`Trainer.train_epoch`) the same dataset produces, in addition: depth targets `depth_gt (6, 8, 22)`, CenterPoint targets on the 100 × 100 head grid (`heatmap`, `ind`, `mask`, `vel_mask`, `reg_target`), the class-agnostic foreground heatmap `fg_heatmap (1, 200, 200)` and the GT future occupancy `future_occ (6, 200, 200)`. `compute_losses()` combines detection, trajectory, feature-selection, fusion (deep supervision) and depth losses with the configured weights; the optimiser updates the trainable parameters.

Program Execution Flow

Dashboard (`python app/streamlit_app.py`)

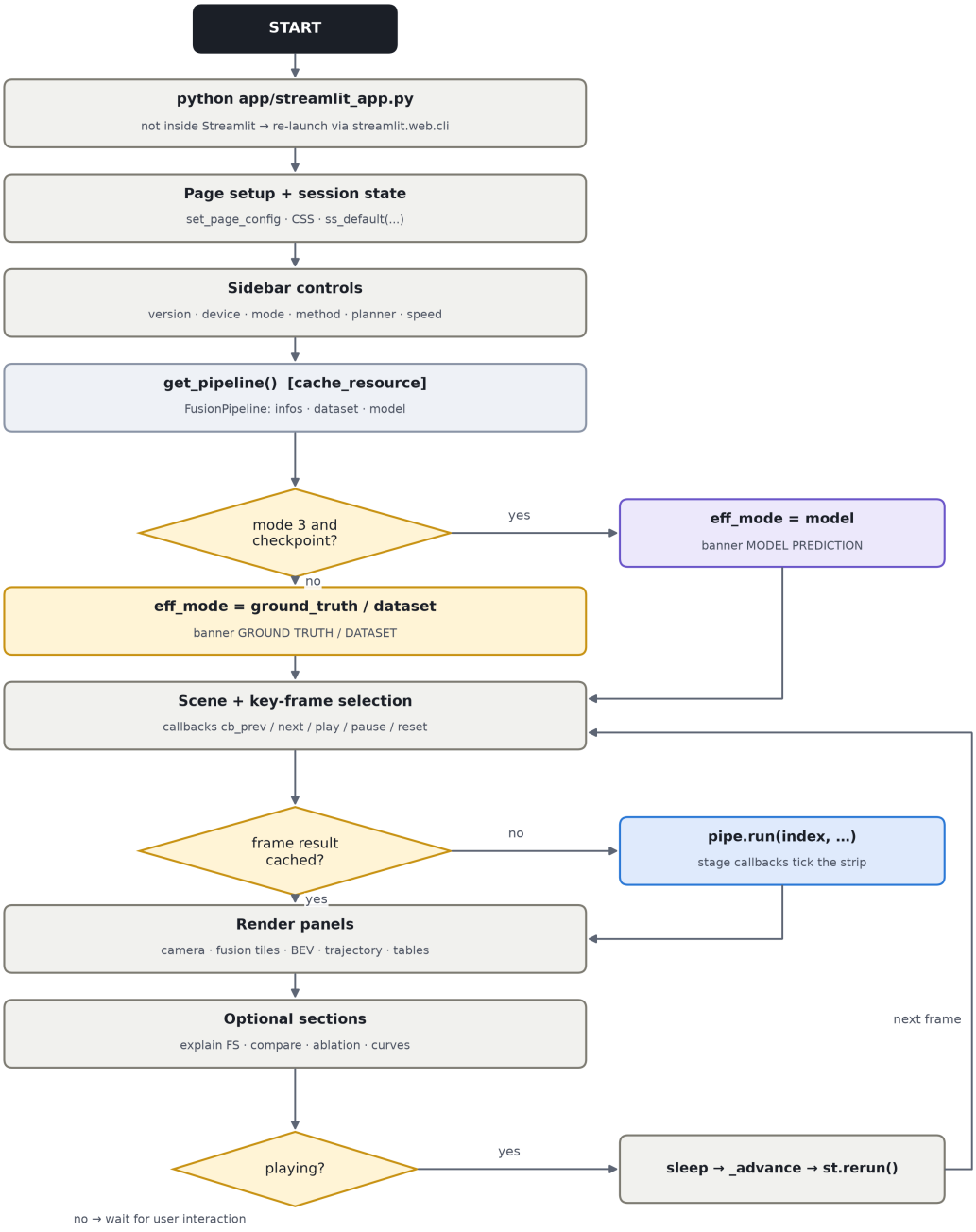


Figure 3 – Execution flow of the Streamlit dashboard. Streamlit re-executes the script on every interaction.

Important properties of this flow:

- **Re-execution model.** Every click re-runs the whole script. Expensive objects are created once through `st.cache_resource` (`get_pipeline`: data, model and checkpoint) and frame results are cached in `st.session_state.results` (LRU, 24 entries). Button callbacks modify the slider state *before* the rerun.
- **Honest mode selection.** Mode 3 uses the network only when a checkpoint exists (or untrained weights are explicitly allowed, with a red banner); otherwise ground truth is shown with the banner "GROUND TRUTH / DATASET VISUALIZATION".
- **Playback.** While playing, the script ends with `sleep(0.5 s / speed − render time)` and `st.rerun()`; the next run advances the slider.

Training (`python main.py train ...`)

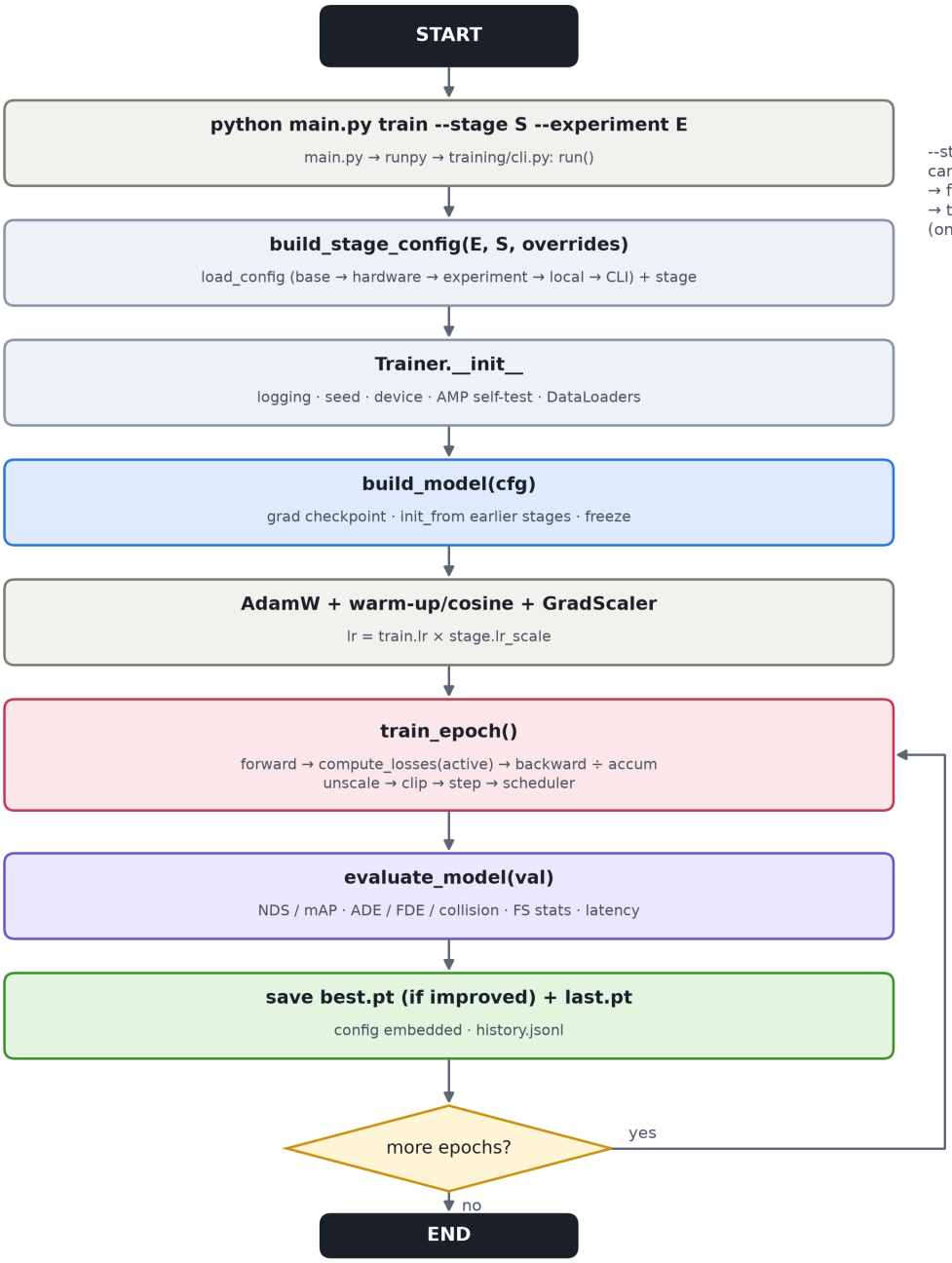


Figure 4 – Execution flow of one training stage.

Evaluation, ablation and scripts

Command	Flow
`main.py evaluate`	load checkpoint → rebuild model from embedded config (local data paths) → `evaluate_model` on the validation split → optional official devkit → JSON report
`main.py ablation`	for each experiment: optional `Trainer(..., 'joint').fit()` → `evaluate_checkpoint(last.pt)` → result row → tables, hypothesis deltas, plot
`main.py infer`	`FusionPipeline(checkpoint)` → for each frame of the scene: `run` → compose 1920×1080 frame → JPEG (+ MP4) → summary
`main.py validate`	load infos → sync report → LiDAR/radar agreement → augmentation round trip → verification images
`main.py smoke`	per experiment: dataset → one training step → timed inference → table

Command	Flow
`main.py preprocess`	load tables → build/cache infos for train, val, all → optional map caches

Tensor Dimension Reference

Symbols

Symbol	Meaning	Default value
B	batch size	1 on the GTX 1650 (`train.batch_size`); 1 at inference
N	number of cameras	6
H, W	network image height/width	128, 352 (GTX 1650); 256, 704 (RTX profiles)
h, w	image feature height/width = H/16, W/16	8, 22
D	depth bins	59 (1–60 m, 1 m steps)
C	channels (context / encoder output)	64
ny, nx	BEV grid rows (y) / columns (x)	200, 200 (0.5 m cells, ±50 m)
P	number of LiDAR points in a batch	≈ 100 000–150 000 per sample (dynamic)
Q	number of radar points in a batch	≈ 400–900 per sample (dynamic)
U	number of occupied pillars	dynamic (`torch.unique`)
K	number of detections	≤ 300 (`max_detections`), dynamic after thresholding
M	number of fused modalities	1–3 (configuration)
T	planning horizon	6 steps of 0.5 s
L	planning-head tokens	13 × 13 = 169

Module-by-module shapes (proposed model F, GTX 1650 profile)

Module	Input	Shape	Output	Shape
CameraLoader	6 JPEG images 1600×900	–	`images`	(B, 6, 3, 128, 352)
ImageBackbone	images flattened	(6B, 3, 128, 352)	neck features	(6B, 256, 8, 22)
DepthNet	features + camera params	(6B, 256, 8, 22), (6B, 16)	depth logits / context	(6B, 59, 8, 22) / (6B, 64, 8, 22)
Lift	softmax(depth) ⊗ context	–	lifted features	(B, 6, 59, 8, 22, 64)
get_geometry	frustum + matrices	(59, 8, 22, 3)	point positions	(B, 6, 59, 8, 22, 3)
voxel_pool + refine	lifted + positions	–	camera BEV `F_cam`	(B, 64, 200, 200)
LidarLoader → collate	sweeps	–	`lidar_points`	(P, 6) = batch, x, y, z, intensity, dt
DynamicPillarVFE (LiDAR)	decorated points	(P, 10)	pillar features	(U, 64) → canvas (B, 64, 200, 200)
LidarEncoder	canvas + raster	(B, 64+4, 200, 200)	`F_lidar`	(B, 64, 200, 200)
RadarLoader → collate	sweeps	–	`radar_points`	(Q, 11)
DynamicPillarVFE (radar)	decorated points	(Q, 15)	pillar features	(U, 64) → canvas
RadarEncoder	canvas + raster	(B, 64+5, 200, 200)	`F_radar`	(B, 64, 200, 200)
Feature selection: channel branch	pooled statistics	(B, 384)	channel gates	(B, 64) per sensor
Feature selection: spatial branch	descriptors	(B, 30, 200, 200)	spatial gates	(B, 1, 200, 200) per sensor
Feature selection: output	F_lidar, F_radar	(B, 64, 200, 200)	selected	(B, 64, 200, 200)
AttentionGatedFusion	3 maps	(B, 64, 200, 200) × 3	fused, weights	(B, 128, 200, 200), (B, 3, 200, 200)
SECONDFPN	fused	(B, 128, 200, 200)	BEV features	(B, 256, 100, 100)
CenterHead	BEV features	(B, 256, 100, 100)	heatmap, reg, height, dim, rot, vel	(B, C_k, 100, 100) with C_k = 10, 2, 1, 3, 2, 2

Module	Input	Shape	Output	Shape
decode_detections	head maps	–	boxes, scores, labels	(K, 9), (K,), (K,)
PlanningHead	BEV + ego state + command	(B, 256, 100, 100), (B, 3), (B,)	waypoints, attention	(B, 6, 2), (B, 13, 13)
BaselinePlanner	detections	(K, 9)	waypoints	(6, 2)

Training targets

Key	Shape	Meaning
`depth_gt`	(B, 6, 8, 22)	nearest LiDAR depth per image feature cell (0 = none)
`heatmap`	(B, 10, 100, 100)	Gaussian class heatmaps (peak = 1)
`ind`, `mask`, `vel_mask`	(B, 500)	flat centre index, object mask, velocity-defined mask
`reg_target`	(B, 500, 10)	dx, dy, z, log l, log w, log h, sin, cos, vx, vy
`fg_heatmap`	(B, 1, 200, 200)	class-agnostic centre heatmap at BEV resolution
`ego_future`, `ego_future_mask`	(B, 6, 2), (B, 6)	logged trajectory in the reference frame
`future_occ`	(B, 6, 200, 200)	GT occupancy of other agents at each future step
`ego_state`, `command`	(B, 3), (B,)	[speed, yaw rate, accel]; 0 right / 1 left / 2 straight

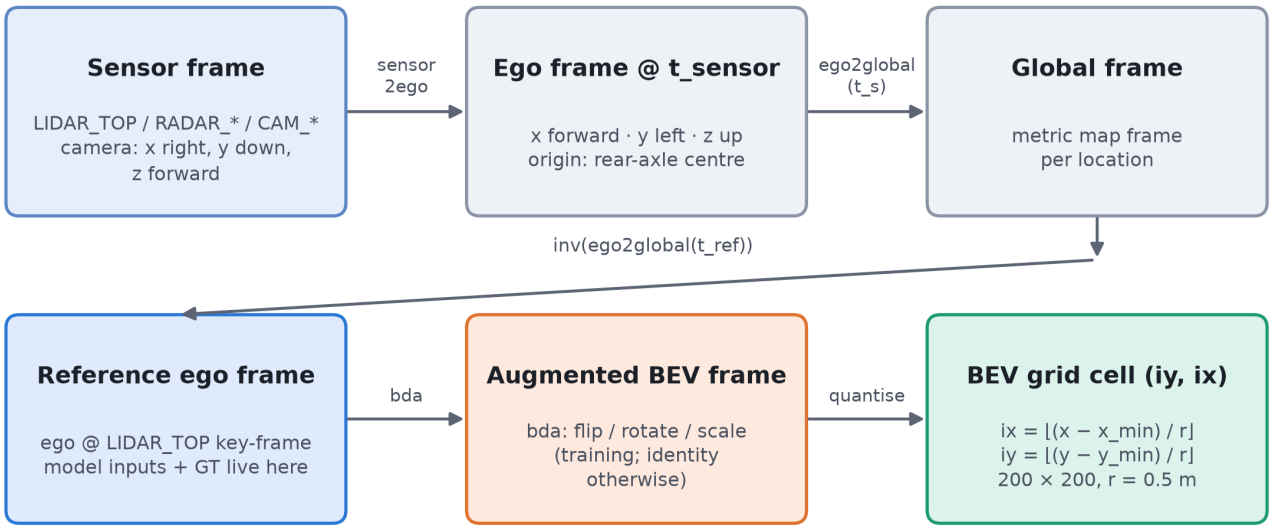
Dynamic shapes

- **P, Q** depend on the scene and the number of sweeps; points outside the grid are dropped and LiDAR is randomly subsampled above `dataset.lidar.max_points`.
- **U** is computed with `torch.unique` on the flat pillar index each forward pass; the canvas size is fixed.
- **K** depends on `score_threshold` and NMS; the evaluator caps at 500 per sample.
- **M** and the feature-selection sizes depend on the enabled sensors: `stat_dim = 2·64·(#inputs)`, spatial input `10·(#inputs)` channels, `#selectable` output maps.
- Image sizes and the BEV grid follow the configuration; the frustum and grids are rebuilt from it.

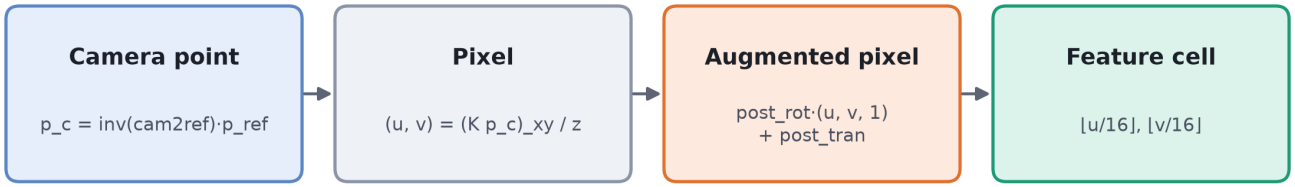
Coordinate Systems & Transformations

```
sensor2ref = inv(ref_ego2global) · ego2global(t_sensor) · sensor2ego
```

computed per sweep in datasets/info_builder.py: _sensor_record



Camera projection (camera_view.project_points, camera_loader.lidar_depth_maps):



LSS lift (lss.get_geometry) runs it backwards:

```
(u', v', d) = post_rot⁻¹((u, v, d) − post_tran)

p_ref = R_cam2ref · K⁻¹ · [u'd, v'd, d]ᵀ + t_cam2ref ;  p_bev = bda · p_ref
```

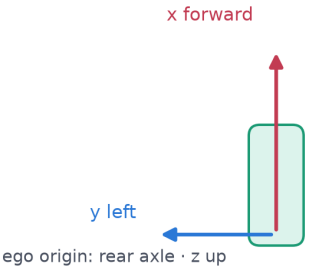


Figure 5 – Frames and transformation chain as implemented.

Frames

Frame	Definition	Where it appears
Camera	x right, y down, z forward (optical axis); pinhole intrinsics `K`	`camera_intrinsic` of calibrated_sensor
LiDAR (LIDAR_TOP)	sensor frame of the spinning LiDAR (roof, z ≈ 1.84 m above ground)	`.pcd.bin` points
Radar (RADAR_*)	sensor frame of each radar; Doppler velocities `vx_comp, vy_comp` also in this frame	`.pcd` points
Ego vehicle	x forward, y left, z up; origin at the rear-axle centre on the ground	`ego_pose` at a timestamp
Global	fixed metric map frame per location	annotations, ego poses, map rasters
Reference	ego frame at the LIDAR_TOP key-frame timestamp	all model inputs, GT boxes, predictions, plans

Frame	Definition	Where it appears
Augmented BEV	reference frame after `bda` (training only)	network inputs/targets in training
BEV grid	cells `(iy, ix)`, rows = y, columns = x	all `(…, ny, nx)` tensors

Homogeneous transformations

A rigid transform from frame *a* to frame *b* is a 4 × 4 matrix with rotation `R` (from a unit quaternion `[w, x, y, z]`, `utils/geometry.py: quat_to_rotmat`) and translation `t`:

$$T_{a\to b} = \begin{bmatrix} R & t \\ \end{bmatrix}, \qquad \tilde{p}_b = T_{a\to b}\,\tilde{p}_a, \qquad \tilde{p} = [x, y, z, 1]^T$$

The inverse is computed analytically (`invert_transform`): rotation R^T, translation $-R^T t$.

Sensor → reference frame (time synchronisation)

Each sensor sample has its own timestamp and ego pose. The chain, computed once per sweep in `datasets/info_builder.py: _sensor_record`, is:

$$T_{s\to ref} = \left(T^{(t_{ref})}_{ego\to global}\right)^{-1} T^{(t_s)}_{ego\to global}\, T_{s\to ego}$$

- `T_s→ego` = calibrated_sensor (extrinsics);
- `T_ego→global(t_s)` = ego_pose at the sensor's own timestamp;
- `T_ego→global(t_ref)` = ego_pose at the LIDAR_TOP key-frame.

Going through the global frame with each sensor's own pose compensates the ego motion between capture times (measured offsets on mini: cameras ≤ 45 ms, radars ≤ 37 ms from the LiDAR key-frame). Sweeps additionally carry `time_lag = t_ref − t_s` (seconds) as a feature.

Ground truth

Annotations are global boxes; `transform_boxes(boxes, inv(T_ego→global(t_ref)))` moves them to the reference frame: centres as points, heading and velocity as vectors. Yaw is re-extracted from the rotated heading vector,

$$\psi = \operatorname{atan2}(R_{10},\, R_{00})$$

which equals the devkit's `quaternion_yaw`. nuScenes sizes `[w, l, h]` are converted to `[l, w, h]`.

Radar velocity

Radar velocities are vectors, so only the rotation is applied (`radar_loader.py: load_sweep`):

$$v_{ref} = R^{(2\times 2)}_{s\to ref}\, v_s$$

Camera projection and lifting

Projection of a reference-frame point (validation, drawing, depth targets):

$$p_c = T_{ref\to c}\, p_{ref}, \qquad [u, v]^T = \tfrac{1}{z_c}\,(K p_c)_{1:2}$$

Image augmentation is an affine map in pixel space (`camera_loader.py: image_aug_matrices`):

$$[u_a, v_a, d]^T = R_{post}\,[u, v, d]^T + t_{post}$$

The LSS encoder lifts every frustum point `(u_a, v_a, d)` back (`lss.py: get_geometry`):

$$[u', v', d]^T = R^{-1}_{post}\big([u_a, v_a, d]^T - t_{post}\big), \qquad p_{ref} = R_{c\to ref} K^{-1}\,[u'd,\ v'd,\ d]^T + t_{c\to ref}$$

followed by the BEV augmentation `p_bev = bda · p_ref`. Verified: projection followed by lifting returns the original point with a maximum error of $6 \cdot 10^{-14}$ m (`scripts/validate_sensors.py`, `tests/test_dataset.py`).

BEV grid

$$i_x = \left\lfloor \frac{x - x_{min}}{r} \right\rfloor, \qquad i_y = \left\lfloor \frac{y - y_{min}}{r} \right\rfloor, \qquad r = 0.5\,\text{m}$$

Tensors are `(…, ny, nx)`. For display, `visualization/common.py: bev_to_display` rotates arrays so that forward points up and left points left. The detection head uses `grid.downsample(2)` (1 m cells, 100 × 100); the planner uses its own grid (x ∈ [−10, 60), y ∈ [−30, 30), 0.5 m).

Where each transformation happens

Transformation	Code
quaternion → matrix, 4×4 build/invert	`utils/geometry.py`
sensor → reference (all sensors)	`datasets/info_builder.py: _sensor_record`
LiDAR points → reference	`datasets/lidar_loader.py: LidarLoader.load_sweep`
radar points + velocities → reference	`datasets/radar_loader.py: RadarLoader.load_sweep`
GT boxes global → reference	`datasets/info_builder.py: _boxes_for_sample` (`transform_boxes`)
ego future / past → reference	`datasets/info_builder.py: build_sample_info`
BEV augmentation	`datasets/nuscenes_dataset.py` (`bev_rotation_matrix`, `augment_boxes_bev`, `_apply_bda_points`), `lss.py`
image augmentation matrices	`datasets/camera_loader.py: image_aug_matrices`
projection into images	`visualization/camera_view.py: project_points`, `camera_loader.lidar_depth_maps`
frustum lifting	`models/camera_bev/lss.py: get_geometry`
metres ↔ grid cells	`utils/bev_grid.py`, `models/common/pillars.py`, `lss.voxel_pool`, `utils/raster.py`
reference → global (submission)	`evaluation/official_eval.py: box_to_global`
reference ↔ global (tracking)	`planning/tracking.py`
map raster → display	`datasets/map_prior.py: crop_to_ego`

Camera → BEV Pipeline

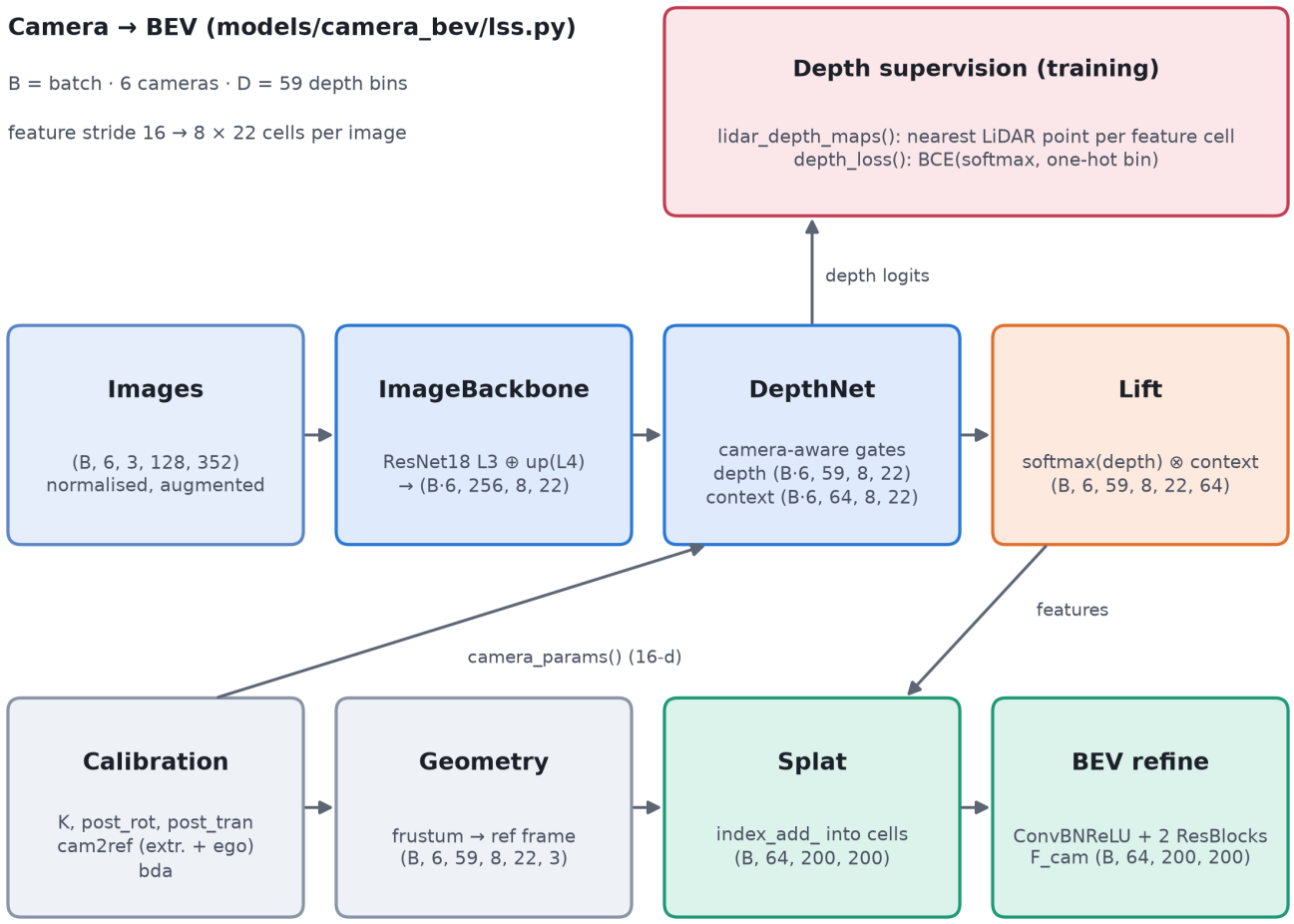


Figure 6 – Camera BEV pipeline (Lift-Splat-Shoot with a camera-aware depth network).

Stage	File · class / function	Shape (GTX 1650 profile)	Operation
Image loading	`camera_loader.py · CameraLoader.load`	6 × (3, 128, 352)	resize to width 352 (factor 0.22), bottom-biased crop of 128 rows, ImageNet normalisation; training: resize jitter 0.94–1.11, crop jitter, flip, ±5.4° rotation
Calibration	`camera_loader.py · image_aug_matrices`; info `sensor2ref`	`K` (6, 3, 3), `post_rot` (6, 3, 3), `post_trans` (6, 3), `cam2ref` (6, 4, 4)	affine image-augmentation matrices; extrinsics + ego motion
Backbone	`image_backbone.py · ImageBackbone.forward`	(6B, 256, 8, 22)	ResNet18 (ImageNet), stem + layer1 frozen, BN in eval mode; layer3 ⊕ upsampled layer4 → neck
Camera descriptor	`lss.py · CameraBEVEncoder.camera_params`	(6B, 16)	normalised fx, fy, cx, cy of `post_rot·K` + 12 extrinsic entries
Depth / context	`lss.py · DepthNet.forward`	(6B, 59, 8, 22), (6B, 64, 8, 22)	camera-aware SE gates; depth logits and context features
Lift	`lss.py · CameraBEVEncoder.forward`	(B, 6, 59, 8, 22, 64)	`softmax(depth) ⊗ context` (outer product per cell)
3-D projection	`lss.py · get_geometry`	(B, 6, 59, 8, 22, 3)	frustum → reference frame (equations in the previous chapter)
BEV pooling	`lss.py · voxel_pool`	(B, 64, 200, 200)	`index_add_` sum-pooling into cells, z collapsed
Refinement	`lss.py · bev_refine`	(B, 64, 200, 200)	ConvBNReLU + 2 ResBlocks

Stage	File · class / function	Shape (GTX 1650 profile)	Operation
Depth supervision	`camera_loader.py` · `lidar_depth_maps`; `lss.py` · `depth_loss`	(B, 6, 8, 22)	nearest LiDAR point per cell; BCE to the one-hot depth bin

Mathematical core. For feature cell `(i, j)` of camera `n` with depth distribution `α_d` and context `c`, the lifted feature at depth `d` is `α_d · c`; its 3-D location is the lifted frustum point; all lifted features that fall into BEV cell `(iy, ix)` are summed:

$$F_{cam}[:, i_y, i_x] = \sum_{n,d,i,j \,:\, cell(p_{n,d,i,j}) = (i_y, i_x)} \alpha_{n,d,i,j}\, c_{n,i,j}$$

Design notes.

- The frustum uses bin *centres* and feature-cell *centre* pixels; the depth targets use the same binning, so geometry and supervision are consistent.
- A missing or unreadable camera image (`cam_valid = False`) multiplies its depth distribution by zero, so it adds nothing to the BEV.
- The camera BEV is bright mostly near the vehicle: many more frustum rays fall into near cells (sum pooling). This is a property of LSS, not an error.
- The dashboard additionally shows a *geometric* camera BEV (`visualization/camera_view.py: camera_ipm_bev`): inverse perspective mapping of the ground plane using only the calibration (no learning, flat-ground assumption, ±25 m). It works before training and illustrates what the learned depth distribution improves on.

LiDAR Pipeline

Stage	Code	Details
Raw LiDAR	`lidar_loader.py · read_lidar_bin`	`.pcd.bin`, float32 × 5: x, y, z, intensity (0–255), ring index
Sweep selection	`info_builder.py · build_sample_info`	key-frame + previous sweeps via the `prev` chain (up to 10 cached; `dataset.lidar.sweeps` = 5 on the GTX 1650, 10 on RTX profiles)
Ego-return removal	`LidarLoader.load_sweep`	drop points within 1.5 m (xy) of the sensor
Coordinate transformation	`LidarLoader.load_sweep`	`sensor2ref` per sweep (motion compensation through the sweep's ego pose); `time_lag` column
Point filtering	`LidarLoader.load`	keep x, y ∈ [−50, 50), z ∈ [−5, 5); random subsampling above 150 000 points (seeded at evaluation)
Batching	`collate_batch`	`(P, 6)` = batch index, x, y, z, intensity, time_lag
Pillarisation	`pillars.py · PillarIndexer`	cell indices, unique pillars, inverse index (dynamic, no point cap)
Point decoration	`DynamicPillarVFE.forward`	`[x/50, y/50, z/5, intensity/255, dt/0.5, Δ to pillar mean (3), Δ to pillar centre (2)]` → 10 features
PointNet	`DynamicPillarVFE`	Linear(10→16)-BN-ReLU → max per pillar → concat → Linear(32→64)-BN-ReLU → max per pillar
Scatter to BEV	`DynamicPillarVFE.forward`	`(U, 64)` → canvas `(B, 64, 200, 200)`
Raster channels	`LidarEncoder.raster`	density `log(1+n)/3`, max height / 3 m, mean height / 3 m, mean intensity / 100
Encoder	`LidarEncoder.forward`	ConvBNReLU(68 → 60) + 2 ResBlocks, then concatenate the 4 raster channels → `F_lidar (B, 64, 200, 200)`

Why dynamic pillars. The original PointPillars caps the points per pillar and the number of pillars. Multi-sweep nuScenes clouds (≈ 110 000 points for 5 sweeps) are dense near the vehicle, and caps would discard information. Scatter operations (`index_add_`, `scatter_reduce_`) handle any number of points in pure PyTorch, so no custom CUDA extension (spconv/mmcv) has to be compiled on Windows.

Why explicit raster channels. The last four output channels are interpretable physical quantities. The feature-selection channel gates on these channels can therefore be read as "how much does the network use LiDAR height / density / intensity". On the trained model F the gate of *mean height* is 0.001 (switched off) while *max height* is 0.98.

Robustness. A missing or corrupt sweep file is skipped with a single warning; with no LiDAR at all the encoder returns an all-zero map, and the rest of the network still runs.

Radar Pipeline

nuScenes radar fields

The radar files are binary PCD files with 18 fields per point (43 bytes, packed). `read_radar_pcd` parses all of them; the following are **used**:

Field	Meaning	Use
`x, y, z`	position in the radar frame (z is unreliable)	transformed to the reference frame
`rcs`	radar cross-section (dBsm)	feature (÷30) and raster channel
`vx_comp, vy_comp`	Doppler velocity compensated for ego motion	rotated to the reference frame; feature (÷10) and raster channels
`vx, vy`	raw (uncompensated) velocity	rotated; feature (÷10)
`dyn_prop`	dynamic property (0 moving … 7 stopped)	filter and feature (÷7)
`invalid_state`	validity flag	filter (keep 0)
`ambig_state`	Doppler ambiguity state	filter (keep 3 = unambiguous)

Not used: `id`, `is_quality_valid`, `x_rms`, `y_rms`, `vx_rms`, `vy_rms`, `pdh0` (parsed but ignored).

Processing chain

Stage	Code	Details
Filtering	`RadarLoader._filter`	devkit defaults: `invalid_state ∈ {0}`, `dyn_prop ∈ {0..6}`, `ambig_state ∈ {3}` (`dataset.radar.filters`)
Sweeps	`info_builder` / `RadarLoader.load`	5 radars × key-frame + previous sweeps (5 on the GTX 1650, 6 cached)
Coordinate transformation	`RadarLoader.load_sweep`	positions with `sensor2ref`; velocity vectors with its rotation only
Velocity processing	`RadarLoader.load_sweep`, `RadarEncoder.raster`	compensated velocity in the reference frame = object motion over ground
Point features	`RADAR_POINT_FEATURES`	x, y, z, rcs, vx_comp, vy_comp, vx, vy, dyn_prop, time_lag → (Q, 10)
BEV representation	`DynamicPillarVFE` + `RadarEncoder.raster`	pillar features (64) + raster (density, vx, vy, RCS, time lag)
Radar features	`RadarEncoder.forward`	ConvBNReLU(69 → 59) + ResBlocks with dilation 1, 2, 4 → concatenate the 5 raster channels → `F_radar (B, 64, 200, 200)`

How velocity is represented

1. **Per point:** `vx_comp, vy_comp` (and raw `vx, vy`) are 2-D vectors in the reference frame, normalised by 10 m/s, inside the PointNet input.
2. **Per BEV cell, explicitly:** the mean compensated velocity of the cell (`radar_vx`, `radar_vy`, ÷10) is appended verbatim as two of the last five output channels. Velocity therefore reaches the feature selection, the fusion, the detection head's velocity branch and the planner without having to survive the learned layers.
3. **Downstream:** the detection head regresses object velocity `(vx, vy)`; the tracker and the planner use it for the motion forecast. Radar measurably helps velocity: adding radar to LiDAR lowered the velocity error (mAVE) from 1.10 to 0.77 m/s on mini-val.

Why the velocity rotation matters. The devkit's `RadarPointCloud.from_file_multisweep` transforms radar positions but leaves the velocity columns in each radar's own frame; with five radars facing different directions the vectors would be inconsistent. The project rotates them explicitly.

Sparsity. After filtering and accumulating 5 sweeps of the 5 radars, a key-frame has ≈ 400–800 radar returns (observed on mini) versus ≈ 110 000 LiDAR points. The convolution stack (one 3×3 conv plus

residual blocks with dilation 1, 2 and 4) has a receptive field of ≈ 31 cells (≈ 15 m), so each isolated return influences its neighbourhood.

Deep-Learning Feature Selection

This chapter documents the project's research component: **Camera-Guided Cross-Modal Feature Selection (CG-CMFS)**, class `CrossModalFeatureSelection` in `models/feature_selection/selection.py`, trained with `models/feature_selection/losses.py`.

Problem it addresses

A naive fusion concatenates every LiDAR and every radar channel at every BEV cell. Part of this information is *redundant* (LiDAR and radar both encode "something is here") and part is *noisy* (radar multipath ghosts, LiDAR returns from vegetation, far-range sparsity). The hypothesis of the project is that a network which **learns to select** informative spatial, semantic and motion features of each sensor before fusion yields better perception and planning.

Where it sits

`F_cam`, `F_lidar`, `F_radar` (each `(B, 64, 200, 200)`) come out of the encoders. The selection module receives `F_lidar` and `F_radar` (the *selectable* modalities) and, if the camera is enabled and `fusion.selection.camera_guided` is true, `F_cam` as a **guide**. It outputs selected maps `F˜_lidar`, `F˜_radar` of the same shapes plus the gates. The fusion module then receives `{camera: F_cam, lidar: F˜_lidar, radar: F˜_radar}` (`MultiSensorFusionNet.forward`).

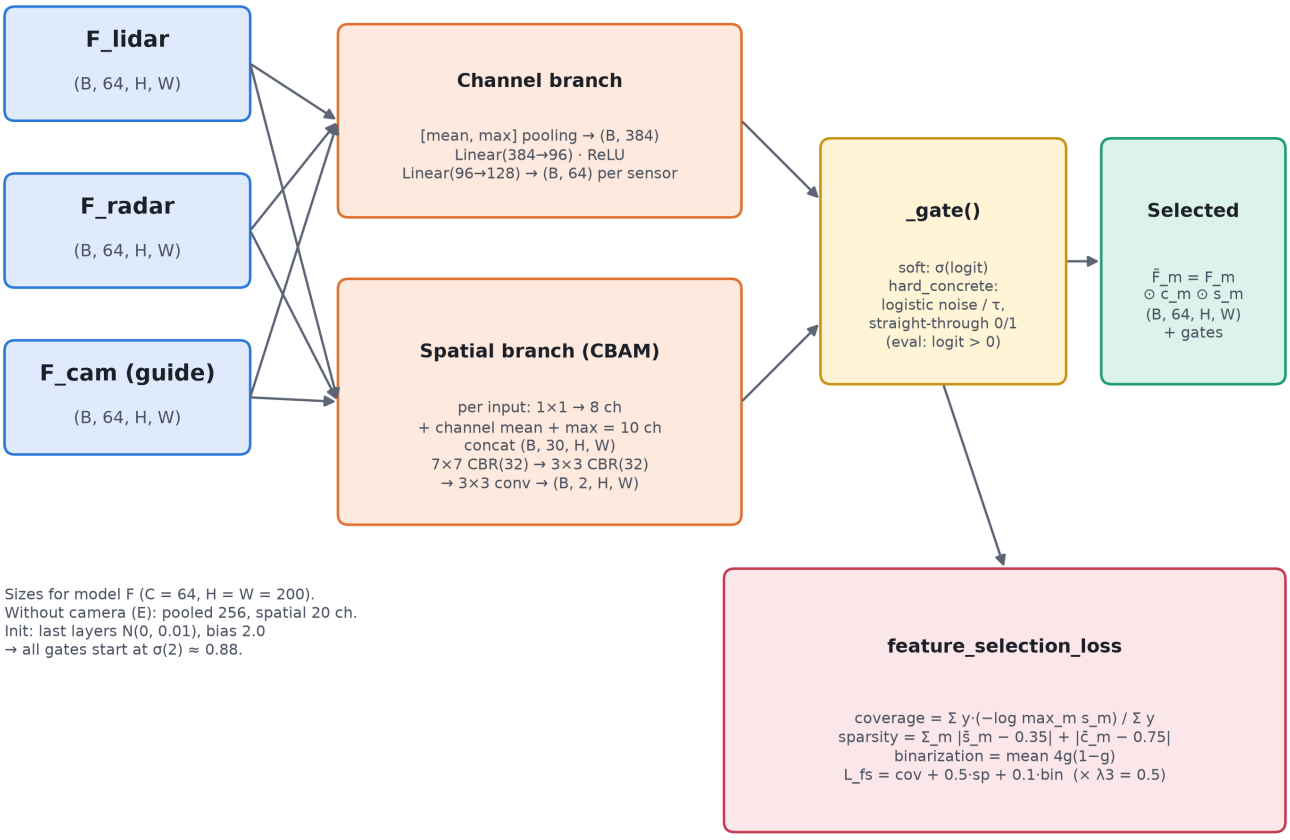


Figure 7 – Structure of the feature-selection module with the default sizes of model F.

Mathematical definition

For each selectable sensor `m ∈ {lidar, radar}` the module predicts a **channel gate** `c_m ∈ (0,1)^C` and a **spatial gate** `s_m ∈ (0,1)^{H×W}`; the selected features are

$$\tilde{F}_m = F_m \odot c_m \odot s_m$$

(`c_m` broadcast over space, `s_m` over channels).

Channel gates (cross-modal squeeze-and-excitation). With `μ(·)` and `max(·)` denoting global average and max pooling over H × W of every input (LiDAR, radar and the camera guide):

$$[\ell^c_{lidar},\ \ell^c_{radar}] = \mathrm{MLP}([\mu(F_l), \max(F_l), \mu(F_r), \max(F_r), \mu(F_c), \max(F_c)]), \qquad c_m = g(\ell^c_m)$$

The MLP is `Linear(384 → 96) → ReLU → Linear(96 → 128)` for model F. Because its input contains statistics of all sensors, the gates of one sensor depend on the content of the others (e.g. radar velocity channels can be kept when LiDAR is sparse).

Spatial gates (CBAM-style, camera-guided). Each input is compressed by a 1×1 ConvBNReLU to 8 channels and complemented by its channel-mean and channel-max maps (10 channels per input, 30 for model F):

$$[\ell^s_{lidar},\ \ell^s_{radar}] = \mathrm{Conv}_{3\times3}\big(\mathrm{CBR}_{3\times3}\big(\mathrm{CBR}_{7\times7}\big([\phi_m(F_m),\ \bar{F}_m,\ \hat{F}_m]_m\big)\big)\big), \qquad s_m = g(\ell^s_m)$$

Gate function `g` (`_gate`, `fusion.selection.gate_mode`).

- `soft` (default, used for all reported experiments): `g(l) = σ(l)`.
- `hard_concrete`: in training a relaxed Bernoulli sample with straight-through gradient, at evaluation `l > 0`:

$$g = \sigma\big(\tfrac{\ell + \log u - \log(1-u)}{\tau}\big),\quad u \sim U(0,1), \qquad g_{hard} = [g > 0.5], \qquad \text{output} = g_{hard} + g - \mathrm{sg}(g)$$

Initialisation. The final layers of both branches have weights `N(0, 0.01)` and bias `init_open = 2.0`, so all gates start at σ(2) ≈ 0.88: training starts from "keep almost everything" and learns what to suppress.

Training objective

`feature_selection_loss` (weights in `loss.feature_selection`, targets in `fusion.selection.target_keep_ratio`):

$$L_{cov} = \frac{\sum_{cells} y\,\big(-\log \max_m s_m\big)}{\sum_{cells} y}$$

$$L_{sp} = \sum_m |\,\overline{s_m} - \rho_s| + |\,\overline{c_m} - \rho_c|, \qquad \rho_s = 0.35,\ \rho_c = 0.75$$

$$L_{bin} = \overline{4\,g\,(1-g)} \quad \text{(over all gates)}$$

$$L_{FS} = 1.0\,L_{cov} + 0.5\,L_{sp} + 0.1\,L_{bin}, \qquad \text{weighted by } \lambda_3 = 0.5 \text{ in the total loss}$$

`y` is the class-agnostic object-centre heatmap (`fg_heatmap`). The coverage term only penalises *missing* selection at objects (at least one sensor must keep its features there); the sparsity term limits how much is kept overall; the binarisation term pushes gates towards crisp 0/1 decisions. The detection, trajectory and fusion losses also flow back through the gates.

Statistics and explainability

`selection_stats()` reports per sensor the fraction of spatial cells / channels with gate > 0.5, mean importances and the *effective selected ratio* (fraction of cells × fraction of channels). The dashboard's **Explain Feature Selection** section and `scripts/render_bev.py` visualise, from the actual network outputs:

View	Function	Meaning
Raw → importance → selected	`visualization/feature_selection.py: energy_map, importance_map`	feature energy before selection, spatial gate, energy after selection
Sensor contribution	`contribution`, `dominance_map`	where LiDAR (blue) or radar (orange) information actually passes the selection
Fusion attention	`modality_weight_map`	per-cell softmax weights of the fusion module (camera green, LiDAR blue, radar orange)
Named channel gates	`channel_gate_table`	gates of the explicit physical channels (height, intensity, velocity, RCS, …)
Sensor trust per class	`object_importance`, `class_importance_summary`	mean gate of each sensor inside detected boxes, per class

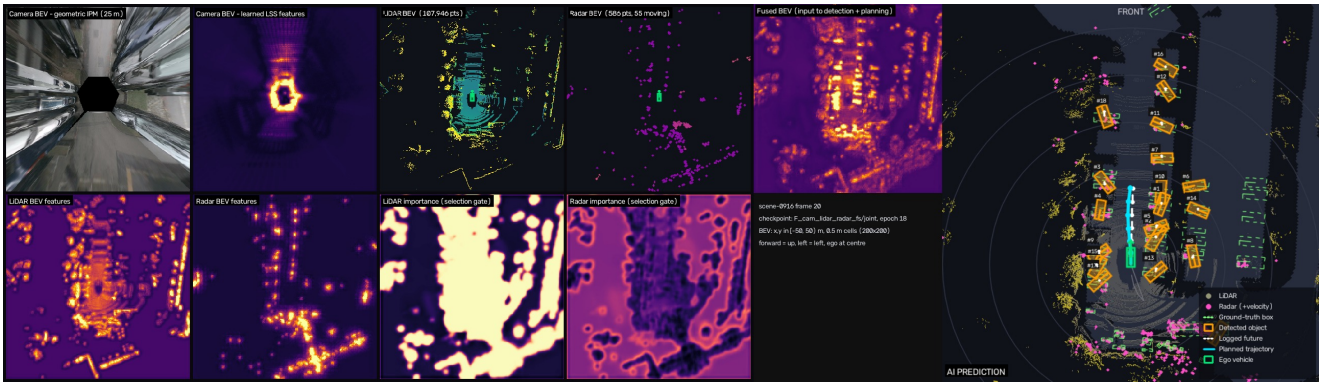


Figure 8 – All BEV representations of a held-out frame (scene-0916) from model F, including the LiDAR and radar importance maps.

What the trained network learned (measured)

Measured on 21 validation frames with the final (epoch 20) weights of model F (camera-guided) and model E (no camera):

Quantity	F (camera + LiDAR + radar)	E (LiDAR + radar)
Effective selected ratio LiDAR / radar	0.34 / 0.11	0.39 / 0.10
LiDAR spatial gate: cells with LiDAR returns / elsewhere	0.99 / 0.42	1.00 / 0.55
Radar spatial gate: cells with radar returns / elsewhere	0.18 / 0.35	0.14 / 0.38
LiDAR gates density / max height / **mean height** / intensity	0.99 / 0.98 / **0.001** / 0.99	0.99 / 0.93 / 0.89 / 0.98
Radar gates density / vx / vy / RCS / time lag	0.99 / 0.98 / 0.99 / 0.98 / 0.98	0.96 / 0.96 / 0.97 / 0.97 / 0.98
Learned channels kept (gate > 0.5) LiDAR / radar	77 % / 73 %	77 % / 75 %

Interpretation (descriptive, not tested causally): LiDAR is kept wherever it has data; radar *velocity* and *RCS* channels are kept; with the camera present the network drops the redundant LiDAR *mean height* channel while keeping *max height*; the radar gate is lower exactly on the cells that contain radar returns than in the surrounding context, i.e. the network relies on the spread radar context rather than on the exact (angularly inaccurate) return positions.

Effect on performance

Comparison (final weights, mini-val)	ΔNDS	ΔmAP	ΔmAVE (m/s, lower = better)	ΔADE (m)
E (feature selection) − C (concatenation)	−0.061	−0.008	+0.255	+0.151
E (feature selection) − D (attention only)	−0.015	+0.007	−0.391	+0.237
F (with camera) − E	+0.039	+0.017	−0.113	+0.015

On nuScenes-mini the hypothesis is not supported: the selection did not beat plain concatenation. With 323 training frames, a single seed and 81 validation frames the comparison is underpowered; E's validation NDS peaked at epoch 6 and then declined, which suggests the extra gating and its regularisers make training on tiny data less stable. Testing the hypothesis properly requires the full dataset and several seeds (see *Known Limitations & Next Steps*).

Configuration knobs

Key	Default	Effect
`fusion.feature_selection`	true (experiment dependent)	enable the module
`fusion.selection.camera_guided`	true	use the camera BEV as guide when the camera is enabled
`fusion.selection.gate_mode`	soft	`soft` or `hard_concrete`
`fusion.selection.temperature`	0.67	hard-concrete temperature τ
`fusion.selection.init_open`	2.0	initial gate logit (σ(2) ≈ 0.88)
`fusion.selection.channel_reduction`	4	MLP bottleneck factor
`fusion.selection.spatial_hidden`, `spatial_kernel`	32, 7	spatial branch width and first kernel
`fusion.selection.threshold`	0.5	gate value counted as "selected" in statistics
`fusion.selection.target_keep_ratio.spatial / channel`	0.35 / 0.75	sparsity targets ρ_s, ρ_c
`loss.lambda_feature_selection`	0.5	λ3 of L_FS in the total loss
`loss.feature_selection.coverage / sparsity / binarization`	1.0 / 0.5 / 0.1	weights inside L_FS

Sensor Fusion & BEV Backbone

Fusion methods

`models/fusion/registry.py` maps `fusion.method` to a class; both classes take `{modality: (B, C_m, H, W)}` and return `(fused (B, 128, H, W), info)`.

`concat` — `ConcatFusion`: concatenation of all maps → 3×3 ConvBNReLU → ResBlock. Used by experiments A, B, C.

`attention_gated` — `AttentionGatedFusion` (experiments D–I). With per-sensor projections `P_m = φ_m(F_m)` (1×1 ConvBNReLU to 128 channels):

$$a = \mathrm{softmax}_m(g([F_1, \ldots, F_M])), \qquad F_w = \sum_m a_m P_m, \qquad F_{fused} = \mathrm{CBAM}(\mathrm{Res}(\mathrm{CBR}_{1\times1}([F_w, P_1, \ldots, P_M])))$$

`g` is a 3×3 ConvBNReLU (→ 64) followed by a 1×1 conv (→ M). `a` is a per-cell probability over sensors ("which sensor to trust where"); keeping the individual projections in the mixer input preserves complementary information. The weights are returned (`info['modality_weights']`) and shown in the dashboard.

Deep supervision (L_fusion)

`MultiSensorFusionNet` adds a 1×1 conv ("aux head", bias −2.19) on every (selected) sensor map and on the fused map. During training each predicts the class-agnostic centre heatmap `fg_heatmap`; the Gaussian focal losses are averaged:

$$L_{fusion} = \frac{1}{M+1}\left(\sum_m L_{focal}(\mathrm{aux}_m(\tilde{F}_m), y) + L_{focal}(\mathrm{aux}_{fused}(F_{fused}), y)\right)$$

This keeps every branch discriminative on its own. Weight `loss.lambda_fusion = 0.25`; disable with `fusion.deep_supervision: false`.

BEV backbone

`SECONDFPN` (`model.bev_backbone`): block 1 = stride-2 conv + 3 convs (128 ch, 100 × 100), block 2 = stride-2 conv + 5 convs (256 ch, 50 × 50); deblock 1 = 1×1 conv, deblock 2 = transposed conv ×2; concatenation → `(B, 256, 100, 100)`. Gradient checkpointing per block when `train.grad_checkpoint` is true.

Adding a fusion method

```python
from models.fusion.registry import register_fusion

@register_fusion("my_fusion")
class MyFusion(nn.Module):
    def __init__(self, in_channels: dict[str, int], out_channels: int): ...
    def forward(self, feats: dict[str, torch.Tensor]):
        return fused, {}            # fused: (B, out_channels, H, W)
```

The module must be imported before the model is built (e.g. add the import in `models/fusion/registry.py`); then set `fusion.method: my_fusion` and add an experiment in `configs/ablation.yaml`.

3D Object Detection

Classes

The ten nuScenes detection classes in index order (`classes` in `configs/dataset.yaml`): car, truck, construction_vehicle, bus, trailer, barrier, motorcycle, bicycle, pedestrian, traffic_cone. nuScenes categories are mapped with the official table `CATEGORY_TO_DETECTION` (`datasets/info_builder.py`); other categories (animals, debris, emergency vehicles, bicycle racks, …) are not detection targets.

Targets (`models/detection/targets.py`)

On the head grid (100 × 100, 1 m): a Gaussian per object in its class heatmap (radius from the CornerNet formula with `gaussian_overlap = 0.1`, at least `min_radius = 2` cells) and, at the centre cell, the regression target `[dx, dy, z, log l, log w, log h, sin ψ, cos ψ, vx, vy]`. GT boxes with no LiDAR/radar return or outside the grid are dropped (`dataset.gt_filter.min_points = 1`). Velocities that nuScenes leaves undefined are masked.

Head and loss (`models/detection/center_head.py`)

Shared 3×3 conv (256 → 64) and six branches. Classification uses the CornerNet focal loss on probabilities `p` with targets `y` (α = 2, γ = 4), normalised by the number of centre cells:

$$L_{cls} = -\frac{1}{N_{pos}}\left[\sum_{y=1}(1-p)^{\alpha}\log p + \sum_{y<1}(1-y)^{\gamma}p^{\alpha}\log(1-p)\right]$$

Regression is an L1 loss at the centre cells weighted by `code_weights` (1 for offset, z, size, sin, cos; 0.2 for velocity) and split into centre, box, orientation and velocity terms:

$$L_{det} = 1.0\,L_{cls} + 0.25\,L_{center} + 0.25\,L_{bbox} + 0.25\,L_{orient} + 0.25\,L_{vel}$$

Decoding (`models/detection/decoder.py`)

Sigmoid → 3×3 max-pool peak extraction → top-300 → metric boxes (`x = (i_x + dx)·1 m + x_min`, sizes `exp(·)` with the log-size clamped at 4, `ψ = atan2(sin, cos)`) → score threshold (`detection_head.score_threshold = 0.1`; evaluation uses `eval.score_threshold = 0.1`, the dashboard shows ≥ 0.3 by default) → class-wise circle NMS with radii `detection_head.nms_radius` (car 1 m, pedestrian 0.25 m, bus 2 m, …).

Attributes

The network does not predict attributes; `evaluation/detection_metrics.py: default_attribute` derives them from the predicted speed (> 0.2 m/s → moving). This affects only mAAE.

Trajectory Planning & Tracking

Inputs

- **Ego state** `[speed, yaw rate, acceleration]`: from the CAN-bus expansion if present, otherwise estimated *causally* from the 20 Hz LiDAR ego poses (`info_builder._ego_state_from_poses`). The CAN-bus folder is not present in the current dataset, so the pose estimate is used.
- **Navigation command** RIGHT / LEFT / STRAIGHT from the lateral offset (±2 m) of the logged future's last point — the standard open-loop protocol (ST-P3, UniAD); it replaces a route planner.
- **Detections** (model mode) or GT boxes (ground-truth mode), the LiDAR key-frame points, optional map prior.

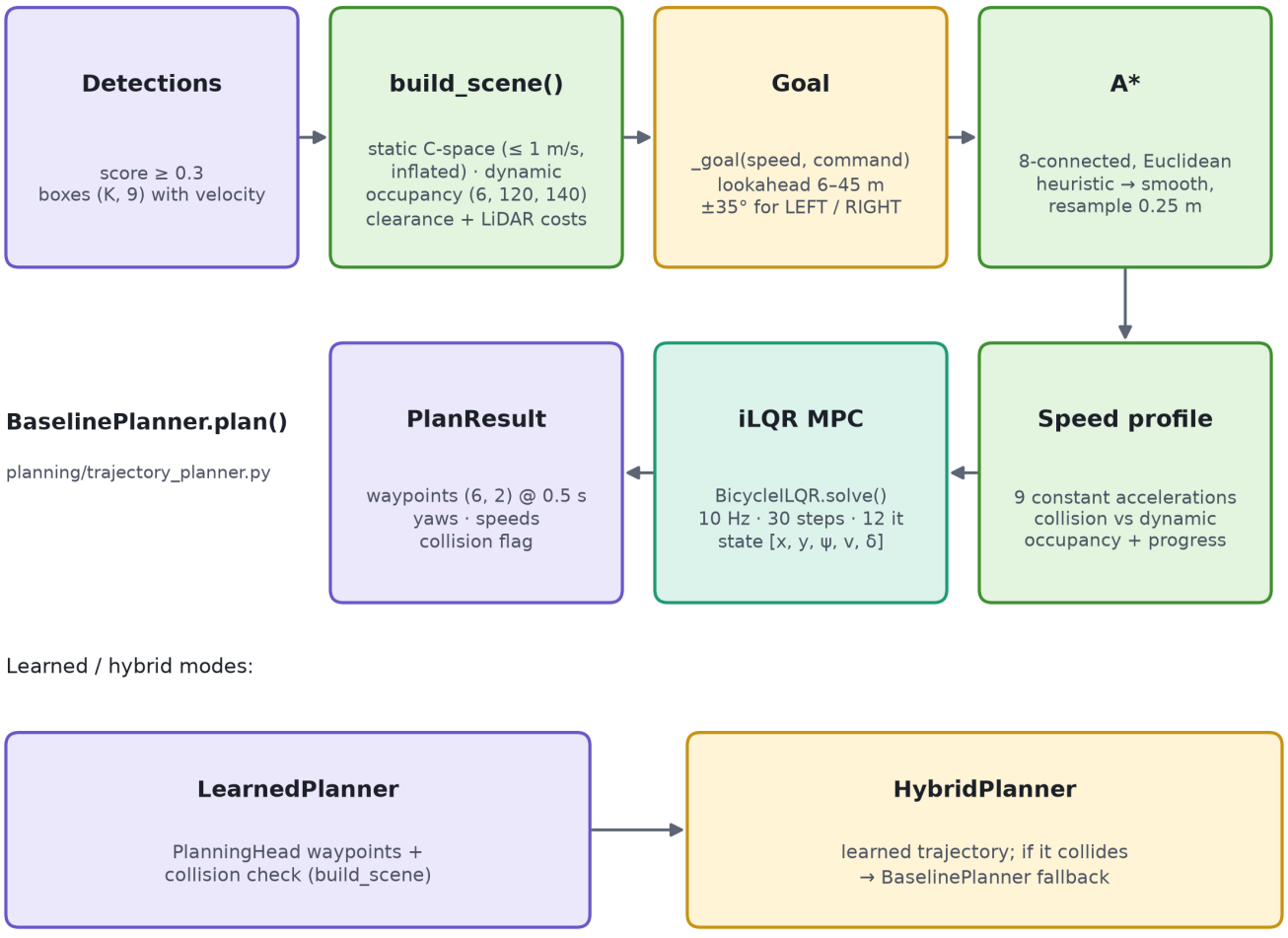


Figure 9 – Planners as implemented.

Baseline planner (planning/trajectory_planner.py: BaselinePlanner)

1. **Scene model** (`build_scene`): static obstacles (detections slower than 1 m/s, score ≥ 0.3) inflated by half the ego width + 0.3 m → blocked cells; dynamic occupancy `(6, 120, 140)` from all detections propagated with their velocity; soft costs from obstacle clearance `exp(−d/1.5 m)` and non-ground LiDAR returns.
2. **Goal** (`_goal`): lookahead `clip(1.2 · 3 s · max(v, 2), 6, 45)` m, bearing 0° / ±35° by command.
3. **A*** (`planning/astar.py`) on the cost grid (8-connected, cost `step·(1 + c)`), then smoothing and 0.25 m resampling.
4. **Speed profile**: nine constant accelerations (−6 … +1.5 m/s²); each is scored by progress error, comfort (a^2) and +1000 per step where the ego footprint hits the dynamic occupancy.
5. **iLQR MPC** (`planning/mpc.py`) on the kinematic bicycle model (state `[x, y, ψ, v, δ]`, control `[a, ω]`, wheelbase 2.588 m, dt 0.1 s, 30 steps, 12 iterations):

$$x_{t+1} = x_t + v_t \cos\psi_t \Delta t, \quad y_{t+1} = y_t + v_t \sin\psi_t \Delta t, \quad \psi_{t+1} = \psi_t + \frac{v_t}{L}\tan\delta_t \Delta t$$

$$v_{t+1} = v_t + a_t \Delta t, \qquad \delta_{t+1} = \delta_t + \omega_t \Delta t, \qquad J = \sum_t e_t^T Q\, e_t + u_t^T R\, u_t$$

6. **Output**: states at 0.5, 1.0, …, 3.0 s → `PlanResult.waypoints (6, 2)` plus a collision flag against the *predicted* occupancy.

Learned and hybrid planners

`LearnedPlanner` uses the `PlanningHead` waypoints (residuals on a constant-velocity prior, trained with L1 to the logged trajectory + collision + smoothness + goal losses). `HybridPlanner` uses the learned trajectory unless it collides with the predicted occupancy, in which case the baseline is used. Select with `planning.planner` or the dashboard dropdown.

Tracking (planning/tracking.py)

Greedy association of detections (score ≥ 0.3) to tracks in the **global** frame, predicted with constant velocity, class-specific gates (car 4 m, pedestrian 1 m, …), track removal after two missed frames, velocity smoothing. Provides the IDs and motion trails in the BEV view.

Measured planning performance (mini-val, 69 frames with a full 3 s future)

Planner and perception	ADE (m)	FDE (m)	Collision rate
Constant velocity	1.77	4.03	14.5 %
Baseline planner with GT objects	1.65	3.48	1.4 %
Logged human trajectory (sanity check of the metric)	0.00	0.00	0.0 %
Learned planning heads (ablation models, final weights)	1.26 – 1.53	2.74 – 3.51	7.2 – 13.0 %
Baseline planner with each model's detections	1.80 – 2.07	–	2.9 – 15.9 %

The learned planners imitate the logged trajectory more closely (lower ADE) than the classical planner but collide more often; the classical planner is safer when perception is good. Collisions are measured against the **real** future positions of other agents (`evaluation/trajectory_metrics.py: collision_flags`).

Loss Functions

`MultiSensorFusionNet.compute_losses()` builds the total loss from the loss groups a training stage activates:

$$L_{total} = \lambda_1 L_{det} + \lambda_2 L_{traj} + \lambda_3 L_{FS} + \lambda_4 L_{fusion} + \lambda_5 L_{depth}$$

Term	λ (key)	Default	Components (weights in `loss.*`)	Defined in
L_det	`lambda_detection`	1.0	heatmap 1.0, center 0.25, bbox 0.25, orientation 0.25, velocity 0.25	`models/detection/center_head.py`
L_traj	`lambda_trajectory`	1.0	regression 1.0, collision 2.0, smoothness 0.1, goal 0.5	`models/trajectory/losses.py`
L_FS	`lambda_feature_selection`	0.5	coverage 1.0, sparsity 0.5, binarization 0.1	`models/feature_selection/losses.py`
L_fusion	`lambda_fusion`	0.25	mean of auxiliary heatmap focal losses	`models/detection/center_head.py: aux_heatmap_loss`
L_depth	`lambda_depth`	1.0	BCE to one-hot LiDAR depth bins	`models/camera_bev/lss.py: depth_loss`

Trajectory loss details

$$L_{traj} = \frac{\sum_t m_t \, \|\hat{p}_t - p_t\|_1}{\sum_t m_t} + 2.0 \, \frac{\sum_t m_t \max_k O_t(f_{t,k})}{\sum_t m_t} + 0.1 \, \overline{\|\Delta^2 \hat{p}\|^2} + 0.5 \, \|\hat{p}_T - p_T\|_2$$

`m_t` masks unknown future steps, `O_t` is the softened GT future occupancy and `f_{t,k}` are five ego-footprint points (bilinear `grid_sample`, differentiable), `Δ²` are second differences including the current velocity.

Depth loss

$$L_{depth} = \frac{1}{|V|} \sum_{cells \in V} \sum_{d} \mathrm{BCE}(\alpha_d, \; [d = \lfloor (z - d_{min}) / \Delta d \rfloor])$$

over cells `V` with a LiDAR depth in [1, 60) m.

Which stage uses which loss

Stage	detection	trajectory	feature_selection	fusion	depth
camera_bev	✓				✓
lidar	✓				
radar	✓				
fusion	✓		✓	✓	
detection	✓		✓	✓	
trajectory		✓			
end_to_end	✓	✓	✓	✓	✓
joint	✓	✓	✓	✓	✓

Training System

Training curriculum

Stage	Description	Losses	Init from	Frozen / trainable	Epochs	LR scale	Overrides
camera_bev (shared)	Stage 1 - Camera -> BEV (LSS + depth supervision, camera-only detection probe)	detection, depth	–	–	12	1.0	{"fusion": {"use_camera_bev": true, "use_lidar": false, "use_radar": false, "feature_selection": false, "method": "concat"}}
lidar (shared)	Stage 2 - LiDAR encoder (PointPillars -> BEV)	detection	–	–	12	1.0	{"fusion": {"use_camera_bev": false, "use_lidar": true, "use_radar": false, "feature_selection": false, "method": "concat"}}
radar (shared)	Stage 3 - Radar encoder (velocity-preserving radar pillars -> BEV)	detection	–	–	12	1.0	{"fusion": {"use_camera_bev": false, "use_lidar": false, "use_radar": true, "feature_selection": false, "method": "concat"}}
fusion	Stage 4 - Feature selection + fusion on frozen sensor encoders	detection, feature_selection, fusion	camera_bev (camera_bev.); lidar (lidar_encoder., bev_backbone., det_head.); radar (radar_encoder.)	camera_bev, lidar_encoder, radar_encoder	10	1.0	–
detection	Stage 5 - Detection fine-tuning (LiDAR/radar encoders unfrozen)	detection, feature_selection, fusion	fusion (all)	camera_bev	8	0.5	–
trajectory	Stage 6 - Learned trajectory planner on frozen perception	trajectory	detection (all)	trainable only: plan_head	10	1.0	{"dataset": {"augmentation": {"rot_range_deg": [0.0, 0.0], "flip_x_prob": 0.0}}}
end_to_end	Stage 7 - Joint fine-tuning of perception + planning	detection, trajectory, feature_selection, fusion, depth	trajectory (all)	camera_bev. backbone	6	0.25	{"dataset": {"augmentation": {"rot_range_deg": [0.0, 0.0], "flip_x_prob": 0.0}}}
joint	Single-stage joint training from scratch (used for the fair ablation study)	detection, trajectory, feature_selection, fusion, depth	–	–	20	1.0	{"dataset": {"augmentation": {"rot_range_deg": [0.0, 0.0], "flip_x_prob": 0.0}}}

Shared single-sensor stages (`shared: true`) are trained once and reused by all experiments (`checkpoints/_shared/<stage>/`). `init_from` copies the parameters whose names start with the given prefixes from the source stage's `best.pt` (falling back to `last.pt`). `frozen` / `trainable_only` control `requires_grad`; frozen BatchNorm layers are kept in eval mode.

The **ablation** uses the single stage `joint` (all losses, from scratch) so that every experiment has an identical recipe. The curriculum was verified mechanically (all seven stages run on 2 samples, weights are handed over and frozen as specified) but has **not** been trained to convergence.

Trainer internals (`training/trainer.py`)

Feature	Implementation
Optimiser	AdamW, `lr = train.lr × stage.lr_scale` (2e-4), weight decay 0.01
Schedule	LambdaLR: linear warm-up (`train.warmup_iters`, ≤ 10 % of steps) × cosine decay to 0
Gradient accumulation	`train.grad_accum` (4 on the GTX 1650 → effective batch 4)
Mixed precision	`torch.autocast` + `GradScaler` if `train.amp` **and** the fp16 self-test passes
Gradient clipping	global norm `train.grad_clip` (35) after unscaling
Robustness	non-finite losses skip the batch (logged); missing sensor files are skipped with one warning
Gradient checkpointing	image backbone layers 2–4 and BEV backbone blocks (`train.grad_checkpoint`)
Data loading	`num_workers` (2), pinned memory, persistent workers, per-worker seeds
Validation	every `val_interval` epochs, `evaluate_model` on the validation split
Checkpoints	`best.pt` (by NDS / −ADE / −loss) and `last.pt`, atomic writes, config embedded, optimiser state included
Logs	`outputs/logs/<experiment>/<stage>/run.log` and `history.jsonl`
Resume	`--resume <checkpoint>` restores model, optimiser, scheduler, scaler, epoch, step, best score
Reproducibility	`runtime.seed` (42) for Python/NumPy/PyTorch; `runtime.deterministic` for cuDNN

Hardware and measured cost (GTX 1650, 4 GB, fp32)

Item	Value
Proposed model F, batch 1, 128×352 images	training peak 3.2 GB, inference 0.5–0.8 GB, ≈ 100 ms per frame
LiDAR/radar-only models	inference 0.2–0.4 GB, 43–71 ms
Training speed	≈ 2.6–2.8 iterations/s (F)
20 epochs on nuScenes-mini	F 39 min; A 19 min; B 18 min; C 24 min; D 25 min; E 28 min
Checkpoint size	65–70 MB (LiDAR/radar models), 258 MB (F) — includes AdamW moments

fp16 note. On this GPU with cuDNN 9.10, fp16 3×3 convolutions with 256 input channels return NaN, and fp32 is also 3.4× faster here (no tensor cores). The `gtx1650` profile sets `train.amp: false`; `utils/device.py: fp16_is_reliable` turns AMP off automatically on any GPU that fails the same probe.

Training commands

```
python main.py train --stage joint --experiment F_cam_lidar_radar_fs --epochs 20
python main.py train --stage curriculum --experiment F_cam_lidar_radar_fs
python training/train_fusion.py --experiment E_lidar_radar_fs --set train.lr=1e-4
python main.py train --stage joint --experiment F_cam_lidar_radar_fs --resume checkpoints/F_cam_lidar_radar_fs/joint
↪    /last.pt
python main.py train --stage joint --set dataset.version=v1.0-trainval dataset.train_split=train dataset.val_split=v
↪    al runtime.hardware_profile=rtx3060
```

Evaluation & Experiments

Detection metrics and their verification

`evaluation/detection_metrics.py` re-implements the official nuScenes detection evaluation (centre-distance matching at 0.5/1/2/4 m, 101-point interpolated precision, minimum recall/precision 0.1, TP errors at 2 m with confidence-aligned cumulative means, class ranges, zero-point and bike-rack filtering, NaN rules). NDS:

$$NDS = \frac{1}{10}\left[5\,mAP + \sum_{k \in \{ATE, ASE, AOE, AVE, AAE\}} (1 - \min(1,\ mTP_k))\right]$$

`scripts/check_eval_vs_devkit.py` evaluates the same (perturbed-GT) predictions with this port and with the official `nuscenes.eval.detection.evaluate.DetectionEval` on mini-val:

Metric	Port	Official devkit	Absolute difference
NDS	0.3566	0.3566	0.0001
mAP	0.2982	0.2984	0.0002
mATE	0.6689	0.6692	0.0003
mASE / mAOE / mAVE / mAAE	0.4345 / 0.4329 / 0.7113 / 0.6776	0.4345 / 0.4329 / 0.7112 / 0.6776	≤ 0.0001

Official devkit on the trained model F (final weights, `scripts/evaluate.py --official`, mini-val, detections exported with score ≥ 0.05): NDS 0.1028, mAP 0.0459, mATE 0.840, mASE 0.715, mAOE 1.341, mAVE 1.608, mAAE 0.647. The in-memory evaluation of the same checkpoint at the training threshold (score ≥ 0.1) gives NDS 0.0943, mAP 0.0440; the difference comes from the lower export threshold (more low-confidence detections raise recall).

mini-val caveat. Three classes (construction vehicle, trailer, barrier) have no GT in the two mini-val scenes; the official rule gives them AP = 0, which caps mAP at 0.7. The project therefore also reports `mAP_present_classes`.

Planning metrics

ADE, FDE, L2 at 1/2/3 s, collision rate (ego polygon vs. real future agent boxes, SAT test), lateral/longitudinal error, jerk and curvature change (`evaluation/trajectory_metrics.py`). Only frames with a complete 3 s future are scored (69 of 81 mini-val frames). The collision check is calibrated: the logged human trajectory scores 0 %.

Ablation study

Design and hypothesis: `docs/EXPERIMENTS.md`. All experiments use stage `joint`, 20 epochs, seed 42, identical data and augmentation; evaluation uses the **final** weights (`--which last`, default) to avoid model selection on the evaluation split. Results generated from `outputs/ablation/results.json`:

Configuration	Ep.	NDS	mAP	mAP pres.	mATE	mASE	mAOE	mAVE	mAAE	ADE	FDE	Coll.	ms	Params M
LiDAR only	20	0.060	0.032	0.046	0.885	0.798	1.138	1.101	0.876	1.374	2.856	0.087	45.0	5.7
Radar only	20	0.053	0.007	0.010	1.032	0.816	1.202	1.476	0.692	1.259	2.849	0.116	43.2	5.7
LiDAR + Radar	20	0.116	0.035	0.050	0.790	0.762	1.215	0.772	0.692	1.364	3.149	0.072	62.6	6.0
LiDAR + Radar + Attention	20	0.070	0.021	0.030	0.886	0.782	1.044	1.418	0.738	1.277	2.741	0.072	67.0	6.0
LiDAR + Radar + Feature Selection	20	0.055	0.027	0.039	0.885	0.807	1.111	1.027	0.895	1.514	3.506	0.072	71.0	6.0

Configuration	Ep.	NDS	mAP	mAP pres.	mATE	mASE	mAOE	mAVE	mAAE	ADE	FDE	Coll.	ms	Params M
Camera + LiDAR + Radar + Feature Selection	20	0.094	0.044	0.063	0.810	0.760	1.238	0.914	0.793	1.529	3.471	0.130	103.2	22.6

Hypothesis deltas (first minus second, from results.md):

Comparison	ΔNDS	ΔmAP	ΔmAVE (↓)	ΔADE (↓)	ΔCollision (↓)	ΔLatency ms
Feature selection vs concatenation (LiDAR+Radar) (E vs C)	-0.0610	-0.0076	+0.2553	+0.1506	+0.0000	+8.4160
Feature selection vs attention-only (LiDAR+Radar) (E vs D)	-0.0149	+0.0065	-0.3908	+0.2372	+0.0000	+3.9604
Adding the camera BEV (F vs E)	+0.0393	+0.0167	-0.1125	+0.0146	+0.0580	+32.2218
Adding radar to LiDAR (C vs A)	+0.0560	+0.0029	-0.3297	-0.0099	-0.0145	+17.5960

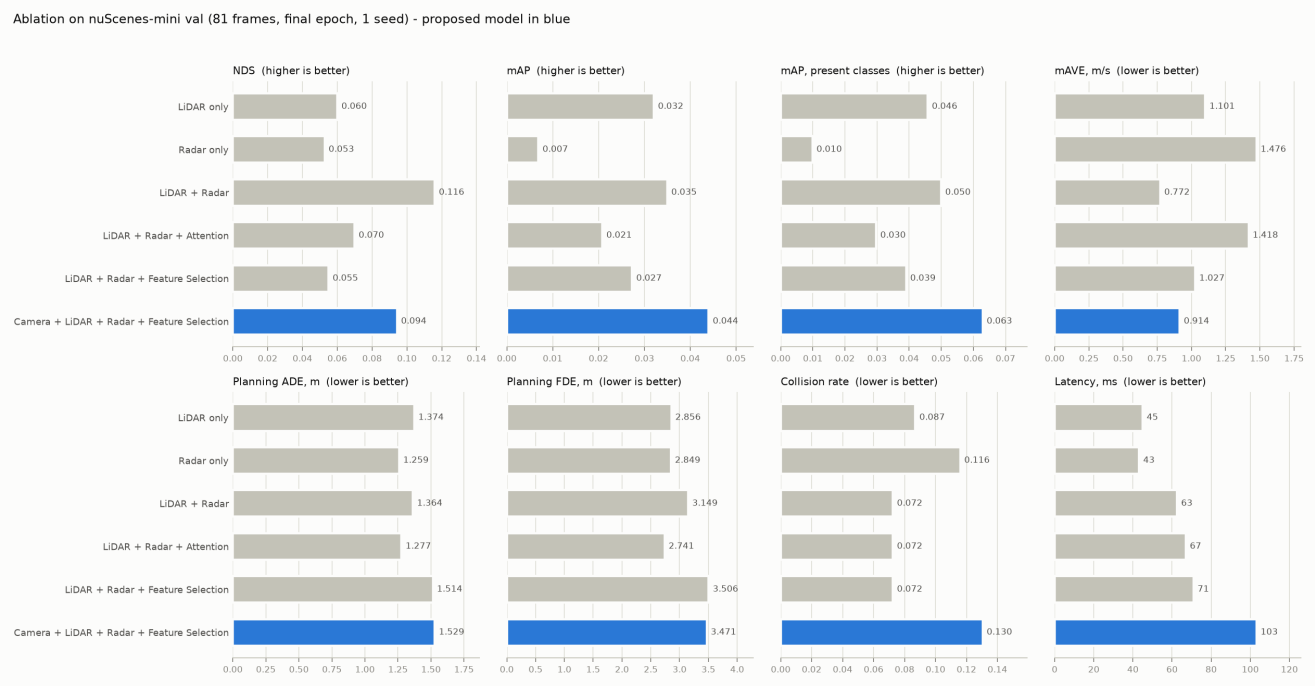


Figure 10 – Ablation results (final weights, mini-val). Proposed model in blue.

Findings. Radar improves LiDAR (C vs A: +0.056 NDS, −0.33 m/s velocity error); the camera BEV improves LiDAR+radar (F vs E: +0.039 NDS, +0.017 mAP; F has the best mAP); learned feature selection does not beat concatenation on mini (E vs C: −0.061 NDS). The best-validation-epoch protocol (archived as `results_best_epoch.*`) gives the same picture. The evaluation is underpowered (81 frames, 2 scenes, one seed).

Visualization & Streamlit Application

Visualisation modules

Module	Main functions	Output
visualization/common.py	BEVCanvas, bev_to_display, heatmap_image, colorize, put_label, stack_images, CLASS_COLORS	drawing primitives (OpenCV, RGB uint8)
visualization/camera_view.py	project_points, draw_boxes_on_image, draw_lidar_on_image, camera_mosaic, camera_ipm_bev	camera images with boxes / points, geometric camera BEV
visualization/bev_view.py	SceneLayers, render_driving_scene	the autonomous-driving BEV
visualization/lidar_view.py, radar_view.py	render_lidar_bev, render_radar_bev	raw-sensor BEV tiles
visualization/feature_selection.py	energy_map, importance_map, contribution, dominance_map, modality_weight_map, object_importance, channel_gate_table	explainability views
visualization/trajectory_view.py	render_trajectory_closeup	planned vs logged trajectory close-up

Display convention: **forward up, left to the left**, ego in the lower-middle of the driving BEV (x ∈ [−25, 55), y ∈ [−40, 40), 8 px/m).

Dashboard structure

Area	Content	Code
Sidebar	faculty toggle, version, device, mode, fusion method ([✓] trained), planner, untrained opt-in, confidence threshold, speed, display options, scene	app/streamlit_app.py
Header	title, scene description, mode banner, warnings	–
Controls	Load Scene, Previous, Play, Pause, Next, Reset, key-frame slider	callbacks cb_*
Pipeline strip (faculty mode)	CAMERA → BEV … DRIVING OUTPUT with live ticks and times	styles.flow_html, on_stage
KPI strip	objects, FPS, NDS, mAP, ADE, collision, fusion method	styles.kpis_html
Panel 1	camera image with predicted (+GT) 3D boxes; all six cameras	panels.camera_panel
Panel 3	camera BEV, LiDAR BEV, radar BEV, LiDAR/radar importance, fused BEV	panels.fusion_tiles
Panel 2	autonomous-driving BEV	panels.driving_bev
Panel 5	ego pose, command, trajectory close-up, ADE/FDE/collision	panels.trajectory_closeup, ego_status
Panel 4	detection table, GT comparison	panels.detection_table, gt_comparison
Explain Feature Selection	raw → importance → selected, contribution, attention, channel gates, class trust	panels.explain_selection
Compare / Ablation / Curves / About	method comparison, results, training curves, system story	–

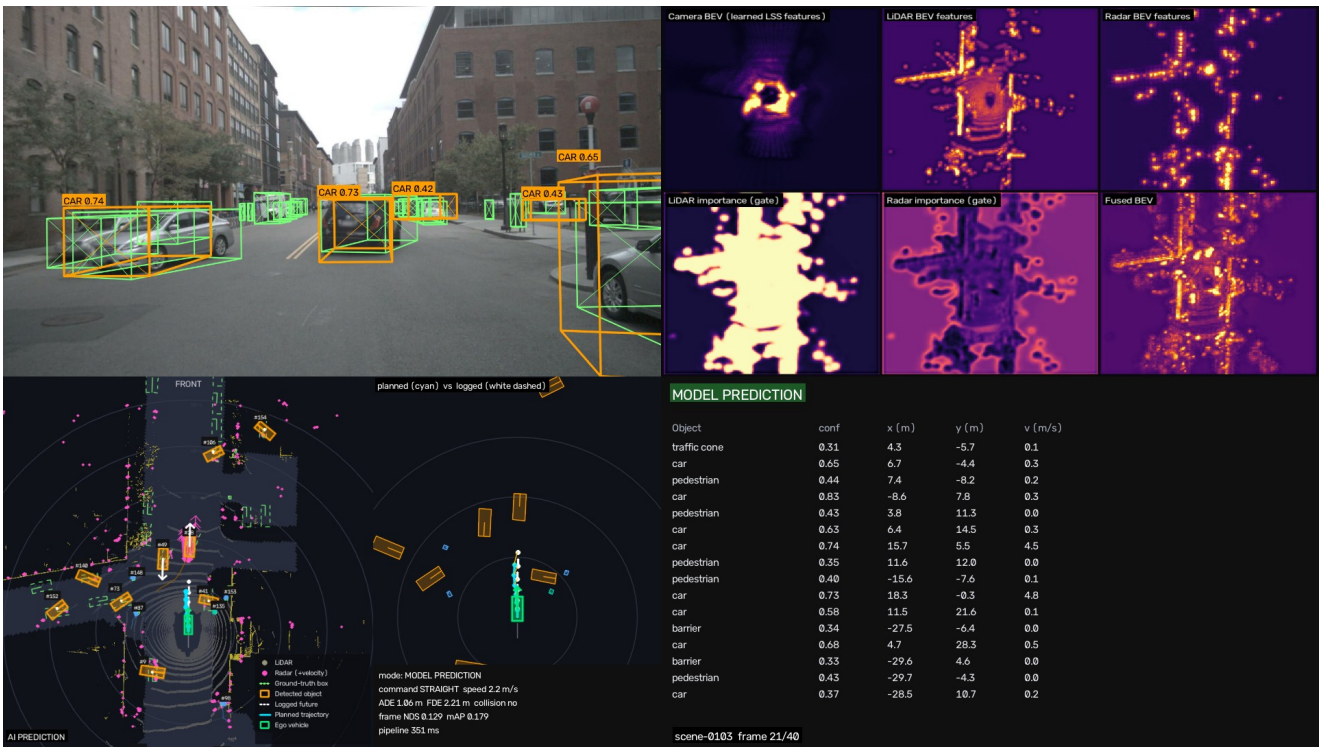


Figure 11 – A frame of the exported replay video (model F, held-out scene-0103): camera with predicted boxes, sensor-fusion tiles, driving BEV, trajectory close-up and detection list.

Modes and honesty rules

Mode	Shows	Requires
1 · Dataset Viewer	raw sensors, GT boxes, logged trajectory	data only
2 · Ground Truth Simulation	GT objects with tracks, logged trajectory, classical planner on GT (labelled)	data only
3 · AI Autonomous Driving	network predictions, gates, plans, metrics vs GT	a checkpoint (else falls back to mode 2 with a warning)

Untrained networks are shown only after ticking "Allow untrained network", with a red UNTRAINED banner. Scenes from the training split are flagged.

Launching

```
python app/streamlit_app.py
streamlit run app/streamlit_app.py
streamlit run app/streamlit_app.py -- --device cpu
```

The first command re-launches itself through Streamlit with the dark theme. `-- --device cpu` sets the default device (useful while the GPU is busy training). Port 8501.

Configuration Reference

Every key of the merged default configuration, generated from the YAML files (comments from the files are shown as descriptions). Merge order: `dataset.yaml`, `model.yaml`, `fusion.yaml`, `planning.yaml`, `training.yaml` → `hardware/<profile>.yaml` → experiment overrides (`ablation.yaml`) → `local.yaml` → `--set key=value`.

Section `dataset`

Key	Default (base YAML)	This machine	Description
dataset.root	/path/to/nuscenes	D:/researcpaper	
dataset.version	v1.0-mini	=	v1.0-mini | v1.0-trainval
dataset.train_split	mini_train	=	mini_train | train
dataset.val_split	mini_val	=	mini_val | val
dataset.cache_dir	outputs/cache	=	pre-computed sample "infos" (relative = project root)
dataset.num_workers	2	=	
dataset.pin_memory	true	=	
dataset.lidar_channel	LIDAR_TOP	=	
dataset.cameras	["CAM_FRONT_LEFT", "CAM_FRONT", "CAM_FRONT_RIGHT", "CAM_BACK_LEFT", "CAM_BACK", "CAM_BACK_RIGHT"]	=	
dataset.radars	["RADAR_FRONT", "RADAR_FRONT_LEFT", "RADAR_FRONT_RIGHT", "RADAR_BACK_LEFT", "RADAR_BACK_RIGHT"]	=	
dataset.max_cached_lidar_sweeps	10	=	sweeps stored in the info cache (the loader may use fewer)
dataset.max_cached_radar_sweeps	6	=	
dataset.history_frames	4	=	past key-frames kept for visualisation (ego trail)
dataset.camera.image_size	[256, 704]	[128, 352]	network input (H, W); hardware profiles may lower it
dataset.camera.resize_jitter	[0.94, 1.11]	=	train-time random resize around the base scale
dataset.camera.bottom_crop_jitter	[0.0, 0.22]	=	
dataset.camera.rotate_deg	5.4	=	
dataset.camera.flip	true	=	
dataset.camera.depth_gt	true	=	LiDAR-projected depth for BEVDepth-style supervision
dataset.lidar.sweeps	10	5	multi-sweep accumulation (adds a time-lag channel)
dataset.lidar.max_points	250000	150000	random subsampling above this
dataset.lidar.remove_close_radius	1.5	=	remove ego-vehicle returns (metres, xy)
dataset.radar.sweeps	6	5	
dataset.radar.filters.invalid_states	[0]	=	
dataset.radar.filters.dynprop_states	[0, 1, 2, 3, 4, 5, 6]	=	
dataset.radar.filters.ambig_states	[3]	=	
dataset.augmentation.enabled	true	=	
dataset.augmentation.flip_y_prob	0.5	=	mirror left/right (driving command is recomputed)

Key	Default (base YAML)	This machine	Description
dataset.augmentation.flip_x_prob	0.0	=	mirror forward/backward (breaks planning; keep 0 when planning)
dataset.augmentation.rot_range_deg	[-22.5, 22.5]	=	
dataset.augmentation.scale_range	[0.95, 1.05]	=	
dataset.augmentation.modality_dropout.camera	0.0	=	
dataset.augmentation.modality_dropout.lidar	0.0	=	
dataset.augmentation.modality_dropout.radar	0.0	=	
dataset.gt_filter.min_points	1	=	boxes with no LiDAR+radar returns are ignored (nuScenes eval rule)

Section bev

Key	Default (base YAML)	This machine	Description
bev.x_range	[-50.0, 50.0]	=	
bev.y_range	[-50.0, 50.0]	=	
bev.z_range	[-5.0, 5.0]	=	
bev.resolution	0.5	=	

Section classes

Key	Default (base YAML)	This machine	Description
classes	["car", "truck", "construction_vehicle", "bus", "trailer", "barrier", "motorcycle", "bicycle", "pedestrian", "traffic_cone"]	=	

Section model

Key	Default (base YAML)	This machine	Description
model.camera_bev.backbone	resnet18	=	resnet18 | resnet34 | resnet50
model.camera_bev.pretrained	true	=	ImageNet weights from torchvision (falls back to random init offline)
model.camera_bev.frozen_stages	1	=	freeze stem + layer1
model.camera_bev.norm_eval	true	=	keep backbone BatchNorm in eval mode (small batches)
model.camera_bev.neck_channels	256	=	
model.camera_bev.feature_stride	16	=	
model.camera_bev.depth_bins	[1.0, 60.0, 1.0]	=	[min, max, step] metres -> 59 bins
model.camera_bev.out_channels	64	=	
model.camera_bev.camera_aware_depth	true	=	condition depth/context on intrinsics + extrinsics
model.camera_bev.bev_refine_blocks	2	=	
model.lidar_encoder.pillar_channels	[32, 64]	=	
model.lidar_encoder.out_channels	64	=	includes 4 explicit raster channels
model.lidar_encoder.num_res_blocks	2	=	
model.lidar_encoder.raster_channels	true	=	density / max height / mean height / intensity
model.radar_encoder.pillar_channels	[32, 64]	=	

Key	Default (base YAML)	This machine	Description
model.radar_encoder.out_channels	64	=	includes 5 explicit raster channels
model.radar_encoder.dilations	[1, 2, 4]	=	
model.radar_encoder.raster_channels	true	=	density / vx / vy / rcs / time-lag (velocity preserved)
model.bev_backbone.layer_nums	[3, 5]	=	
model.bev_backbone.layer_strides	[2, 2]	=	
model.bev_backbone.num_filters	[128, 256]	=	
model.bev_backbone.upsample_strides	[1, 2]	=	
model.bev_backbone.num_upsample_filters	[128, 128]	=	
model.detection_head.head_channels	64	=	
model.detection_head.out_stride	2	=	relative to the BEV grid -> 100 x 100 at 1.0 m
model.detection_head.min_radius	2	=	
model.detection_head.gaussian_overlap	0.1	=	
model.detection_head.max_objects	500	=	
model.detection_head.code_weights	[1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 0.2, 0.2]	=	dx dy z l w h sin cos vx vy
model.detection_head.score_threshold	0.1	=	
model.detection_head.max_detections	300	=	
model.detection_head.nms_radius.car	1.0	=	
model.detection_head.nms_radius.truck	1.5	=	
model.detection_head.nms_radius.construction_vehicle	1.5	=	
model.detection_head.nms_radius.bus	2.0	=	
model.detection_head.nms_radius.trailer	2.0	=	
model.detection_head.nms_radius.barrier	0.5	=	
model.detection_head.nms_radius.motorcycle	0.5	=	
model.detection_head.nms_radius.bicycle	0.5	=	
model.detection_head.nms_radius.pedestrian	0.25	=	
model.detection_head.nms_radius.traffic_cone	0.25	=	
model.trajectory_head.enabled	true	=	
model.trajectory_head.hidden	128	=	
model.trajectory_head.num_heads	4	=	

Section `fusion`

Key	Default (base YAML)	This machine	Description
fusion.method	attention_gated	=	concat | attention_gated (registry: models/fusion/registry.py)
fusion.use_camera_bev	true	=	
fusion.use_lidar	true	=	
fusion.use_radar	true	=	

Key	Default (base YAML)	This machine	Description
fusion.feature_selection	true	=	Camera-Guided Cross-Modal Feature Selection before fusion
fusion.out_channels	128	=	
fusion.deep_supervision	true	=	auxiliary per-modality heatmaps (L_fusion)
fusion.selection.camera_guided	true	=	camera BEV conditions the LiDAR/radar gates
fusion.selection.channel_reduction	4	=	cross-modal squeeze-excitation bottleneck
fusion.selection.spatial_hidden	32	=	
fusion.selection.spatial_kernel	7	=	CBAM-style spatial attention kernel
fusion.selection.gate_mode	soft	=	soft | hard_concrete (stochastic straight-through binary masks)
fusion.selection.temperature	0.67	=	hard_concrete temperature
fusion.selection.init_open	2.0	=	final-layer bias: gates start open (sigmoid(2) = 0.88)
fusion.selection.threshold	0.5	=	a feature counts as "selected" when its gate > threshold
fusion.selection.target_keep_ratio.spatial	0.35	=	
fusion.selection.target_keep_ratio.channel	0.75	=	

Section planning

Key	Default (base YAML)	This machine	Description
planning.planner	baseline	=	baseline | learned | hybrid (learned, verified by the baseline's collision check)
planning.horizon	6	=	future steps (nuScenes key-frames are 2 Hz)
planning.dt	0.5	=	seconds per step -> 3 s horizon
planning.command_threshold	2.0	=	|y_T| (m) above which the navigation command is LEFT / RIGHT
planning.ego.length	4.084	=	
planning.ego.width	1.85	=	
planning.ego.rear_axle_to_center	1.4	=	
planning.ego.wheelbase	2.588	=	
planning.ego.max_steer	0.6	=	rad
planning.ego.max_accel	3.0	=	
planning.ego.max_decel	6.0	=	
planning.ego.max_speed	20.0	=	
planning.collision_classes	["car", "truck", "construction_vehicle", "bus", "trailer", "motorcycle", "bicycle", "pedestrian"]	=	
planning.grid.x_range	[-10.0, 60.0]	=	
planning.grid.y_range	[-30.0, 30.0]	=	
planning.grid.resolution	0.5	=	
planning.occupancy.min_score	0.3	=	detections below this score are ignored by the planner
planning.occupancy.inflation	0.3	=	extra safety margin around obstacles (m)
planning.occupancy.use_lidar_obstacles	true	=	non-ground LiDAR points become a soft cost (covers missed detections)
planning.occupancy.lidar_z_band	[0.4, 2.5]	=	
planning.occupancy.lidar_cost	3.0	=	

Key	Default (base YAML)	This machine	Description
planning.occupancy.map_prior	false	=	use the nuScenes raster map prior as an off-road cost (loads large PNGs)
planning.occupancy.map_cost	2.0	=	
planning.astar.lookahead_min	6.0	=	goal distance bounds (m); the goal scales with speed * horizon
planning.astar.lookahead_max	45.0	=	
planning.astar.turn_angle_deg	35.0	=	goal bearing for LEFT / RIGHT commands
planning.astar.obstacle_clearance_cost	4.0	=	
planning.astar.max_expansions	60000	=	
planning.speed.accel_candidates	[-6.0, -4.0, -2.5, -1.5, -0.75, 0.0, 0.5, 1.0, 1.5]	=	
planning.speed.comfort_weight	0.05	=	
planning.speed.progress_weight	1.0	=	
planning.mpc.dt	0.1	=	
planning.mpc.iterations	12	=	
planning.mpc.w_pos	4.0	=	
planning.mpc.w_yaw	1.0	=	
planning.mpc.w_speed	0.5	=	
planning.mpc.w_accel	0.1	=	
planning.mpc.w_steer	2.0	=	
planning.mpc.w_steer_rate	4.0	=	

Section runtime

Key	Default (base YAML)	This machine	Description
runtime.seed	42	=	
runtime.deterministic	false	=	
runtime.device	auto	=	auto | cuda | cpu
runtime.hardware_profile	null	gtx1650	gtx1650 | rtx3060 | rtx4060 | rtx4070 | jetson_orin_nano
runtime.default_experiment	null	=	e.g. F_cam_lidar_radar_fs (configs/ablation.yaml)
runtime.output_dir	outputs	=	
runtime.checkpoint_dir	checkpoints	=	

Section train

Key	Default (base YAML)	This machine	Description
train.batch_size	2	1	
train.grad_accum	1	4	effective batch = batch_size * grad_accum
train.epochs	20	=	
train.lr	0.0002	=	
train.weight_decay	0.01	=	
train.amp	true	false	fp16 mixed precision on CUDA
train.grad_clip	35.0	=	
train.grad_checkpoint	false	true	trade compute for memory in the image backbone / BEV backbone
train.warmup_iters	100	=	
train.log_interval	20	=	
train.val_interval	1	=	epochs between validations
train.max_val_samples	null	=	cap validation samples for quick runs (null = all)
train.max_train_samples	null	=	

Section `eval`

Key	Default (base YAML)	This machine	Description
`eval.score_threshold`	`0.1`	=	
`eval.official_devkit`	`false`	=	also run the official nuScenes DetectionEval (needs nuscenes-devkit)

Section `loss`

Key	Default (base YAML)	This machine	Description
`loss.lambda_detection`	`1.0`	=	
`loss.lambda_trajectory`	`1.0`	=	
`loss.lambda_feature_selection`	`0.5`	=	
`loss.lambda_fusion`	`0.25`	=	
`loss.lambda_depth`	`1.0`	=	
`loss.detection.heatmap`	`1.0`	=	classification (Gaussian focal loss)
`loss.detection.center`	`0.25`	=	sub-cell offset + height
`loss.detection.bbox`	`0.25`	=	log-size regression
`loss.detection.orientation`	`0.25`	=	sin/cos yaw
`loss.detection.velocity`	`0.25`	=	
`loss.feature_selection.coverage`	`1.0`	=	selected features must cover objects (foreground guidance)
`loss.feature_selection.sparsity`	`0.5`	=	keep-ratio regulariser
`loss.feature_selection.binarization`	`0.1`	=	push gates towards 0/1 (interpretable masks)
`loss.trajectory.regression`	`1.0`	=	L1 to the logged human trajectory
`loss.trajectory.collision`	`2.0`	=	occupancy under the ego footprint (GT future occupancy)
`loss.trajectory.smoothness`	`0.1`	=	squared second differences
`loss.trajectory.goal`	`0.5`	=	final-point / progress term

Hardware profiles

Profile	Overrides

gtx1650	`dataset.num_workers = 2`
	`dataset.camera.image_size = [128, 352]`
	`dataset.lidar.sweeps = 5`
	`dataset.lidar.max_points = 150000`
	`dataset.radar.sweeps = 5`
	`train.batch_size = 1`
	`train.grad_accum = 4`
	`train.amp = false`
	`train.grad_checkpoint = true`

jetson_orin_nano	`dataset.num_workers = 2`
	`dataset.camera.image_size = [128, 352]`
	`dataset.lidar.sweeps = 3`
	`dataset.lidar.max_points = 100000`
	`dataset.radar.sweeps = 3`
	`train.batch_size = 1`
	`train.grad_accum = 4`
	`train.amp = true`
	`train.grad_checkpoint = true`

rtx3060	`dataset.num_workers = 4`
	`dataset.camera.image_size = [256, 704]`
	`dataset.lidar.sweeps = 10`
	`dataset.lidar.max_points = 250000`
	`dataset.radar.sweeps = 6`
	`train.batch_size = 4`
	`train.grad_accum = 1`
	`train.amp = true`
	`train.grad_checkpoint = false`

Profile	Overrides

rtx4060	`dataset.num_workers = 4`
	`dataset.camera.image_size = [256, 704]`
	`dataset.lidar.sweeps = 10`
	`dataset.lidar.max_points = 250000`
	`dataset.radar.sweeps = 6`
	`train.batch_size = 2`
	`train.grad_accum = 2`
	`train.amp = true`
	`train.grad_checkpoint = false`

rtx4070	`dataset.num_workers = 6`
	`dataset.camera.image_size = [256, 704]`
	`dataset.lidar.sweeps = 10`
	`dataset.lidar.max_points = 300000`
	`dataset.radar.sweeps = 6`
	`model.camera_bev.backbone = "resnet34"`
	`train.batch_size = 4`
	`train.grad_accum = 1`
	`train.amp = true`
	`train.grad_checkpoint = false`

Experiments

Experiment	Label	Description	Fusion overrides
`A_lidar_only`	LiDAR only	Experiment A - LiDAR only	`use_camera_bev=False, use_lidar=True, use_radar=False, method=concat, feature_selection=False`
`B_radar_only`	Radar only	Experiment B - Radar only	`use_camera_bev=False, use_lidar=False, use_radar=True, method=concat, feature_selection=False`
`C_lidar_radar_concat`	LiDAR + Radar	Experiment C - LiDAR + Radar, simple concatenation	`use_camera_bev=False, use_lidar=True, use_radar=True, method=concat, feature_selection=False`
`D_lidar_radar_attention`	LiDAR + Radar + Attention	Experiment D - LiDAR + Radar, attention-gated fusion (no feature selection)	`use_camera_bev=False, use_lidar=True, use_radar=True, method=attention_gated, feature_selection=False`
`E_lidar_radar_fs`	LiDAR + Radar + Feature Selection	Experiment E - LiDAR + Radar, feature selection + attention fusion	`use_camera_bev=False, use_lidar=True, use_radar=True, method=attention_gated, feature_selection=True`
`F_cam_lidar_radar_fs`	Camera + LiDAR + Radar + Feature Selection	Experiment F - Camera BEV + LiDAR + Radar, camera-guided feature selection + fusion (proposed)	`use_camera_bev=True, use_lidar=True, use_radar=True, method=attention_gated, feature_selection=True`
`G_cam_lidar_fs`	Camera + LiDAR	Camera BEV + LiDAR with feature selection	`use_camera_bev=True, use_lidar=True, use_radar=False, method=attention_gated, feature_selection=True`
`H_cam_radar_fs`	Camera + Radar	Camera BEV + Radar with feature selection	`use_camera_bev=True, use_lidar=False, use_radar=True, method=attention_gated, feature_selection=True`
`I_cam_lidar_radar_attention`	Camera + LiDAR + Radar	Camera BEV + LiDAR + Radar, attention fusion without feature selection	`use_camera_bev=True, use_lidar=True, use_radar=True, method=attention_gated, feature_selection=False`

Default run: `A_lidar_only`, `B_radar_only`, `C_lidar_radar_concat`, `D_lidar_radar_attention`, `E_lidar_radar_fs`, `F_cam_lidar_radar_fs`

Dashboard label	Experiment
1. LiDAR Only	A_lidar_only
2. Radar Only	B_radar_only
3. LiDAR + Radar	C_lidar_radar_concat
4. LiDAR + Radar + Attention	D_lidar_radar_attention
5. LiDAR + Radar + Feature Selection	E_lidar_radar_fs
6. Camera + LiDAR + Radar + Feature Selection	F_cam_lidar_radar_fs

Command-Line Reference

`python main.py <command> [arguments]` dispatches to the scripts below; every script can also be run directly. The arguments are extracted from the `argparse` definitions in the code.

main.py

```
Single entry point for the whole project.

python main.py preprocess [--maps]                    build sample-info caches (+ map caches)
python main.py validate                               Phase 2: sensor sync / calibration report + images
python main.py smoke [--experiments all]              shapes / losses / memory / latency on real data
python main.py train --stage joint --experiment F_cam_lidar_radar_fs [--epochs 20]
python main.py train --stage curriculum               the 7-stage curriculum (stages 1-7 in order)
python main.py evaluate --checkpoint checkpoints/F_cam_lidar_radar_fs/joint/best.pt [--official]
python main.py ablation [--train] [--epochs 20]       train/evaluate experiments A-F + plots
python main.py infer --scene scene-0103 [--video]     render a scene replay to images / MP4
python main.py bev --scene scene-0103 --frame 12      every BEV of one frame in a single image
python main.py planner --planner baseline             evaluate planners with GT perception
python main.py benchmark                              latency / memory per experiment
python main.py app                                    launch the Streamlit dashboard
python main.py test                                   run the test-suite
```

evaluation/ablation.py

Ablation study runner: train (optional) + evaluate every experiment with one recipe, then plot.

Argument	Default	Type / action	Choices	Help
`--experiments`	None			default: ablation.yaml default_run
`--train`		'store_true'		train experiments that have no checkpoint yet
`--retrain`		'store_true'		train even if a checkpoint exists
`--epochs`	None	int		
`--set`	[]			training config overrides key=value
`--proposed`	'F_cam_lidar_radar_fs'			
`--which`	'last'		['last', 'best']	evaluate the final weights (fair, default) or the best-validation-NDS epoch
`--plot-only`		'store_true'		redraw table/plot from results.json

scripts/benchmark.py

Latency / memory benchmark of every experiment configuration (inference, batch 1, real nuScenes frames).

Argument	Default	Type / action	Choices	Help
`--experiments`	None			
`--frames`	20	int		
`--set`	[]			

scripts/download_dataset.py

Help obtaining nuScenes.

Argument	Default	Type / action	Choices	Help
`--download-mini`		'store_true'		
`--dest`	None			
`--i-accept-the-nuscenes-terms`		'store_true'		

scripts/eval_planner.py

Evaluate planners on a split with *ground-truth* objects as perception (oracle upper bound), or with the constant-velocity baseline, without needing a trained network.

Argument	Default	Type / action	Choices	Help
--planner	'baseline'		['baseline', 'constant_velocity', 'logged']	
--split	None			
--set	[]			

scripts/evaluate.py

Evaluate a checkpoint on the validation split (all metric families) and save a JSON report.

Argument	Default	Type / action	Choices	Help
--checkpoint				
--official		'store_true'		also run nuscenes-devkit DetectionEval
--no-baseline-planner		'store_true'		
--set	[]			

scripts/inference.py

Run the full pipeline over a nuScenes scene and export composite frames, an MP4 and results JSON.

Argument	Default	Type / action	Choices	Help
--scene	'scene-0103'			
--experiment	'F_cam_lidar_radar_fs'			
--checkpoint	None			
--mode	'model'		['model', 'ground_truth', 'dataset']	
--planner	'baseline'		['baseline', 'learned', 'hybrid']	
--score	0.3	float		
--video		'store_true'		also write an MP4 (2 fps = real time)
--fps	2.0	float		
--set	[]			

scripts/preprocess.py

Pre-compute the per-sample info caches (and optionally the reduced map rasters).

Argument	Default	Type / action	Choices	Help
--maps		'store_true'		cache reduced map priors
--rebuild		'store_true'		ignore existing caches
--set	[]			

scripts/render_bev.py

Render every Bird's-Eye-View representation of one nuScenes frame into a single image.

Argument	Default	Type / action	Choices	Help
--scene	'scene-0103'			
--frame	12	int		key-frame index inside the scene
--experiment	'F_cam_lidar_radar_fs'			
--device	'cpu'			
--set	[]			

scripts/smoke_test.py

End-to-end smoke test on real nuScenes data: shapes, losses, backward, memory, latency.

Argument	Default	Type / action	Choices	Help
--experiments	['F_cam_lidar_radar_fs']			names or 'all
--set	[]			config overrides key=value
--quiet		'store_true'		

scripts/validate_sensors.py

Phase 2 - verify sensor synchronisation and calibration on real nuScenes data.

Argument	Default	Type / action	Choices	Help
--split	None			split to validate (default: dataset.val_split)
--samples	60	int		samples used for the numeric checks
--render	2	int		number of samples rendered to images
--set	[]			config overrides key=value

training/cli.py

Shared command-line entry point for all training scripts.

Argument	Default	Type / action	Choices	Help
--experiment	'F_cam_lidar_radar_fs'			entry of configs/ablation.yaml
--epochs	None	int		override the stage's epoch count
--resume	None			checkpoint to resume from
--set	[]			config overrides, e.g. train.batch_size=2
--stage			f"one of {CURRICULUM + ('joint', 'curriculum')}	

Line-by-Line Code Walkthroughs

This chapter explains the important source files **block by block**. Each code listing is read from the file at build time and shows the original line numbers; the explanation below it describes what the lines do, why they are needed, what data and tensor shapes they operate on and how they connect to the next block. Trivial lines (imports, simple assignments) are explained in groups; tensor transformations, geometry, attention, selection, fusion, losses, decoding and planning logic are explained in detail. Files not covered here (small scripts, tests, styles) are fully described in the *API Reference*. Emoji characters inside string literals (the dashboard's button labels) are printed as `[name]` placeholders because the PDF fonts contain no emoji glyphs; the source files contain the emoji.

Utilities and data layer

utils/config.py

Layered YAML configuration. Every tunable value of the project flows through `load_config()`; no path or hyper-parameter is hard-coded in Python. (203 lines)

```
"""Layered YAML configuration.

Merge order (later wins):

1. ``configs/{dataset,model,fusion,planning,training}.yaml``  (committed defaults)
2. ``configs/hardware/<profile>.yaml``                         (GPU-specific sizes)
3. experiment overrides from ``configs/ablation.yaml``         (fusion ablations)
4. ``configs/local.yaml``                                      (machine-specific, git-ignored)
5. command-line ``--set a.b.c=value`` overrides

String values may reference environment variables with ``${env:NAME,default}``.
No filesystem path is hard-coded in Python code; everything flows from here.
"""
```

Docstring: the merge order (base YAML files → hardware profile → experiment overrides → `configs/local.yaml` → command-line `--set key=value`) and the `${env:NAME,default}` syntax.

```
from __future__ import annotations

import copy
import os
import re
from pathlib import Path
from typing import Any, Iterable, Mapping, MutableMapping, Optional

import yaml
```

Imports: `copy` (deep copies of nested dicts), `os` (environment variables, user paths), `re` (environment-variable interpolation), `pathlib.Path`, typing helpers and PyYAML.

```
PROJECT_ROOT = Path(__file__).resolve().parents[1]
CONFIG_DIR = PROJECT_ROOT / "configs"
BASE_FILES = ("dataset.yaml", "model.yaml", "fusion.yaml", "planning.yaml", "training.yaml")
_ENV_PATTERN = re.compile(r"\$\{env:([A-Za-z_][A-Za-z0-9_]*)(?:,([^}]*))?\}")
```

Module constants. `PROJECT_ROOT` is computed from this file's location (`utils/` → parent), so the project works from any working directory. `CONFIG_DIR` = `configs/`. `BASE_FILES` fixes the order in which the five base YAML files are merged. `_ENV_PATTERN` matches `${env:NAME}` or `${env:NAME,default}`.

```
class Config(dict):
    """A ``dict`` with attribute access, used for every configuration object."""

    def __init__(self, *args: Any, **kwargs: Any) -> None:
        super().__init__(*args, **kwargs)
        for key, value in list(self.items()):
            self[key] = _wrap(value)

    def __getattr__(self, key: str) -> Any:
        try:
            return self[key]
        except KeyError as exc:  # keep hasattr()/pickle semantics correct
            raise AttributeError(key) from exc

    def __setattr__(self, key: str, value: Any) -> None:
        self[key] = _wrap(value)
```

```
46
47      def __deepcopy__(self, memo: dict) -> "Config":
48          return Config(copy.deepcopy(dict(self), memo))
49
50      def get_path(self, dotted: str, default: Any = None) -> Any:
51          """Return ``cfg['a']['b']['c']`` for ``dotted='a.b.c'`` or ``default``."""
52          node: Any = self
53          for part in dotted.split("."):
54              if not isinstance(node, Mapping) or part not in node:
55                  return default
56              node = node[part]
57          return node
58
59      def set_path(self, dotted: str, value: Any) -> None:
60          """Set a nested key, creating intermediate dictionaries as needed."""
61          parts = dotted.split(".")
62          node: MutableMapping = self
63          for part in parts[:-1]:
64              if part not in node or not isinstance(node[part], MutableMapping):
65                  node[part] = Config()
66              node = node[part]
67          node[parts[-1]] = _wrap(value)
68
69      def to_dict(self) -> dict:
70          """Plain-``dict`` deep copy (safe for YAML/JSON/pickle in checkpoints)."""
71          return _unwrap(self)
```

`Config` is a `dict` subclass with attribute access (`cfg.dataset.root` instead of `cfg['dataset']['root']`).

- `__init__` recursively wraps nested dicts/lists via `_wrap`.
- `__getattr__` converts a missing key into `AttributeError` (required for `hasattr`, `copy`, `pickle`).
- `__setattr__` stores attributes as keys.
- `get_path('a.b.c', default)` / `set_path('a.b.c', value)` read and write nested keys (used for command-line overrides and to patch machine-specific keys into checkpoint configs).
- `to_dict()` returns plain dicts (stored inside checkpoints).

```
74 def _wrap(value: Any) -> Any:
75      if isinstance(value, Config):
76          return value
77      if isinstance(value, Mapping):
78          return Config(value)
79      if isinstance(value, list):
80          return [_wrap(v) for v in value]
81      return value
82
83
84 def _unwrap(value: Any) -> Any:
85      if isinstance(value, Mapping):
86          return {k: _unwrap(v) for k, v in value.items()}
87      if isinstance(value, list):
88          return [_unwrap(v) for v in value]
89      return value
```

`_wrap` / `_unwrap`: recursive conversion between plain containers and `Config`.

```
92 def deep_merge(base: MutableMapping, update: Mapping) -> MutableMapping:
93      """Recursively merge ``update`` into ``base`` (in place) and return ``base``."""
94      for key, value in update.items():
95          if isinstance(value, Mapping) and isinstance(base.get(key), MutableMapping):
96              deep_merge(base[key], value)
97          else:
98              base[key] = copy.deepcopy(value)
99      return base
```

`deep_merge(base, update)`: recursive in-place merge; nested mappings are merged key by key, everything else (including lists) is replaced by a deep copy. This is what lets a hardware profile change only `dataset.camera.image_size` without repeating the other camera settings.

```
102 def _interpolate_env(value: Any) -> Any:
103      if isinstance(value, str):
104          def repl(match: re.Match) -> str:
105              name, default = match.group(1), match.group(2)
106              return os.environ.get(name, default if default is not None else "")
107          return _ENV_PATTERN.sub(repl, value)
108      if isinstance(value, dict):
109          return {k: _interpolate_env(v) for k, v in value.items()}
110      if isinstance(value, list):
111          return [_interpolate_env(v) for v in value]
112      return value
```

`_interpolate_env` walks the merged structure and replaces `${env:NAME,default}` inside strings with the environment value (or the default). `dataset.root` defaults to `${env:NUSCENES_ROOT,/path/to/nuscenes}`.

```
def read_yaml(path: Path) -> dict:
    with open(path, "r", encoding="utf-8") as f:
        data = yaml.safe_load(f)
    return data or {}


def parse_override(item: str) -> tuple[str, Any]:
    """Parse ``'a.b=3'`` into ``('a.b', 3)``; the value is parsed as YAML."""
    if "=" not in item:
        raise ValueError(f"Override must look like key=value, got {item!r}")
    key, raw = item.split("=", 1)
    return key.strip(), yaml.safe_load(raw)


def list_experiments(config_dir: Path = CONFIG_DIR) -> dict[str, dict]:
    """Return the experiment table from ``configs/ablation.yaml``."""
    path = config_dir / "ablation.yaml"
    if not path.exists():
        return {}
    return read_yaml(path).get("experiments", {})
```

`read_yaml(path)` → dict (empty dict for an empty file). `parse_override('a.b=3')` → `('a.b', 3)`; the value is parsed as YAML so `true`, `[128,352]`, `0.5` get their proper types. `list_experiments()` returns the `experiments` table of `configs/ablation.yaml`.

```
def load_config(
    experiment: Optional[str] = None,
    hardware: Optional[str] = None,
    overrides: Optional[Iterable[str]] = None,
    config_dir: Path = CONFIG_DIR,
    use_local: bool = True,
) -> Config:
    """Build the full configuration for a run.

    Args:
        experiment: name of an entry in ``configs/ablation.yaml`` (e.g. ``"F_cam_lidar_radar_fs"``).
            ``None`` uses ``runtime.default_experiment``.
        hardware: hardware profile name (``configs/hardware/<name>.yaml``). ``None`` uses
            ``runtime.hardware_profile``.
        overrides: ``key=value`` strings applied last.
        config_dir: directory containing the YAML files.
        use_local: merge ``configs/local.yaml`` when present.
    """
    merged: dict = {}
    for name in BASE_FILES:
        path = config_dir / name
        if path.exists():
            deep_merge(merged, read_yaml(path))

    local: dict = {}
    local_path = config_dir / "local.yaml"
    if use_local and local_path.exists():
        local = read_yaml(local_path)

    cli = [parse_override(o) for o in (overrides or [])]
    cli_dict = Config()
    for key, value in cli:
        cli_dict.set_path(key, value)

    hw_name = (
        hardware
        or Config(cli_dict).get_path("runtime.hardware_profile")
        or Config(local).get_path("runtime.hardware_profile")
        or Config(merged).get_path("runtime.hardware_profile")
    )
    if hw_name:
        hw_path = config_dir / "hardware" / f"{hw_name}.yaml"
        if not hw_path.exists():
            raise FileNotFoundError(f"Unknown hardware profile '{hw_name}' ({hw_path})")
        deep_merge(merged, read_yaml(hw_path))
        merged.setdefault("runtime", {})["hardware_profile"] = hw_name

    exp_name = experiment or Config(cli_dict).get_path("runtime.default_experiment") or Config(merged).get_path(
        "runtime.default_experiment"
    )
    if exp_name:
        experiments = list_experiments(config_dir)
        if exp_name not in experiments:
            raise KeyError(f"Unknown experiment '{exp_name}'. Available: {sorted(experiments)}")
        deep_merge(merged, experiments[exp_name].get("overrides", {}))
```

```
192         merged.setdefault("runtime", {})["experiment"] = exp_name
193         merged["runtime"]["experiment_description"] = experiments[exp_name].get("description", "")
194 
195     deep_merge(merged, local)
196     deep_merge(merged, cli_dict.to_dict())
197     return Config(_interpolate_env(merged))
```

`load_config(experiment, hardware, overrides, config_dir, use_local)` — the central function:

1. merge the base files in `BASE_FILES` order;
2. read `local.yaml` (optional) and parse the CLI overrides into a nested dict;
3. choose the hardware profile: explicit argument > CLI > local.yaml > base; merge `configs/hardware/<name>.yaml` (unknown name → `FileNotFoundError`);
4. choose the experiment the same way and merge its `overrides` (unknown → `KeyError` listing the available names); record `runtime.experiment` and its description;
5. merge local.yaml, then the CLI overrides (highest priority);
6. interpolate environment variables and return a `Config`.

```
200 def resolve_path(path_like: str | os.PathLike, base: Path = PROJECT_ROOT) -> Path:
201     """Resolve a config path: absolute paths are kept, relative ones are anchored at the project root."""
202     path = Path(os.path.expanduser(str(path_like)))
203     return path if path.is_absolute() else (base / path)
```

`resolve_path(p)`: absolute paths are returned unchanged, relative ones are anchored at the project root (used for `outputs/`, `checkpoints/`, the cache directory).

utils/geometry.py

NumPy rigid-body geometry. The docstring defines the coordinate conventions used everywhere (quaternion order, matrix naming, reference frame, box layout). (204 lines)

```
1 """Rigid-body geometry helpers (NumPy).
2 
3 Conventions used across the whole project
4 ----------------------------------------
5 * Quaternions follow nuScenes / pyquaternion order ``[w, x, y, z]``.
6 * 4x4 homogeneous matrices are named ``<dst>_from_<src>`` or ``<src>2<dst>``:
7   ``ego2global`` maps a point in the ego frame to the global frame.
8 * The **reference frame** of every sample is the ego-vehicle frame at the
9   timestamp of the LIDAR_TOP key-frame (x forward, y left, z up, origin at the
10  centre of the rear axle on the ground).
11 * 3D boxes are stored as ``[x, y, z, l, w, h, yaw, vx, vy]``. Note that nuScenes
12  stores sizes as ``[w, l, h]``; conversion happens once in the info builder.
13 """
14 from __future__ import annotations
15 
16 from typing import Sequence
17 
18 import numpy as np
```

Conventions (must be respected by any new code): quaternions `[w, x, y, z]`; matrices named `<src>2<dst>`; reference frame = ego at the LIDAR_TOP key-frame; boxes `[x, y, z, l, w, h, yaw, vx, vy]` (nuScenes stores `[w, l, h]` — converted once in the info builder).

```
21 def quat_to_rotmat(q: Sequence[float]) -> np.ndarray:
22     """Convert a unit quaternion ``[w, x, y, z]`` to a 3x3 rotation matrix."""
23     w, x, y, z = np.asarray(q, dtype=np.float64) / np.linalg.norm(q)
24     return np.array(
25         [
26             [1 - 2 * (y * y + z * z), 2 * (x * y - w * z), 2 * (x * z + w * y)],
27             [2 * (x * y + w * z), 1 - 2 * (x * x + z * z), 2 * (y * z - w * x)],
28             [2 * (x * z - w * y), 2 * (y * z + w * x), 1 - 2 * (x * x + y * y)],
29         ]
30     )
31 
32 
33 def rotmat_to_quat(rot: np.ndarray) -> np.ndarray:
34     """Convert a 3x3 rotation matrix to a unit quaternion ``[w, x, y, z]`` (w >= 0)."""
35     m = np.asarray(rot, dtype=np.float64)
36     trace = np.trace(m)
37     if trace > 0:
38         s = 0.5 / np.sqrt(trace + 1.0)
39         q = [0.25 / s, (m[2, 1] - m[1, 2]) * s, (m[0, 2] - m[2, 0]) * s, (m[1, 0] - m[0, 1]) * s]
40     elif m[0, 0] > m[1, 1] and m[0, 0] > m[2, 2]:
41         s = 2.0 * np.sqrt(1.0 + m[0, 0] - m[1, 1] - m[2, 2])
42         q = [(m[2, 1] - m[1, 2]) / s, 0.25 * s, (m[0, 1] + m[1, 0]) / s, (m[0, 2] + m[2, 0]) / s]
43     elif m[1, 1] > m[2, 2]:
```

```
        s = 2.0 * np.sqrt(1.0 + m[1, 1] - m[0, 0] - m[2, 2])
        q = [(m[0, 2] - m[2, 0]) / s, (m[0, 1] + m[1, 0]) / s, 0.25 * s, (m[1, 2] + m[2, 1]) / s]
    else:
        s = 2.0 * np.sqrt(1.0 + m[2, 2] - m[0, 0] - m[1, 1])
        q = [(m[1, 0] - m[0, 1]) / s, (m[0, 2] + m[2, 0]) / s, (m[1, 2] + m[2, 1]) / s, 0.25 * s]
    q = np.asarray(q)
    q /= np.linalg.norm(q)
    return q if q[0] >= 0 else -q
```

`quat_to_rotmat(q)`: normalises the quaternion and returns the standard 3×3 rotation matrix. `rotmat_to_quat(R)`: inverse conversion (Shepperd's method with the four branches for numerical stability; returns w ≥ 0). Used to write rotations in the nuScenes submission format.

```
def transform_matrix(translation: Sequence[float], rotation: Sequence[float] | np.ndarray) -> np.ndarray:
    """Build a 4x4 matrix from a translation and a quaternion (or 3x3 rotation)."""
    rot = np.asarray(rotation, dtype=np.float64)
    if rot.shape != (3, 3):
        rot = quat_to_rotmat(rot)
    mat = np.eye(4)
    mat[:3, :3] = rot
    mat[:3, 3] = np.asarray(translation, dtype=np.float64)
    return mat


def invert_transform(mat: np.ndarray) -> np.ndarray:
    """Invert a rigid 4x4 transform without a general matrix inverse."""
    inv = np.eye(4)
    rot_t = mat[:3, :3].T
    inv[:3, :3] = rot_t
    inv[:3, 3] = -rot_t @ mat[:3, 3]
    return inv


def apply_transform(mat: np.ndarray, points: np.ndarray) -> np.ndarray:
    """Apply a 4x4 transform to ``(N, 3+)`` points; extra columns are kept unchanged."""
    if points.shape[0] == 0:
        return points.copy()
    out = points.copy()
    out[:, :3] = points[:, :3] @ mat[:3, :3].T + mat[:3, 3]
    return out
```

`transform_matrix(t, q_or_R)` builds a 4×4 homogeneous transform; `invert_transform(T)` inverts a rigid transform analytically ($[R^T, -R^T t]$), cheaper and more accurate than `np.linalg.inv`; `apply_transform(T, points)` transforms the first three columns of an `(N, 3+)` array and keeps extra columns (intensity, time-lag) untouched.

```
def yaw_from_rotmat(rot: np.ndarray) -> float:
    """Heading of the x-axis of ``rot`` projected onto the ground plane (nuScenes ``quaternion_yaw``)."""
    return float(np.arctan2(rot[1, 0], rot[0, 0]))


def yaw_from_quat(q: Sequence[float]) -> float:
    return yaw_from_rotmat(quat_to_rotmat(q))


def wrap_angle(angle: np.ndarray | float) -> np.ndarray | float:
    """Wrap an angle (array) to ``[-pi, pi)``."""
    return (np.asarray(angle) + np.pi) % (2 * np.pi) - np.pi


def rot2d(theta: float) -> np.ndarray:
    c, s = np.cos(theta), np.sin(theta)
    return np.array([[c, -s], [s, c]])
```

`yaw_from_rotmat(R) = atan2(R[1,0], R[0,0])` — heading of the rotated x-axis projected onto the ground plane (identical to the devkit's `quaternion_yaw`). `wrap_angle` maps angles to [−π, π); `rot2d(θ)` is a 2×2 rotation.

```
def box_corners_bev(boxes: np.ndarray) -> np.ndarray:
    """BEV corners of boxes ``(N, >=7)`` ``[x, y, z, l, w, h, yaw, ...]`` -> ``(N, 4, 2)``.

    Corner order: front-left, front-right, rear-right, rear-left (counter-clockwise from front).
    """
    if boxes.shape[0] == 0:
        return np.zeros((0, 4, 2))
    x, y, l, w, yaw = boxes[:, 0], boxes[:, 1], boxes[:, 3], boxes[:, 4], boxes[:, 6]
    local = np.stack(
        [
            np.stack([l / 2, w / 2], -1),
            np.stack([l / 2, -w / 2], -1),
            np.stack([-l / 2, -w / 2], -1),
```

```
            np.stack([-l / 2, w / 2], -1),
        ],
        axis=1,
    )  # (N, 4, 2)
    c, s = np.cos(yaw), np.sin(yaw)
    rot = np.stack([np.stack([c, -s], -1), np.stack([s, c], -1)], axis=1)  # (N, 2, 2)
    return np.einsum("nij,nkj->nki", rot, local) + np.stack([x, y], -1)[:, None, :]


def box_corners_3d(boxes: np.ndarray) -> np.ndarray:
    """3D corners ``(N, 8, 3)``; first four on the top face, last four on the bottom face."""
    if boxes.shape[0] == 0:
        return np.zeros((0, 8, 3))
    bev = box_corners_bev(boxes)
    z, h = boxes[:, 2], boxes[:, 5]
    top = np.concatenate([bev, np.repeat((z + h / 2)[:, None, None], 4, 1)], -1)
    bottom = np.concatenate([bev, np.repeat((z - h / 2)[:, None, None], 4, 1)], -1)
    return np.concatenate([top, bottom], axis=1)
```

`box_corners_bev(boxes)` → `(N, 4, 2)`: corners front-left, front-right, rear-right, rear-left in the local box frame `(±l/2, ±w/2)`, rotated by yaw (batched with `einsum`) and translated. `box_corners_3d` adds top (z + h/2) and bottom (z − h/2) faces → `(N, 8, 3)` (used to draw boxes in camera images).

```
def points_in_boxes(points: np.ndarray, boxes: np.ndarray) -> np.ndarray:
    """Boolean mask ``(N_points, N_boxes)`` of points inside 3D yaw-rotated boxes."""
    if points.shape[0] == 0 or boxes.shape[0] == 0:
        return np.zeros((points.shape[0], boxes.shape[0]), dtype=bool)
    rel = points[:, None, :3] - boxes[None, :, :3]  # (P, B, 3)
    c, s = np.cos(-boxes[:, 6]), np.sin(-boxes[:, 6])
    lx = rel[..., 0] * c - rel[..., 1] * s
    ly = rel[..., 0] * s + rel[..., 1] * c
    return (
        (np.abs(lx) <= boxes[:, 3] / 2)
        & (np.abs(ly) <= boxes[:, 4] / 2)
        & (np.abs(rel[..., 2]) <= boxes[:, 5] / 2)
    )
```

`points_in_boxes(points (P, 3), boxes (B, 7+))` → boolean `(P, B)`: each point is rotated into each box's frame by −yaw and tested against the half-extents. Used by the calibration validation (points inside GT boxes vs. `num_lidar_pts`) and the bike-rack rule of the evaluator.

```
def transform_boxes(boxes: np.ndarray, mat: np.ndarray) -> np.ndarray:
    """Transform ``[x, y, z, l, w, h, yaw, vx, vy]`` boxes with a rigid 4x4 transform.

    Velocities are rotated (not translated); yaw is re-extracted from the rotated heading
    vector so pitch/roll in ``mat`` cannot corrupt it.
    """
    if boxes.shape[0] == 0:
        return boxes.copy()
    out = boxes.copy()
    out[:, :3] = boxes[:, :3] @ mat[:3, :3].T + mat[:3, 3]
    heading = np.stack([np.cos(boxes[:, 6]), np.sin(boxes[:, 6]), np.zeros(len(boxes))], -1) @ mat[:3, :3].T
    out[:, 6] = np.arctan2(heading[:, 1], heading[:, 0])
    if boxes.shape[1] >= 9:
        vel = np.concatenate([boxes[:, 7:9], np.zeros((len(boxes), 1))], -1) @ mat[:3, :3].T
        out[:, 7:9] = vel[:, :2]
    return out
```

`transform_boxes(boxes, T)`: centres are transformed as points, the heading as a *vector* `(cos yaw, sin yaw, 0)` rotated by `R` (so pitch/roll in `T` cannot corrupt yaw), velocities as vectors (rotation only). Used to bring every annotation from the global frame into the reference frame.

```
def project_to_image(points_cam: np.ndarray, intrinsic: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
    """Pinhole projection of camera-frame points ``(N, 3)`` -> pixels ``(N, 2)`` and depths ``(N,)``."""
    depth = points_cam[:, 2]
    uvw = points_cam[:, :3] @ intrinsic.T
    with np.errstate(divide="ignore", invalid="ignore"):
        uv = uvw[:, :2] / uvw[:, 2:3]
    return uv, depth
```

`project_to_image(p_cam, K)` → pixels `(N, 2)` and depths `(N,)` (pinhole model, division by z; invalid divisions are silenced).

```
def bev_rotation_matrix(rot_angle: float, scale: float, flip_x: bool, flip_y: bool) -> np.ndarray:
    """4x4 BEV data-augmentation matrix (``bda``): flips, then rotation about z, then isotropic scale.

    ``flip_y`` mirrors the world left/right (y -> -y); ``flip_x`` mirrors forward/backward.
    """
    flip = np.diag([-1.0 if flip_x else 1.0, -1.0 if flip_y else 1.0, 1.0])
    rot = np.eye(3)
    rot[:2, :2] = rot2d(rot_angle)
```

```
    mat = np.eye(4)
    mat[:3, :3] = scale * rot @ flip
    return mat


def augment_boxes_bev(boxes: np.ndarray, bda: np.ndarray) -> np.ndarray:
    """Apply a BEV augmentation matrix (possibly with reflection and scale) to boxes."""
    if boxes.shape[0] == 0:
        return boxes.copy()
    out = boxes.copy()
    lin = bda[:3, :3]
    scale = float(np.cbrt(abs(np.linalg.det(lin))))
    out[:, :3] = boxes[:, :3] @ lin.T + bda[:3, 3]
    out[:, 3:6] = boxes[:, 3:6] * scale
    heading = np.stack([np.cos(boxes[:, 6]), np.sin(boxes[:, 6]), np.zeros(len(boxes))], -1) @ lin.T
    out[:, 6] = np.arctan2(heading[:, 1], heading[:, 0])
    if boxes.shape[1] >= 9:
        vel = np.concatenate([boxes[:, 7:9], np.zeros((len(boxes), 1))], -1) @ lin.T
        out[:, 7:9] = vel[:, :2]
    return out
```

`bev_rotation_matrix(rot, scale, flip_x, flip_y)` builds the 4×4 BEV augmentation matrix `bda = scale · R_z(rot) · Flip`. `augment_boxes_bev(boxes, bda)` applies it to boxes: centres transformed, sizes multiplied by the scale (the cube root of |det|), heading and velocity transformed as vectors — so a mirror (`flip_y`) correctly negates yaw and vy.

utils/bev_grid.py

The single BEV grid definition shared by every module — the key to keeping camera, LiDAR, radar, targets, planning and visualisation aligned. (66 lines)

```
"""The single Bird's-Eye-View grid definition shared by every module.

Layout convention (identical for camera, LiDAR, radar, fusion, detection, planning):

* BEV tensors are ``(B, C, H, W)`` with ``H = ny`` (rows index **y**, left axis) and
  ``W = nx`` (columns index **x**, forward axis).
* Cell ``(iy, ix)`` covers ``x in [x_min + ix*r, x_min + (ix+1)*r)`` and
  ``y in [y_min + iy*r, y_min + (iy+1)*r)``.
* For display, :func:`visualization.common.bev_to_display` rotates arrays so that
  forward points up and left points left.
"""
from __future__ import annotations

from dataclasses import dataclass
from typing import Any, Mapping

import numpy as np
```

Docstring: tensors are `(B, C, H = ny, W = nx)`; rows index y (left), columns index x (forward); cell `(iy, ix)` covers `[x_min + ix·r, x_min + (ix+1)·r) × [y_min + iy·r, …)`.

```
@dataclass(frozen=True)
class BEVGrid:
    x_range: tuple[float, float]
    y_range: tuple[float, float]
    z_range: tuple[float, float]
    resolution: float

    @property
    def nx(self) -> int:
        return int(round((self.x_range[1] - self.x_range[0]) / self.resolution))

    @property
    def ny(self) -> int:
        return int(round((self.y_range[1] - self.y_range[0]) / self.resolution))

    @property
    def shape(self) -> tuple[int, int]:
        """``(H, W) = (ny, nx)``."""
        return self.ny, self.nx

    def downsample(self, stride: int) -> "BEVGrid":
        """Same extent with ``stride``-times coarser cells (e.g. the detection-head grid)."""
        return BEVGrid(self.x_range, self.y_range, self.z_range, self.resolution * stride)

    def world_to_grid(self, x: np.ndarray, y: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
        """Continuous grid coordinates (``ix``, ``iy``); cell centres are at ``k + 0.5``."""
        return (np.asarray(x) - self.x_range[0]) / self.resolution, (np.asarray(y) - self.y_range[0]) / self.reso
            lution

```

```
    def grid_to_world(self, ix: np.ndarray, iy: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
        return np.asarray(ix) * self.resolution + self.x_range[0], np.asarray(iy) * self.resolution + self.y_rang
            e[0]

    def in_range(self, x: np.ndarray, y: np.ndarray) -> np.ndarray:
        return (x >= self.x_range[0]) & (x < self.x_range[1]) & (y >= self.y_range[0]) & (y < self.y_range[1])

    def cell_centers(self) -> tuple[np.ndarray, np.ndarray]:
        xs = self.x_range[0] + (np.arange(self.nx) + 0.5) * self.resolution
        ys = self.y_range[0] + (np.arange(self.ny) + 0.5) * self.resolution
        return xs, ys

    @classmethod
    def from_config(cls, bev_cfg: Mapping[str, Any]) -> "BEVGrid":
        return cls(
            tuple(float(v) for v in bev_cfg["x_range"]),
            tuple(float(v) for v in bev_cfg["y_range"]),
            tuple(float(v) for v in bev_cfg["z_range"]),
            float(bev_cfg["resolution"]),
        )
```

`BEVGrid` is an immutable dataclass with `x_range`, `y_range`, `z_range`, `resolution`. Properties `nx`, `ny`, `shape`; `downsample(stride)` returns the same extent with coarser cells (the detection-head grid: stride 2 → 100 × 100); `world_to_grid` / `grid_to_world` convert between metres and continuous cell coordinates (cell centres at k + 0.5); `in_range` masks points; `cell_centers` returns the metric centres; `from_config(cfg.bev)` builds the grid from YAML.

utils/raster.py

Rasterisation helpers (OpenCV) for boxes and points on a `BEVGrid`. (45 lines)

```
"""Rasterisation of oriented boxes / points onto a :class:`BEVGrid` (array layout ``[iy, ix]``)."""
from __future__ import annotations

import cv2
import numpy as np

from utils.bev_grid import BEVGrid
from utils.geometry import box_corners_bev

_SHIFT = 4  # cv2 fixed-point sub-pixel bits
```

Imports and `_SHIFT = 4`: OpenCV fixed-point precision (coordinates are multiplied by 2^4 for sub-pixel accurate polygon filling).

```
def rasterize_boxes(boxes: np.ndarray, grid: BEVGrid, inflate: float = 0.0, values: np.ndarray | None = None,
                    out: np.ndarray | None = None) -> np.ndarray:
    """Fill oriented BEV boxes ``[x, y, z, l, w, h, yaw, ...]`` into an ``(ny, nx)`` array.

    ``inflate`` grows length and width by ``2 * inflate`` metres (safety margin).
    """
    canvas = out if out is not None else np.zeros(grid.shape, dtype=np.uint8)
    if boxes.shape[0] == 0:
        return canvas
    b = boxes.astype(np.float64).copy()
    # cv2.fillPoly also fills the pixels on the polygon boundary, which adds ~half a cell on every side;
    # shrinking by one cell makes the filled area match the metric box area.
    b[:, 3:5] = np.maximum(b[:, 3:5] + 2.0 * inflate - grid.resolution, 0.01 * grid.resolution)
    corners = box_corners_bev(b)  # (N, 4, 2) metres
    cols = (corners[..., 0] - grid.x_range[0]) / grid.resolution
    rows = (corners[..., 1] - grid.y_range[0]) / grid.resolution
    poly = np.round(np.stack([cols, rows], -1) * (1 << _SHIFT) - 0.5 * (1 << _SHIFT)).astype(np.int32)
    for i in range(len(poly)):
        value = 1 if values is None else values[i]
        cv2.fillPoly(canvas, [poly[i]], color=value, lineType=cv2.LINE_8, shift=_SHIFT)
    return canvas
```

`rasterize_boxes(boxes, grid, inflate, values, out)` fills oriented boxes into an `(ny, nx)` uint8 array.

- `cv2.fillPoly` also fills pixels on the polygon boundary, which adds about half a cell on every side; the code therefore shrinks length and width by one cell (`− grid.resolution`, never below 1 % of a cell) so the filled area matches the metric area (verified by `tests/test_geometry.py`).
- Corners are converted to grid coordinates (x → column, y → row), shifted by −0.5 cell because OpenCV pixel centres sit on integer coordinates, and drawn with sub-pixel precision.
- `inflate` grows the box by a safety margin on each side (used by the planner).

```
def rasterize_points(points: np.ndarray, grid: BEVGrid) -> np.ndarray:
    """Count of points per cell ``(ny, nx)`` (float32)."""
```

```
    counts = np.zeros(grid.shape, dtype=np.float32)
    if points.shape[0] == 0:
        return counts
    ix = np.floor((points[:, 0] - grid.x_range[0]) / grid.resolution).astype(np.int64)
    iy = np.floor((points[:, 1] - grid.y_range[0]) / grid.resolution).astype(np.int64)
    ok = (ix >= 0) & (ix < grid.nx) & (iy >= 0) & (iy < grid.ny)
    np.add.at(counts, (iy[ok], ix[ok]), 1.0)
    return counts
```

`rasterize_points(points, grid)` → `(ny, nx)` float32 point counts (`np.add.at` handles repeated cells).

utils/device.py

Device selection, the fp16 reliability self-test and GPU memory reporting. (78 lines)

```
"""Device selection and GPU memory reporting."""
from __future__ import annotations

import logging
from typing import Any

import torch
```

Imports.

```
def get_device(preference: str = "auto") -> torch.device:
    """Use CUDA when available (or requested) and fall back to CPU gracefully."""
    pref = (preference or "auto").lower()
    if pref.startswith("cuda") and not torch.cuda.is_available():
        return torch.device("cpu")
    if pref == "auto":
        return torch.device("cuda" if torch.cuda.is_available() else "cpu")
    return torch.device(pref)
```

`get_device(preference)`: `auto` → CUDA if available else CPU; a requested CUDA device falls back to CPU when CUDA is unavailable (the program never crashes for lack of a GPU).

```
_FP16_OK: dict[str, bool] = {}


def fp16_is_reliable(device: torch.device) -> bool:
    """Numerically test fp16 convolutions on this GPU (cached).

    Some GPU / cuDNN combinations return NaN for fp16 convolutions: on a GTX 1650 with
    cuDNN 9.10 every 3x3 conv with 256 input channels does (measured in this project,
    see README "Hardware notes"). The test runs exactly that layer type in fp16 and fp32.
    """
    key = str(device)
    if key not in _FP16_OK:
        g = torch.Generator(device="cpu").manual_seed(0)
        conv = torch.nn.Conv2d(256, 256, 3, padding=1).to(device)
        ok = True
        # Shapes that occur in this model (image backbone layer3, BEV backbone block2, head input);
        # cuDNN picks different kernels per shape, so all of them are probed.
        for shape in ((6, 256, 8, 22), (1, 256, 50, 50), (1, 256, 100, 100)):
            x = torch.randn(*shape, generator=g).to(device)
            with torch.no_grad():
                ref = conv(x)
                with torch.autocast(device_type="cuda", dtype=torch.float16):
                    y = conv(x).float()
            ok &= bool(torch.isfinite(y).all()) and float((y - ref).abs().max() / ref.abs().max()) < 0.02
        _FP16_OK[key] = ok
        if not ok:
            logging.getLogger(__name__).warning(
                "fp16 convolutions are numerically broken on %s (cuDNN %s); mixed precision disabled, using fp32.
                    ",
                torch.cuda.get_device_name(device), torch.backends.cudnn.version())
    return _FP16_OK[key]
```

`fp16_is_reliable(device)` — added after a real failure on the development machine: with cuDNN 9.10 the GTX 1650 returns NaN for fp16 3×3 convolutions with 256 input channels. The function runs exactly that convolution type in fp16 and fp32 on three tensor shapes that occur in the model and compares the outputs (finite and < 2 % relative error). The result is cached per device; a warning is logged when fp16 is broken.

```
def amp_enabled(device: torch.device, requested: bool) -> bool:
    """Use fp16 mixed precision only on CUDA devices where fp16 convolutions are verified correct."""
    return bool(requested) and device.type == "cuda" and fp16_is_reliable(device)
```

`amp_enabled(device, requested)`: mixed precision is used only if requested, on CUDA, and when the self-test passes. Every training/evaluation/inference code path calls this function, so a broken GPU silently falls back to fp32 instead of producing NaN losses.

```
57 def gpu_memory_report(device: torch.device) -> dict[str, Any]:
58     """Current / peak allocated memory in MiB (empty dict on CPU)."""
59     if device.type != "cuda":
60         return {}
61     return {
62         "allocated_mb": torch.cuda.memory_allocated(device) / 2**20,
63         "peak_allocated_mb": torch.cuda.max_memory_allocated(device) / 2**20,
64         "reserved_mb": torch.cuda.memory_reserved(device) / 2**20,
65         "total_mb": torch.cuda.get_device_properties(device).total_memory / 2**20,
66         "name": torch.cuda.get_device_name(device),
67     }
68
69
70 def move_to_device(obj: Any, device: torch.device, non_blocking: bool = True) -> Any:
71     """Recursively move tensors inside dicts/lists/tuples to ``device``."""
72     if torch.is_tensor(obj):
73         return obj.to(device, non_blocking=non_blocking)
74     if isinstance(obj, dict):
75         return {k: move_to_device(v, device, non_blocking) for k, v in obj.items()}
76     if isinstance(obj, (list, tuple)):
77         return type(obj)(move_to_device(v, device, non_blocking) for v in obj)
78     return obj
```

`gpu_memory_report` returns allocated / peak / reserved / total MiB and the GPU name (empty on CPU). `move_to_device` recursively moves tensors inside dicts/lists/tuples (batches contain nested lists and metadata dicts).

datasets/nuscenes_db.py

A lightweight replacement for the devkit's `NuScenes` class: loads the 13 JSON tables and builds the reverse indexes the rest of the code needs. The official devkit is only needed for the official evaluation and the full train/val split lists. (119 lines)

```
1  """Lightweight nuScenes table loader.
2
3  Reads the raw JSON tables of any nuScenes version (``v1.0-mini``, ``v1.0-trainval``)
4  and builds the same reverse indexes as the official devkit (``sample['data']``,
5  ``sample['anns']``, ``sample_data['channel']``), without importing the devkit.
6  The devkit stays an *optional* dependency, used only for the official evaluation.
7  """
8  from __future__ import annotations
9
10 import json
11 import logging
12 import time
13 from pathlib import Path
14 from typing import Any, Iterator
15
16 logger = logging.getLogger(__name__)
17
18 TABLES = (
19     "attribute", "calibrated_sensor", "category", "ego_pose", "instance", "log", "map",
20     "sample", "sample_annotation", "sample_data", "scene", "sensor", "visibility",
21 )
```

Docstring and the list `TABLES` of the 13 nuScenes tables.

```
24 class NuScenesDB:
25     """In-memory nuScenes database with token lookup.
26
27     Args:
28         root: dataset root containing ``samples/``, ``sweeps/``, ``maps/`` and ``<version>/``.
29         version: table folder name, e.g. ``"v1.0-mini"``.
30     """
31
32     def __init__(self, root: str | Path, version: str = "v1.0-mini", verbose: bool = True) -> None:
33         self.root = Path(root)
34         self.version = version
35         table_dir = self.root / version
36         if not table_dir.is_dir():
37             raise FileNotFoundError(
38                 f"nuScenes tables not found at '{table_dir}'. Set dataset.root / dataset.version "
39                 f"(configs/local.yaml) to the folder that contains '{version}/'."
40             )
41         start = time.time()
42         self.tables: dict[str, list[dict]] = {}
```

```
        self._index: dict[str, dict[str, dict]] = {}
        for name in TABLES:
            path = table_dir / f"{name}.json"
            if not path.exists():
                raise FileNotFoundError(f"Missing nuScenes table {path}")
            with open(path, "r", encoding="utf-8") as f:
                records = json.load(f)
            self.tables[name] = records
            self._index[name] = {r["token"]: r for r in records}
        self._build_reverse_index()
        if verbose:
            logger.info(
                "Loaded nuScenes %s from %s: %d scenes, %d samples, %d annotations (%.1fs)",
                version, self.root, len(self.tables["scene"]), len(self.tables["sample"]),
                len(self.tables["sample_annotation"]), time.time() - start,
            )

    # ------------------------------------------------------------------ index
```

Constructor: checks that `<root>/<version>/` exists (clear `FileNotFoundError` telling the user to set `dataset.root`), loads every table with `json.load` into `self.tables[name]` (list of records) and `self._index[name]` (token → record dict), builds the reverse index and logs a summary. For v1.0-mini this takes ≈ 0.5 s.

```
    def _build_reverse_index(self) -> None:
        for rec in self.tables["sample"]:
            rec["data"] = {}
            rec["anns"] = []
        for sd in self.tables["sample_data"]:
            cs = self.get("calibrated_sensor", sd["calibrated_sensor_token"])
            sensor = self.get("sensor", cs["sensor_token"])
            sd["channel"] = sensor["channel"]
            sd["sensor_modality"] = sensor["modality"]
            if sd["is_key_frame"]:
                self.get("sample", sd["sample_token"])["data"][sensor["channel"]] = sd["token"]
        for ann in self.tables["sample_annotation"]:
            self.get("sample", ann["sample_token"])["anns"].append(ann["token"])
            inst = self.get("instance", ann["instance_token"])
            ann["category_name"] = self.get("category", inst["category_token"])["name"]
        self._scene_by_name = {s["name"]: s for s in self.tables["scene"]}
        self._log_to_map: dict[str, dict] = {}
        for m in self.tables["map"]:
            for log_token in m["log_tokens"]:
                self._log_to_map[log_token] = m

    # ------------------------------------------------------------------ access
```

`_build_reverse_index()` adds the same derived fields as the devkit:

- `sample['data'] = {channel: sample_data_token}` for key-frames and `sample['anns'] = [...]`;
- `sample_data['channel']` and `['sensor_modality']` (via calibrated_sensor → sensor);
- `annotation['category_name']` (via instance → category);
- `scene name → scene` and `log token → map` lookups.

```
    def get(self, table: str, token: str) -> dict[str, Any]:
        try:
            return self._index[table][token]
        except KeyError as exc:
            raise KeyError(f"Token {token!r} not found in table {table!r}") from exc

    def has(self, table: str, token: str) -> bool:
        return token in self._index[table]

    @property
    def scenes(self) -> list[dict]:
        return self.tables["scene"]

    def scene_by_name(self, name: str) -> dict:
        return self._scene_by_name[name]

    def scene_sample_tokens(self, scene: dict) -> list[str]:
        """Key-frame sample tokens of a scene in temporal order."""
        tokens, token = [], scene["first_sample_token"]
        while token:
            tokens.append(token)
            token = self.get("sample", token)["next"]
        return tokens

    def iter_sweeps(self, sd_token: str, direction: str = "prev") -> Iterator[dict]:
        """Walk the ``prev``/``next`` chain of a sample_data record (including itself)."""
        token = sd_token
```

```
        while token:
            rec = self.get("sample_data", token)
            yield rec
            token = rec[direction]

    def map_for_log(self, log_token: str) -> dict | None:
        return self._log_to_map.get(log_token)

    def file_path(self, sample_data: dict) -> Path:
        return self.root / sample_data["filename"]
```

Accessors: `get(table, token)` (with a helpful `KeyError`), `has`, `scenes`, `scene_by_name`, `scene_sample_tokens(scene)` (follows the `next` chain from the first sample → key-frames in temporal order), `iter_sweeps(sd_token, 'prev'|'next')` (walks a sensor's sample_data chain including non-key-frame sweeps), `map_for_log`, `file_path`.

datasets/info_builder.py

Pre-computes one self-contained *info* dictionary per key-frame: file paths, calibration and ego-pose matrices, ground truth in the reference frame, ego state, ego future/past and future agents. The result is cached as a pickle so training and the dashboard never parse the tables again. (407 lines)

```
"""Pre-compute per-sample "infos" from the nuScenes tables.

An *info* is a self-contained dictionary with everything a training / inference
step needs for one key-frame: file paths of all sensors (+ sweeps), calibration and
ego-pose matrices, ground-truth boxes **already expressed in the reference frame**,
ego state, ego future/past trajectory and future agent boxes.

Building infos needs the full table set (slow for trainval); the result is cached as a
pickle, so the training dataset and the dashboard never touch the raw tables again.

Coordinate frames
-----------------
``sensor --(sensor2ego)--> ego(t_sensor) --(ego2global)--> global --(inv ref_ego2global)--> ref``

The reference frame ``ref`` is the ego frame at the LIDAR_TOP key-frame timestamp.
Cameras, radars and LiDAR sweeps are captured at slightly different times; going
through the global frame with each sensor's *own* ego pose compensates for ego motion
between the capture times (this is the time synchronisation).
"""
```

Docstring: the transformation chain `sensor → ego(t_sensor) → global → ref` and why going through the global frame with each sensor's own ego pose performs the time synchronisation.

```
from __future__ import annotations

import hashlib
import json
import logging
import pickle
from pathlib import Path
from typing import Any, Optional

import numpy as np

from datasets.nuscenes_db import NuScenesDB
from datasets.splits import get_split_scenes
from utils.config import Config, resolve_path
from utils.geometry import invert_transform, quat_to_rotmat, transform_boxes, transform_matrix, wrap_angle, yaw_f
    rom_rotmat

logger = logging.getLogger(__name__)

INFO_FORMAT_VERSION = 4

# Official nuScenes category -> detection class mapping (nuscenes.eval.detection.utils).
CATEGORY_TO_DETECTION = {
    "movable_object.barrier": "barrier",
    "vehicle.bicycle": "bicycle",
    "vehicle.bus.bendy": "bus",
    "vehicle.bus.rigid": "bus",
    "vehicle.car": "car",
    "vehicle.construction": "construction_vehicle",
    "vehicle.motorcycle": "motorcycle",
    "human.pedestrian.adult": "pedestrian",
    "human.pedestrian.child": "pedestrian",
    "human.pedestrian.construction_worker": "pedestrian",
    "human.pedestrian.police_officer": "pedestrian",
    "movable_object.trafficcone": "traffic_cone",
```

```
    "vehicle.trailer": "trailer",
    "vehicle.truck": "truck",
}

# Navigation command derived from the logged future (ST-P3 / UniAD convention).
COMMANDS = ("RIGHT", "LEFT", "STRAIGHT")
```

Imports, `INFO_FORMAT_VERSION` (bumped when the info layout changes → invalidates old caches), the official `CATEGORY_TO_DETECTION` mapping from nuScenes categories to the 10 detection classes, and `COMMANDS` (index 0 = RIGHT, 1 = LEFT, 2 = STRAIGHT).

```
def _sd_matrices(db: NuScenesDB, sd: dict) -> tuple[np.ndarray, np.ndarray, dict]:
    cs = db.get("calibrated_sensor", sd["calibrated_sensor_token"])
    pose = db.get("ego_pose", sd["ego_pose_token"])
    return transform_matrix(cs["translation"], cs["rotation"]), transform_matrix(pose["translation"], pose["rotat
        ion"]), cs
```

`_sd_matrices(db, sd)` returns `sensor2ego` (calibrated_sensor) and `ego2global` (ego_pose at this sample_data's timestamp) as 4×4 matrices.

```
def _sensor_record(db: NuScenesDB, sd: dict, ref_global2ego: np.ndarray, ref_timestamp: int) -> dict[str, Any]:
    sensor2ego, ego2global, cs = _sd_matrices(db, sd)
    rec = {
        "token": sd["token"],
        "path": sd["filename"],  # relative to dataset root (keeps the cache portable)
        "timestamp": sd["timestamp"],
        "time_lag": (ref_timestamp - sd["timestamp"]) * 1e-6,
        "sensor2ego": sensor2ego,
        "ego2global": ego2global,
        "sensor2ref": ref_global2ego @ ego2global @ sensor2ego,
    }
    if cs.get("camera_intrinsic"):
        rec["intrinsic"] = np.asarray(cs["camera_intrinsic"], dtype=np.float64)
        rec["width"], rec["height"] = sd["width"], sd["height"]
    return rec
```

`_sensor_record()` builds the per-sweep record: relative file path (keeps the cache portable across machines), timestamp, `time_lag` = (t_ref − t_sweep) in seconds, both matrices and the combined `sensor2ref = inv(ref_ego2global) · ego2global · sensor2ego`. Cameras also get the intrinsic matrix and the image size.

```
def annotation_velocity(db: NuScenesDB, ann: dict, max_time_diff: float = 1.5) -> np.ndarray:
    """Global-frame velocity of an annotation (identical to ``NuScenes.box_velocity``)."""
    has_prev, has_next = ann["prev"] != "", ann["next"] != ""
    if not has_prev and not has_next:
        return np.full(3, np.nan)
    first = db.get("sample_annotation", ann["prev"]) if has_prev else ann
    last = db.get("sample_annotation", ann["next"]) if has_next else ann
    pos_diff = np.asarray(last["translation"]) - np.asarray(first["translation"])
    t_last = 1e-6 * db.get("sample", last["sample_token"])["timestamp"]
    t_first = 1e-6 * db.get("sample", first["sample_token"])["timestamp"]
    time_diff = t_last - t_first
    if has_prev and has_next:
        max_time_diff *= 2
    if time_diff <= 0 or time_diff > max_time_diff:
        return np.full(3, np.nan)
    return pos_diff / time_diff
```

`annotation_velocity()` reproduces `NuScenes.box_velocity`: central difference of the annotation positions using the previous and next annotation of the same instance (one-sided at the ends), `nan` when the instance has a single annotation or the time gap exceeds 1.5 s (3 s for a central difference).

```
def _ann_box_global(ann: dict) -> np.ndarray:
    """nuScenes annotation -> ``[x, y, z, l, w, h, yaw]`` in the global frame (size is ``[w, l, h]``)."""
    w, l, h = ann["size"]
    yaw = yaw_from_rotmat(quat_to_rotmat(ann["rotation"]))
    return np.array([*ann["translation"], l, w, h, yaw], dtype=np.float64)
```

`_ann_box_global(ann)` converts a nuScenes annotation to `[x, y, z, l, w, h, yaw]` in the global frame — note the swap from nuScenes `size = [w, l, h]`.

```
def _boxes_for_sample(db: NuScenesDB, sample: dict, ref_global2ego: np.ndarray, classes: list[str],
                      with_velocity: bool = True) -> dict[str, Any]:
    boxes, labels, names, lidar_pts, radar_pts, attrs, vel_valid, inst, tokens, vis = ([] for _ in range(10))
    for ann_token in sample["anns"]:
        ann = db.get("sample_annotation", ann_token)
        det_name = CATEGORY_TO_DETECTION.get(ann["category_name"])
        if det_name is None or det_name not in classes:
            continue
        box = np.zeros(9)
```

```
        box[:7] = _ann_box_global(ann)
        if with_velocity:
            vel = annotation_velocity(db, ann)
            valid = bool(np.all(np.isfinite(vel[:2])))
            box[7:9] = vel[:2] if valid else 0.0
        else:
            valid = False
        boxes.append(box)
        labels.append(classes.index(det_name))
        names.append(det_name)
        lidar_pts.append(ann["num_lidar_pts"])
        radar_pts.append(ann["num_radar_pts"])
        attrs.append(db.get("attribute", ann["attribute_tokens"][0])["name"] if ann["attribute_tokens"] else "")
        vel_valid.append(valid)
        inst.append(ann["instance_token"])
        tokens.append(ann["token"])
        vis.append(int(ann["visibility_token"]) if str(ann["visibility_token"]).isdigit() else 0)
    global_boxes = np.asarray(boxes, dtype=np.float64).reshape(-1, 9)
    return {
        "boxes": transform_boxes(global_boxes, ref_global2ego).astype(np.float32),
        "labels": np.asarray(labels, dtype=np.int64),
        "names": names,
        "num_lidar_pts": np.asarray(lidar_pts, dtype=np.int32),
        "num_radar_pts": np.asarray(radar_pts, dtype=np.int32),
        "attributes": attrs,
        "velocity_valid": np.asarray(vel_valid, dtype=bool),
        "instance_tokens": inst,
        "ann_tokens": tokens,
        "visibility": np.asarray(vis, dtype=np.int32),
    }
```

`_boxes_for_sample()` collects all annotations of a sample that map to one of the configured classes: global boxes + velocity (or 0 with `velocity_valid = False`), label index, name, LiDAR/radar point counts, attribute name, instance and annotation tokens, visibility; finally the boxes are transformed to the reference frame with `transform_boxes`.

```
def _lidar_pose(db: NuScenesDB, sample: dict, lidar_channel: str) -> tuple[dict, np.ndarray]:
    sd = db.get("sample_data", sample["data"][lidar_channel])
    pose = db.get("ego_pose", sd["ego_pose_token"])
    return sd, transform_matrix(pose["translation"], pose["rotation"])
```

`_lidar_pose(db, sample, channel)` returns the LIDAR_TOP key-frame sample_data and its ego pose — this pose *defines* the reference frame of the sample.

```
class CanBus:
    """Optional reader for the nuScenes CAN-bus expansion (``<root>/can_bus/<scene>_pose.json``)."""

    def __init__(self, root: Path) -> None:
        self.dir = Path(root) / "can_bus"
        self.available = self.dir.is_dir()
        self._cache: dict[str, Optional[list[dict]]] = {}

    def pose_messages(self, scene_name: str) -> Optional[list[dict]]:
        if not self.available:
            return None
        if scene_name not in self._cache:
            path = self.dir / f"{scene_name}_pose.json"
            self._cache[scene_name] = json.loads(path.read_text()) if path.exists() else None
        return self._cache[scene_name]

    def ego_state(self, scene_name: str, timestamp: int) -> Optional[dict[str, float]]:
        msgs = self.pose_messages(scene_name)
        if not msgs:
            return None
        past = [m for m in msgs if m["utime"] <= timestamp]
        if not past:
            return None
        m = past[-1]
        return {
            "speed": float(np.hypot(m["vel"][0], m["vel"][1])),
            "yaw_rate": float(m["rotation_rate"][2]),
            "accel": float(m["accel"][0]),
            "source": "can_bus",
        }
```

`CanBus`: optional reader for the CAN-bus expansion (`<root>/can_bus/<scene>_pose.json`). If the folder exists, `ego_state()` returns speed, yaw rate and longitudinal acceleration of the last message before the timestamp. The CAN-bus expansion is **not present** in the current dataset folder, so the pose-based estimate below is used.

```python
def _ego_state_from_poses(db: NuScenesDB, lidar_sd: dict, window: float = 0.2, accel_window: float = 0.5) -> dict[str, float]:
    """Causal (backward-difference) ego speed, yaw rate and acceleration from 20 Hz LiDAR poses."""

    def pose_at(sd: dict) -> tuple[np.ndarray, float, float]:
        pose = db.get("ego_pose", sd["ego_pose_token"])
        rot = quat_to_rotmat(pose["rotation"])
        return np.asarray(pose["translation"][:2]), yaw_from_rotmat(rot), sd["timestamp"] * 1e-6

    def velocity(sd: dict) -> Optional[tuple[float, float, float]]:
        p0, yaw0, t0 = pose_at(sd)
        for prev in db.iter_sweeps(sd["prev"]) if sd["prev"] else []:
            p1, yaw1, t1 = pose_at(prev)
            if t0 - t1 >= window:
                dt = t0 - t1
                v_global = (p0 - p1) / dt
                # Speed along the current heading (signed: negative when reversing).
                speed = float(v_global @ np.array([np.cos(yaw0), np.sin(yaw0)]))
                return speed, float(wrap_angle(yaw0 - yaw1) / dt), t0
        return None

    current = velocity(lidar_sd)
    if current is None:  # first sweeps of a log: fall back to a forward difference
        nxt = db.get("sample_data", lidar_sd["next"]) if lidar_sd["next"] else None
        if nxt is None:
            return {"speed": 0.0, "yaw_rate": 0.0, "accel": 0.0, "source": "none"}
        p0, yaw0, t0 = pose_at(lidar_sd)
        p1, yaw1, t1 = pose_at(nxt)
        dt = max(t1 - t0, 1e-3)
        speed = float(((p1 - p0) / dt) @ np.array([np.cos(yaw0), np.sin(yaw0)]))
        return {"speed": speed, "yaw_rate": float(wrap_angle(yaw1 - yaw0) / dt), "accel": 0.0, "source": "ego_pose_fwd"}
    speed, yaw_rate, t0 = current
    accel = 0.0
    for prev in db.iter_sweeps(lidar_sd["prev"]) if lidar_sd["prev"] else []:
        if t0 - prev["timestamp"] * 1e-6 >= accel_window:
            past = velocity(prev)
            if past is not None:
                accel = (speed - past[0]) / (t0 - past[2])
            break
    return {"speed": speed, "yaw_rate": yaw_rate, "accel": float(accel), "source": "ego_pose"}
```

`_ego_state_from_poses()` — causal ego-state estimation from the 20 Hz LiDAR ego poses: speed along the current heading and yaw rate from a backward difference over ≥ 0.2 s, acceleration from the speed change over ≥ 0.5 s; a forward difference is used only for the very first sweeps of a log. Being causal (no future poses) keeps the planner input honest.

```python
def driving_command(future_xy: np.ndarray, mask: np.ndarray, threshold: float) -> int:
    """0 = RIGHT, 1 = LEFT, 2 = STRAIGHT from the lateral offset of the last valid future point."""
    if not mask.any():
        return 2
    y = future_xy[np.nonzero(mask)[0][-1], 1]
    if y >= threshold:
        return 1
    if y <= -threshold:
        return 0
    return 2
```

`driving_command(future_xy, mask, threshold)`: the navigation command used by the planners — LEFT if the last valid future point is ≥ 2 m to the left, RIGHT if ≤ −2 m, else STRAIGHT (ST-P3/UniAD open-loop protocol; stands in for a route planner).

```python
def build_sample_info(db: NuScenesDB, sample_token: str, cfg: Config, can_bus: Optional[CanBus] = None) -> dict[str, Any]:
    """Build the info dictionary of one key-frame sample."""
    dcfg, pcfg = cfg.dataset, cfg.planning
    classes: list[str] = list(cfg.classes)
    sample = db.get("sample", sample_token)
    scene = db.get("scene", sample["scene_token"])
    log = db.get("log", scene["log_token"])
    map_rec = db.map_for_log(scene["log_token"])

    lidar_sd, ref_ego2global = _lidar_pose(db, sample, dcfg.lidar_channel)
    ref_global2ego = invert_transform(ref_ego2global)
    ref_ts = lidar_sd["timestamp"]

    info: dict[str, Any] = {
        "token": sample_token,
        "timestamp": sample["timestamp"],
        "scene_token": scene["token"],
        "scene_name": scene["name"],
        "scene_description": scene["description"],
```

```
        "location": log["location"],
        "log_token": log["token"],
        "map_filename": map_rec["filename"] if map_rec else None,
        "prev": sample["prev"],
        "next": sample["next"],
        "ref_ego2global": ref_ego2global,
    }

    # LiDAR key-frame + previous sweeps.
    sweeps = []
    for i, sd in enumerate(db.iter_sweeps(lidar_sd["token"])):
        if i >= int(dcfg.max_cached_lidar_sweeps):
            break
        sweeps.append(_sensor_record(db, sd, ref_global2ego, ref_ts))
    info["lidar"] = sweeps

    # Cameras (key-frame image of each camera).
    info["cams"] = {}
    for cam in dcfg.cameras:
        if cam in sample["data"]:
            info["cams"][cam] = _sensor_record(db, db.get("sample_data", sample["data"][cam]), ref_global2ego, r
    ↪               ef_ts)

    # Radars: key-frame sweep + previous sweeps per radar.
    info["radars"] = {}
    for radar in dcfg.radars:
        if radar not in sample["data"]:
            continue
        recs = []
        for i, sd in enumerate(db.iter_sweeps(sample["data"][radar])):
            if i >= int(dcfg.max_cached_radar_sweeps):
                break
            recs.append(_sensor_record(db, sd, ref_global2ego, ref_ts))
        info["radars"][radar] = recs
```

`build_sample_info()`, part 1: looks up sample, scene, log and map; computes the reference pose; records the LiDAR key-frame plus up to `max_cached_lidar_sweeps` previous sweeps, the six camera key-frames and, per radar, the key-frame plus `max_cached_radar_sweeps − 1` previous sweeps.

```
    # Ground truth in the reference frame.
    info["gt"] = _boxes_for_sample(db, sample, ref_global2ego, classes)
    # Bicycle racks: the official evaluation ignores bicycles / motorcycles parked inside them.
    racks = [_ann_box_global(a) for a in (db.get("sample_annotation", t) for t in sample["anns"])
             if a["category_name"] == "static_object.bicycle_rack"]
    racks_global = np.zeros((len(racks), 9))
    if racks:
        racks_global[:, :7] = np.asarray(racks)
    info["gt"]["bike_racks"] = transform_boxes(racks_global, ref_global2ego).astype(np.float32)

    # Ego state (CAN bus when available, otherwise causal finite differences of ego poses).
    state = can_bus.ego_state(scene["name"], ref_ts) if can_bus is not None else None
    info["ego_state"] = state or _ego_state_from_poses(db, lidar_sd)
```

Part 2: ground truth (`_boxes_for_sample`) and the bicycle racks (category `static_object.bicycle_rack`) needed for the official evaluation rule that ignores bicycles and motorcycles parked inside racks; then the ego state (CAN bus if available, else poses).

```
    # Ego future / past trajectory and future agent boxes (key-frames, 2 Hz).
    horizon = int(pcfg.horizon)
    fut_xy = np.zeros((horizon, 2), dtype=np.float32)
    fut_yaw = np.zeros(horizon, dtype=np.float32)
    fut_mask = np.zeros(horizon, dtype=bool)
    agent_future: list[dict[str, Any]] = []
    token = sample["next"]
    for k in range(horizon):
        if not token:
            agent_future.append({"boxes": np.zeros((0, 9), np.float32), "labels": np.zeros(0, np.int64), "instan
    ↪               ce_tokens": []})
            continue
        fsample = db.get("sample", token)
        _, f_ego2global = _lidar_pose(db, fsample, dcfg.lidar_channel)
        rel = ref_global2ego @ f_ego2global
        fut_xy[k] = rel[:2, 3]
        fut_yaw[k] = yaw_from_rotmat(rel[:3, :3])
        fut_mask[k] = True
        fboxes = _boxes_for_sample(db, fsample, ref_global2ego, classes, with_velocity=False)
        agent_future.append({k2: fboxes[k2] for k2 in ("boxes", "labels", "instance_tokens")})
        token = fsample["next"]
    info["ego_future"] = {"xy": fut_xy, "yaw": fut_yaw, "mask": fut_mask}
    info["agent_future"] = agent_future
    info["command"] = driving_command(fut_xy, fut_mask, float(pcfg.command_threshold))

    past = []
```

```
    token = sample["prev"]
    for _ in range(int(dcfg.history_frames)):
        if not token:
            break
        psample = db.get("sample", token)
        _, p_ego2global = _lidar_pose(db, psample, dcfg.lidar_channel)
        past.append((ref_global2ego @ p_ego2global)[:2, 3])
        token = psample["prev"]
    info["ego_past"] = np.asarray(past, dtype=np.float32).reshape(-1, 2)
```

Part 3: walks the next `planning.horizon` (6) key-frames: ego position and yaw in the current reference frame (the logged trajectory, `mask` False past the scene end) and all future agent boxes (also in the current reference frame) for the collision metric/loss; the command is derived from the future; then up to `history_frames` past ego positions.

```
    # Future trajectories of the current GT objects (for visualisation of object motion).
    n_obj = len(info["gt"]["instance_tokens"])
    inst_xy = np.zeros((n_obj, horizon, 2), dtype=np.float32)
    inst_mask = np.zeros((n_obj, horizon), dtype=bool)
    index = {tok: i for i, tok in enumerate(info["gt"]["instance_tokens"])}
    for k, fut in enumerate(agent_future):
        for j, tok in enumerate(fut["instance_tokens"]):
            if tok in index:
                inst_xy[index[tok], k] = fut["boxes"][j, :2]
                inst_mask[index[tok], k] = True
    info["gt"]["future_xy"] = inst_xy
    info["gt"]["future_mask"] = inst_mask
    return info
```

Part 4: for every current GT object, its future positions are looked up by instance token in the future agent lists → `future_xy (M, T, 2)` and `future_mask (M, T)` (drawn as 'logged object futures' in the dashboard).

```
def build_infos(db: NuScenesDB, scene_names: list[str], cfg: Config) -> list[dict[str, Any]]:
    can_bus = CanBus(db.root)
    infos = []
    for name in scene_names:
        scene = db.scene_by_name(name)
        tokens = db.scene_sample_tokens(scene)
        for idx, token in enumerate(tokens):
            info = build_sample_info(db, token, cfg, can_bus)
            info["frame_idx"] = idx
            info["num_frames"] = len(tokens)
            infos.append(info)
    return infos


def _cache_key(cfg: Config, split: str) -> str:
    relevant = {
        "fmt": INFO_FORMAT_VERSION,
        "version": cfg.dataset.version,
        "split": split,
        "classes": list(cfg.classes),
        "cams": list(cfg.dataset.cameras),
        "radars": list(cfg.dataset.radars),
        "lidar_sweeps": cfg.dataset.max_cached_lidar_sweeps,
        "radar_sweeps": cfg.dataset.max_cached_radar_sweeps,
        "history": cfg.dataset.history_frames,
        "horizon": cfg.planning.horizon,
        "cmd": cfg.planning.command_threshold,
    }
    return hashlib.md5(json.dumps(relevant, sort_keys=True).encode()).hexdigest()[:10]
```

`build_infos(db, scene_names, cfg)` loops over scenes and key-frames, adds `frame_idx` and `num_frames`. `_cache_key()` hashes every setting that affects the infos (format version, version, split, classes, sensors, sweep caps, horizon, command threshold) so a config change automatically produces a new cache file.

```
def load_infos(cfg: Config, split: str, db: Optional[NuScenesDB] = None, rebuild: bool = False) -> list[dict[str
    , Any]]:
    """Load cached infos for ``split`` or build (and cache) them from the tables."""
    cache_dir = resolve_path(cfg.dataset.cache_dir)
    cache_dir.mkdir(parents=True, exist_ok=True)
    path = cache_dir / f"infos_{cfg.dataset.version}_{split}_{_cache_key(cfg, split)}.pkl"
    if path.exists() and not rebuild:
        with open(path, "rb") as f:
            infos = pickle.load(f)
        logger.info("Loaded %d cached infos from %s", len(infos), path)
        return infos
    db = db or NuScenesDB(cfg.dataset.root, cfg.dataset.version)
    scenes = get_split_scenes(split, [s["name"] for s in db.scenes])
    logger.info("Building infos for split %s (%d scenes) ...", split, len(scenes))
    infos = build_infos(db, scenes, cfg)
    with open(path, "wb") as f:
```

```
405         pickle.dump(infos, f, protocol=pickle.HIGHEST_PROTOCOL)
406     logger.info("Cached %d infos to %s", len(infos), path)
407     return infos
```

`load_infos(cfg, split, db, rebuild)` returns the cached infos for a split (`outputs/cache/infos_<version>_<split>_<hash>.pkl`) or builds them (loading the tables only when needed) and writes the cache.

datasets/camera_loader.py

Camera image loading with augmentation and the matrices the LSS encoder needs. (171 lines)

```
1  """Camera loading, image-space augmentation and calibration matrices.
2
3  For every camera the loader returns the matrices that the Lift-Splat camera BEV
4  encoder needs to place image features in the reference BEV frame:
5
6  * ``intrinsic`` ``K`` (3x3) of the original 1600x900 image,
7  * ``post_rot`` / ``post_tran``: the affine image augmentation (resize, crop, flip,
8    rotate) so that ``p_aug = post_rot @ p_orig + post_tran`` in pixel space,
9  * ``cam2ref`` (4x4): camera -> ego(t_cam) -> global -> ego(t_ref), i.e. the calibrated
10   extrinsics *and* the ego-motion between the camera and LiDAR capture times.
11
12 A pixel ``(u, v)`` with depth ``d`` in the augmented image maps to the reference frame as
13 ``cam2ref · K⁻¹ · [d·u', d·v', d]`` with ``(u', v') = post_rot⁻¹ ((u, v) − post_tran)``.
14 """
```

Docstring: the meaning of `intrinsic`, `post_rot/post_tran` and `cam2ref`, and the lifting equation.

```
15 from __future__ import annotations
16
17 import logging
18 from dataclasses import dataclass
19 from pathlib import Path
20 from typing import Any, Optional
21
22 import numpy as np
23 from PIL import Image
24
25 from utils.geometry import invert_transform
26 from utils.logging_utils import WarnOnce
27
28 logger = logging.getLogger(__name__)
29 _warn = WarnOnce(logger)
30
31 IMAGENET_MEAN = np.array([0.485, 0.456, 0.406], dtype=np.float32)
32 IMAGENET_STD = np.array([0.229, 0.224, 0.225], dtype=np.float32)
```

Imports and the ImageNet normalisation constants.

```
35 @dataclass
36 class ImageAug:
37     resize: float
38     resize_dims: tuple[int, int]          # (W, H) after resizing
39     crop: tuple[int, int, int, int]       # (x0, y0, x1, y1) in the resized image
40     flip: bool
41     rotate: float                         # degrees, counter-clockwise
```

`ImageAug`: the sampled augmentation of one image — resize factor, resized size, crop box, horizontal flip, rotation (degrees).

```
44 def _rot2(theta: float) -> np.ndarray:
45     # Image-space rotation matching PIL.Image.rotate (counter-clockwise on screen, y down).
46     return np.array([[np.cos(theta), np.sin(theta)], [-np.sin(theta), np.cos(theta)]])
```

`_rot2(θ)`: 2×2 rotation matching `PIL.Image.rotate` (counter-clockwise on screen, image y axis pointing down).

```
49 def image_aug_matrices(aug: ImageAug) -> tuple[np.ndarray, np.ndarray]:
50     """3x3 ``post_rot`` and 3-vector ``post_tran`` of an :class:`ImageAug` (LSS convention)."""
51     rot = np.eye(2) * aug.resize
52     tran = -np.asarray(aug.crop[:2], dtype=np.float64)
53     width, height = aug.crop[2] - aug.crop[0], aug.crop[3] - aug.crop[1]
54     if aug.flip:
55         a = np.array([[-1.0, 0.0], [0.0, 1.0]])
56         rot, tran = a @ rot, a @ tran + np.array([width, 0.0])
57     a = _rot2(np.deg2rad(aug.rotate))
58     centre = np.array([width, height]) / 2.0
59     b = a @ -centre + centre
60     rot, tran = a @ rot, a @ tran + b
61     post_rot, post_tran = np.eye(3), np.zeros(3)
```

```
    post_rot[:2, :2], post_tran[:2] = rot, tran
    return post_rot, post_tran
```

`image_aug_matrices(aug)` computes `post_rot (3×3)` and `post_tran (3,)` so that `p_aug = post_rot·p + post_tran` for pixel coordinates (LSS convention): scale by the resize factor, subtract the crop offset, mirror around the crop width if flipped, rotate around the crop centre. Verified numerically (round trip error $6 \cdot 10^{-14}$ m).

```
def apply_image_aug(img: Image.Image, aug: ImageAug) -> Image.Image:
    img = img.resize(aug.resize_dims, Image.BILINEAR).crop(aug.crop)
    if aug.flip:
        img = img.transpose(Image.FLIP_LEFT_RIGHT)
    if aug.rotate:
        img = img.rotate(aug.rotate, resample=Image.BILINEAR)
    return img
```

`apply_image_aug(img, aug)` applies the same operations to the PIL image in the same order (resize → crop → flip → rotate).

```
class CameraLoader:
    """Load the surround-view images of one sample with consistent augmentation matrices."""

    def __init__(self, root: Path, cameras: list[str], image_size: tuple[int, int], camera_cfg: Any) -> None:
        self.root = Path(root)
        self.cameras = list(cameras)
        self.final_h, self.final_w = int(image_size[0]), int(image_size[1])
        self.cfg = camera_cfg
```

`CameraLoader.__init__` stores the dataset root, camera list, final network input size (H, W) and the `dataset.camera` config.

```
    def sample_aug(self, orig_w: int, orig_h: int, train: bool, rng: np.random.Generator) -> ImageAug:
        fh, fw = self.final_h, self.final_w
        base = fw / orig_w
        bot = self.cfg.bottom_crop_jitter
        if train:
            resize = base * rng.uniform(*self.cfg.resize_jitter)
            dims = (int(orig_w * resize), int(orig_h * resize))
            crop_h = int((1 - rng.uniform(*bot)) * dims[1]) - fh
            crop_w = int(rng.uniform(0, max(0, dims[0] - fw)))
            flip = bool(self.cfg.flip) and rng.random() < 0.5
            rotate = float(rng.uniform(-self.cfg.rotate_deg, self.cfg.rotate_deg))
        else:
            resize = base
            dims = (int(orig_w * resize), int(orig_h * resize))
            crop_h = int((1 - float(np.mean(bot))) * dims[1]) - fh
            crop_w = int(max(0, dims[0] - fw) / 2)
            flip, rotate = False, 0.0
        crop_h = max(crop_h, 0)
        return ImageAug(resize, dims, (crop_w, crop_h, crop_w + fw, crop_h + fh), flip, rotate)
```

`sample_aug(W, H, train, rng)`: base resize = target width / original width. In training a random resize jitter, a random bottom-biased crop (keeps the road, drops sky), a random flip and a random rotation (±5.4°) are drawn; at test time the resize is the base, the crop is centred horizontally and placed at the mean bottom ratio, no flip/rotation.

```
    def load(self, info: dict[str, Any], train: bool, rng: np.random.Generator) -> dict[str, np.ndarray]:
        n = len(self.cameras)
        imgs = np.zeros((n, 3, self.final_h, self.final_w), dtype=np.float32)
        intrinsics = np.tile(np.eye(3), (n, 1, 1))
        post_rots = np.tile(np.eye(3), (n, 1, 1))
        post_trans = np.zeros((n, 3))
        cam2ref = np.tile(np.eye(4), (n, 1, 1))
        valid = np.zeros(n, dtype=bool)
        for i, cam in enumerate(self.cameras):
            rec = info["cams"].get(cam)
            if rec is None:
                _warn(f"{info['token']}:{cam}", f"Camera {cam} missing in sample {info['token']}")
                continue
            intrinsics[i] = rec["intrinsic"]
            cam2ref[i] = rec["sensor2ref"]
            path = self.root / rec["path"]
            try:
                with Image.open(path) as im:
                    img = im.convert("RGB")
            except OSError as exc:
                _warn(str(path), f"Unreadable image {path}: {exc} (camera disabled for this sample)")
                aug = self.sample_aug(rec["width"], rec["height"], False, rng)
                post_rots[i], post_trans[i] = image_aug_matrices(aug)
                continue
```

```
            aug = self.sample_aug(img.width, img.height, train, rng)
            img = apply_image_aug(img, aug)
            post_rots[i], post_trans[i] = image_aug_matrices(aug)
            arr = (np.asarray(img, dtype=np.float32) / 255.0 - IMAGENET_MEAN) / IMAGENET_STD
            imgs[i] = arr.transpose(2, 0, 1)
            valid[i] = True
        return {
            "images": imgs,
            "intrinsics": intrinsics.astype(np.float32),
            "post_rots": post_rots.astype(np.float32),
            "post_trans": post_trans.astype(np.float32),
            "cam2ref": cam2ref.astype(np.float32),
            "cam_valid": valid,
        }
```

`load(info, train, rng)` → arrays for the six cameras: normalised images `(6, 3, H, W)`, intrinsics, post_rots, post_trans, cam2ref and `cam_valid`. A missing camera record or an unreadable file is logged once (`WarnOnce`) and leaves a zero image with `cam_valid = False`, which the camera encoder uses to ignore that camera.

```
def lidar_depth_maps(points_ref: np.ndarray, cams: dict[str, np.ndarray], image_size: tuple[int, int],
                     stride: int, depth_range: tuple[float, float]) -> np.ndarray:
    """Sparse LiDAR depth per camera at feature resolution ``(N, H/stride, W/stride)``.

    Each feature cell keeps the *closest* projected LiDAR point (0 = no supervision),
    exactly the BEVDepth depth-supervision target.
    """
    h, w = image_size[0] // stride, image_size[1] // stride
    n = cams["images"].shape[0]
    depth = np.zeros((n, h, w), dtype=np.float32)
    if points_ref.shape[0] == 0:
        return depth
    xyz1 = np.concatenate([points_ref[:, :3], np.ones((points_ref.shape[0], 1), np.float32)], 1).astype(np.float
    ↪       64)
    for i in range(n):
        if not cams["cam_valid"][i]:
            continue
        pc = (xyz1 @ invert_transform(cams["cam2ref"][i].astype(np.float64)).T)[:, :3]
        d = pc[:, 2]
        keep = (d > depth_range[0]) & (d < depth_range[1])
        pc, d = pc[keep], d[keep]
        uv = (pc @ cams["intrinsics"][i].T)[:, :2] / d[:, None]
        uv = uv @ cams["post_rots"][i][:2, :2].T + cams["post_trans"][i][:2]
        col, row = np.floor(uv[:, 0] / stride).astype(np.int64), np.floor(uv[:, 1] / stride).astype(np.int64)
        ok = (col >= 0) & (col < w) & (row >= 0) & (row < h)
        col, row, d = col[ok], row[ok], d[ok]
        order = np.argsort(-d)  # far first so the nearest point is written last
        depth[i, row[order], col[order]] = d[order]
    return depth
```

`lidar_depth_maps(points_ref, cams, image_size, stride, depth_range)` → `(6, h, w)` depth targets: each LiDAR point is transformed into the camera (`inv(cam2ref)`), projected with K, moved into the augmented image with post_rot/post_tran, binned into feature cells (÷16), and the **closest** depth per cell is kept (points are written far-to-near). 0 means no supervision.

datasets/lidar_loader.py

Multi-sweep LiDAR accumulation in the reference frame. (90 lines)

```
"""LiDAR loading: multi-sweep accumulation in the reference frame.

Pipeline: ``.pcd.bin`` (sensor frame) -> remove ego returns -> sensor->ref transform
(motion-compensated through the global frame, per sweep) -> time-lag channel -> range filter.

Output points: ``(N, 5)`` float32 ``[x, y, z, intensity, time_lag]`` in the reference frame.
"""
from __future__ import annotations

import logging
from pathlib import Path
from typing import Any, Optional

import numpy as np

from utils.bev_grid import BEVGrid
from utils.geometry import apply_transform
from utils.logging_utils import WarnOnce

logger = logging.getLogger(__name__)
_warn = WarnOnce(logger)
```

```
22
23 LIDAR_POINT_FEATURES = ("x", "y", "z", "intensity", "time_lag")
```

Docstring with the processing chain; `LIDAR_POINT_FEATURES` = x, y, z, intensity, time_lag.

```
26 def read_lidar_bin(path: Path) -> np.ndarray:
27     """Read a nuScenes LiDAR file -> ``(N, 5)`` ``[x, y, z, intensity, ring]`` in the sensor frame."""
28     data = np.fromfile(path, dtype=np.float32)
29     if data.size % 5 != 0:
30         raise ValueError(f"Corrupt LiDAR file (size {data.size} not divisible by 5): {path}")
31     return data.reshape(-1, 5)
```

`read_lidar_bin(path)` reads a nuScenes `.pcd.bin` (float32 × 5: x, y, z, intensity, ring index) and validates the size.

```
34 class LidarLoader:
35     """Load and accumulate LiDAR sweeps for one sample.
36
37     Missing or corrupt sweep files are skipped with a warning; if every sweep fails the
38     loader returns an empty ``(0, 5)`` array so the model can still run on other sensors.
39     """
40
41     def __init__(self, root: Path, grid: BEVGrid, num_sweeps: int = 10, max_points: Optional[int] = None,
42                  remove_close_radius: float = 1.5, range_margin: float = 0.0) -> None:
43         self.root = Path(root)
44         self.grid = grid
45         self.num_sweeps = num_sweeps
46         self.max_points = max_points
47         self.remove_close_radius = remove_close_radius
48         self.range_margin = range_margin
```

Constructor: root, grid, number of sweeps (`dataset.lidar.sweeps`), optional `max_points`, `remove_close_radius` (1.5 m: drops returns from the ego vehicle itself).

```
50     def load_sweep(self, rec: dict[str, Any]) -> Optional[np.ndarray]:
51         path = self.root / rec["path"]
52         try:
53             pts = read_lidar_bin(path)
54         except (OSError, ValueError) as exc:
55             _warn(str(path), f"Skipping LiDAR sweep {path}: {exc}")
56             return None
57         if self.remove_close_radius > 0:
58             far = np.hypot(pts[:, 0], pts[:, 1]) > self.remove_close_radius
59             pts = pts[far]
60         out = np.empty((pts.shape[0], 5), dtype=np.float32)
61         out[:, :3] = apply_transform(rec["sensor2ref"], pts[:, :3].astype(np.float64))[:, :3]
62         out[:, 3] = pts[:, 3]
63         out[:, 4] = rec["time_lag"]
64         return out
```

`load_sweep(rec)`: read, remove close points (sensor frame), transform xyz with `sensor2ref` (motion-compensated through the sweep's own ego pose), keep intensity, set the time-lag column. Errors are logged once and return `None`.

```
66     def load(self, info: dict[str, Any], num_sweeps: Optional[int] = None, rng: Optional[np.random.Generator] = N
   ↪       one,
67              filter_range: bool = True) -> np.ndarray:
68         n = num_sweeps or self.num_sweeps
69         chunks = [p for p in (self.load_sweep(rec) for rec in info["lidar"][:n]) if p is not None]
70         if not chunks:
71             _warn(info["token"], f"No LiDAR data for sample {info['token']}")
72             return np.zeros((0, 5), dtype=np.float32)
73         points = np.concatenate(chunks, axis=0)
74         if filter_range:
75             g, m = self.grid, self.range_margin
76             keep = (
77                 (points[:, 0] >= g.x_range[0] - m) & (points[:, 0] < g.x_range[1] + m)
78                 & (points[:, 1] >= g.y_range[0] - m) & (points[:, 1] < g.y_range[1] + m)
79                 & (points[:, 2] >= g.z_range[0]) & (points[:, 2] < g.z_range[1])
80             )
81             points = points[keep]
82         if self.max_points and points.shape[0] > self.max_points:
83             rng = rng or np.random.default_rng()
84             points = points[rng.choice(points.shape[0], self.max_points, replace=False)]
85         return points
```

`load(info, num_sweeps, rng, filter_range)`: concatenates the first *n* sweeps (key-frame first), filters to the BEV x/y range and z range, and randomly subsamples above `max_points` (seeded rng → deterministic at evaluation). Returns `(P, 5)` or an empty `(0, 5)` array.

```
    def load_keyframe(self, info: dict[str, Any]) -> np.ndarray:
        """Only the key-frame sweep (used for depth supervision and calibration checks)."""
        pts = self.load_sweep(info["lidar"][0])
        return pts if pts is not None else np.zeros((0, 5), dtype=np.float32)
```

`load_keyframe(info)`: only the key-frame sweep (depth supervision, calibration checks, planner obstacle layer).

datasets/radar_loader.py

Radar loading: own binary-PCD parser, the devkit's default filters, multi-sweep accumulation, and rotation of the velocity vectors into the reference frame. (109 lines)

```
"""Radar loading: binary PCD parsing, filtering, multi-sweep accumulation.

nuScenes radars (Continental ARS 408) report ~100 detections per sweep with
ego-motion-compensated Doppler velocity (``vx_comp``, ``vy_comp``) in the *sensor*
frame. The official devkit's multi-sweep loader transforms positions but not the
velocity vectors; here velocities are rotated into the reference frame as well,
because velocity is the radar's most valuable cue for dynamic objects.

Output points: ``(N, 10)`` float32 with columns :data:`RADAR_POINT_FEATURES`.
"""
from __future__ import annotations

import logging
from pathlib import Path
from typing import Any, Iterable, Optional

import numpy as np

from utils.bev_grid import BEVGrid
from utils.logging_utils import WarnOnce

logger = logging.getLogger(__name__)
_warn = WarnOnce(logger)

RADAR_POINT_FEATURES = ("x", "y", "z", "rcs", "vx_comp", "vy_comp", "vx", "vy", "dyn_prop", "time_lag")
_PCD_TYPES = {("F", 4): "<f4", ("F", 8): "<f8", ("I", 1): "i1", ("I", 2): "<i2", ("I", 4): "<i4",
              ("U", 1): "u1", ("U", 2): "<u2", ("U", 4): "<u4"}
```

Docstring: why velocities must be rotated (the devkit's multisweep loader leaves them in the sensor frame). `RADAR_POINT_FEATURES` defines the 10 output columns; `_PCD_TYPES` maps PCD (TYPE, SIZE) pairs to NumPy dtypes.

```
def read_radar_pcd(path: Path) -> np.ndarray:
    """Parse a nuScenes radar ``.pcd`` (``DATA binary``) into a NumPy structured array."""
    raw = Path(path).read_bytes()
    marker = raw.find(b"DATA binary")
    if marker < 0:
        raise ValueError(f"Unsupported PCD (expected 'DATA binary'): {path}")
    header: dict[str, list[str]] = {}
    for line in raw[:marker].decode("ascii", errors="replace").splitlines():
        if line and not line.startswith("#"):
            key, *values = line.split()
            header[key] = values
    fields, sizes, types = header["FIELDS"], header["SIZE"], header["TYPE"]
    dtype = np.dtype([(f, _PCD_TYPES[(t, int(s))]) for f, s, t in zip(fields, sizes, types)])  # packed
    n_points = int(header["POINTS"][0])
    start = raw.index(b"\n", marker) + 1
    if len(raw) - start < n_points * dtype.itemsize:
        raise ValueError(f"Truncated radar file {path}")
    return np.frombuffer(raw, dtype=dtype, count=n_points, offset=start)
```

`read_radar_pcd(path)`: parses the ASCII header up to `DATA binary` (FIELDS, SIZE, TYPE, POINTS), builds a *packed* structured dtype (43 bytes per point for nuScenes radar) and reads exactly `POINTS` records with `np.frombuffer` (the files contain trailing padding). Raises `ValueError` for non-binary or truncated files.

```
class RadarLoader:
    """Load all radars of a sample (key-frame + previous sweeps) in the reference frame."""

    def __init__(self, root: Path, grid: BEVGrid, num_sweeps: int = 6,
                 invalid_states: Optional[Iterable[int]] = (0,),
                 dynprop_states: Optional[Iterable[int]] = range(7),
                 ambig_states: Optional[Iterable[int]] = (3,)) -> None:
        self.root = Path(root)
        self.grid = grid
        self.num_sweeps = num_sweeps
        self.invalid_states = None if invalid_states is None else np.asarray(list(invalid_states))
        self.dynprop_states = None if dynprop_states is None else np.asarray(list(dynprop_states))
```

```
62         self.ambig_states = None if ambig_states is None else np.asarray(list(ambig_states))
```

Constructor: the three filter lists (`None` disables a filter). Defaults from `configs/dataset.yaml` = devkit defaults: `invalid_state ∈ {0}`, `dyn_prop ∈ {0..6}`, `ambig_state ∈ {3}`.

```
64     def _filter(self, pts: np.ndarray) -> np.ndarray:
65         keep = np.ones(len(pts), dtype=bool)
66         if self.invalid_states is not None:
67             keep &= np.isin(pts["invalid_state"], self.invalid_states)
68         if self.dynprop_states is not None:
69             keep &= np.isin(pts["dyn_prop"], self.dynprop_states)
70         if self.ambig_states is not None:
71             keep &= np.isin(pts["ambig_state"], self.ambig_states)
72         return pts[keep]
```

`_filter(pts)` keeps points whose states are in the allowed sets (`np.isin`).

```
74     def load_sweep(self, rec: dict[str, Any]) -> Optional[np.ndarray]:
75         path = self.root / rec["path"]
76         try:
77             pts = self._filter(read_radar_pcd(path))
78         except (OSError, ValueError, KeyError) as exc:
79             _warn(str(path), f"Skipping radar sweep {path}: {exc}")
80             return None
81         n = len(pts)
82         out = np.zeros((n, len(RADAR_POINT_FEATURES)), dtype=np.float32)
83         if n == 0:
84             return out
85         mat = rec["sensor2ref"]
86         xyz = np.stack([pts["x"], pts["y"], pts["z"]], -1).astype(np.float64)
87         out[:, 0:3] = xyz @ mat[:3, :3].T + mat[:3, 3]
88         out[:, 3] = pts["rcs"]
89         rot = mat[:2, :2]  # velocities are vectors: rotate only
90         out[:, 4:6] = np.stack([pts["vx_comp"], pts["vy_comp"]], -1) @ rot.T
91         out[:, 6:8] = np.stack([pts["vx"], pts["vy"]], -1) @ rot.T
92         out[:, 8] = pts["dyn_prop"]
93         out[:, 9] = rec["time_lag"]
94         return out
```

`load_sweep(rec)` → `(N, 10)`: positions transformed with `sensor2ref`; RCS copied; `(vx_comp, vy_comp)` (ego-motion compensated) and raw `(vx, vy)` rotated with the 2×2 rotation part of `sensor2ref`; dynamic property; time-lag.

```
96     def load(self, info: dict[str, Any], num_sweeps: Optional[int] = None, filter_range: bool = True) -> np.ndar
       ↪ ray:
97         n = num_sweeps or self.num_sweeps
98         chunks = []
99         for recs in info["radars"].values():
100            for rec in recs[:n]:
101                pts = self.load_sweep(rec)
102                if pts is not None and len(pts):
103                    chunks.append(pts)
104        if not chunks:
105            return np.zeros((0, len(RADAR_POINT_FEATURES)), dtype=np.float32)
106        points = np.concatenate(chunks, 0)
107        if filter_range:
108            points = points[self.grid.in_range(points[:, 0], points[:, 1])]
109        return points
```

`load(info, num_sweeps)`: loops over the 5 radars and their first *n* sweeps, concatenates, and filters to the BEV range.

datasets/nuscenes_dataset.py

The PyTorch `Dataset` used for training, evaluation and inference, plus the batch collation. (231 lines)

```
1  """PyTorch dataset for multi-sensor fusion, detection and planning on nuScenes.
2
3  One item = one key-frame, with every quantity in the reference frame *after* the BEV
4  augmentation matrix ``bda`` (which is also handed to the camera encoder so the lifted
5  camera features land in the same augmented frame):
6
7  ====================  ========================  ======================================
8  key                   shape                     meaning
9  ====================  ========================  ======================================
10 images                (N_cam, 3, H, W)          normalised images
11 intrinsics            (N_cam, 3, 3)             K of the original image
12 post_rots/post_trans  (N_cam, 3, 3) / (N_cam, 3)  image augmentation
13 cam2ref               (N_cam, 4, 4)             camera -> reference ego frame
14 cam_valid             (N_cam,)                  image successfully loaded
15 depth_gt              (N_cam, H/16, W/16)       LiDAR depth for depth supervision
```

```
bda                   (4, 4)                    BEV augmentation matrix
lidar_points          (P, 5)                    x y z intensity time_lag
radar_points          (Q, 10)                   see RADAR_POINT_FEATURES
heatmap ... reg_target CenterPoint targets       see models/detection/targets.py
fg_heatmap            (1, ny, nx)               class-agnostic, BEV resolution
gt_boxes / gt_labels  (M, 9) / (M,)             filtered GT (for evaluation)
ego_future(_mask)     (T, 2) / (T,)             logged ego trajectory
future_occ            (T, ny, nx) uint8         GT agent occupancy at future steps
ego_state             (3,)                      speed, yaw-rate, acceleration
command               ()                        0 right / 1 left / 2 straight
====================  ========================  ======================================
"""
```

Docstring: table of every key in a dataset item with its shape and meaning (reproduced in the *Tensor Dimension Reference* chapter).

```python
from __future__ import annotations

import logging
from pathlib import Path
from typing import Any, Optional

import numpy as np
import torch
from torch.utils.data import Dataset

from datasets.camera_loader import CameraLoader, lidar_depth_maps
from datasets.info_builder import driving_command, load_infos
from datasets.lidar_loader import LidarLoader
from datasets.radar_loader import RadarLoader
from models.detection.targets import build_center_targets, foreground_heatmap
from utils.bev_grid import BEVGrid
from utils.config import Config
from utils.geometry import augment_boxes_bev, bev_rotation_matrix
from utils.raster import rasterize_boxes

logger = logging.getLogger(__name__)

POINT_KEYS = ("lidar_points", "radar_points")
LIST_KEYS = ("gt_boxes", "gt_labels", "meta", "raw_lidar_points", "raw_radar_points")
```

Imports and the key groups used by `collate_batch` (`POINT_KEYS` are concatenated with a batch-index column, `LIST_KEYS` stay Python lists).

```python
def enabled_modalities(cfg: Config) -> dict[str, bool]:
    f = cfg.fusion
    return {"camera": bool(f.use_camera_bev), "lidar": bool(f.use_lidar), "radar": bool(f.use_radar)}
```

`enabled_modalities(cfg)` → `{camera, lidar, radar}: bool` from `fusion.use_*`.

```python
class NuScenesFusionDataset(Dataset):
    """Multi-modal nuScenes key-frame dataset.

    Args:
        cfg: full configuration.
        split: ``mini_train`` / ``mini_val`` / ``train`` / ``val``.
        train: enables augmentation.
        infos: pre-loaded infos (otherwise loaded from the cache / built).
        load_all_sensors: always return raw LiDAR and radar points (dashboard visualisation),
            even for modalities the model does not use.
        with_targets: build detection / planning targets.
    """

    def __init__(self, cfg: Config, split: str, train: bool = False, infos: Optional[list[dict]] = None,
                 load_all_sensors: bool = False, with_targets: bool = True) -> None:
        self.cfg = cfg
        self.split = split
        self.train = train
        self.infos = infos if infos is not None else load_infos(cfg, split)
        self.load_all_sensors = load_all_sensors
        self.with_targets = with_targets
        self.modalities = enabled_modalities(cfg)
        self.classes = list(cfg.classes)
        self.grid = BEVGrid.from_config(cfg.bev)
        self.head_grid = self.grid.downsample(int(cfg.model.detection_head.out_stride))
        dcfg = cfg.dataset
        root = Path(dcfg.root)
        self.image_size = tuple(int(v) for v in dcfg.camera.image_size)
        self.feature_stride = int(cfg.model.camera_bev.feature_stride)
        self.camera_loader = CameraLoader(root, dcfg.cameras, self.image_size, dcfg.camera)
        self.lidar_loader = LidarLoader(root, self.grid, int(dcfg.lidar.sweeps), dcfg.lidar.max_points,
                                        float(dcfg.lidar.remove_close_radius))
        f = dcfg.radar.filters
        self.radar_loader = RadarLoader(root, self.grid, int(dcfg.radar.sweeps), f.invalid_states, f.dynprop_sta
```

```
            tes,
                                    f.ambig_states)
        self.collision_labels = np.array([self.classes.index(c) for c in cfg.planning.collision_classes
                                          if c in self.classes])
        max_samples = cfg.train.get("max_train_samples") if train else cfg.train.get("max_val_samples")
        if max_samples:
            self.infos = self.infos[: int(max_samples)]

    def __len__(self) -> int:
        return len(self.infos)

    # ------------------------------------------------------------------ helpers
```

Constructor: loads (or receives) the infos, stores modality flags, classes, the BEV grid and the head grid (`downsample(out_stride)`), creates the three loaders from the config, precomputes the label indices of the collision classes and optionally truncates the dataset (`train.max_train_samples` / `max_val_samples`) for quick runs.

```
    def _sample_bda(self, rng: np.random.Generator) -> np.ndarray:
        aug = self.cfg.dataset.augmentation
        if not (self.train and aug.enabled):
            return np.eye(4)
        rot = np.deg2rad(rng.uniform(*aug.rot_range_deg))
        scale = rng.uniform(*aug.scale_range)
        return bev_rotation_matrix(rot, scale, flip_x=rng.random() < aug.flip_x_prob, flip_y=rng.random() < aug.
            flip_y_prob)
```

`_sample_bda(rng)`: identity unless training with augmentation enabled; otherwise a random rotation, scale and flips → `bev_rotation_matrix`.

```
    @staticmethod
    def _apply_bda_points(points: np.ndarray, bda: np.ndarray, vel_cols: tuple[tuple[int, int], ...] = ()) -> np
        .ndarray:
        if points.shape[0] == 0 or np.allclose(bda, np.eye(4)):
            return points
        out = points.copy()
        out[:, :3] = points[:, :3] @ bda[:3, :3].T + bda[:3, 3]
        for a, b in vel_cols:  # velocity vectors rotate/reflect/scale with the world
            out[:, [a, b]] = points[:, [a, b]] @ bda[:2, :2].T
        return out
```

`_apply_bda_points(points, bda, vel_cols)`: applies the augmentation to xyz and to the listed velocity column pairs (radar `vx_comp, vy_comp` and `vx, vy`).

```
    def _gt_mask(self, gt: dict[str, Any]) -> np.ndarray:
        boxes = gt["boxes"]
        if boxes.shape[0] == 0:
            return np.zeros(0, dtype=bool)
        pts = gt["num_lidar_pts"] + gt["num_radar_pts"]
        return (pts >= int(self.cfg.dataset.gt_filter.min_points)) & self.grid.in_range(boxes[:, 0], boxes[:, 1]
            )

    # ------------------------------------------------------------------ item
```

`_gt_mask(gt)`: keeps GT boxes with ≥ `gt_filter.min_points` LiDAR+radar returns (nuScenes evaluation rule) and centres inside the grid.

```
    def __getitem__(self, idx: int) -> dict[str, Any]:
        info = self.infos[idx]
        rng = np.random.default_rng(np.random.randint(0, 2**31 - 1) if self.train else idx)
        bda = self._sample_bda(rng)
        mods = self.modalities
        item: dict[str, Any] = {"bda": bda.astype(np.float32)}

        need_depth = mods["camera"] and self.cfg.dataset.camera.depth_gt and self.with_targets
        lidar = None
        if mods["lidar"] or self.load_all_sensors:
            lidar = self.lidar_loader.load(info, rng=rng)
        if mods["camera"]:
            cams = self.camera_loader.load(info, self.train, rng)
            item.update(cams)
            if need_depth:
                key_pts = self.lidar_loader.load_keyframe(info)
                dbound = self.cfg.model.camera_bev.depth_bins
                item["depth_gt"] = lidar_depth_maps(key_pts, cams, self.image_size, self.feature_stride,
                                                    (float(dbound[0]), float(dbound[1])))
        if lidar is not None:
            lidar = self._apply_bda_points(lidar, bda)
            if self.load_all_sensors:
                item["raw_lidar_points"] = lidar
            if mods["lidar"]:
```

```
                item["lidar_points"] = lidar
        if mods["radar"] or self.load_all_sensors:
            radar = self._apply_bda_points(self.radar_loader.load(info), bda, vel_cols=((4, 5), (6, 7)))
            if self.load_all_sensors:
                item["raw_radar_points"] = radar
            if mods["radar"]:
                item["radar_points"] = radar
```

`__getitem__(idx)`, part 1 — sensors: a per-item RNG (random in training, seeded by the index otherwise), the augmentation matrix, LiDAR (if used or needed for display), cameras with depth targets from the key-frame LiDAR, radar; augmentation is applied to point clouds. `load_all_sensors` (dashboard) also returns the raw clouds for drawing.

```
        # Simulated sensor failure (training robustness).
        if self.train:
            drop = self.cfg.dataset.augmentation.modality_dropout
            if mods["lidar"] and rng.random() < float(drop.lidar):
                item["lidar_points"] = item["lidar_points"][:0]
            if mods["radar"] and rng.random() < float(drop.radar):
                item["radar_points"] = item["radar_points"][:0]
            if mods["camera"] and rng.random() < float(drop.camera):
                item["cam_valid"] = np.zeros_like(item["cam_valid"])
```

Part 2 — optional modality dropout (training only): empties a point cloud or invalidates all cameras with the configured probabilities (0 by default).

```
        gt = info["gt"]
        keep = self._gt_mask(gt)
        boxes = augment_boxes_bev(gt["boxes"][keep].astype(np.float64), bda).astype(np.float32)
        labels = gt["labels"][keep]
        item["gt_boxes"], item["gt_labels"] = boxes, labels
        mirrored = np.linalg.det(bda[:2, :2]) < 0

        # Planning inputs / targets.
        fut = info["ego_future"]
        fut_xy = (fut["xy"].astype(np.float64) @ bda[:2, :2].T + bda[:2, 3]).astype(np.float32)
        item["ego_future"] = fut_xy
        item["ego_future_mask"] = fut["mask"].astype(np.float32)
        item["command"] = np.int64(driving_command(fut_xy, fut["mask"], float(self.cfg.planning.command_threshol
            d)))
        es = info["ego_state"]
        item["ego_state"] = np.array([es["speed"], -es["yaw_rate"] if mirrored else es["yaw_rate"], es["accel"]]
            ,
                                     dtype=np.float32)

        if self.with_targets:
            hcfg = self.cfg.model.detection_head
            targets = build_center_targets(boxes, labels, gt["velocity_valid"][keep], self.head_grid, len(self.c
                lasses),
                                           int(hcfg.max_objects), int(hcfg.min_radius), float(hcfg.gaussian_over
                                               lap))
            item.update(targets)
            item["fg_heatmap"] = foreground_heatmap(boxes, self.grid, int(hcfg.min_radius), float(hcfg.gaussian_
                overlap))
            occ = np.zeros((len(info["agent_future"]),) + self.grid.shape, dtype=np.uint8)
            for t, agents in enumerate(info["agent_future"]):
                sel = np.isin(agents["labels"], self.collision_labels)
                if sel.any():
                    rasterize_boxes(augment_boxes_bev(agents["boxes"][sel].astype(np.float64), bda), self.grid,
                        out=occ[t])
            item["future_occ"] = occ

        item["meta"] = {
            "index": idx,
            "token": info["token"],
            "scene_name": info["scene_name"],
            "frame_idx": info.get("frame_idx", 0),
            "timestamp": info["timestamp"],
            "mirrored": bool(mirrored),
        }
        return item
```

Part 3 — targets: GT boxes filtered and augmented; ego future transformed by `bda`, command recomputed from the augmented future, ego state (yaw rate negated when mirrored); CenterPoint targets on the head grid, the class-agnostic foreground heatmap on the BEV grid, and the future occupancy `(T, ny, nx)` of collision-class agents; `meta` carries token, scene, frame index.

```
def collate_batch(samples: list[dict[str, Any]]) -> dict[str, Any]:
    """Stack fixed-size arrays, concatenate point clouds with a leading batch-index column."""
    batch: dict[str, Any] = {}
    keys = set().union(*(s.keys() for s in samples))
```

```
    for key in keys:
        values = [s.get(key) for s in samples]
        if key in POINT_KEYS:
            width = max((v.shape[1] for v in values if v is not None), default=1)
            chunks = []
            for b, pts in enumerate(values):
                pts = pts if pts is not None else np.zeros((0, width), np.float32)
                chunks.append(np.concatenate([np.full((pts.shape[0], 1), b, np.float32), pts], 1))
            batch[key] = torch.from_numpy(np.concatenate(chunks, 0))
        elif key in LIST_KEYS:
            batch[key] = [torch.from_numpy(v) if isinstance(v, np.ndarray) else v for v in values]
        else:
            batch[key] = torch.from_numpy(np.stack(values, 0))
    batch["batch_size"] = len(samples)
    return batch
```

`collate_batch(samples)`: point clouds are concatenated with a leading batch-index column (`(Σ P_b, 1 + F)`), `LIST_KEYS` stay lists (variable-length GT), all other arrays are stacked with a new batch dimension; `batch_size` is added.

Neural-network modules

models/common/blocks.py

Small reusable building blocks used by every BEV network in the project. Nothing here is sensor-specific; the blocks operate on 4-D feature maps `(B, C, H, W)`. (71 lines)

```
"""Reusable convolutional / attention building blocks."""
from __future__ import annotations

import torch
from torch import nn
```

Module docstring and imports. `torch.nn` provides the layer classes; `from __future__ import annotations` lets the type hints use modern syntax on Python 3.10/3.11.

```
class ConvBNReLU(nn.Sequential):
    def __init__(self, in_ch: int, out_ch: int, kernel: int = 3, stride: int = 1, dilation: int = 1, relu: bool =
        True) -> None:
        layers: list[nn.Module] = [
            nn.Conv2d(in_ch, out_ch, kernel, stride, padding=dilation * (kernel // 2), dilation=dilation, bias=Fa
                lse),
            nn.BatchNorm2d(out_ch),
        ]
        if relu:
            layers.append(nn.ReLU(inplace=True))
        super().__init__(*layers)
```

`ConvBNReLU` is a `nn.Sequential` of Conv2d → BatchNorm2d → (optional) ReLU.

- `padding=dilation * (kernel // 2)` keeps the spatial size unchanged for odd kernels at stride 1 (also for dilated kernels), so `(B, in_ch, H, W) → (B, out_ch, H/stride, W/stride)`.
- `bias=False` because the following BatchNorm has its own learnable shift; a conv bias would be redundant.
- `relu=False` is used as the last layer of a residual branch (the ReLU is applied after the addition).

```
class ResBlock(nn.Module):
    """Basic residual block (two 3x3 convs) with optional dilation."""

    def __init__(self, channels: int, dilation: int = 1) -> None:
        super().__init__()
        self.conv1 = ConvBNReLU(channels, channels, 3, dilation=dilation)
        self.conv2 = ConvBNReLU(channels, channels, 3, dilation=dilation, relu=False)
        self.act = nn.ReLU(inplace=True)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.act(x + self.conv2(self.conv1(x)))
```

Basic residual block: two 3×3 `ConvBNReLU` (the second without ReLU), then `ReLU(x + branch(x))`. Input and output shapes are identical `(B, C, H, W)`. The `dilation` argument enlarges the receptive field without downsampling — used by the radar encoder (d = 1, 2, 4) and the depth network (d = 2).

```
class SqueezeExcitation(nn.Module):
    """Channel attention (SE): global pooling -> bottleneck MLP -> sigmoid channel weights."""

```

```
    def __init__(self, channels: int, reduction: int = 4) -> None:
        super().__init__()
        hidden = max(channels // reduction, 8)
        self.mlp = nn.Sequential(nn.Linear(2 * channels, hidden), nn.ReLU(inplace=True), nn.Linear(hidden, channe
            ls))

    def weights(self, x: torch.Tensor) -> torch.Tensor:
        stats = torch.cat([x.mean((2, 3)), x.amax((2, 3))], 1)
        return torch.sigmoid(self.mlp(stats))

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return x * self.weights(x)[:, :, None, None]
```

Channel attention (Squeeze-and-Excitation). `weights()` pools every channel globally with both mean and max (`x.mean((2, 3))`, `x.amax((2, 3))` → `(B, 2C)`), passes the result through a bottleneck MLP `2C → max(C/reduction, 8) → C` and a sigmoid, giving one weight in (0, 1) per channel `(B, C)`. `forward()` multiplies the feature map by these weights broadcast over H and W. Used inside `CBAM`.

```
class SpatialAttention(nn.Module):
    """CBAM spatial attention: [mean_c, max_c] -> 7x7 conv -> sigmoid map."""

    def __init__(self, kernel: int = 7) -> None:
        super().__init__()
        self.conv = nn.Conv2d(2, 1, kernel, padding=kernel // 2)

    def weights(self, x: torch.Tensor) -> torch.Tensor:
        return torch.sigmoid(self.conv(torch.cat([x.mean(1, keepdim=True), x.amax(1, keepdim=True)], 1)))

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return x * self.weights(x)
```

CBAM spatial attention. The channel-wise mean and max maps (`(B, 1, H, W)` each) are concatenated and convolved with a 7×7 kernel to a single map; the sigmoid gives a per-pixel weight `(B, 1, H, W)` which `forward()` multiplies onto every channel.

```
class CBAM(nn.Module):
    """Convolutional Block Attention Module (channel attention followed by spatial attention)."""

    def __init__(self, channels: int, reduction: int = 4, kernel: int = 7) -> None:
        super().__init__()
        self.channel = SqueezeExcitation(channels, reduction)
        self.spatial = SpatialAttention(kernel)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.spatial(self.channel(x))
```

Convolutional Block Attention Module = channel attention followed by spatial attention. Used as the final refinement of `AttentionGatedFusion` (models/fusion/fusion_methods.py). Shape preserving.

models/common/pillars.py

Pure-PyTorch *dynamic pillar* encoding shared by the LiDAR and radar encoders. It turns an unordered point list into a dense BEV canvas without custom CUDA kernels (important on Windows). Point clouds arrive as one tensor for the whole batch with a leading batch-index column (see `collate_batch`). (122 lines)

```
"""Dynamic pillar feature encoding (pure PyTorch, no custom CUDA ops).

"Dynamic voxelisation" (Zhou et al., MVF 2019) assigns *every* point to its pillar
without a max-points-per-pillar cap and aggregates with scatter-max. This keeps all
returns of the dense multi-sweep LiDAR and works identically for sparse radar.

Input:  ``points (P, 1 + F)`` with a leading batch-index column (see ``collate_batch``).
Output: BEV canvas ``(B, C, ny, nx)`` on the shared :class:`BEVGrid`.
"""
from __future__ import annotations

import torch
from torch import nn

from utils.bev_grid import BEVGrid
```

Docstring: explains *dynamic voxelisation* — every point is kept (no maximum number of points per pillar, unlike the original PointPillars) and features are aggregated with a scatter-max. Input `points (P, 1+F)`, output `(B, C, ny, nx)`.

```
def scatter_mean(src: torch.Tensor, index: torch.Tensor, size: int) -> torch.Tensor:
    out = torch.zeros(size, src.shape[1], device=src.device, dtype=src.dtype).index_add_(0, index, src)
    count = torch.zeros(size, device=src.device, dtype=src.dtype).index_add_(0, index, torch.ones_like(index, dty
```

```
        pe=src.dtype))
    return out / count.clamp(min=1).unsqueeze(1)
```

`scatter_mean(src (P, F), index (P,), size U) → (U, F)`: per-group mean.

1. `index_add_` sums the rows of `src` into `out[index]` (one row per pillar).
2. A second `index_add_` of ones counts the points per pillar.
3. Division by `count.clamp(min=1)` avoids division by zero for (impossible) empty groups.

Used for the per-pillar mean xyz (cluster offsets) and for the raster statistics.

```
def scatter_max(src: torch.Tensor, index: torch.Tensor, size: int) -> torch.Tensor:
    """Per-group max for non-negative ``src`` (inputs are post-ReLU)."""
    out = torch.zeros(size, src.shape[1], device=src.device, dtype=src.dtype)
    return out.scatter_reduce_(0, index.unsqueeze(1).expand(-1, src.shape[1]), src, reduce="amax", include_self=T
        rue)
```

`scatter_max(src (P, C), index (P,), size U) → (U, C)`: per-group maximum using `Tensor.scatter_reduce_(..., reduce='amax', include_self=True)`. The output is initialised with zeros, which is correct only because `src` is post-ReLU (non-negative) — documented in the docstring. This is the PointNet max-pooling inside each pillar.

```
class PillarIndexer:
    """Computes pillar membership of points on a BEV grid."""

    def __init__(self, grid: BEVGrid) -> None:
        self.grid = grid

    def __call__(self, points: torch.Tensor, batch_size: int) -> dict[str, torch.Tensor]:
        g = self.grid
        b = points[:, 0].long()
        ix = torch.floor((points[:, 1] - g.x_range[0]) / g.resolution).long()
        iy = torch.floor((points[:, 2] - g.y_range[0]) / g.resolution).long()
        keep = (ix >= 0) & (ix < g.nx) & (iy >= 0) & (iy < g.ny) & (b >= 0) & (b < batch_size)
        b, ix, iy, pts = b[keep], ix[keep], iy[keep], points[keep]
        flat = (b * g.ny + iy) * g.nx + ix
        unique, inverse = torch.unique(flat, return_inverse=True)
        return {"points": pts, "ix": ix, "iy": iy, "flat": unique, "inverse": inverse}
```

`PillarIndexer` maps points to pillars on a `BEVGrid`.

- Column 0 is the batch index `b`; columns 1 and 2 are x and y in metres (reference frame).
- `ix = floor((x − x_min)/r)`, `iy = floor((y − y_min)/r)` give the cell indices (r = 0.5 m → 200 × 200).
- `keep` drops points outside the grid or with an invalid batch index.
- `flat = (b·ny + iy)·nx + ix` gives one integer id per (sample, cell).
- `torch.unique(flat, return_inverse=True)` returns the U occupied pillars (`unique`) and, for every point, the index of its pillar (`inverse`, shape `(P',)`). This inverse index drives all scatter operations.

Returns the filtered points and the index tensors in a dict.

```
class DynamicPillarVFE(nn.Module):
    """PointNet over pillars: per-point MLP layers with scatter-max aggregation.

    Point decoration (``in_features + 5`` dims): raw features, offset to the pillar's point
    mean (3), offset to the pillar centre in x/y (2).
    """

    def __init__(self, in_features: int, channels: tuple[int, ...], grid: BEVGrid) -> None:
        super().__init__()
        self.grid = grid
        self.indexer = PillarIndexer(grid)
        self.layers = nn.ModuleList()
        c_in = in_features + 5
        for i, c_out in enumerate(channels):
            last = i == len(channels) - 1
            # Every layer except the last concatenates the pillar-max feature (PFN in PointPillars).
            self.layers.append(nn.Sequential(nn.Linear(c_in, c_out if last else c_out // 2, bias=False),
                                             nn.BatchNorm1d(c_out if last else c_out // 2), nn.ReLU(inplace=True)
                                                 ))
            c_in = c_out
        self.out_channels = channels[-1]
```

`DynamicPillarVFE` (Voxel/Pillar Feature Encoder). Constructor builds a small PointNet: for `channels = (32, 64)` the layers are `Linear(F+5 → 16)` and `Linear(32 → 64)`, each followed by BatchNorm1d and ReLU. Non-final layers output `c_out // 2` features because the pillar-max feature (another `c_out // 2`) is concatenated afterwards (PointPillars PFN design), so the next layer again sees `c_out` inputs. `in_features`

`+ 5` = raw point features + 3 offsets to the pillar mean + 2 offsets to the pillar centre.

```
    def forward(self, points: torch.Tensor, batch_size: int, normalizer: torch.Tensor | None = None) -> torch.Tensor:
        """Return the BEV canvas ``(B, C, ny, nx)``. ``normalizer`` rescales raw feature columns."""
        g = self.grid
        canvas = points.new_zeros(batch_size, self.out_channels, g.ny, g.nx, dtype=torch.float32)
        if points.shape[0] == 0:
            return canvas
        idx = self.indexer(points.float(), batch_size)
        pts, inv, n_pillars = idx["points"], idx["inverse"], idx["flat"].numel()
        if n_pillars == 0 or (self.training and pts.shape[0] < 2):  # BatchNorm1d needs >1 sample
            return canvas
        feats = pts[:, 1:]
        xyz = feats[:, :3]
        mean = scatter_mean(xyz, inv, n_pillars)[inv]
        centre_x = (idx["ix"].float() + 0.5) * g.resolution + g.x_range[0]
        centre_y = (idx["iy"].float() + 0.5) * g.resolution + g.y_range[0]
        raw = feats / normalizer if normalizer is not None else feats
        x = torch.cat([raw, xyz - mean, (xyz[:, 0] - centre_x).unsqueeze(1), (xyz[:, 1] - centre_y).unsqueeze(1)], 1)
        for i, layer in enumerate(self.layers):
            x = layer(x)
            pooled = scatter_max(x, inv, n_pillars)
            if i < len(self.layers) - 1:
                x = torch.cat([x, pooled[inv]], 1)
        flat = idx["flat"]
        b = flat // (g.ny * g.nx)
        rem = flat % (g.ny * g.nx)
        canvas[b, :, rem // g.nx, rem % g.nx] = pooled.to(canvas.dtype)
        return canvas
```

Forward pass, step by step:

1. `canvas (B, C, ny, nx)` of zeros is created first, so empty inputs (e.g. a missing radar) return a valid all-zero feature map instead of crashing.
2. `self.indexer` assigns pillars. If no point is inside the grid — or, during training, fewer than two points exist (BatchNorm1d cannot normalise a single sample) — the empty canvas is returned.
3. `feats = pts[:, 1:]` drops the batch column; `xyz` are the first three feature columns.
4. `mean = scatter_mean(xyz, inv, U)[inv]` broadcasts each pillar's mean back to its points.
5. `centre_x/centre_y` are the metric centres of each point's cell.
6. `raw = feats / normalizer` rescales raw columns (e.g. x/50 m, intensity/255) so inputs are O(1).
7. The decorated point feature is `[raw, xyz − mean, x − centre_x, y − centre_y]` → `(P', F+5)`.
8. For each PFN layer: Linear-BN-ReLU per point, then `scatter_max` per pillar → `pooled (U, C_i)`; for non-final layers the pooled feature is concatenated back to every point.
9. The final `pooled (U, C)` is written into the canvas at `[b, :, iy, ix]` (recovered from `flat` by integer division / modulo). `.to(canvas.dtype)` avoids an fp16/fp32 mismatch under autocast.

Output: `(B, C, ny, nx)`; cells without points stay zero.

```
def bev_raster_stats(points: torch.Tensor, batch_size: int, grid: BEVGrid, value_cols: list[int],
                     count_scale: float = 3.0) -> torch.Tensor:
    """Hand-crafted BEV raster: ``log(1+count)/count_scale`` plus the mean of selected point columns.

    Returns ``(B, 1 + len(value_cols), ny, nx)``. These explicit channels keep physically
    meaningful quantities (density, height, intensity, radar velocity) visible to the
    feature-selection gates, which makes the learned importances interpretable.
    """
    g = grid
    out = points.new_zeros(batch_size, 1 + len(value_cols), g.ny, g.nx, dtype=torch.float32)
    if points.shape[0] == 0:
        return out
    idx = PillarIndexer(g)(points.float(), batch_size)
    pts, inv, flat = idx["points"], idx["inverse"], idx["flat"]
    if flat.numel() == 0:
        return out
    count = torch.zeros(flat.numel(), device=pts.device).index_add_(0, inv, torch.ones_like(inv, dtype=torch.float32))
    stats = [torch.log1p(count).unsqueeze(1) / count_scale]
    if value_cols:
        stats.append(scatter_mean(pts[:, [c + 1 for c in value_cols]], inv, flat.numel()))
    vals = torch.cat(stats, 1)
    b = flat // (g.ny * g.nx)
    rem = flat % (g.ny * g.nx)
    out[b, :, rem // g.nx, rem % g.nx] = vals
    return out
```

`bev_raster_stats` builds *hand-crafted* BEV channels: `log(1 + count) / 3` (point density) plus the per-cell mean of selected point columns (`value_cols`, 0-based feature indices; `+1` skips the batch column). Output `(B, 1 + len(value_cols), ny, nx)`. These explicit, physically meaningful channels (density, height, intensity, radar velocity, RCS, time lag) are appended to the learned encoder outputs so that the feature-selection channel gates can be interpreted per physical quantity.

models/camera_bev/image_backbone.py

The image feature extractor of the camera branch: a torchvision ResNet trunk with a light two-level FPN neck that outputs one feature map at stride 16. (74 lines)

```python
"""Image backbone: torchvision ResNet + a light FPN neck at stride 16."""
from __future__ import annotations

import logging

import torch
import torch.nn.functional as F
import torchvision
from torch import nn
from torch.utils.checkpoint import checkpoint

from models.common.blocks import ConvBNReLU

logger = logging.getLogger(__name__)
_WEIGHTS = {"resnet18": "ResNet18_Weights", "resnet34": "ResNet34_Weights", "resnet50": "ResNet50_Weights"}
```

Imports. `torchvision` provides the ResNet definitions and ImageNet weights; `torch.utils.checkpoint.checkpoint` implements gradient checkpointing; `_WEIGHTS` maps the backbone name in the config (`model.camera_bev.backbone`) to the torchvision weights enum.

```python
class ImageBackbone(nn.Module):
    """ResNet trunk (layer1-4) whose stride-32 output is upsampled and merged with stride 16.

    Input ``(BN, 3, H, W)`` -> output ``(BN, out_channels, H/16, W/16)``.
    """

    def __init__(self, name: str = "resnet18", pretrained: bool = True, out_channels: int = 256,
                 frozen_stages: int = 1, norm_eval: bool = True) -> None:
        super().__init__()
        weights = None
        if pretrained:
            try:
                weights = getattr(torchvision.models, _WEIGHTS[name]).IMAGENET1K_V1
                net = getattr(torchvision.models, name)(weights=weights)
            except Exception as exc:  # offline / download failure: keep running with random init
                logger.warning("Could not load ImageNet weights for %s (%s); using random init.", name, exc)
                net = getattr(torchvision.models, name)(weights=None)
        else:
            net = getattr(torchvision.models, name)(weights=None)
        self.stem = nn.Sequential(net.conv1, net.bn1, net.relu, net.maxpool)
        self.layer1, self.layer2, self.layer3, self.layer4 = net.layer1, net.layer2, net.layer3, net.layer4
        c3 = self.layer3[-1].conv1.in_channels if name != "resnet50" else 1024
        c4 = self.layer4[-1].conv1.in_channels if name != "resnet50" else 2048
        self.neck = nn.Sequential(ConvBNReLU(c3 + c4, out_channels, 1), ConvBNReLU(out_channels, out_channels, 3)
            )
        self.frozen_stages = frozen_stages
        self.norm_eval = norm_eval
        self.grad_checkpoint = False
        self._freeze()
```

Constructor.

- Loads `resnet18/34/50` with ImageNet weights when `pretrained=True`. The `try/except` keeps the program running offline: on any download error a warning is logged and the network is randomly initialised.
- `stem` = conv1+bn1+relu+maxpool (stride 4); `layer1..layer4` have strides 4, 8, 16, 32.
- `c3`/`c4` read the channel counts of layer3/layer4 from the network (256/512 for ResNet18/34, 1024/2048 for ResNet50).
- `neck`: 1×1 ConvBNReLU on `concat(c3, upsample(c4))` → `out_channels` (256), then a 3×3 ConvBNReLU.
- `frozen_stages` / `norm_eval` / `grad_checkpoint` are stored and `_freeze()` is applied.

```python
    def _freeze(self) -> None:
        stages = [self.stem, self.layer1, self.layer2, self.layer3, self.layer4]
        for stage in stages[: self.frozen_stages + 1 if self.frozen_stages > 0 else 0]:
```

```
            stage.eval()
            for p in stage.parameters():
                p.requires_grad_(False)
```

`_freeze()` puts the first `frozen_stages + 1` stages (stem, layer1 for the default `frozen_stages = 1`) into eval mode and disables their gradients. Low-level ImageNet filters are generic; freezing them saves memory and reduces over-fitting on nuScenes-mini.

```
    def train(self, mode: bool = True) -> "ImageBackbone":
        super().train(mode)
        self._freeze()
        if mode and self.norm_eval:
            for m in self.modules():
                if isinstance(m, nn.BatchNorm2d):
                    m.eval()
        return self
```

Overrides `nn.Module.train()` so that (a) frozen stages stay frozen after `model.train()` is called by the trainer and (b) with `norm_eval=True` every BatchNorm2d stays in eval mode — with batch size 1 × 6 images, batch statistics would be noisy, so the ImageNet running statistics are kept.

```
    def _run(self, layer: nn.Module, x: torch.Tensor) -> torch.Tensor:
        if self.grad_checkpoint and self.training and torch.is_grad_enabled():
            return checkpoint(layer, x, use_reentrant=False)
        return layer(x)
```

`_run(layer, x)` executes a stage either normally or through `checkpoint(layer, x)` when gradient checkpointing is enabled (`train.grad_checkpoint`, on in the GTX 1650 profile). Checkpointing discards intermediate activations in the forward pass and recomputes them in the backward pass: less GPU memory, ~20–30 % more compute.

```
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.layer1(self.stem(x))
        x = self._run(self.layer2, x)
        c3 = self._run(self.layer3, x)
        c4 = self._run(self.layer4, c3)
        c4 = F.interpolate(c4, size=c3.shape[-2:], mode="bilinear", align_corners=False)
        return self.neck(torch.cat([c3, c4], 1))
```

Forward: `(B·N, 3, H, W)` → stem+layer1 → layer2 → `c3 (B·N, 256, H/16, W/16)` → `c4 (B·N, 512, H/32, W/32)`; `c4` is bilinearly upsampled to `c3`'s size, concatenated (`768` channels for ResNet18) and fused by the neck → `(B·N, 256, H/16, W/16)`. For 128×352 inputs this is `(6, 256, 8, 22)`.

models/camera_bev/lss.py

Camera → BEV transformation (Lift-Splat-Shoot with a BEVDepth-style camera-aware depth network). This is the most geometry-heavy module; the docstring at the top contains the exact equations used by `get_geometry()`. (170 lines)

```
"""Camera -> BEV: Lift-Splat-Shoot with a BEVDepth-style camera-aware depth network.

Data flow (B batch, N=6 cameras, D depth bins, h×w = H/16 × W/16 feature cells)::

    images (B, N, 3, H, W)
      └─ ImageBackbone ............................ (B·N, 256, h, w)
          └─ DepthNet (conditioned on K, extrinsics)  depth (B·N, D, h, w) softmax
                                                      context (B·N, C, h, w)
              └─ LIFT: outer product ................ (B, N, D, h, w, C)
                 geometry (frustum -> ref frame) .... (B, N, D, h, w, 3)
                  └─ SPLAT: sum-pool into BEV cells . (B, C, ny, nx)
                      └─ BEV refinement convs ....... (B, C, ny, nx)

Geometry of frustum point ``(u, v, d)`` (augmented-image pixel, metric depth)::

    p_img = post_rot⁻¹ ((u, v, d) − post_tran)          # undo resize / crop / flip / rotate
    p_cam = K⁻¹ (p_img.u · d, p_img.v · d, d)            # back-project with the intrinsics
    p_ref = cam2ref · p_cam                              # extrinsics + ego motion to the LiDAR time
    p_bev = bda · p_ref                                  # BEV augmentation (identity at test time)
"""
from __future__ import annotations

import math
from typing import Any, Optional

import torch
import torch.nn.functional as F
from torch import nn
```

```
29
30 from models.camera_bev.image_backbone import ImageBackbone
31 from models.common.blocks import ConvBNReLU, ResBlock
32 from utils.bev_grid import BEVGrid
```

Docstring with the data flow and the frustum-to-reference-frame equations (see the *Camera → BEV* chapter for the derivation). Imports: `math` for the bin-centre helper, `torch.nn.functional` for one-hot encoding and BCE, the backbone and the shared blocks.

```
35 class DepthNet(nn.Module):
36     """Predicts a categorical depth distribution and context features per image cell.
37
38     ``camera_aware``: an MLP over intrinsics/extrinsics produces SE-style gates for the
39     depth and context branches (BEVDepth), so one network serves all six cameras.
40     """
41
42     def __init__(self, in_channels: int, mid_channels: int, depth_bins: int, context_channels: int,
43                  camera_aware: bool = True, cam_param_dim: int = 16) -> None:
44         super().__init__()
45         self.reduce = ConvBNReLU(in_channels, mid_channels, 3)
46         self.camera_aware = camera_aware
47         if camera_aware:
48             self.cam_bn = nn.BatchNorm1d(cam_param_dim)
49             self.cam_mlp = nn.Sequential(nn.Linear(cam_param_dim, mid_channels), nn.ReLU(inplace=True),
50                                          nn.Linear(mid_channels, 2 * mid_channels))
51         self.context = nn.Sequential(ResBlock(mid_channels), nn.Conv2d(mid_channels, context_channels, 1))
52         self.depth = nn.Sequential(ResBlock(mid_channels), ResBlock(mid_channels, dilation=2), nn.Conv2d(mid_chan
   ↪             nels, depth_bins, 1))
```

`DepthNet` predicts, for every image feature cell, a categorical depth distribution over D bins and a C-dimensional context feature.

- `reduce`: 3×3 ConvBNReLU 256 → 256.
- If `camera_aware`: `cam_bn` (BatchNorm1d over the 16-d camera descriptor) and `cam_mlp` (16 → 256 → 512) produce two sets of 256 SE-style gates — one for the depth branch, one for the context branch. This lets a single network serve six cameras with different intrinsics/extrinsics (BEVDepth idea).
- `context`: ResBlock + 1×1 conv → C = 64 channels.
- `depth`: ResBlock + dilated ResBlock (larger receptive field) + 1×1 conv → D = 59 logits.

```
54     def forward(self, x: torch.Tensor, cam_params: Optional[torch.Tensor]) -> tuple[torch.Tensor, torch.Tensor]:
55         x = self.reduce(x)
56         x_ctx, x_dep = x, x
57         if self.camera_aware and cam_params is not None:
58             gates = torch.sigmoid(self.cam_mlp(self.cam_bn(cam_params.float()))).to(x.dtype)
59             g_ctx, g_dep = gates.chunk(2, dim=1)
60             x_ctx = x * g_ctx[:, :, None, None]
61             x_dep = x * g_dep[:, :, None, None]
62         return self.depth(x_dep), self.context(x_ctx)
```

`DepthNet.forward(x (B·N, 256, h, w), cam_params (B·N, 16))`: reduce, compute sigmoid gates from the camera descriptor (cast to float for numerical stability, then back to the activation dtype), multiply them channel-wise into two copies of the features and return `(depth_logits (B·N, D, h, w), context (B·N, C, h, w))`.

```
65 class CameraBEVEncoder(nn.Module):
66     """Lift-Splat camera BEV encoder producing ``(B, C, ny, nx)`` on the shared grid."""
67
68     def __init__(self, cfg: Any, grid: BEVGrid, image_size: tuple[int, int]) -> None:
69         super().__init__()
70         self.grid = grid
71         self.image_size = (int(image_size[0]), int(image_size[1]))
72         self.stride = int(cfg.feature_stride)
73         dmin, dmax, dstep = (float(v) for v in cfg.depth_bins)
74         self.dbound = (dmin, dmax, dstep)
75         self.num_bins = int(round((dmax - dmin) / dstep))
76         self.out_channels = int(cfg.out_channels)
77         self.backbone = ImageBackbone(cfg.backbone, bool(cfg.pretrained), int(cfg.neck_channels),
78                                       int(cfg.frozen_stages), bool(cfg.norm_eval))
79         self.depth_net = DepthNet(int(cfg.neck_channels), int(cfg.neck_channels), self.num_bins, self.out_channel
   ↪             s,
80                                   bool(cfg.camera_aware_depth))
81         self.bev_refine = nn.Sequential(ConvBNReLU(self.out_channels, self.out_channels, 3),
82                                         *[ResBlock(self.out_channels) for _ in range(int(cfg.bev_refine_blocks))]
   ↪                                         )
83         self.register_buffer("frustum", self._create_frustum(), persistent=False)
84
85     # ------------------------------------------------------------------ geometry
```

Constructor of the full camera branch. Reads from `model.camera_bev`:

- `feature_stride` (16) and `depth_bins [1, 60, 1]` → `dbound`, `num_bins = 59`;
- builds `ImageBackbone`, `DepthNet` and `bev_refine` (ConvBNReLU + `bev_refine_blocks` ResBlocks);
- `register_buffer('frustum', ..., persistent=False)`: the frustum is a constant tensor that moves with `.to(device)` but is not saved in checkpoints (it is recomputed from the config).

```
86      def _create_frustum(self) -> torch.Tensor:
87          fh, fw = self.image_size
88          h, w = fh // self.stride, fw // self.stride
89          dmin, _, dstep = self.dbound
90          depths = dmin + (torch.arange(self.num_bins, dtype=torch.float32) + 0.5) * dstep  # bin centres
91          xs = (torch.arange(w, dtype=torch.float32) + 0.5) * self.stride - 0.5               # cell-centre pixels
92          ys = (torch.arange(h, dtype=torch.float32) + 0.5) * self.stride - 0.5
93          d = depths.view(-1, 1, 1).expand(-1, h, w)
94          x = xs.view(1, 1, w).expand(self.num_bins, h, w)
95          y = ys.view(1, h, 1).expand(self.num_bins, h, w)
96          return torch.stack([x, y, d], -1)  # (D, h, w, 3)
```

`_create_frustum()` → `(D, h, w, 3)`. For every depth bin and feature cell it stores `(u, v, d)`: `u, v` are the pixel coordinates of the cell centre in the *augmented* network input image (`(k + 0.5)·16 − 0.5`) and `d` the metric depth of the bin centre (`1.5, 2.5, … 59.5 m`). Using bin centres keeps the geometry consistent with the depth-supervision binning (`floor((d − d_min)/step)`).

```
98      def get_geometry(self, intrinsics: torch.Tensor, post_rots: torch.Tensor, post_trans: torch.Tensor,
99                       cam2ref: torch.Tensor, bda: torch.Tensor) -> torch.Tensor:
100         """Reference-frame location of every frustum point: ``(B, N, D, h, w, 3)``."""
101         b, n = intrinsics.shape[:2]
102         pts = self.frustum.view(1, 1, *self.frustum.shape) - post_trans.view(b, n, 1, 1, 1, 3)
103         pts = torch.linalg.inv(post_rots).view(b, n, 1, 1, 1, 3, 3).matmul(pts.unsqueeze(-1))
104         pts = torch.cat([pts[..., :2, :] * pts[..., 2:3, :], pts[..., 2:3, :]], -2)
105         combine = cam2ref[..., :3, :3].matmul(torch.linalg.inv(intrinsics))
106         pts = combine.view(b, n, 1, 1, 1, 3, 3).matmul(pts).squeeze(-1) + cam2ref[..., :3, 3].view(b, n, 1, 1, 1
    ↪           , 3)
107         pts = bda[:, :3, :3].view(b, 1, 1, 1, 1, 3, 3).matmul(pts.unsqueeze(-1)).squeeze(-1) + bda[:, :3, 3].vie
    ↪           w(b, 1, 1, 1, 1, 3)
108         return pts
```

`get_geometry()` lifts every frustum point to the BEV frame → `(B, N, D, h, w, 3)`.

1. `pts = frustum − post_trans` and multiplication with `inv(post_rots)` undo the image augmentation (resize, crop, flip, rotation) — back to original 1600×900 pixel coordinates `(u', v', d)`.
2. `cat([u'·d, v'·d], d)` forms the homogeneous image point scaled by depth.
3. `combine = R_cam2ref · K⁻¹` back-projects into the camera frame and rotates into the reference frame in one matrix product; adding `t_cam2ref` translates. `cam2ref` already contains the extrinsics *and* the ego motion between the camera and LiDAR timestamps.
4. `bda` (BEV data augmentation: flip/rotate/scale, identity at test time) is applied last so that camera features land in the same augmented frame as the LiDAR/radar points and GT boxes.

Broadcasting via `.view(b, n, 1, 1, 1, 3, 3)` applies each camera's matrices to all D·h·w points.

```
110     def voxel_pool(self, geom: torch.Tensor, feats: torch.Tensor) -> torch.Tensor:
111         """Sum-pool lifted features ``(B, N, D, h, w, C)`` into BEV cells ``(B, C, ny, nx)`` (z collapsed)."""
112         g = self.grid
113         b, c = geom.shape[0], feats.shape[-1]
114         geom = geom.reshape(b, -1, 3)
115         feats = feats.reshape(b, -1, c).float()
116         ix = torch.floor((geom[..., 0] - g.x_range[0]) / g.resolution).long()
117         iy = torch.floor((geom[..., 1] - g.y_range[0]) / g.resolution).long()
118         z = geom[..., 2]
119         keep = (ix >= 0) & (ix < g.nx) & (iy >= 0) & (iy < g.ny) & (z >= g.z_range[0]) & (z < g.z_range[1])
120         batch_idx = torch.arange(b, device=geom.device).view(b, 1).expand_as(ix)
121         flat = (batch_idx * g.ny + iy) * g.nx + ix
122         bev = torch.zeros(b * g.ny * g.nx, c, device=feats.device, dtype=torch.float32)
123         bev.index_add_(0, flat[keep], feats[keep])
124         return bev.view(b, g.ny, g.nx, c).permute(0, 3, 1, 2).contiguous()
```

`voxel_pool()` — the *splat*. Flattens geometry to `(B, N·D·h·w, 3)` and features to `(B, N·D·h·w, C)` (in fp32 to avoid overflow), computes integer cells `ix, iy`, keeps points inside the grid and z-range, builds a flat index `(b·ny + iy)·nx + ix` and sums all features falling into the same cell with `index_add_` → `(B·ny·nx, C)` → reshaped/permuted to `(B, C, ny, nx)`. The height dimension is collapsed (sum over all z). This replaces the CUDA "cumsum trick" of the original LSS with a single, deterministic-enough PyTorch op.

```
126     @staticmethod
127     def camera_params(intrinsics: torch.Tensor, post_rots: torch.Tensor, cam2ref: torch.Tensor,
```

```
                      image_size: tuple[int, int]) -> torch.Tensor:
        """16-d camera descriptor: augmented focal lengths / principal point (normalised) + extrinsics."""
        k_aug = post_rots.matmul(intrinsics)
        h, w = image_size
        intr = torch.stack([k_aug[..., 0, 0] / w, k_aug[..., 1, 1] / h, k_aug[..., 0, 2] / w, k_aug[..., 1, 2] /
            h], -1)
        extr = cam2ref[..., :3, :4].reshape(*cam2ref.shape[:2], 12)
        return torch.cat([intr, extr], -1).flatten(0, 1)

    # ------------------------------------------------------------ forward
```

`camera_params()` builds the 16-d camera descriptor for DepthNet: the augmented intrinsics `K_aug = post_rot · K` give fx/W, fy/H, cx/W, cy/H (normalised), plus the 12 entries of `cam2ref[:3, :4]`. Flattened to `(B·N, 16)`.

```
    def forward(self, images: torch.Tensor, intrinsics: torch.Tensor, post_rots: torch.Tensor, post_trans: torch
        .Tensor,
                cam2ref: torch.Tensor, bda: torch.Tensor, cam_valid: Optional[torch.Tensor] = None) -> dict[str,
                    torch.Tensor]:
        b, n = images.shape[:2]
        feats = self.backbone(images.flatten(0, 1))
        cam_params = self.camera_params(intrinsics, post_rots, cam2ref, self.image_size)
        depth_logits, context = self.depth_net(feats, cam_params)
        depth = depth_logits.float().softmax(1)
        if cam_valid is not None:  # a missing / unreadable camera contributes nothing
            depth = depth * cam_valid.flatten().float().view(-1, 1, 1, 1)
        lifted = depth.unsqueeze(1) * context.float().unsqueeze(2)  # (BN, C, D, h, w)
        lifted = lifted.view(b, n, *lifted.shape[1:]).permute(0, 1, 3, 4, 5, 2)  # (B, N, D, h, w, C)
        geom = self.get_geometry(intrinsics.float(), post_rots.float(), post_trans.float(), cam2ref.float(), bda
            .float())
        bev = self.voxel_pool(geom, lifted)
        bev = self.bev_refine(bev)
        return {"bev": bev, "depth_logits": depth_logits, "depth": depth}
```

Forward pass:

1. `images (B, N, 3, H, W)` → `flatten(0, 1)` → backbone → `feats (B·N, 256, h, w)`.
2. DepthNet → `depth_logits (B·N, D, h, w)`, `context (B·N, C, h, w)`; `softmax(1)` over the depth bins gives a probability distribution per cell.
3. `cam_valid` (False for a missing/unreadable image) zeroes that camera's depth so it contributes nothing.
4. Lift: `depth.unsqueeze(1) * context.unsqueeze(2)` is the outer product `(B·N, C, D, h, w)`; it is reshaped and permuted to `(B, N, D, h, w, C)`.
5. `get_geometry` + `voxel_pool` → `(B, C, ny, nx)`; `bev_refine` convolutions smooth the splatted map.

Returns `bev`, `depth_logits` (for the loss) and `depth` (probabilities).

```
    def depth_loss(self, depth_logits: torch.Tensor, depth_gt: torch.Tensor) -> torch.Tensor:
        """BEVDepth loss: BCE between the predicted distribution and the one-hot LiDAR depth bin."""
        dmin, dmax, dstep = self.dbound
        gt = depth_gt.reshape(-1)
        valid = (gt >= dmin) & (gt < dmax)
        if not valid.any():
            return depth_logits.sum() * 0.0
        probs = depth_logits.float().softmax(1).permute(0, 2, 3, 1).reshape(-1, self.num_bins)[valid]
        idx = ((gt[valid] - dmin) / dstep).floor().long().clamp(0, self.num_bins - 1)
        target = F.one_hot(idx, self.num_bins).float()
        with torch.autocast(device_type=probs.device.type, enabled=False):
            loss = F.binary_cross_entropy(probs.clamp(1e-6, 1 - 1e-6), target, reduction="none").sum()
        return loss / max(1.0, float(valid.sum()))
```

`depth_loss()` — BEVDepth supervision. `depth_gt (B, N, h, w)` holds the distance of the closest LiDAR point per feature cell (0 = none). Valid cells within `[d_min, d_max)` are converted to one-hot bin targets; the loss is the binary cross-entropy between the predicted probabilities and the one-hot vector, summed over bins and averaged over valid cells. Autocast is disabled because `F.binary_cross_entropy` is unsafe in fp16. If no cell is valid, `logits.sum()·0` returns a zero that is still connected to the graph.

```
def depth_bin_centers(dbound: tuple[float, float, float]) -> list[float]:
    dmin, dmax, dstep = dbound
    return [dmin + (k + 0.5) * dstep for k in range(int(math.floor((dmax - dmin) / dstep)))]
```

Helper returning the list of depth-bin centres for a `dbound`; not used by the model itself (kept for analysis / plotting).

models/lidar_encoder/pillar_encoder.py

LiDAR branch: dynamic pillars + explicit raster channels + a small residual CNN at full BEV resolution. (63 lines)

```
"""LiDAR encoder: dynamic PointPillars -> BEV feature map.

    lidar_points (P, 1+5) [batch, x, y, z, intensity, time_lag]   (reference frame)
      ├─ DynamicPillarVFE (PointNet per 0.5 m pillar) ........ (B, 64, ny, nx)
      ├─ explicit raster: density, max/mean height, intensity  (B, 4, ny, nx)
      └─ concat -> ConvBNReLU -> ResBlocks ................... (B, C-4, ny, nx)
    output = concat(learned, raster) ......................... (B, C, ny, nx)

Compute: one linear layer per point (~150k-250k points) + ~3 conv layers at 200x200.
"""
from __future__ import annotations

from typing import Any

import torch
from torch import nn

from models.common.blocks import ConvBNReLU, ResBlock
from models.common.pillars import DynamicPillarVFE, PillarIndexer, bev_raster_stats
from utils.bev_grid import BEVGrid

LIDAR_RASTER_NAMES = ("lidar_density", "lidar_max_height", "lidar_mean_height", "lidar_intensity")
```

Docstring: input `(P, 1+5)` = batch, x, y, z, intensity, time-lag; output `(B, 64, ny, nx)` = 60 learned + 4 raster channels. `LIDAR_RASTER_NAMES` names the raster channels; these names appear in the dashboard's channel-gate chart.

```
class LidarEncoder(nn.Module):
    in_features = 5

    def __init__(self, cfg: Any, grid: BEVGrid) -> None:
        super().__init__()
        self.grid = grid
        self.use_raster = bool(cfg.raster_channels)
        n_raster = len(LIDAR_RASTER_NAMES) if self.use_raster else 0
        self.out_channels = int(cfg.out_channels)
        learned = self.out_channels - n_raster
        self.vfe = DynamicPillarVFE(self.in_features, tuple(int(c) for c in cfg.pillar_channels), grid)
        self.conv = nn.Sequential(ConvBNReLU(self.vfe.out_channels + n_raster, learned, 3),
                                  *[ResBlock(learned) for _ in range(int(cfg.num_res_blocks))])
        zmax = max(abs(grid.z_range[0]), abs(grid.z_range[1]))
        self.register_buffer("normalizer", torch.tensor([grid.x_range[1], grid.y_range[1], zmax, 255.0, 0.5]), pe
    ↪           rsistent=False)
        self.channel_names = [f"lidar_learned_{i}" for i in range(learned)] + list(LIDAR_RASTER_NAMES[:n_raster])
```

Constructor. `in_features = 5`. `learned = out_channels − 4`. Builds:

- `vfe`: `DynamicPillarVFE(5, (32, 64), grid)`;
- `conv`: ConvBNReLU(64 + 4 → 60) followed by `num_res_blocks` ResBlocks (2);
- `normalizer` buffer `[x_max, y_max, z_max, 255, 0.5]` — divides x, y, z, intensity and time-lag before the PointNet;
- `channel_names`: 60 × `lidar_learned_i` + the 4 raster names (used for explainability).

```
    def raster(self, points: torch.Tensor, batch_size: int) -> torch.Tensor:
        stats = bev_raster_stats(points, batch_size, self.grid, value_cols=[2, 3])  # density, mean z, mean inten
    ↪           sity
        zmax = torch.zeros_like(stats[:, :1])
        if points.shape[0]:
            idx = PillarIndexer(self.grid)(points.float(), batch_size)
            if idx["flat"].numel():
                vals = torch.zeros(idx["flat"].numel(), device=points.device).scatter_reduce_(
                    0, idx["inverse"], idx["points"][:, 3], reduce="amax", include_self=False)
                flat = idx["flat"]
                b, rem = flat // (self.grid.ny * self.grid.nx), flat % (self.grid.ny * self.grid.nx)
                zmax[b, 0, rem // self.grid.nx, rem % self.grid.nx] = vals
        return torch.cat([stats[:, :1], zmax / 3.0, stats[:, 1:2] / 3.0, stats[:, 2:3] / 100.0], 1)
```

`raster()` builds the 4 explicit channels:

- `bev_raster_stats(..., value_cols=[2, 3])` → density, mean z, mean intensity;
- max height per cell via `scatter_reduce_(reduce='amax', include_self=False)` on the z column (column 3 of the batched point tensor) — `include_self=False` makes negative heights work;
- scaling: density as is, heights / 3 m, intensity / 100.

Output `(B, 4, ny, nx)` ordered as `LIDAR_RASTER_NAMES` (density, max height, mean height, intensity).

```
    def forward(self, points: torch.Tensor, batch_size: int) -> dict[str, torch.Tensor]:
        pillars = self.vfe(points, batch_size, self.normalizer)
        if self.use_raster:
            raster = self.raster(points, batch_size)
            feats = torch.cat([self.conv(torch.cat([pillars, raster], 1)), raster], 1)
        else:
            raster = pillars.new_zeros(batch_size, 0, *self.grid.shape)
            feats = self.conv(pillars)
        return {"bev": feats, "raster": raster, "pillars": pillars}
```

`forward(points, batch_size)`: pillar features `(B, 64, ny, nx)` + raster `(B, 4, ny, nx)` → `conv(cat)` → `(B, 60, ny, nx)` → concatenated with the raster again → `(B, 64, ny, nx)`. Returning the raster unchanged in the last 4 channels guarantees that these physical quantities reach the feature-selection gates verbatim.

models/radar_encoder/radar_encoder.py

Radar branch. Same dynamic-pillar idea as LiDAR, adapted to very sparse points (≈ 100× fewer) and with the ego-motion-compensated Doppler velocity kept explicitly. (59 lines)

```
"""Radar encoder: velocity-preserving radar pillars -> BEV feature map.

    radar_points (Q, 1+10) [batch, x, y, z, rcs, vx_c, vy_c, vx, vy, dyn_prop, time_lag]
      ├─ DynamicPillarVFE ..................................... (B, 64, ny, nx)
      ├─ explicit raster: density, vx_comp, vy_comp, rcs, time-lag .... (B, 5, ny, nx)
      └─ concat -> ConvBNReLU -> dilated ResBlocks (d = 1, 2, 4) ....... (B, C-5, ny, nx)
    output = concat(learned, raster) ................................ (B, C, ny, nx)

Radar returns are ~100x sparser than LiDAR, so dilated convolutions spread each return
over a larger receptive field. The motion-compensated velocity components are kept
*verbatim* in the output (raster channels), guaranteeing that velocity information
reaches fusion, detection (velocity regression) and planning.
"""
from __future__ import annotations

from typing import Any

import torch
from torch import nn

from models.common.blocks import ConvBNReLU, ResBlock
from models.common.pillars import DynamicPillarVFE, bev_raster_stats
from utils.bev_grid import BEVGrid

RADAR_RASTER_NAMES = ("radar_density", "radar_vx", "radar_vy", "radar_rcs", "radar_time_lag")
```

Docstring: the 10 radar point features are x, y, z, rcs, vx_comp, vy_comp, vx, vy, dyn_prop, time_lag (see `datasets/radar_loader.py: RADAR_POINT_FEATURES`). `RADAR_RASTER_NAMES` lists the 5 explicit channels.

```
class RadarEncoder(nn.Module):
    in_features = 10

    def __init__(self, cfg: Any, grid: BEVGrid) -> None:
        super().__init__()
        self.grid = grid
        self.use_raster = bool(cfg.raster_channels)
        n_raster = len(RADAR_RASTER_NAMES) if self.use_raster else 0
        self.out_channels = int(cfg.out_channels)
        learned = self.out_channels - n_raster
        self.vfe = DynamicPillarVFE(self.in_features, tuple(int(c) for c in cfg.pillar_channels), grid)
        self.conv = nn.Sequential(ConvBNReLU(self.vfe.out_channels + n_raster, learned, 3),
                                  *[ResBlock(learned, dilation=int(d)) for d in cfg.dilations])
        self.register_buffer("normalizer", torch.tensor([grid.x_range[1], grid.y_range[1], 5.0, 30.0, 10.0, 10.0,
                                                         10.0, 10.0, 7.0, 0.5]), persistent=False)
        self.register_buffer("raster_scale", torch.tensor([1.0, 10.0, 10.0, 30.0, 0.5]).view(1, -1, 1, 1), persis
    ↪           tent=False)
        self.channel_names = [f"radar_learned_{i}" for i in range(learned)] + list(RADAR_RASTER_NAMES[:n_raster])
```

Constructor. `learned = 64 − 5 = 59`. `conv` = ConvBNReLU(64 + 5 → 59) + one ResBlock per entry of `dilations` (1, 2, 4): dilated convolutions spread each isolated radar return over a larger neighbourhood (receptive field ≈ 31 cells ≈ 15 m) without downsampling. `normalizer` rescales the 10 raw columns (positions /50 m, RCS /30, velocities /10 m/s, dyn_prop /7, time-lag /0.5 s). `raster_scale` rescales the raster channels.

```
    def raster(self, points: torch.Tensor, batch_size: int) -> torch.Tensor:
        # value columns (0-based, without batch col): vx_comp=4, vy_comp=5, rcs=3, time_lag=9
        stats = bev_raster_stats(points, batch_size, self.grid, value_cols=[4, 5, 3, 9])
        return stats / self.raster_scale
```

`raster()` → `(B, 5, ny, nx)`: density, mean vx_comp, mean vy_comp, mean RCS, mean time-lag (value columns 4, 5, 3, 9), divided by `[1, 10, 10, 30, 0.5]`. Velocities stay in the reference frame (rotated by the loader), so a cell's (vx, vy) is directly the object's motion over ground.

```
    def forward(self, points: torch.Tensor, batch_size: int) -> dict[str, torch.Tensor]:
        pillars = self.vfe(points, batch_size, self.normalizer)
        if self.use_raster:
            raster = self.raster(points, batch_size)
            feats = torch.cat([self.conv(torch.cat([pillars, raster], 1)), raster], 1)
        else:
            raster = pillars.new_zeros(batch_size, 0, *self.grid.shape)
            feats = self.conv(pillars)
        return {"bev": feats, "raster": raster, "pillars": pillars}
```

Same structure as the LiDAR encoder: VFE + raster → conv → concatenate raster → `(B, 64, ny, nx)`. The last five channels are exactly the radar raster (velocity preserved).

models/feature_selection/selection.py

The research component. Learned, camera-guided, cross-modal selection of LiDAR and radar BEV features before fusion. See the *Deep-Learning Feature Selection* chapter for the method, the losses and what the trained network actually learned. (142 lines)

```
"""Camera-Guided Cross-Modal Feature Selection (CG-CMFS) — the research component.

Problem
-------
Plain concatenation hands every LiDAR and radar channel at every BEV cell to the fusion
network, including redundant channels (e.g. LiDAR and radar both encoding "something is
here") and noisy ones (radar multipath ghosts, LiDAR returns from vegetation).

Mechanism
---------
For every selectable modality ``m ∈ {lidar, radar}`` the module predicts

* a **channel gate** ``c_m ∈ (0,1)^C`` — *which features* of the modality are useful for
  this scene (cross-modal squeeze-and-excitation: the gate of LiDAR depends on global
  statistics of LiDAR, radar *and* the camera BEV, so e.g. radar velocity channels can be
  up-weighted when LiDAR is sparse);
* a **spatial gate** ``s_m ∈ (0,1)^{H×W}`` — *where* the modality is informative
  (CBAM-style spatial attention over compressed features of all modalities, with the
  camera BEV as the spatial reference / guide).

Selected features: ``F̃_m = F_m ⊙ c_m ⊙ s_m``.

Why this design (vs. alternatives)
----------------------------------
* Channel + spatial gating (SE/CBAM) is cheap at 200×200 BEV resolution, unlike global
  cross-attention (40k tokens → O(1.6e9) attention entries) which does not fit a 4 GB GPU.
* Gates are *explicit* per-modality numbers in [0, 1] → directly interpretable as
  "feature importance" maps and channel importance bars (explainability requirement).
* ``hard_concrete`` mode turns the gates into stochastic binary masks with a
  straight-through estimator: genuine *selection* (0/1) instead of soft re-weighting.
* Supervision: a coverage loss (objects must stay selected by at least one sensor) and a
  sparsity loss (keep only a target ratio) — see ``losses.py``.

Shapes: inputs ``(B, C_m, H, W)`` per modality, guide ``(B, C_cam, H, W)`` or ``None``;
outputs the same shapes plus gate tensors.
"""
```

The long docstring is the design rationale: the problem (redundant / noisy channels when concatenating), the mechanism (channel gates `c_m` and spatial gates `s_m`, `F̃_m = F_m ⊙ c_m ⊙ s_m`), why SE/CBAM-style gating instead of global cross-attention (memory: 40 000 BEV tokens), the optional hard-concrete binary masks, and the shapes.

```
from __future__ import annotations

from typing import Any, Optional

import torch
from torch import nn

from models.common.blocks import ConvBNReLU
```

Imports: `torch`, `nn` and the shared `ConvBNReLU` block.

```
class CrossModalFeatureSelection(nn.Module):
    """Learned channel + spatial feature selection for LiDAR / radar BEV features.

```

```
    Args:
        channels: channels per selectable modality, e.g. ``{"lidar": 64, "radar": 64}``.
        guide_channels: channels of the camera BEV guide (``None`` = no guide).
        cfg: ``fusion.selection`` config.
    """
```

Class docstring: `channels` maps each *selectable* modality to its channel count (`{'lidar': 64, 'radar': 64}` for the full model; only one entry for camera+LiDAR or camera+radar), `guide_channels` is the camera BEV channel count or `None`.

```
    def __init__(self, channels: dict[str, int], guide_channels: Optional[int], cfg: Any) -> None:
        super().__init__()
        self.modalities = list(channels)
        self.channels = dict(channels)
        self.has_guide = guide_channels is not None
        self.gate_mode = str(cfg.gate_mode)
        self.temperature = float(cfg.temperature)
        self.threshold = float(cfg.threshold)
        init_open = float(cfg.init_open)

        all_channels = dict(channels)
        if self.has_guide:
            all_channels["camera"] = int(guide_channels)
        stat_dim = 2 * sum(all_channels.values())
        hidden = max(stat_dim // int(cfg.channel_reduction), 32)
        self.channel_mlp = nn.Sequential(nn.Linear(stat_dim, hidden), nn.ReLU(inplace=True),
                                         nn.Linear(hidden, sum(channels.values())))
```

Constructor, part 1 — configuration and the **channel branch**:

- stores the modality list, `gate_mode` (`soft` | `hard_concrete`), `temperature`, `threshold` (0.5; a gate above it counts as *selected*) and `init_open` (2.0);
- `all_channels` = selectable modalities + `camera` if a guide is used;
- `stat_dim = 2 · Σ channels` — each input contributes a mean and a max per channel (2 · 3 · 64 = 384 for model F);
- `hidden = max(stat_dim // channel_reduction, 32)` = 96;
- `channel_mlp`: Linear(384 → 96) → ReLU → Linear(96 → Σ selectable channels = 128). Because the input contains statistics of *all* sensors, the LiDAR gates can depend on the radar and camera content (cross-modal squeeze-and-excitation).

```
        compress = 8
        self.compress = nn.ModuleDict({m: ConvBNReLU(c, compress, 1) for m, c in all_channels.items()})
        sp_hidden = int(cfg.spatial_hidden)
        k = int(cfg.spatial_kernel)
        self.spatial = nn.Sequential(
            ConvBNReLU((compress + 2) * len(all_channels), sp_hidden, k),
            ConvBNReLU(sp_hidden, sp_hidden, 3),
            nn.Conv2d(sp_hidden, len(self.modalities), 3, padding=1),
        )
        for head in (self.channel_mlp[-1], self.spatial[-1]):
            nn.init.normal_(head.weight, std=0.01)
            nn.init.constant_(head.bias, init_open)

    # ------------------------------------------------------------------ gates
```

Constructor, part 2 — the **spatial branch** and initialisation:

- `compress`: per input a 1×1 ConvBNReLU 64 → 8 channels;
- the spatial network input is, per input, 8 compressed channels + channel-mean + channel-max = 10 channels → `(compress + 2) · len(all_channels)` = 30 for model F;
- `spatial`: 7×7 ConvBNReLU(30 → 32) → 3×3 ConvBNReLU(32 → 32) → 3×3 Conv(32 → #selectable) = one logit map per selectable sensor;
- both final layers get weights `N(0, 0.01)` and bias `init_open = 2.0`, so at the start of training every gate is ≈ σ(2) = 0.88: the network starts from "keep almost everything" and learns what to suppress, instead of starting from random masks that would destroy the signal.

```
    def _gate(self, logits: torch.Tensor) -> torch.Tensor:
        if self.gate_mode == "soft":
            return torch.sigmoid(logits)
        if self.gate_mode == "hard_concrete":
            if self.training:
                u = torch.rand_like(logits).clamp(1e-6, 1 - 1e-6)
                soft = torch.sigmoid((logits + torch.log(u) - torch.log1p(-u)) / self.temperature)
                hard = (soft > 0.5).to(soft.dtype)
```

```
96                return hard + soft - soft.detach()  # straight-through estimator
97            return (logits > 0).to(logits.dtype)
98        raise ValueError(f"Unknown gate_mode {self.gate_mode!r}")
```

`_gate(logits)` converts logits to gates:

- `soft` (default): `sigmoid(logits)` — continuous importance in (0, 1).
- `hard_concrete`, training: logistic noise `log u − log(1 − u)` is added, divided by the temperature and squashed (a Gumbel/Concrete relaxation); `hard = (soft > 0.5)`; the returned value `hard + soft − soft.detach()` equals the binary mask in the forward pass but has the gradient of `soft` (straight-through estimator).
- `hard_concrete`, evaluation: deterministic `logits > 0`.
- Any other mode raises `ValueError`.

```
100    def forward(self, feats: dict[str, torch.Tensor], guide: Optional[torch.Tensor] = None
101                ) -> tuple[dict[str, torch.Tensor], dict[str, Any]]:
102        inputs = {m: feats[m] for m in self.modalities}
103        if self.has_guide:
104            if guide is None:
105                raise ValueError("Camera-guided selection needs the camera BEV guide")
106            inputs["camera"] = guide
107        # Channel gates from global statistics of all modalities.
108        stats = torch.cat([torch.cat([x.float().mean((2, 3)), x.float().amax((2, 3))], 1) for x in inputs.values
    ↪           ()], 1)
109        ch_logits = self.channel_mlp(stats)
110        ch_logits_split = torch.split(ch_logits, [self.channels[m] for m in self.modalities], dim=1)
111        # Spatial gates from compressed features + channel-pooled descriptors (CBAM).
112        sp_in = torch.cat([torch.cat([self.compress[m](x), x.mean(1, keepdim=True), x.amax(1, keepdim=True)], 1)
113                           for m, x in inputs.items()], 1)
114        sp_logits = self.spatial(sp_in)
```

`forward(feats, guide)`:

1. Collects the selectable inputs and, if configured, the camera guide (a `ValueError` is raised when a guide is expected but missing).
2. **Channel statistics**: for every input, `mean` and `amax` over H and W (in fp32) → concatenated `(B, stat_dim)`; `channel_mlp` → `(B, Σ C_m)` logits, split per sensor into `(B, C_m)`.
3. **Spatial descriptor**: for every input `[compress(x) (8 ch), mean_c(x), max_c(x)]` → concatenated `(B, 30, H, W)` → `spatial` → `sp_logits (B, M, H, W)`.

```
116        selected, channel_gates, spatial_gates = {}, {}, {}
117        for i, m in enumerate(self.modalities):
118            c_gate = self._gate(ch_logits_split[i])            # (B, C_m)
119            s_gate = self._gate(sp_logits[:, i:i + 1])          # (B, 1, H, W)
120            channel_gates[m], spatial_gates[m] = c_gate, s_gate
121            selected[m] = feats[m] * c_gate[:, :, None, None].to(feats[m].dtype) * s_gate.to(feats[m].dtype)
122        info = {
123            "channel_gates": channel_gates,
124            "spatial_gates": spatial_gates,
125            "channel_logits": dict(zip(self.modalities, ch_logits_split)),
126            "spatial_logits": sp_logits,
127        }
128        return selected, info
```

4. For each selectable sensor `m` (index `i`): `c_gate = _gate(channel logits) (B, C_m)`, `s_gate = _gate(sp_logits[:, i:i+1]) (B, 1, H, W)`, and the selected features `F_m · c_gate[:, :, None, None] · s_gate (B, C_m, H, W)` — the channel gate broadcasts over space, the spatial gate over channels. Gates are cast to the feature dtype for AMP.
5. `info` returns gates and logits; the gates feed `feature_selection_loss`, the dashboard's explainability views and `selection_stats`.

```
130    @torch.no_grad()
131    def selection_stats(self, info: dict[str, Any]) -> dict[str, float]:
132        """Selected-feature ratios (fraction of gates above threshold) and mean importances."""
133        out: dict[str, float] = {}
134        for m in self.modalities:
135            s, c = info["spatial_gates"][m].float(), info["channel_gates"][m].float()
136            out[f"{m}_spatial_selected_ratio"] = float((s > self.threshold).float().mean())
137            out[f"{m}_channel_selected_ratio"] = float((c > self.threshold).float().mean())
138            out[f"{m}_mean_spatial_importance"] = float(s.mean())
139            out[f"{m}_mean_channel_importance"] = float(c.mean())
140            out[f"{m}_effective_selected_ratio"] = float(((s > self.threshold).float().mean((1, 2, 3))
141                                                         * (c > self.threshold).float().mean(1)).mean())
142        return out
```

`selection_stats(info)` (no gradients) summarises the gates per sensor: fraction of spatial cells / channels with gate > threshold, mean spatial / channel importance, and the *effective selected ratio* = (fraction of selected cells) × (fraction of selected channels), averaged over the batch. These numbers are logged during validation, stored in the ablation results and shown in the dashboard.

models/feature_selection/losses.py

Training signal for the selection gates (`L_feature_selection` in the total loss). (39 lines)

```
"""Feature-selection losses.

``L_feature_selection = w_cov · L_coverage + w_sp · L_sparsity + w_bin · L_binarization``

* **Coverage** (foreground guidance): at object centres (class-agnostic Gaussian
  heatmap ``y``) at least one sensor must keep its features:
  ``L_cov = Σ y · −log(max_m s_m) / Σ y``. Only the positive term is used, so the loss
  never forces background to zero — that is the job of the sparsity term.
* **Sparsity**: ``Σ_m |mean(s_m) − ρ_s| + |mean(c_m) − ρ_c|`` — keep only a target ratio of
  spatial cells / channels, forcing the network to choose.
* **Binarisation**: ``mean(4 g (1 − g))`` pushes gates towards 0/1 so the selection is
  crisp and interpretable.
"""
from __future__ import annotations

from typing import Any

import torch
```

Docstring with the three terms and their rationale.

```
def feature_selection_loss(info: dict[str, Any], fg_heatmap: torch.Tensor, cfg_weights: Any,
                           target_keep: Any) -> dict[str, torch.Tensor]:
    spatial = torch.cat([info["spatial_gates"][m].float() for m in info["spatial_gates"]], 1)  # (B, M, H, W)
    combined = spatial.amax(1, keepdim=True).clamp(1e-4, 1.0)
    y = fg_heatmap.float()
    coverage = (y * -torch.log(combined)).sum() / y.sum().clamp(min=1.0)

    rho_s, rho_c = float(target_keep.spatial), float(target_keep.channel)
    sparsity = spatial.new_zeros(())
    gates = []
    for m in info["spatial_gates"]:
        s, c = info["spatial_gates"][m].float(), info["channel_gates"][m].float()
        sparsity = sparsity + (s.mean() - rho_s).abs() + (c.mean() - rho_c).abs()
        gates += [s.flatten(), c.flatten()]
    all_gates = torch.cat(gates)
    binarization = (4.0 * all_gates * (1.0 - all_gates)).mean()
    total = (float(cfg_weights.coverage) * coverage + float(cfg_weights.sparsity) * sparsity
             + float(cfg_weights.binarization) * binarization)
    return {"fs_coverage": coverage, "fs_sparsity": sparsity, "fs_binarization": binarization, "feature_selection
    ↪       ": total}
```

Implementation:

- `spatial` stacks the spatial gates of all selectable sensors → `(B, M, H, W)`; `combined = max over sensors` (clamped to ≥ 1e-4 for the log).
- **coverage** `Σ y · (−log combined) / Σ y` with `y = fg_heatmap (B, 1, H, W)` (class-agnostic Gaussian object-centre heatmap built by the dataset). Only positives are penalised: near objects, at least one sensor must keep its features; background is left to the sparsity term.
- **sparsity** `Σ_m |mean(s_m) − ρ_s| + |mean(c_m) − ρ_c|` with `ρ_s = 0.35`, `ρ_c = 0.75` (`fusion.selection.target_keep_ratio`): forces the network to keep only a target fraction.
- **binarization** `mean(4g(1 − g))` over all gates: 0 when every gate is 0 or 1, 1 when all are 0.5 → pushes towards crisp, interpretable decisions.
- `total = w_cov·coverage + w_sp·sparsity + w_bin·binarization` with weights from `loss.feature_selection` (1.0 / 0.5 / 0.1). All components are returned for logging.

models/fusion/fusion_methods.py

The interchangeable fusion modules. Both take a dict of BEV maps and return `(fused, info)`. (56 lines)

```
"""BEV fusion methods.

All methods take ``{modality: (B, C_m, H, W)}`` and return ``(B, C_out, H, W)`` plus an
info dict, so they are interchangeable (see :mod:`models.fusion.registry`).

* :class:`ConcatFusion` — baseline: concatenate -> conv (Experiment C).
* :class:`AttentionGatedFusion` — per-cell softmax attention over modalities (which
  sensor to trust *where*), weighted sum + residual mixing + CBAM refinement
  (Experiments D-F). Combined with feature selection it forms the proposed method.
"""
from __future__ import annotations

from typing import Any

import torch
from torch import nn

from models.common.blocks import CBAM, ConvBNReLU, ResBlock
```

Docstring and imports (CBAM, ConvBNReLU, ResBlock).

```
class ConcatFusion(nn.Module):
    """``concat(F_m) -> 3x3 conv -> residual block``."""

    def __init__(self, in_channels: dict[str, int], out_channels: int) -> None:
        super().__init__()
        self.modalities = list(in_channels)
        self.fuse = nn.Sequential(ConvBNReLU(sum(in_channels.values()), out_channels, 3), ResBlock(out_channels))

    def forward(self, feats: dict[str, torch.Tensor]) -> tuple[torch.Tensor, dict[str, Any]]:
        return self.fuse(torch.cat([feats[m] for m in self.modalities], 1)), {}
```

`ConcatFusion` (baseline, experiments A–C): concatenate all inputs along the channel axis (`Σ C_m` channels, e.g. 128 for LiDAR + radar), 3×3 ConvBNReLU to `out_channels` (128) and one ResBlock. `info` is empty.

```
class AttentionGatedFusion(nn.Module):
    """Gated attention fusion.

    1. Project each modality to a common space: ``P_m = φ_m(F_m)``.
    2. Per-cell modality attention ``a = softmax_m(g([F_1..F_M]))`` ∈ Δ^M.
    3. ``F_w = Σ_m a_m · P_m`` (sensor reliability weighting).
    4. ``F = CBAM(ResBlock(conv1x1([F_w, P_1..P_M])))`` keeps complementary detail.
    """

    def __init__(self, in_channels: dict[str, int], out_channels: int) -> None:
        super().__init__()
        self.modalities = list(in_channels)
        m = len(self.modalities)
        self.proj = nn.ModuleDict({k: ConvBNReLU(c, out_channels, 1) for k, c in in_channels.items()})
        self.attention = nn.Sequential(ConvBNReLU(sum(in_channels.values()), 64, 3), nn.Conv2d(64, m, 1))
        self.mix = nn.Sequential(ConvBNReLU(out_channels * (m + 1), out_channels, 1), ResBlock(out_channels),
                                 CBAM(out_channels))
```

`AttentionGatedFusion` (experiments D–I).

- `proj[m]`: 1×1 ConvBNReLU projecting each sensor to the common space `(B, 128, H, W)` (`P_m`).
- `attention`: 3×3 ConvBNReLU over the concatenated raw inputs → 1×1 conv → `M` logits per cell.
- `mix`: 1×1 ConvBNReLU over `128·(M+1)` channels → ResBlock → CBAM.

```
    def forward(self, feats: dict[str, torch.Tensor]) -> tuple[torch.Tensor, dict[str, Any]]:
        projected = [self.proj[k](feats[k]) for k in self.modalities]
        weights = torch.softmax(self.attention(torch.cat([feats[k] for k in self.modalities], 1)).float(), dim=1)
        weighted = sum(weights[:, i:i + 1].to(p.dtype) * p for i, p in enumerate(projected))
        fused = self.mix(torch.cat([weighted] + projected, 1))
        return fused, {"modality_weights": weights, "modality_order": self.modalities}
```

Forward: `weights = softmax_m(attention(...)) (B, M, H, W)` — a per-cell probability over sensors (computed in fp32); `weighted = Σ_m a_m · P_m`; the mixer sees `[weighted, P_1, …, P_M]` so it can still use complementary information that the attention weighting would average away. Returns the fused map `(B, 128, H, W)` and `info = {modality_weights, modality_order}` (visualised as the "fusion attention" RGB map in the dashboard).

models/fusion/registry.py

A tiny registry that decouples the fusion choice (`fusion.method` in YAML) from the model code. (30 lines)

```python
"""Fusion-method registry: add a new fusion by decorating a class with ``@register_fusion("name")``.

A fusion class must accept ``(in_channels: dict[str, int], out_channels: int)`` and its
``forward(feats: dict[str, Tensor])`` must return ``(fused (B, C_out, H, W), info dict)``.
"""
from __future__ import annotations

from typing import Callable

from torch import nn

from models.fusion.fusion_methods import AttentionGatedFusion, ConcatFusion
```

Docstring: the contract a fusion class must satisfy.

```python
FUSION_REGISTRY: dict[str, type[nn.Module]] = {
    "concat": ConcatFusion,
    "attention_gated": AttentionGatedFusion,
}


def register_fusion(name: str) -> Callable[[type[nn.Module]], type[nn.Module]]:
    def deco(cls: type[nn.Module]) -> type[nn.Module]:
        FUSION_REGISTRY[name] = cls
        return cls
    return deco


def build_fusion(method: str, in_channels: dict[str, int], out_channels: int) -> nn.Module:
    if method not in FUSION_REGISTRY:
        raise KeyError(f"Unknown fusion method {method!r}; available: {sorted(FUSION_REGISTRY)}")
    return FUSION_REGISTRY[method](in_channels, out_channels)
```

`FUSION_REGISTRY` maps names to classes. `register_fusion(name)` is a decorator to add new methods from any module; `build_fusion(method, in_channels, out_channels)` instantiates the configured class or raises `KeyError` listing the available names.

models/fusion/bev_backbone.py

SECOND-FPN 2-D backbone on the fused BEV map (shared by every experiment). (47 lines)

```python
"""SECOND-FPN BEV backbone on the fused BEV map.

    fused (B, 128, 200, 200)
      ├─ block1: stride 2, 3 convs ....... (B, 128, 100, 100) ─ deblock ×1 ─┐
      └─ block2: stride 2, 5 convs ....... (B, 256,  50,  50) ─ deblock ×2 ─┤
                                                                  concat (B, 256, 100, 100)
"""
from __future__ import annotations

from typing import Any

import torch
from torch import nn
from torch.utils.checkpoint import checkpoint

from models.common.blocks import ConvBNReLU
```

Docstring with the default shapes: `(B, 128, 200, 200)` → block1 `(B, 128, 100, 100)` → block2 `(B, 256, 50, 50)`; deblocks bring both to 100 × 100 and the outputs are concatenated → `(B, 256, 100, 100)`.

```python
class SECONDFPN(nn.Module):
    def __init__(self, in_channels: int, cfg: Any) -> None:
        super().__init__()
        self.blocks = nn.ModuleList()
        self.deblocks = nn.ModuleList()
        c_in = in_channels
        for n, stride, c_out, up, c_up in zip(cfg.layer_nums, cfg.layer_strides, cfg.num_filters,
                                              cfg.upsample_strides, cfg.num_upsample_filters):
            layers = [ConvBNReLU(c_in, c_out, 3, stride=stride)] + [ConvBNReLU(c_out, c_out, 3) for _ in range(n)
                ]
            self.blocks.append(nn.Sequential(*layers))
            if up == 1:
                self.deblocks.append(ConvBNReLU(c_out, c_up, 1))
            else:
                self.deblocks.append(nn.Sequential(nn.ConvTranspose2d(c_out, c_up, up, stride=up, bias=False),
                                                   nn.BatchNorm2d(c_up), nn.ReLU(inplace=True)))
            c_in = c_out
```

```
        self.out_channels = int(sum(cfg.num_upsample_filters))
        self.out_stride = int(cfg.layer_strides[0])
        self.grad_checkpoint = False
```

Constructor iterates over the zipped config lists (`layer_nums [3, 5]`, `layer_strides [2, 2]`, `num_filters [128, 256]`, `upsample_strides [1, 2]`, `num_upsample_filters [128, 128]`): each block = stride-s ConvBNReLU + n ConvBNReLU; each deblock = 1×1 ConvBNReLU (upsample 1) or ConvTranspose2d (upsample 2). `out_channels = Σ num_upsample_filters = 256`; `out_stride = first stride = 2` (checked against `detection_head.out_stride` in the model).

```
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        outs = []
        for block, deblock in zip(self.blocks, self.deblocks):
            if self.grad_checkpoint and self.training:
                x = checkpoint(block, x, use_reentrant=False)
            else:
                x = block(x)
            outs.append(deblock(x))
        return torch.cat(outs, 1)
```

Runs the blocks sequentially (optionally through gradient checkpointing), applies each deblock to the corresponding block output and concatenates along channels.

`models/detection/targets.py`

CenterPoint training targets, generated with NumPy inside the DataLoader workers (`NuScenesFusionDataset.__getitem__`). (95 lines)

```
"""CenterPoint training targets (NumPy, generated inside the dataset workers).

For each GT box the class heatmap receives a 2D Gaussian at the box centre; regression
targets are stored at the integer centre cell:
``[dx, dy, z, log l, log w, log h, sin yaw, cos yaw, vx, vy]``.
"""
from __future__ import annotations

import numpy as np

from utils.bev_grid import BEVGrid
```

Docstring: what each regression channel means.

```
def gaussian_radius(height: float, width: float, min_overlap: float = 0.1) -> float:
    """CornerNet/CenterPoint Gaussian radius such that a shifted box keeps ``min_overlap`` IoU."""
    a1, b1 = 1, height + width
    c1 = width * height * (1 - min_overlap) / (1 + min_overlap)
    r1 = (b1 + np.sqrt(b1**2 - 4 * a1 * c1)) / 2
    a2, b2 = 4, 2 * (height + width)
    c2 = (1 - min_overlap) * width * height
    r2 = (b2 + np.sqrt(b2**2 - 4 * a2 * c2)) / 2
    a3, b3 = 4 * min_overlap, -2 * min_overlap * (height + width)
    c3 = (min_overlap - 1) * width * height
    r3 = (b3 + np.sqrt(b3**2 - 4 * a3 * c3)) / 2
    return float(min(r1, r2, r3))
```

`gaussian_radius(height, width, min_overlap)` — the CornerNet/CenterPoint formula: solves three quadratic equations for the largest centre shift that still keeps an IoU of `min_overlap` (0.1) with the GT box, and returns the minimum radius (in cells).

```
def draw_gaussian(heatmap: np.ndarray, center: tuple[int, int], radius: int) -> None:
    """Draw (max-merge) a Gaussian of ``radius`` cells at ``center = (ix, iy)`` into ``heatmap[iy, ix]``."""
    diameter = 2 * radius + 1
    sigma = diameter / 6
    ys, xs = np.ogrid[-radius:radius + 1, -radius:radius + 1]
    gaussian = np.exp(-(xs * xs + ys * ys) / (2 * sigma * sigma))
    x, y = center
    h, w = heatmap.shape
    left, right = min(x, radius), min(w - x, radius + 1)
    top, bottom = min(y, radius), min(h - y, radius + 1)
    if left + right <= 0 or top + bottom <= 0:
        return
    region = heatmap[y - top:y + bottom, x - left:x + right]
    g = gaussian[radius - top:radius + bottom, radius - left:radius + right]
    np.maximum(region, g, out=region)
```

`draw_gaussian(heatmap, center, radius)` renders a 2-D Gaussian (σ = diameter / 6) with peak 1.0 at `center = (ix, iy)` into `heatmap[iy, ix]` with element-wise maximum (overlapping objects keep the larger value). Clipping handles boxes near the grid border.

```python
def build_center_targets(boxes: np.ndarray, labels: np.ndarray, velocity_valid: np.ndarray, grid: BEVGrid,
                         num_classes: int, max_objects: int = 500, min_radius: int = 2,
                         gaussian_overlap: float = 0.1) -> dict[str, np.ndarray]:
    """Targets on the detection-head grid.

    Args:
        boxes: ``(M, 9)`` reference-frame boxes ``[x, y, z, l, w, h, yaw, vx, vy]``.
        labels: ``(M,)`` class indices.
        velocity_valid: ``(M,)`` whether the GT velocity is defined (single-frame tracks are not).
        grid: the head grid (BEV grid downsampled by the head stride).
    """
    ny, nx = grid.shape
    heatmap = np.zeros((num_classes, ny, nx), dtype=np.float32)
    ind = np.zeros(max_objects, dtype=np.int64)
    mask = np.zeros(max_objects, dtype=np.float32)
    vel_mask = np.zeros(max_objects, dtype=np.float32)
    reg = np.zeros((max_objects, 10), dtype=np.float32)
    k = 0
    for box, label, vel_ok in zip(boxes, labels, velocity_valid):
        if k >= max_objects:
            break
        cx, cy = grid.world_to_grid(box[0], box[1])
        ix, iy = int(np.floor(cx)), int(np.floor(cy))
        if not (0 <= ix < nx and 0 <= iy < ny):
            continue
        length, width = box[3] / grid.resolution, box[4] / grid.resolution
        if length <= 0 or width <= 0:
            continue
        radius = max(min_radius, int(gaussian_radius(length, width, gaussian_overlap)))
        draw_gaussian(heatmap[label], (ix, iy), radius)
        ind[k] = iy * nx + ix
        mask[k] = 1.0
        vel_mask[k] = float(vel_ok)
        reg[k] = [cx - ix, cy - iy, box[2], np.log(max(box[3], 1e-3)), np.log(max(box[4], 1e-3)),
                  np.log(max(box[5], 1e-3)), np.sin(box[6]), np.cos(box[6]), box[7], box[8]]
        k += 1
    return {"heatmap": heatmap, "ind": ind, "mask": mask, "vel_mask": vel_mask, "reg_target": reg}
```

`build_center_targets(boxes (M, 9), labels, velocity_valid, grid, ...)` on the **head grid** (100 × 100, 1 m cells):

- `heatmap (K=10, ny, nx)`: one Gaussian per object in its class channel; radius from the box length/width in cells, at least `min_radius` (2);
- `ind (max_objects,)`: flat index `iy·nx + ix` of each object's centre cell;
- `mask`: 1 for real objects; `vel_mask`: 1 if the annotation velocity is defined;
- `reg (max_objects, 10)`: `[dx, dy, z, log l, log w, log h, sin yaw, cos yaw, vx, vy]` where `dx, dy` are the sub-cell offsets of the true centre.

Objects outside the grid or with non-positive size are skipped; at most `max_objects` (500).

```python
def foreground_heatmap(boxes: np.ndarray, grid: BEVGrid, min_radius: int = 2, gaussian_overlap: float = 0.1) -> n
    p.ndarray:
    """Class-agnostic centre heatmap ``(1, ny, nx)`` at BEV resolution (feature-selection guidance)."""
    ny, nx = grid.shape
    heatmap = np.zeros((ny, nx), dtype=np.float32)
    for box in boxes:
        cx, cy = grid.world_to_grid(box[0], box[1])
        ix, iy = int(np.floor(cx)), int(np.floor(cy))
        if not (0 <= ix < nx and 0 <= iy < ny):
            continue
        radius = max(min_radius, int(gaussian_radius(box[3] / grid.resolution, box[4] / grid.resolution, gaussian
            _overlap)))
        draw_gaussian(heatmap, (ix, iy), radius)
    return heatmap[None]
```

`foreground_heatmap(boxes, grid)` → `(1, ny, nx)` class-agnostic Gaussians at the **BEV** resolution (0.5 m, 200 × 200). Used by the feature-selection coverage loss and the fusion deep-supervision loss.

models/detection/center_head.py

Detection head and its losses. (93 lines)

```
"""CenterPoint-style 3D detection head on the BEV feature map.

Input ``(B, 256, 100, 100)`` -> per-cell predictions:

=========  ======  ====================================================
branch     ch      meaning
=========  ======  ====================================================
```

```
heatmap    10      class centre likelihood (logits)
reg        2       sub-cell centre offset (dx, dy)
height     1       box-centre z (m)
dim        3       log(l), log(w), log(h)
rot        2       sin(yaw), cos(yaw)
vel        2       vx, vy (m/s, reference frame)
=========  ======  ==================================================

Loss = Gaussian focal loss (classification) + L1 on centre / size / orientation /
velocity at GT centre cells.
"""
from __future__ import annotations

from typing import Any

import torch
import torch.nn.functional as F
from torch import nn

from models.common.blocks import ConvBNReLU

HEAD_BRANCHES = {"reg": 2, "height": 1, "dim": 3, "rot": 2, "vel": 2}
```

Docstring: output branches and the loss summary. `HEAD_BRANCHES` lists the regression branches and their channel counts.

```
class CenterHead(nn.Module):
    def __init__(self, in_channels: int, num_classes: int, head_channels: int = 64) -> None:
        super().__init__()
        self.num_classes = num_classes
        self.shared = ConvBNReLU(in_channels, head_channels, 3)
        self.heads = nn.ModuleDict()
        for name, out in {"heatmap": num_classes, **HEAD_BRANCHES}.items():
            self.heads[name] = nn.Sequential(ConvBNReLU(head_channels, head_channels, 3), nn.Conv2d(head_channels
, out, 1))
        nn.init.constant_(self.heads["heatmap"][-1].bias, -2.19)  # prior probability 0.1

    def forward(self, x: torch.Tensor) -> dict[str, torch.Tensor]:
        x = self.shared(x)
        return {name: head(x) for name, head in self.heads.items()}
```

`CenterHead`: a shared 3×3 ConvBNReLU (256 → 64) followed by one small head per branch (3×3 ConvBNReLU + 1×1 conv). The heatmap bias is initialised to −2.19 = logit(0.1) so the initial predicted object probability is 0.1 everywhere (CenterNet trick; prevents a huge initial focal loss). `forward` returns a dict of `(B, ch, 100, 100)` maps.

```
def gaussian_focal_loss(pred: torch.Tensor, target: torch.Tensor, alpha: float = 2.0, gamma: float = 4.0) -> tupl
e[torch.Tensor, torch.Tensor]:
    """CornerNet focal loss on probabilities; returns (summed loss, number of positives)."""
    pred = pred.float().clamp(1e-4, 1 - 1e-4)
    target = target.float()
    pos = target.eq(1).float()
    neg = 1.0 - pos
    pos_loss = -torch.log(pred) * (1 - pred).pow(alpha) * pos
    neg_loss = -torch.log(1 - pred) * pred.pow(alpha) * (1 - target).pow(gamma) * neg
    return pos_loss.sum() + neg_loss.sum(), pos.sum()
```

`gaussian_focal_loss(pred, target)` — CornerNet focal loss on probabilities. Positives are cells where the target equals exactly 1 (Gaussian peaks): `−log(p)·(1 − p)^α`. Negatives: `−log(1 − p)·p^α·(1 − y)^γ` — the `(1 − y)^γ` term (γ = 4) reduces the penalty near object centres. Returns the summed loss and the number of positives (for normalisation).

```
def _gather(feat: torch.Tensor, ind: torch.Tensor) -> torch.Tensor:
    """``feat (B, C, H, W)``, ``ind (B, K)`` flat indices -> ``(B, K, C)``."""
    b, c = feat.shape[:2]
    feat = feat.view(b, c, -1).permute(0, 2, 1)
    return feat.gather(1, ind.unsqueeze(-1).expand(-1, -1, c))
```

`_gather(feat (B, C, H, W), ind (B, K))` → `(B, K, C)`: picks the predictions at the object-centre cells using `gather` on the flattened spatial dimension.

```
def detection_loss(preds: dict[str, torch.Tensor], batch: dict[str, Any], weights: Any,
                   code_weights: list[float]) -> dict[str, torch.Tensor]:
    """CenterPoint losses. ``weights`` holds heatmap/center/bbox/orientation/velocity factors."""
    hm_sum, num_pos = gaussian_focal_loss(torch.sigmoid(preds["heatmap"].float()), batch["heatmap"])
    loss_cls = hm_sum / num_pos.clamp(min=1.0)

    ind, mask, vel_mask = batch["ind"], batch["mask"].float(), batch["vel_mask"].float()
    target = batch["reg_target"].float()
    pred = torch.cat([_gather(preds[k].float(), ind) for k in ("reg", "height", "dim", "rot", "vel")], -1)  # (B,
```

```
        K, 10)
    cw = pred.new_tensor(code_weights)
    l1 = (pred - target).abs() * cw
    norm = mask.sum().clamp(min=1.0)
    m = mask.unsqueeze(-1)
    loss_center = (l1[..., 0:3] * m).sum() / norm
    loss_bbox = (l1[..., 3:6] * m).sum() / norm
    loss_rot = (l1[..., 6:8] * m).sum() / norm
    loss_vel = (l1[..., 8:10] * vel_mask.unsqueeze(-1)).sum() / vel_mask.sum().clamp(min=1.0)
    total = (float(weights.heatmap) * loss_cls + float(weights.center) * loss_center + float(weights.bbox) * loss_bbox
             + float(weights.orientation) * loss_rot + float(weights.velocity) * loss_vel)
    return {"det_cls": loss_cls, "det_center": loss_center, "det_bbox": loss_bbox, "det_orientation": loss_rot,
            "det_velocity": loss_vel, "detection": total}
```

`detection_loss()`:

- classification: focal loss on `sigmoid(heatmap)` normalised by the number of positives;
- regression: predictions gathered at `ind` and concatenated in the same order as the targets `(B, K, 10)`; L1 error weighted by `code_weights` (velocity 0.2);
- split into `center` (dx, dy, z), `bbox` (log sizes), `orientation` (sin, cos) and `velocity` (masked by `vel_mask`), each normalised by the number of objects;
- `detection` = weighted sum with `loss.detection` weights (1.0 heatmap, 0.25 for the others).

```
def aux_heatmap_loss(logits: torch.Tensor, target: torch.Tensor) -> torch.Tensor:
    """Class-agnostic deep-supervision heatmap loss (resizes the target if needed)."""
    if logits.shape[-2:] != target.shape[-2:]:
        target = F.adaptive_max_pool2d(target, logits.shape[-2:])
    loss, num_pos = gaussian_focal_loss(torch.sigmoid(logits.float()), target)
    return loss / num_pos.clamp(min=1.0)
```

`aux_heatmap_loss(logits, target)` — class-agnostic heatmap loss used for fusion deep supervision; adaptive max-pooling resizes the target if the prediction has a different resolution.

models/detection/decoder.py

Converts head outputs to metric 3-D boxes in the reference frame. (69 lines)

```
"""Decode CenterHead outputs into 3D boxes ``[x, y, z, l, w, h, yaw, vx, vy]`` + score + label."""
from __future__ import annotations

from typing import Optional

import numpy as np
import torch
import torch.nn.functional as F

from utils.bev_grid import BEVGrid
```

Imports; NumPy is used for the greedy NMS loop on the CPU.

```
def circle_nms(centers: torch.Tensor, scores: torch.Tensor, radius: float) -> torch.Tensor:
    """Greedy distance-based NMS (CenterPoint). Returns kept indices (sorted by score)."""
    order = scores.argsort(descending=True)
    if radius <= 0 or order.numel() <= 1:
        return order
    c = centers[order].detach().cpu().double()
    close = (torch.cdist(c, c) < radius).numpy()  # greedy loop on CPU: no per-step GPU sync
    suppressed = np.zeros(order.numel(), dtype=bool)
    keep = []
    for i in range(order.numel()):
        if suppressed[i]:
            continue
        keep.append(i)
        suppressed |= close[i]
    return order[torch.as_tensor(keep, device=order.device, dtype=torch.long)]
```

`circle_nms(centers, scores, radius)` — CenterPoint's distance-based NMS: sort by score, compute all pairwise centre distances once (`torch.cdist` on CPU), then greedily keep a box and suppress every other box within `radius` metres. Running the loop on the CPU avoids one GPU synchronisation per iteration.

```
@torch.no_grad()
def decode_detections(preds: dict[str, torch.Tensor], grid: BEVGrid, classes: list[str], score_threshold: float = 0.1,
                      max_detections: int = 300, nms_radius: Optional[dict[str, float]] = None) -> list[dict[str, torch.Tensor]]:
    """Per-sample detections: ``{"boxes": (K, 9), "scores": (K,), "labels": (K,)}`` in the reference frame."""
    heat = torch.sigmoid(preds["heatmap"].float())
    b, c, h, w = heat.shape
```

```
    peak = F.max_pool2d(heat, 3, stride=1, padding=1)
    heat = heat * (heat == peak)
    k = min(max_detections, c * h * w)
    scores, idx = heat.view(b, -1).topk(k)
    labels = idx // (h * w)
    pix = idx % (h * w)
    iy, ix = (pix // w).float(), (pix % w).float()

    def gather(name: str) -> torch.Tensor:
        t = preds[name].float()
        return t.view(b, t.shape[1], -1).gather(2, pix.unsqueeze(1).expand(-1, t.shape[1], -1)).permute(0, 2, 1)

    reg, height, dim, rot, vel = gather("reg"), gather("height"), gather("dim"), gather("rot"), gather("vel")
    x = (ix + reg[..., 0]) * grid.resolution + grid.x_range[0]
    y = (iy + reg[..., 1]) * grid.resolution + grid.y_range[0]
    dims = dim.clamp(max=4.0).exp()  # log-size clamp avoids exp overflow from an untrained head
    yaw = torch.atan2(rot[..., 0], rot[..., 1])
    boxes = torch.cat([x.unsqueeze(-1), y.unsqueeze(-1), height, dims, yaw.unsqueeze(-1), vel], -1)

    results = []
    for i in range(b):
        keep = scores[i] > score_threshold
        bx, sc, lb = boxes[i][keep], scores[i][keep], labels[i][keep]
        if nms_radius and bx.shape[0]:
            kept = []
            for cls_id in lb.unique():
                sel = torch.nonzero(lb == cls_id).squeeze(1)
                r = float(nms_radius.get(classes[int(cls_id)], 0.0))
                kept.append(sel[circle_nms(bx[sel, :2], sc[sel], r)])
            order = torch.cat(kept)
            order = order[sc[order].argsort(descending=True)]
            bx, sc, lb = bx[order], sc[order], lb[order]
        results.append({"boxes": bx, "scores": sc, "labels": lb})
    return results
```

`decode_detections()`:

1. `heat = sigmoid(heatmap)`; a 3×3 max-pool keeps only local peaks (`heat == peak`) — the CenterNet replacement for NMS.
2. `topk(max_detections)` over the flattened `C·H·W` scores → `labels = idx // (H·W)`, `pix = idx % (H·W)`, `iy, ix`.
3. `gather` reads the regression branches at those pixels.
4. Metric box: `x = (ix + dx)·r + x_min`, `y = (iy + dy)·r + y_min`, `z`, `l, w, h = exp(log size)` (log-size clamped at 4 → at most $e^4 \approx 55$ m, protects against an untrained head), `yaw = atan2(sin, cos)`, `vx, vy`.
5. Per sample: score threshold, then class-wise circle NMS with radii from `model.detection_head.nms_radius`; results sorted by score.

Returns a list (one per sample) of `{boxes (K, 9), scores (K,), labels (K,)}`.

models/trajectory/planning_head.py

The learned planner (Stage 6) — predicts six future ego positions from the BEV features. (59 lines)

```
"""Learned trajectory planner (replaces / complements the classical baseline planner).

    BEV features (B, 256, 100, 100) + coord channels
      └ 3 × stride-2 conv ............................... (B, 128, 13, 13) -> 169 tokens
    ego query = MLP([speed, yaw_rate, accel, one_hot(command)])   (B, 1, 128)
      └ multi-head cross-attention (query -> BEV tokens) ..... context (B, 128)
          └ GRU decoder, T steps: residual waypoint deltas
    waypoints_t = constant-velocity prior_t + Σ residuals ....... (B, T, 2)

Predicting a residual on top of the constant-velocity prior makes the head trainable on
the small nuScenes-mini split. The attention weights show *where* the planner looked.
"""
from __future__ import annotations

import torch
import torch.nn.functional as F
from torch import nn

from models.common.blocks import ConvBNReLU
```

Docstring with the architecture and the reason for residual prediction on top of a constant-velocity prior (trainable on mini).

```
class PlanningHead(nn.Module):
    def __init__(self, in_channels: int, hidden: int = 128, horizon: int = 6, dt: float = 0.5, num_heads: int = 4
```

```
        ,
                num_commands: int = 3, ego_state_dim: int = 3) -> None:
        super().__init__()
        self.horizon, self.dt, self.num_commands = horizon, dt, num_commands
        self.reduce = nn.Sequential(ConvBNReLU(in_channels + 2, hidden, 3, stride=2), ConvBNReLU(hidden, hidden,
            3, stride=2),
                                    ConvBNReLU(hidden, hidden, 3, stride=2))
        self.query = nn.Sequential(nn.Linear(ego_state_dim + num_commands, hidden), nn.ReLU(inplace=True), nn.Lin
            ear(hidden, hidden))
        self.attn = nn.MultiheadAttention(hidden, num_heads, batch_first=True)
        self.gru = nn.GRUCell(hidden + 2, hidden)
        self.out = nn.Linear(hidden, 2)
        nn.init.zeros_(self.out.weight)
        nn.init.zeros_(self.out.bias)
        self.register_buffer("state_scale", torch.tensor([10.0, 0.5, 2.0]), persistent=False)
```

Constructor:

- `reduce`: three stride-2 ConvBNReLU layers on `[BEV features, x-coord, y-coord]` (two coordinate channels give the attention a sense of position): 100 → 50 → 25 → 13, so 169 tokens of 128 channels;
- `query`: MLP over `[speed, yaw rate, accel] / state_scale` + one-hot command (3);
- `attn`: 4-head cross-attention (query → BEV tokens);
- `gru`: GRUCell over `[context, current residual]`;
- `out`: Linear(128 → 2) **initialised to zero**, so an untrained head outputs exactly the constant-velocity prior.

```
    def forward(self, bev: torch.Tensor, ego_state: torch.Tensor, command: torch.Tensor) -> dict[str, torch.Tenso
        r]:
        b, _, h, w = bev.shape
        ys = torch.linspace(-1, 1, h, device=bev.device, dtype=bev.dtype).view(1, 1, h, 1).expand(b, 1, h, w)
        xs = torch.linspace(-1, 1, w, device=bev.device, dtype=bev.dtype).view(1, 1, 1, w).expand(b, 1, h, w)
        tokens = self.reduce(torch.cat([bev, xs, ys], 1))
        tokens = tokens.flatten(2).transpose(1, 2).float()                      # (B, L, hidden)
        q_in = torch.cat([ego_state.float() / self.state_scale, F.one_hot(command.long(), self.num_commands).floa
            t()], 1)
        query = self.query(q_in).unsqueeze(1)                                    # (B, 1, hidden)
        ctx, attn_w = self.attn(query, tokens, tokens, need_weights=True)       # (B, 1, hidden), (B, 1, L)
        ctx = ctx.squeeze(1)
        speed = ego_state[:, 0].float().clamp(min=0.0)
        t = torch.arange(1, self.horizon + 1, device=bev.device, dtype=torch.float32) * self.dt
        prior = torch.stack([speed[:, None] * t[None], torch.zeros_like(speed[:, None] * t[None])], -1)  # (B, T,
             2)
        hidden = ctx
        residual = torch.zeros(b, 2, device=bev.device)
        outs = []
        for _ in range(self.horizon):
            hidden = self.gru(torch.cat([ctx, residual], 1), hidden)
            residual = residual + self.out(hidden)
            outs.append(residual)
        waypoints = prior + torch.stack(outs, 1)
        side = int(round(tokens.shape[1] ** 0.5))
        return {"waypoints": waypoints, "prior": prior, "attention": attn_w.view(b, side, -1) if side * side == t
            okens.shape[1] else attn_w}
```

Forward:

1. Normalised coordinate channels are concatenated to the BEV map; `reduce` → tokens `(B, L=169, 128)`.
2. The ego query `(B, 1, 128)` attends over the tokens → context `(B, 128)` and attention weights `(B, 1, 169)` (reshaped to 13 × 13 for visualisation).
3. Prior: `prior_t = (speed · t·dt, 0)` for t = 1…6 (speed clamped ≥ 0).
4. The GRU unrolls six steps; each step adds a predicted delta to the running residual.
5. `waypoints = prior + residuals (B, 6, 2)` in the reference frame (x forward, y left).

`models/trajectory/losses.py`

Losses of the learned planner (`L_trajectory`). (74 lines)

```
"""Trajectory-planning losses.

``L_trajectory = w_reg · L1(pred, logged) + w_col · collision + w_smooth · smoothness + w_goal · goal``

* **Collision penalty**: the GT future occupancy of other agents ``(B, T, H, W)`` is
  softened (3×3 average) and sampled with ``grid_sample`` at 5 points of the ego
  footprint (rear, centre, front, left, right) placed at each predicted waypoint with the
  heading of the trajectory — differentiable w.r.t. the waypoints.
* **Smoothness**: squared second differences including the current pose (and the
```

```
  current velocity), penalising jerky trajectories.
* **Goal / progress**: Euclidean error of the final valid waypoint.
"""
from __future__ import annotations

from typing import Any

import torch
import torch.nn.functional as F

from utils.bev_grid import BEVGrid
```

Docstring describing the four terms.

```
def ego_footprint_points(waypoints: torch.Tensor, ego_cfg: Any) -> torch.Tensor:
    """Sample points on the ego body ``(B, T, 5, 2)`` for waypoints ``(B, T, 2)`` (rear-axle positions)."""
    prev = torch.cat([torch.zeros_like(waypoints[:, :1]), waypoints[:, :-1]], 1)
    delta = waypoints - prev
    heading = torch.atan2(delta[..., 1], delta[..., 0] + 1e-6)
    still = delta.norm(dim=-1) < 0.05
    heading = torch.where(still, torch.zeros_like(heading), heading)
    fwd = torch.stack([heading.cos(), heading.sin()], -1)
    left = torch.stack([-heading.sin(), heading.cos()], -1)
    l, w, c = float(ego_cfg.length), float(ego_cfg.width), float(ego_cfg.rear_axle_to_center)
    centre = waypoints + c * fwd
    offsets = [(-l / 2 + 0.3, 0.0), (0.0, 0.0), (l / 2 - 0.3, 0.0), (0.0, w / 2 - 0.2), (0.0, -w / 2 + 0.2)]
    return torch.stack([centre + a * fwd + b * left for a, b in offsets], 2)
```

`ego_footprint_points(waypoints (B, T, 2), ego_cfg)` → `(B, T, 5, 2)`: the heading of each step is the direction from the previous waypoint (0 when standing still); five points are placed on the ego body (rear, centre, front, left, right) using the vehicle length/width and the rear-axle-to-centre offset — waypoints are rear-axle positions.

```
def sample_occupancy(occ: torch.Tensor, points: torch.Tensor, grid: BEVGrid) -> torch.Tensor:
    """Bilinear sample ``occ (B, T, H, W)`` at metric ``points (B, T, P, 2)`` -> ``(B, T, P)``."""
    b, t, h, w = occ.shape
    gx = (points[..., 0] - grid.x_range[0]) / (grid.x_range[1] - grid.x_range[0]) * 2 - 1  # x -> width axis
    gy = (points[..., 1] - grid.y_range[0]) / (grid.y_range[1] - grid.y_range[0]) * 2 - 1  # y -> height axis
    coords = torch.stack([gx, gy], -1).reshape(b * t, 1, -1, 2)
    sampled = F.grid_sample(occ.reshape(b * t, 1, h, w), coords, align_corners=False, padding_mode="zeros")
    return sampled.view(b, t, -1)
```

`sample_occupancy(occ (B, T, H, W), points (B, T, P, 2), grid)` converts metric points to normalised grid coordinates (x → width axis, y → height axis, `align_corners=False`) and bilinearly samples the occupancy with `grid_sample`. Differentiable w.r.t. the points → the collision penalty pushes waypoints out of occupied cells.

```
def trajectory_loss(pred: torch.Tensor, batch: dict[str, Any], grid: BEVGrid, ego_cfg: Any, weights: Any,
                    dt: float) -> dict[str, torch.Tensor]:
    gt = batch["ego_future"].float()
    mask = batch["ego_future_mask"].float()
    pred = pred.float()
    valid = mask.sum().clamp(min=1.0)
    reg = ((pred - gt).abs().sum(-1) * mask).sum() / valid

    occ = F.avg_pool2d(batch["future_occ"].float(), 3, stride=1, padding=1)
    occ_at = sample_occupancy(occ, ego_footprint_points(pred, ego_cfg), grid).amax(-1)  # (B, T)
    collision = (occ_at * mask).sum() / valid

    speed = batch["ego_state"][:, 0].float()
    start = torch.stack([-speed * dt, torch.zeros_like(speed)], -1).unsqueeze(1)
    path = torch.cat([start, torch.zeros_like(start), pred], 1)
    smooth = (path[:, 2:] - 2 * path[:, 1:-1] + path[:, :-2]).pow(2).sum(-1).mean()

    last = (mask.cumsum(1) == mask.sum(1, keepdim=True)) & (mask > 0)
    has_last = last.any(1)
    if has_last.any():
        goal = ((pred - gt).norm(dim=-1) * last.float()).sum() / has_last.float().sum()
    else:
        goal = pred.sum() * 0.0
    total = (float(weights.regression) * reg + float(weights.collision) * collision
             + float(weights.smoothness) * smooth + float(weights.goal) * goal)
    return {"traj_regression": reg, "traj_collision": collision, "traj_smoothness": smooth, "traj_goal": goal,
            "trajectory": total}
```

`trajectory_loss()`:

- `reg`: L1 distance to the logged trajectory, masked where the future is unknown (end of a scene);
- `collision`: GT future occupancy of other agents, softened with a 3×3 average pool, sampled at the 5 footprint points; the max over points is averaged over valid steps;

- `smooth`: squared second differences of `[−v·dt, 0, waypoints]` (includes the current velocity);
- `goal`: Euclidean error of the last valid waypoint;
- `trajectory` = weighted sum (`loss.trajectory`: 1.0 / 2.0 / 0.1 / 0.5).

models/fusion_model.py

`MultiSensorFusionNet` wires all sub-networks together according to the configuration and implements the configurable total loss. Every training script, evaluator and the dashboard use this class. (203 lines)

```
"""MultiSensorFusionNet — the complete network.

::

    Camera → CameraBEVEncoder (LSS) ──────────────┐ F_cam (B,64,200,200) ── guide ──┐
    LiDAR  → LidarEncoder (dynamic pillars) ──────┤ F_lidar (B,64,200,200) ─┐       │
    Radar  → RadarEncoder (velocity pillars) ─────┤ F_radar (B,64,200,200) ─┴→ CrossModalFeatureSelection
                                                  │                              | F~_lidar, F~_radar
                                                  └──────────→ Fusion (concat | attention_gated)
                                                                    | fused BEV (B,128,200,200)
                                                               SECOND-FPN (B,256,100,100)
                                                                    ┴
                                                          ┌─────────┴──────────┐
                                                     CenterHead           PlanningHead
                                                   3D boxes + velocity    ego waypoints (B,T,2)

Total loss (weights in configs/training.yaml)::

    L = λ1 L_detection + λ2 L_trajectory + λ3 L_feature_selection + λ4 L_fusion + λ5 L_depth
"""
```

Docstring: ASCII architecture and the total-loss formula.

```
from __future__ import annotations

from typing import Any, Iterable, Optional

import torch
from torch import nn

from models.camera_bev.lss import CameraBEVEncoder
from models.detection.center_head import CenterHead, aux_heatmap_loss, detection_loss
from models.detection.decoder import decode_detections
from models.feature_selection.losses import feature_selection_loss
from models.feature_selection.selection import CrossModalFeatureSelection
from models.fusion.bev_backbone import SECONDFPN
from models.fusion.registry import build_fusion
from models.lidar_encoder.pillar_encoder import LidarEncoder
from models.radar_encoder.radar_encoder import RadarEncoder
from models.trajectory.losses import trajectory_loss
from models.trajectory.planning_head import PlanningHead
from utils.bev_grid import BEVGrid
from utils.config import Config
from utils.timer import NullTimer, StageTimer

ALL_LOSSES = ("detection", "trajectory", "feature_selection", "fusion", "depth")
```

Imports of every sub-module. `ALL_LOSSES` lists the loss groups a training stage can activate (`configs/training.yaml → stages.<name>.losses`).

```
class MultiSensorFusionNet(nn.Module):
    def __init__(self, cfg: Config) -> None:
        super().__init__()
        self.cfg = cfg
        f, m = cfg.fusion, cfg.model
        self.grid = BEVGrid.from_config(cfg.bev)
        self.classes = list(cfg.classes)
        self.use_camera, self.use_lidar, self.use_radar = bool(f.use_camera_bev), bool(f.use_lidar), bool(f.use_r
    ↪           adar)
        if not (self.use_camera or self.use_lidar or self.use_radar):
            raise ValueError("At least one modality must be enabled in fusion.use_*")

        channels: dict[str, int] = {}
        if self.use_camera:
            self.camera_bev = CameraBEVEncoder(m.camera_bev, self.grid, tuple(cfg.dataset.camera.image_size))
            channels["camera"] = self.camera_bev.out_channels
        if self.use_lidar:
            self.lidar_encoder = LidarEncoder(m.lidar_encoder, self.grid)
            channels["lidar"] = self.lidar_encoder.out_channels
        if self.use_radar:
            self.radar_encoder = RadarEncoder(m.radar_encoder, self.grid)
            channels["radar"] = self.radar_encoder.out_channels
        self.modalities = list(channels)
```

Constructor, part 1 — **sensor encoders**. Reads `fusion.use_camera_bev / use_lidar / use_radar` (set per experiment in `configs/ablation.yaml`). At least one must be on (`ValueError` otherwise). Each enabled encoder is created and its output channel count recorded in `channels` (insertion order camera, lidar, radar defines `self.modalities`). Disabled encoders are simply not created, so a LiDAR-only model has no camera parameters at all.

```
68          selectable = {k: c for k, c in channels.items() if k in ("lidar", "radar")}
69          self.use_feature_selection = bool(f.feature_selection) and len(selectable) > 0
70          if self.use_feature_selection:
71              guide = channels.get("camera") if (self.use_camera and f.selection.camera_guided) else None
72              self.feature_selection = CrossModalFeatureSelection(selectable, guide, f.selection)
73
74          self.fusion_method = str(f.method)
75          self.fusion = build_fusion(self.fusion_method, channels, int(f.out_channels))
76          self.deep_supervision = bool(f.deep_supervision)
77          if self.deep_supervision:
78              heads = {k: nn.Conv2d(c, 1, 1) for k, c in channels.items()}
79              heads["fused"] = nn.Conv2d(int(f.out_channels), 1, 1)
80              self.aux_heads = nn.ModuleDict(heads)
81              for h in self.aux_heads.values():
82                  nn.init.constant_(h.bias, -2.19)
```

Constructor, part 2 — **feature selection and fusion**:

- `selectable` = LiDAR and/or radar channels; feature selection is built only if `fusion.feature_selection` is true *and* at least one of them is enabled; the camera is used as guide if enabled and `selection.camera_guided`;
- `build_fusion(fusion.method, channels, 128)` creates the fusion module;
- `aux_heads`: a 1×1 conv per sensor + one for the fused map (deep supervision, bias −2.19).

```
84          self.bev_backbone = SECONDFPN(int(f.out_channels), m.bev_backbone)
85          hcfg = m.detection_head
86          self.head_grid = self.grid.downsample(int(hcfg.out_stride))
87          if self.bev_backbone.out_stride != int(hcfg.out_stride):
88              raise ValueError("detection_head.out_stride must equal bev_backbone.layer_strides[0]")
89          self.det_head = CenterHead(self.bev_backbone.out_channels, len(self.classes), int(hcfg.head_channels))
90          self.use_planning_head = bool(m.trajectory_head.enabled)
91          if self.use_planning_head:
92              self.plan_head = PlanningHead(self.bev_backbone.out_channels, int(m.trajectory_head.hidden),
93                                            int(cfg.planning.horizon), float(cfg.planning.dt), int(m.trajectory_hea
    ↪                                             d.num_heads))
94
95      # ------------------------------------------------------------------- utils
```

Constructor, part 3 — **backbone and heads**: SECOND-FPN, the head grid (`grid.downsample(out_stride)` = 100 × 100), a consistency check between the backbone stride and the head stride, `CenterHead` with 10 classes, and the optional `PlanningHead` (horizon 6, dt 0.5 s).

```
96      def set_grad_checkpoint(self, enabled: bool) -> None:
97          if self.use_camera:
98              self.camera_bev.backbone.grad_checkpoint = enabled
99          self.bev_backbone.grad_checkpoint = enabled
100
101     def module_groups(self) -> dict[str, nn.Module]:
102         """Top-level sub-networks by name (used for staged freezing)."""
103         return {name: mod for name, mod in self.named_children()}
104
105     def freeze(self, names: Iterable[str]) -> list[str]:
106         """Freeze parameters whose name starts with any of ``names`` (e.g. ``camera_bev.backbone``)."""
107         prefixes = tuple(n.rstrip(".") + "." for n in names)
108         frozen = []
109         for pname, p in self.named_parameters():
110             if pname.startswith(prefixes):
111                 p.requires_grad_(False)
112                 frozen.append(pname)
113         return frozen
114
115     # ------------------------------------------------------------------- forward
```

Utilities. `set_grad_checkpoint` toggles checkpointing in the image backbone and the BEV backbone. `module_groups` lists top-level children. `freeze(names)` disables gradients for every parameter whose name starts with one of the prefixes (e.g. `camera_bev.backbone`) and returns their names — used by the staged training.

```
116     def forward(self, batch: dict[str, Any], timer: Optional[StageTimer] = None) -> dict[str, Any]:
117         timer = timer or NullTimer()
118         bsz = int(batch["batch_size"])
```

```
        out: dict[str, Any] = {"raw": {}}
        feats: dict[str, torch.Tensor] = {}
        if self.use_camera:
            with timer.stage("camera_bev"):
                cam = self.camera_bev(batch["images"], batch["intrinsics"], batch["post_rots"], batch["post_trans"],
                                      batch["cam2ref"], batch["bda"], batch.get("cam_valid"))
            feats["camera"] = cam["bev"]
            out["depth_logits"], out["depth"] = cam["depth_logits"], cam["depth"]
        if self.use_lidar:
            with timer.stage("lidar_encoder"):
                lid = self.lidar_encoder(batch["lidar_points"], bsz)
            feats["lidar"] = lid["bev"]
            out["lidar_raster"] = lid["raster"]
        if self.use_radar:
            with timer.stage("radar_encoder"):
                rad = self.radar_encoder(batch["radar_points"], bsz)
            feats["radar"] = rad["bev"]
            out["radar_raster"] = rad["raster"]
        out["raw"] = dict(feats)
```

Forward pass, part 1 — encoders. A `StageTimer` (or a no-op `NullTimer`) measures each stage (used by the dashboard's live pipeline strip). Each enabled encoder is run on its batch entries; the camera encoder also returns `depth_logits`/`depth` for the depth loss; LiDAR/radar rasters are kept in the output for analysis. `out['raw']` keeps the unselected maps.

```
        selected = dict(feats)
        if self.use_feature_selection:
            with timer.stage("feature_selection"):
                sel, info = self.feature_selection({k: feats[k] for k in self.feature_selection.modalities},
                                                   feats.get("camera"))
            selected.update(sel)
            out["selection"] = info
        out["selected"] = selected

        with timer.stage("fusion"):
            fused, fusion_info = self.fusion(selected)
        out["fused"], out["fusion_info"] = fused, fusion_info
        if self.deep_supervision and self.training:
            out["aux"] = {k: self.aux_heads[k](selected[k]) for k in self.modalities}
            out["aux"]["fused"] = self.aux_heads["fused"](fused)

        with timer.stage("bev_backbone"):
            bev = self.bev_backbone(fused)
        out["bev_features"] = bev
        with timer.stage("detection_head"):
            out["det"] = self.det_head(bev)
        if self.use_planning_head and "ego_state" in batch:
            with timer.stage("planning_head"):
                out["plan"] = self.plan_head(bev, batch["ego_state"], batch["command"])
        return out

    # ------------------------------------------------------------------ losses
```

Forward pass, part 2 — selection, fusion, heads:

- `selected` starts as a copy of the raw maps; with feature selection, the LiDAR/radar entries are replaced by their selected versions and the gate info stored in `out['selection']`;
- fusion → `out['fused'] (B, 128, 200, 200)` and `fusion_info`;
- in training mode with deep supervision, auxiliary heatmap logits are computed for every (selected) sensor and for the fused map;
- backbone → `bev_features (B, 256, 100, 100)` → `det` dict;
- the planning head runs only if the batch contains `ego_state` (always true for dataset batches).

```
    def compute_losses(self, out: dict[str, Any], batch: dict[str, Any], loss_cfg: Config,
                       active: Iterable[str] = ALL_LOSSES) -> dict[str, torch.Tensor]:
        active = set(active)
        losses: dict[str, torch.Tensor] = {}
        total = out["fused"].new_zeros((), dtype=torch.float32)
        if "detection" in active:
            det = detection_loss(out["det"], batch, loss_cfg.detection, list(self.cfg.model.detection_head.code_weights))
            losses.update(det)
            total = total + float(loss_cfg.lambda_detection) * det["detection"]
        if "trajectory" in active and "plan" in out:
            traj = trajectory_loss(out["plan"]["waypoints"], batch, self.grid, self.cfg.planning.ego, loss_cfg.trajectory,
                                   float(self.cfg.planning.dt))
            losses.update(traj)
```

```
            total = total + float(loss_cfg.lambda_trajectory) * traj["trajectory"]
        if "feature_selection" in active and "selection" in out:
            fs = feature_selection_loss(out["selection"], batch["fg_heatmap"], loss_cfg.feature_selection,
                                        self.cfg.fusion.selection.target_keep_ratio)
            losses.update(fs)
            total = total + float(loss_cfg.lambda_feature_selection) * fs["feature_selection"]
        if "fusion" in active and "aux" in out:
            aux = [aux_heatmap_loss(logits, batch["fg_heatmap"]) for logits in out["aux"].values()]
            losses["fusion"] = torch.stack(aux).mean()
            total = total + float(loss_cfg.lambda_fusion) * losses["fusion"]
        if "depth" in active and self.use_camera and "depth_gt" in batch:
            losses["depth"] = self.camera_bev.depth_loss(out["depth_logits"], batch["depth_gt"])
            total = total + float(loss_cfg.lambda_depth) * losses["depth"]
        losses["total"] = total
        return losses

    # ------------------------------------------------------------------ decode
```

`compute_losses(out, batch, loss_cfg, active)` builds $L = \lambda_{det} \cdot L_{detection} + \lambda_{traj} \cdot L_{trajectory} + \lambda_{fs} \cdot L_{feature_selection} + \lambda_{fus} \cdot L_{fusion} + \lambda_{depth} \cdot L_{depth}$, adding only the groups listed in `active` and only when their inputs exist (e.g. no trajectory loss without a planning head, no depth loss without a camera or depth targets). Every sub-loss is returned for logging together with `total` (fp32 scalar).

```
    def decode(self, out: dict[str, Any], score_threshold: Optional[float] = None) -> list[dict[str, torch.Tenso
        r]]:
        h = self.cfg.model.detection_head
        thr = float(h.score_threshold if score_threshold is None else score_threshold)
        return decode_detections(out["det"], self.head_grid, self.classes, thr, int(h.max_detections), dict(h.nm
            s_radius))


def build_model(cfg: Config) -> MultiSensorFusionNet:
    return MultiSensorFusionNet(cfg)
```

`decode(out, score_threshold)` calls `decode_detections` with the head grid, class list, threshold, `max_detections` and per-class NMS radii from the config. `build_model(cfg)` is the factory used everywhere.

Planning, training, evaluation, inference and application

planning/occupancy_grid.py

Occupancy and cost layers used by the classical planner (ego frame, layout `[iy, ix]`). (98 lines)

```
"""Occupancy / cost grids for planning (ego frame, layout ``[iy, ix]``).

* **Dynamic occupancy** ``(T, ny, nx)``: detected objects propagated with their
  predicted velocity (constant-velocity motion forecast) to every planning step.
* **Static geometry**: slow / static objects at t=0 inflated by the ego half-width
  (configuration space for a point-robot A*).
* **Soft costs**: obstacle clearance (distance transform), non-ground LiDAR returns
  (covers missed detections, curbs, poles), and an optional map prior.
"""
from __future__ import annotations

from typing import Any, Optional

import cv2
import numpy as np

from utils.bev_grid import BEVGrid
from utils.raster import rasterize_boxes
```

Docstring: dynamic occupancy (constant-velocity forecast), static configuration space, soft costs.

```
def planning_grid(cfg_planning: Any) -> BEVGrid:
    g = cfg_planning.grid
    return BEVGrid(tuple(g.x_range), tuple(g.y_range), (-5.0, 5.0), float(g.resolution))


def propagate_boxes(boxes: np.ndarray, t: float) -> np.ndarray:
    """Constant-velocity forecast of ``[x, y, z, l, w, h, yaw, vx, vy]`` boxes by ``t`` seconds."""
    out = boxes.copy()
    out[:, 0] += boxes[:, 7] * t
    out[:, 1] += boxes[:, 8] * t
    return out
```

`planning_grid(cfg.planning)` → the planner's own `BEVGrid` ($x \in [-10, 60)$, $y \in [-30, 30)$, 0.5 m → 140 × 120 cells): forward-biased because the plan only covers 3 s ahead. `propagate_boxes(boxes, t)` shifts box

centres by `velocity · t` (constant-velocity motion forecast).

```
class OccupancyBuilder:
    def __init__(self, cfg_planning: Any) -> None:
        self.cfg = cfg_planning
        self.grid = planning_grid(cfg_planning)
        self.horizon = int(cfg_planning.horizon)
        self.dt = float(cfg_planning.dt)
        ego = cfg_planning.ego
        self.ego_box = np.array([[float(ego.rear_axle_to_center), 0.0, 0.0, float(ego.length), float(ego.width),
            1.6, 0.0, 0.0, 0.0]])
```

Constructor stores the grid, horizon (6), dt (0.5 s) and the ego box `[rear_axle_to_center, 0, 0, length, width, …]` in the ego frame (the ego frame origin is the rear axle, so the body centre is 1.4 m ahead).

```
    def dynamic_occupancy(self, boxes: np.ndarray, inflate: float = 0.0, times: Optional[np.ndarray] = None) -> n
        p.ndarray:
        """``(T, ny, nx)`` uint8 occupancy at ``t = dt, 2dt, ... T·dt`` (CV motion forecast)."""
        times = times if times is not None else (np.arange(1, self.horizon + 1) * self.dt)
        occ = np.zeros((len(times),) + self.grid.shape, dtype=np.uint8)
        for k, t in enumerate(times):
            if boxes.shape[0]:
                rasterize_boxes(propagate_boxes(boxes, float(t)), self.grid, inflate=inflate, out=occ[k])
        return occ

    def static_occupancy(self, boxes: np.ndarray, inflate: float) -> np.ndarray:
        occ = np.zeros(self.grid.shape, dtype=np.uint8)
        if boxes.shape[0]:
            rasterize_boxes(boxes, self.grid, inflate=inflate, out=occ)
        return occ
```

`dynamic_occupancy(boxes, inflate, times)` → `(T, ny, nx)` uint8: for every future time `t = dt, 2dt, …, T·dt` the boxes are propagated and rasterised (with optional inflation). `static_occupancy(boxes, inflate)` → `(ny, nx)` at t = 0.

```
    def lidar_obstacles(self, points: Optional[np.ndarray], z_band: tuple[float, float]) -> np.ndarray:
        """Cells with non-ground LiDAR returns (excluding the ego vehicle itself)."""
        occ = np.zeros(self.grid.shape, dtype=np.uint8)
        if points is None or points.shape[0] == 0:
            return occ
        p = points[(points[:, 2] > z_band[0]) & (points[:, 2] < z_band[1])]
        ego = self.ego_box[0]
        in_ego = (np.abs(p[:, 0] - ego[0]) < ego[3] / 2 + 0.5) & (np.abs(p[:, 1]) < ego[4] / 2 + 0.5)
        p = p[~in_ego]
        ix = np.floor((p[:, 0] - self.grid.x_range[0]) / self.grid.resolution).astype(np.int64)
        iy = np.floor((p[:, 1] - self.grid.y_range[0]) / self.grid.resolution).astype(np.int64)
        ok = (ix >= 0) & (ix < self.grid.nx) & (iy >= 0) & (iy < self.grid.ny)
        occ[iy[ok], ix[ok]] = 1
        return occ
```

`lidar_obstacles(points, z_band)`: cells containing non-ground LiDAR returns (0.4 m < z < 2.5 m), excluding points on the ego vehicle — catches obstacles the detector missed, curbs, poles and vegetation.

```
    def clearance_cost(self, occ: np.ndarray, scale_m: float = 1.5) -> np.ndarray:
        """``exp(-d / scale)`` where ``d`` is the distance (m) to the nearest occupied cell."""
        if not occ.any():
            return np.zeros(occ.shape, dtype=np.float32)
        dist = cv2.distanceTransform((occ == 0).astype(np.uint8), cv2.DIST_L2, 5) * self.grid.resolution
        return np.exp(-dist / scale_m).astype(np.float32)
```

`clearance_cost(occ)`: `exp(−d / 1.5 m)` where `d` is the OpenCV distance transform (distance to the nearest occupied cell) — a soft cost that keeps paths away from obstacles.

```
    def footprint_points(self, xy: np.ndarray, yaw: np.ndarray, margin: float = 0.0) -> np.ndarray:
        """Points sampled inside the ego footprint ``(N, 15, 2)`` for rear-axle poses ``xy (N, 2)``."""
        ego = self.ego_box[0]
        ls = np.linspace(-ego[3] / 2 - margin, ego[3] / 2 + margin, 5) + ego[0]
        ws = np.linspace(-ego[4] / 2 - margin, ego[4] / 2 + margin, 3)
        local = np.stack(np.meshgrid(ls, ws, indexing="ij"), -1).reshape(-1, 2)
        c, s = np.cos(yaw)[:, None], np.sin(yaw)[:, None]
        x = xy[:, 0:1] + local[None, :, 0] * c - local[None, :, 1] * s
        y = xy[:, 1:2] + local[None, :, 0] * s + local[None, :, 1] * c
        return np.stack([x, y], -1)

    def occupied_at(self, occ: np.ndarray, pts: np.ndarray) -> np.ndarray:
        """Look up ``occ (ny, nx)`` at metric points ``(..., 2)`` (outside the grid = free)."""
        ix = np.floor((pts[..., 0] - self.grid.x_range[0]) / self.grid.resolution).astype(np.int64)
        iy = np.floor((pts[..., 1] - self.grid.y_range[0]) / self.grid.resolution).astype(np.int64)
        ok = (ix >= 0) & (ix < self.grid.nx) & (iy >= 0) & (iy < self.grid.ny)
        out = np.zeros(pts.shape[:-1], dtype=bool)
        out[ok] = occ[iy[ok], ix[ok]] > 0
        return out
```

`footprint_points(xy (N, 2), yaw (N,))` → `(N, 15, 2)`: a 5 × 3 grid of points covering the ego body (optionally enlarged by a margin) for each rear-axle pose. `occupied_at(occ, pts)` looks the points up in an occupancy grid (outside the grid = free).

planning/astar.py

Grid A* used by the baseline planner to find a geometric path on the static cost map. (92 lines)

```python
"""Grid A* on a BEV cost map (8-connected, Euclidean heuristic)."""
from __future__ import annotations

import heapq
import math
from typing import Optional

import numpy as np

_MOVES = [(-1, 0, 1.0), (1, 0, 1.0), (0, -1, 1.0), (0, 1, 1.0),
          (-1, -1, math.sqrt(2)), (-1, 1, math.sqrt(2)), (1, -1, math.sqrt(2)), (1, 1, math.sqrt(2))]
```

Imports and `_MOVES`: the 8-neighbourhood with step lengths 1 and √2.

```python
def astar(cost: np.ndarray, start: tuple[int, int], goal: tuple[int, int], max_expansions: int = 60000,
          heuristic_weight: float = 1.0) -> tuple[list[tuple[int, int]], bool]:
    """Shortest path on a cost grid.

    Args:
        cost: ``(H, W)`` per-cell traversal cost (``>= 0``, ``np.inf`` = blocked). Moving into a
            cell costs ``step_length * (1 + cost[cell])``.
        start, goal: ``(row, col)`` cells.

    Returns:
        ``(path, reached)`` — if the goal is unreachable the path leads to the explored cell
        closest to the goal (best effort), and ``reached`` is ``False``.
    """
    h, w = cost.shape
    if not (0 <= goal[0] < h and 0 <= goal[1] < w):
        raise ValueError("goal outside grid")
    g_score = np.full(cost.shape, np.inf)
    parent = -np.ones(cost.shape + (2,), dtype=np.int32)
    g_score[start] = 0.0

    def heuristic(r: int, c: int) -> float:
        return heuristic_weight * math.hypot(r - goal[0], c - goal[1])

    open_heap = [(heuristic(*start), 0.0, start)]
    best, best_h = start, heuristic(*start)
    closed = np.zeros(cost.shape, dtype=bool)
    expansions = 0
    while open_heap and expansions < max_expansions:
        _, g, node = heapq.heappop(open_heap)
        if closed[node]:
            continue
        closed[node] = True
        expansions += 1
        hn = heuristic(*node)
        if hn < best_h:
            best, best_h = node, hn
        if node == goal:
            best = goal
            break
        r, c = node
        for dr, dc, step in _MOVES:
            nr, nc = r + dr, c + dc
            if not (0 <= nr < h and 0 <= nc < w) or closed[nr, nc]:
                continue
            cell_cost = cost[nr, nc]
            if not np.isfinite(cell_cost):
                continue
            ng = g + step * (1.0 + cell_cost)
            if ng < g_score[nr, nc]:
                g_score[nr, nc] = ng
                parent[nr, nc] = (r, c)
                heapq.heappush(open_heap, (ng + heuristic(nr, nc), ng, (nr, nc)))
    path = [best]
    while path[-1] != start:
        pr, pc = parent[path[-1]]
        if pr < 0:
            break
        path.append((int(pr), int(pc)))
    path.reverse()
    return path, best == goal
```

`astar(cost, start, goal, max_expansions, heuristic_weight)`:

- `cost` contains per-cell traversal costs; `inf` marks blocked cells. Moving into a cell costs `step · (1 + cost)` — with zero cost this is the Euclidean path length.
- Classic A* with a binary heap (`heapq`) of `(f = g + h, g, node)`, `g_score` and `parent` arrays and a `closed` set; the heuristic is the Euclidean distance (admissible).
- The best node seen so far (closest to the goal) is tracked; if the goal is unreachable or the expansion budget runs out, the path to that node is returned with `reached = False` (best effort instead of failure).
- The path is reconstructed backwards through `parent` and reversed.

```
76 def nearest_free(cost: np.ndarray, cell: tuple[int, int], max_radius: int = 20) -> Optional[tuple[int, int]]:
77     """Closest finite-cost cell to ``cell`` within ``max_radius`` (square rings)."""
78     h, w = cost.shape
79     r0, c0 = cell
80     if 0 <= r0 < h and 0 <= c0 < w and np.isfinite(cost[r0, c0]):
81         return cell
82     for rad in range(1, max_radius + 1):
83         best, best_d = None, np.inf
84         for r in range(r0 - rad, r0 + rad + 1):
85             for c in (c0 - rad, c0 + rad) if abs(r - r0) != rad else range(c0 - rad, c0 + rad + 1):
86                 if 0 <= r < h and 0 <= c < w and np.isfinite(cost[r, c]):
87                     d = math.hypot(r - r0, c - c0)
88                     if d < best_d:
89                         best, best_d = (r, c), d
90         if best is not None:
91             return best
92     return None
```

`nearest_free(cost, cell, max_radius)`: searches square rings of increasing radius for the closest finite-cost cell — used when the goal falls inside an obstacle.

planning/mpc.py

Model-predictive control by iterative LQR on a kinematic bicycle model. Turns the geometric path and speed profile into a dynamically feasible, smooth trajectory. (135 lines)

```
1  """Model Predictive Control: iterative LQR (iLQR) on a kinematic bicycle model.
2
3  State ``s = [x, y, yaw, v, δ]`` (rear-axle position, heading, speed, steering angle),
4  control ``u = [a, ω]`` (acceleration, steering rate)::
5
6      x'   = x + v cos(yaw) dt
7      y'   = y + v sin(yaw) dt
8      yaw' = yaw + v / L · tan(δ) dt
9      v'   = v + a dt
10     δ'   = δ + ω dt
11
12 Stage cost ``w_pos‖p − p_ref‖² + w_yaw Δyaw² + w_speed (v − v_ref)² + w_steer δ² + w_accel a² +
13 w_steer_rate ω²``. Controls and states are clamped to the vehicle limits after every
14 forward pass (projected iLQR), which is sufficient for this open-loop tracking problem.
15 """
16 from __future__ import annotations
17
18 from dataclasses import dataclass
19 from typing import Any
20
21 import numpy as np
22
23 from utils.geometry import wrap_angle
```

Docstring: state `[x, y, yaw, v, δ]`, control `[a, ω]` (acceleration, steering rate), the discrete dynamics and the stage cost.

```
26 @dataclass
27 class MPCResult:
28     states: np.ndarray    # (N+1, 5)
29     controls: np.ndarray  # (N, 2)
30     cost: float
31     iterations: int
```

`MPCResult`: optimised states `(N+1, 5)`, controls `(N, 2)`, final cost and iterations used.

```
34 class BicycleILQR:
35     def __init__(self, cfg_mpc: Any, ego_cfg: Any) -> None:
36         self.dt = float(cfg_mpc.dt)
37         self.iterations = int(cfg_mpc.iterations)
38         self.L = float(ego_cfg.wheelbase)
39         self.max_steer = float(ego_cfg.max_steer)
```

```
40          self.max_accel, self.max_decel = float(ego_cfg.max_accel), float(ego_cfg.max_decel)
41          self.max_speed = float(ego_cfg.max_speed)
42          self.Q = np.diag([cfg_mpc.w_pos, cfg_mpc.w_pos, cfg_mpc.w_yaw, cfg_mpc.w_speed, cfg_mpc.w_steer]).astype(
    ↪           float)
43          self.R = np.diag([cfg_mpc.w_accel, cfg_mpc.w_steer_rate]).astype(float)
44
45      # ------------------------------------------------------------------ model
```

Constructor: `dt` (0.1 s), iterations (12), wheelbase L (2.588 m), steering / acceleration / speed limits from `planning.ego`, and the diagonal cost matrices `Q` (position 4, yaw 1, speed 0.5, steering 2) and `R` (acceleration 0.1, steering rate 4).

```
46      def step(self, s: np.ndarray, u: np.ndarray) -> np.ndarray:
47          x, y, yaw, v, d = s
48          a, w = u
49          dt = self.dt
50          nxt = np.array([x + v * np.cos(yaw) * dt, y + v * np.sin(yaw) * dt, yaw + v / self.L * np.tan(d) * dt,
51                          v + a * dt, d + w * dt])
52          nxt[3] = np.clip(nxt[3], 0.0, self.max_speed)
53          nxt[4] = np.clip(nxt[4], -self.max_steer, self.max_steer)
54          return nxt
```

`step(s, u)`: one Euler step of the bicycle model — position moves along the heading, `yaw += v/L · tan(δ) · dt`, `v += a·dt`, `δ += ω·dt`; speed is clipped to [0, v_max] and steering to ±δ_max (no reversing).

```
56      def jacobians(self, s: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
57          _, _, yaw, v, d = s
58          dt = self.dt
59          A = np.eye(5)
60          A[0, 2], A[0, 3] = -v * np.sin(yaw) * dt, np.cos(yaw) * dt
61          A[1, 2], A[1, 3] = v * np.cos(yaw) * dt, np.sin(yaw) * dt
62          A[2, 3], A[2, 4] = np.tan(d) / self.L * dt, v / (self.L * np.cos(d) ** 2) * dt
63          B = np.zeros((5, 2))
64          B[3, 0], B[4, 1] = dt, dt
65          return A, B
```

`jacobians(s)`: analytic linearisation `A = ∂f/∂s` (5 × 5) and `B = ∂f/∂u` (5 × 2) around the current state, needed by the Riccati recursion.

```
67      def rollout(self, s0: np.ndarray, controls: np.ndarray) -> np.ndarray:
68          states = [s0]
69          for u in controls:
70              states.append(self.step(states[-1], u))
71          return np.asarray(states)
72
73      def _clip(self, u: np.ndarray) -> np.ndarray:
74          u = u.copy()
75          u[:, 0] = np.clip(u[:, 0], -self.max_decel, self.max_accel)
76          u[:, 1] = np.clip(u[:, 1], -1.0, 1.0)
77          return u
78
79      def _err(self, s: np.ndarray, ref: np.ndarray) -> np.ndarray:
80          e = s - ref
81          e[..., 2] = wrap_angle(e[..., 2])
82          return e
83
84      def cost(self, states: np.ndarray, controls: np.ndarray, ref: np.ndarray) -> float:
85          e = self._err(states[1:], ref)
86          return float(np.einsum("ni,ij,nj->", e, self.Q, e) + np.einsum("ni,ij,nj->", controls, self.R, controls))
87
88      # ------------------------------------------------------------------ solve
```

`rollout`, `_clip` (control limits), `_err` (state error with the yaw difference wrapped to [−π, π)) and `cost` (Σ $e^T Q e + u^T R u$ over the horizon).

```
89       def solve(self, s0: np.ndarray, ref: np.ndarray, u_init: np.ndarray | None = None) -> MPCResult:
90          """Track ``ref (N, 5)`` (``[x, y, yaw, v, 0]`` per step) from initial state ``s0``."""
91          n = ref.shape[0]
92          controls = np.zeros((n, 2)) if u_init is None else self._clip(u_init)
93          states = self.rollout(s0, controls)
94          cost = self.cost(states, controls, ref)
95          it = 0
96          for it in range(1, self.iterations + 1):
97              # Backward pass (Riccati recursion on the quadratic cost-to-go).
98              Vx, Vxx = np.zeros(5), np.zeros((5, 5))
99              ks, Ks = np.zeros((n, 2)), np.zeros((n, 2, 5))
100             for t in reversed(range(n)):
101                 A, B = self.jacobians(states[t])
102                 e = self._err(states[t + 1], ref[t])
103                 lx = 2 * self.Q @ e + Vx
104                 lxx = 2 * self.Q + Vxx
105                 Qx = A.T @ lx
```

```
                Qu = B.T @ lx + 2 * self.R @ controls[t]
                Qxx = A.T @ lxx @ A
                Quu = B.T @ lxx @ B + 2 * self.R + 1e-6 * np.eye(2)
                Qux = B.T @ lxx @ A
                Quu_inv = np.linalg.inv(Quu)
                ks[t] = -Quu_inv @ Qu
                Ks[t] = -Quu_inv @ Qux
                Vx = Qx + Ks[t].T @ Quu @ ks[t] + Ks[t].T @ Qu + Qux.T @ ks[t]
                Vxx = Qxx + Ks[t].T @ Quu @ Ks[t] + Ks[t].T @ Qux + Qux.T @ Ks[t]
                Vxx = 0.5 * (Vxx + Vxx.T)
            # Forward pass with backtracking line search.
            improved = False
            for alpha in (1.0, 0.5, 0.25, 0.1):
                new_u = np.zeros_like(controls)
                new_s = [s0]
                for t in range(n):
                    dx = self._err(new_s[-1], states[t])
                    new_u[t] = controls[t] + alpha * ks[t] + Ks[t] @ dx
                    new_u[t] = self._clip(new_u[t:t + 1])[0]
                    new_s.append(self.step(new_s[-1], new_u[t]))
                new_s = np.asarray(new_s)
                new_cost = self.cost(new_s, new_u, ref)
                if new_cost < cost:
                    states, controls, improved = new_s, new_u, True
                    converged = cost - new_cost < 1e-4 * max(cost, 1.0)
                    cost = new_cost
                    break
            if not improved or converged:
                break
        return MPCResult(states, controls, cost, it)
```

`solve(s0, ref, u_init)` — iLQR:

1. Initial rollout with the given controls (the planner passes the chosen constant acceleration).
2. **Backward pass** (Gauss–Newton): from the last step to the first, the quadratic expansion of the cost-to-go gives feedforward $k_t = -Quu^{-1}\ Qu$ and feedback $K_t = -Quu^{-1}\ Qux$; the value function derivatives `Vx, Vxx` are propagated (symmetrised for stability; $1e\text{-}6 \cdot I$ regularises `Quu`).
3. **Forward pass** with a backtracking line search over $\alpha \in \{1, 0.5, 0.25, 0.1\}$: $u_t = u_t + \alpha \cdot k_t + K_t \cdot (x_t - \hat{x}_t)$, clipped, re-simulated; accepted if the cost decreases.
4. Stops when no step improves or the relative improvement is < 1e-4.

planning/tracking.py

Online multi-object tracking used by the replay (track IDs, smoothed velocities, motion trails). (102 lines)

```
"""Online multi-object tracking for scene replay (CenterPoint-style greedy association).

Detections of each key-frame are moved to the global frame; existing tracks are
predicted with their velocity, and detections are greedily matched (highest score
first) to the nearest compatible track within a class-specific distance. Tracks give
persistent IDs, smoothed velocities and motion histories for the visualisation and the
planner's motion forecast.
"""
from __future__ import annotations

from dataclasses import dataclass, field
from typing import Optional

import numpy as np

from utils.geometry import invert_transform

# Max centre distance (m) between a predicted track and a detection (CenterPoint nuScenes values).
MATCH_DISTANCE = {"car": 4.0, "truck": 4.0, "bus": 5.5, "trailer": 3.0, "construction_vehicle": 3.0,
                  "pedestrian": 1.0, "motorcycle": 13.0, "bicycle": 3.0, "barrier": 1.0, "traffic_cone": 1.0}
```

Docstring and `MATCH_DISTANCE`: CenterPoint's class-specific maximum association distances (e.g. car 4 m, pedestrian 1 m, motorcycle 13 m).

```
@dataclass
class Track:
    track_id: int
    label: int
    position: np.ndarray            # global xy
    velocity: np.ndarray            # global vxy
    score: float
    age: int = 1
    misses: int = 0
    history: list = field(default_factory=list)
```

`Track`: id, label, global position and velocity, score, age, consecutive misses and the position history.

```
class MultiObjectTracker:
    def __init__(self, classes: list[str], max_misses: int = 2, history: int = 8) -> None:
        self.classes = classes
        self.max_misses = max_misses
        self.history_len = history
        self.reset()

    def reset(self) -> None:
        self.tracks: list[Track] = []
        self.next_id = 1
        self.last_time: Optional[float] = None
```

Constructor and `reset()` (clears tracks, restarts ids).

```
    def update(self, boxes_ref: np.ndarray, labels: np.ndarray, scores: np.ndarray, ref_ego2global: np.ndarray,
               timestamp_us: int) -> np.ndarray:
        """Associate detections; returns track ids ``(N,)`` aligned with the detections."""
        t = timestamp_us * 1e-6
        dt = 0.5 if self.last_time is None else max(t - self.last_time, 1e-3)
        if self.last_time is not None and (t - self.last_time > 2.0 or t < self.last_time):
            self.reset()  # jumped in the replay: start fresh
            dt = 0.5
        self.last_time = t
        rot = ref_ego2global[:2, :2]
        pos = boxes_ref[:, :2] @ rot.T + ref_ego2global[:2, 3] if len(boxes_ref) else np.zeros((0, 2))
        vel = boxes_ref[:, 7:9] @ rot.T if len(boxes_ref) else np.zeros((0, 2))
        predicted = [tr.position + tr.velocity * dt for tr in self.tracks]
        ids = np.zeros(len(boxes_ref), dtype=np.int64)
        used = set()
        for i in np.argsort(-scores):
            name = self.classes[int(labels[i])]
            best, best_d = None, MATCH_DISTANCE.get(name, 2.0)
            for j, (tr, p) in enumerate(zip(self.tracks, predicted)):
                if j in used or tr.label != int(labels[i]):
                    continue
                d = float(np.hypot(*(pos[i] - p)))
                if d < best_d:
                    best, best_d = j, d
            if best is None:
                tr = Track(self.next_id, int(labels[i]), pos[i].copy(), vel[i].copy(), float(scores[i]))
                self.next_id += 1
                self.tracks.append(tr)
                used.add(len(self.tracks) - 1)
            else:
                tr = self.tracks[best]
                measured = (pos[i] - tr.position) / dt
                tr.velocity = 0.5 * vel[i] + 0.5 * measured
                tr.position, tr.score, tr.age, tr.misses = pos[i].copy(), float(scores[i]), tr.age + 1, 0
                used.add(best)
            tr.history = (tr.history + [tr.position.copy()])[-self.history_len:]
            ids[i] = tr.track_id
        for j, tr in enumerate(self.tracks):
            if j not in used:
                tr.misses += 1
        self.tracks = [tr for tr in self.tracks if tr.misses <= self.max_misses]
        return ids
```

`update(boxes_ref, labels, scores, ref_ego2global, timestamp_us)` → track ids:

1. Time step `dt` from the timestamps; a jump backwards or > 2 s resets the tracker (scene change / seek).
2. Detections are moved to the **global** frame (tracks must survive ego motion).
3. Existing tracks are predicted with `position + velocity · dt`.
4. Greedy association in descending score order: nearest unused track of the same class within the class distance.
5. Matched: velocity = ½ (detected velocity) + ½ (finite-difference velocity); unmatched detection → new track; unmatched tracks accumulate misses and are removed after `max_misses` (2).
6. Histories are truncated to the last 8 positions.

```
    def histories_ref(self, ids: np.ndarray, ref_ego2global: np.ndarray) -> list[np.ndarray]:
        """Motion history of the given track ids in the current reference frame."""
        g2r = invert_transform(ref_ego2global)
        by_id = {tr.track_id: tr for tr in self.tracks}
        out = []
        for tid in ids:
            tr = by_id.get(int(tid))
            if tr is None or not tr.history:
```

```
                out.append(np.zeros((0, 2)))
                continue
            h = np.asarray(tr.history)
            out.append(h @ g2r[:2, :2].T + g2r[:2, 3])
        return out
```

`histories_ref(ids, ref_ego2global)`: track histories transformed back into the current reference frame for drawing motion trails.

planning/trajectory_planner.py

The planners. `BaselinePlanner` needs no training; `LearnedPlanner` and `HybridPlanner` use the network's planning head. All share the `plan(...) -> PlanResult` interface. (250 lines)

```
"""Trajectory planners.

``BaselinePlanner`` (classical, no training required) — path/velocity decomposition::

    detections ─► occupancy (static C-space + CV-forecast dynamic occupancy + LiDAR/map costs)
    command ────► goal point (lookahead ∝ speed, bearing from LEFT/RIGHT/STRAIGHT)
                  A* geometric path on the static cost map
                  speed profile: best constant acceleration that keeps the ego footprint
                                 free of the dynamic occupancy at every step
                  iLQR MPC tracks (path, speed) with a kinematic bicycle model
    output: waypoints (T, 2) at dt = 0.5 s + collision flag

``LearnedPlanner`` wraps the network's planning head; ``HybridPlanner`` uses the learned
trajectory but falls back to the baseline whenever the learned one collides with the
predicted occupancy (safety layer). All planners share the ``plan()`` interface, so a
learned planner can replace the baseline by changing ``planning.planner`` in the config.
"""
```

Docstring with the path/velocity-decomposition pipeline.

```
from __future__ import annotations

import time
from dataclasses import dataclass, field
from typing import Any, Optional

import cv2
import numpy as np

from planning.astar import astar, nearest_free
from planning.mpc import BicycleILQR
from planning.occupancy_grid import OccupancyBuilder

COMMAND_NAMES = ("RIGHT", "LEFT", "STRAIGHT")
```

Imports and `COMMAND_NAMES`.

```
@dataclass
class PlanResult:
    waypoints: np.ndarray                       # (T, 2) rear-axle positions at t = dt..T·dt
    yaws: np.ndarray                            # (T,)
    speeds: np.ndarray                          # (T,)
    collision: bool                             # footprint intersects predicted occupancy
    planner: str
    command: int
    path: np.ndarray = field(default_factory=lambda: np.zeros((0, 2)))      # geometric (A*) path
    dense: np.ndarray = field(default_factory=lambda: np.zeros((0, 5)))     # MPC states at 10 Hz
    goal: Optional[np.ndarray] = None
    accel: float = 0.0
    time_ms: float = 0.0
    debug: dict[str, Any] = field(default_factory=dict)
```

`PlanResult`: waypoints `(T, 2)` at t = 0.5 … 3.0 s (rear-axle positions in the reference frame), yaws, speeds, the collision flag against *predicted* occupancy, planner name, command, the A* path, the dense 10 Hz MPC states, goal, chosen acceleration, run time and a debug dict (occupancy layers).

```
def _smooth_path(path: np.ndarray, window: int = 5) -> np.ndarray:
    if len(path) < 3:
        return path
    k = min(window, len(path))
    pad = np.concatenate([np.repeat(path[:1], k // 2, 0), path, np.repeat(path[-1:], k // 2, 0)])
    kernel = np.ones(k) / k
    out = np.stack([np.convolve(pad[:, i], kernel, mode="valid") for i in range(2)], -1)
    out[0] = path[0]
    return out


```

```python
def _resample(path: np.ndarray, step: float = 0.25) -> tuple[np.ndarray, np.ndarray]:
    seg = np.linalg.norm(np.diff(path, axis=0), axis=1)
    s = np.concatenate([[0.0], np.cumsum(seg)])
    if s[-1] < 1e-6:
        return path[:1], np.zeros(1)
    grid = np.arange(0.0, s[-1] + 1e-9, step)
    return np.stack([np.interp(grid, s, path[:, i]) for i in range(2)], -1), grid


def _pose_at(path: np.ndarray, arclen: np.ndarray, s: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
    """Interpolate positions/headings at arc lengths ``s`` (extrapolates along the last heading)."""
    heading = np.arctan2(np.gradient(path[:, 1]), np.gradient(path[:, 0])) if len(path) > 1 else np.zeros(1)
    xy = np.stack([np.interp(s, arclen, path[:, 0]), np.interp(s, arclen, path[:, 1])], -1)
    over = s > arclen[-1]
    if over.any():
        h = heading[-1]
        xy[over] = path[-1] + (s[over] - arclen[-1])[:, None] * np.array([np.cos(h), np.sin(h)])
    yaw = np.interp(s, arclen, np.unwrap(heading))
    return xy, yaw
```

Helpers: `_smooth_path` (moving average, first point fixed), `_resample` (equal arc-length steps of 0.25 m; returns points and cumulative arc length), `_pose_at(path, arclen, s)` (position and heading at arc lengths `s`, extrapolated straight beyond the end of the path).

```python
class BaselinePlanner:
    name = "baseline"

    def __init__(self, cfg: Any) -> None:
        self.cfg = cfg
        p = cfg.planning
        self.p = p
        self.occ = OccupancyBuilder(p)
        self.grid = self.occ.grid
        self.mpc = BicycleILQR(p.mpc, p.ego)
        self.horizon, self.dt = int(p.horizon), float(p.dt)
        self.classes = list(cfg.classes)

    # ------------------------------------------------------------ helpers
```

Constructor: occupancy builder, planning grid, iLQR solver, horizon, dt, classes.

```python
    def _goal(self, speed: float, command: int) -> np.ndarray:
        a = self.p.astar
        dist = float(np.clip(max(speed, 2.0) * self.horizon * self.dt * 1.2, a.lookahead_min, a.lookahead_max))
        theta = {0: -1.0, 1: 1.0, 2: 0.0}[int(command)] * np.deg2rad(float(a.turn_angle_deg))
        return np.array([dist * np.cos(theta), dist * np.sin(theta)])

    def _to_cell(self, xy: np.ndarray) -> tuple[int, int]:
        ix = int(np.clip(np.floor((xy[0] - self.grid.x_range[0]) / self.grid.resolution), 0, self.grid.nx - 1))
        iy = int(np.clip(np.floor((xy[1] - self.grid.y_range[0]) / self.grid.resolution), 0, self.grid.ny - 1))
        return iy, ix

    def _to_metric(self, cells: list[tuple[int, int]]) -> np.ndarray:
        c = np.asarray(cells, dtype=np.float64)
        return np.stack([(c[:, 1] + 0.5) * self.grid.resolution + self.grid.x_range[0],
                         (c[:, 0] + 0.5) * self.grid.resolution + self.grid.y_range[0]], -1)
```

`_goal(speed, command)`: lookahead distance `max(speed, 2) · 3 s · 1.2`, clipped to [6, 45] m; direction 0° for STRAIGHT, ±35° for LEFT/RIGHT. Coordinate helpers `_to_cell` / `_to_metric`.

```python
    def collision_check(self, waypoints: np.ndarray, yaws: np.ndarray, dyn_occ: np.ndarray) -> np.ndarray:
        """Per-step collision flags ``(T,)`` of the ego footprint against ``dyn_occ (T, ny, nx)``."""
        pts = self.occ.footprint_points(waypoints, yaws)
        return np.array([self.occ.occupied_at(dyn_occ[t], pts[t]).any() for t in range(len(waypoints))])
```

`collision_check(waypoints, yaws, dyn_occ)` → per-step booleans: ego footprint points at step t tested against the dynamic occupancy at step t.

```python
    def build_scene(self, detections: dict[str, np.ndarray], lidar_points: Optional[np.ndarray] = None,
                    map_drivable: Optional[np.ndarray] = None) -> dict[str, np.ndarray]:
        """Occupancy layers used by planning (also returned for visualisation)."""
        occ_cfg = self.p.occupancy
        boxes = detections["boxes"]
        keep = detections["scores"] >= float(occ_cfg.min_score) if len(boxes) else np.zeros(0, bool)
        boxes = boxes[keep]
        speed = np.hypot(boxes[:, 7], boxes[:, 8]) if len(boxes) else np.zeros(0)
        moving = speed > 1.0
        half_w = float(self.p.ego.width) / 2
        static_occ = self.occ.static_occupancy(boxes[~moving], inflate=half_w + float(occ_cfg.inflation))
        dyn_occ = self.occ.dynamic_occupancy(boxes, inflate=float(occ_cfg.inflation))
        cost = 0.5 * float(self.p.astar.obstacle_clearance_cost) * self.occ.clearance_cost(static_occ)
        lidar_occ = np.zeros(self.grid.shape, np.uint8)
        if occ_cfg.use_lidar_obstacles and lidar_points is not None:
```

```
            lidar_occ = self.occ.lidar_obstacles(lidar_points, tuple(occ_cfg.lidar_z_band))
            lidar_occ = cv2.dilate(lidar_occ, np.ones((3, 3), np.uint8))
            cost += float(occ_cfg.lidar_cost) * lidar_occ
        if map_drivable is not None:
            cost += float(occ_cfg.map_cost) * (1 - map_drivable.astype(np.float32))
        cost = cost.astype(np.float64)
        cost[static_occ > 0] = np.inf
        return {"static_occ": static_occ, "dynamic_occ": dyn_occ, "lidar_occ": lidar_occ, "cost": cost}

    # ------------------------------------------------------------------ plan
```

`build_scene(detections, lidar_points, map_drivable)`:

- keeps detections with score ≥ `occupancy.min_score` (0.3);
- *static* obstacles = objects slower than 1 m/s, rasterised inflated by half the ego width + margin (configuration space for a point robot);
- *dynamic* occupancy over the horizon for all objects (constant-velocity forecast);
- cost = ½·clearance cost + LiDAR obstacle cost (dilated) + optional map cost; static cells → `inf`.

Moving objects are deliberately *not* blocked geometrically (the ego should follow a moving car, not plan around it); they are handled by the speed profile.

```
    def plan(self, ego_state: np.ndarray, command: int, detections: dict[str, np.ndarray],
             lidar_points: Optional[np.ndarray] = None, map_drivable: Optional[np.ndarray] = None,
             **_: Any) -> PlanResult:
        t0 = time.perf_counter()
        speed, yaw_rate, accel0 = (float(v) for v in ego_state[:3])
        speed = max(speed, 0.0)
        scene = self.build_scene(detections, lidar_points, map_drivable)
        cost = scene["cost"].copy()
        start = self._to_cell(np.zeros(2))
        # The ego may start inside an inflated obstacle (close parked car): clear its own footprint.
        r, c = start
        cost[max(r - 3, 0):r + 4, max(c - 2, 0):c + 8] = np.where(
            np.isfinite(cost[max(r - 3, 0):r + 4, max(c - 2, 0):c + 8]), cost[max(r - 3, 0):r + 4, max(c - 2, 0)
                :c + 8], 50.0)

        goal_xy = self._goal(speed, command)
        goal_cell = nearest_free(cost, self._to_cell(goal_xy)) or self._to_cell(goal_xy)
        cells, reached = astar(cost, start, goal_cell, int(self.p.astar.max_expansions))
        path = self._to_metric(cells) if len(cells) > 1 else np.stack([np.zeros(2), goal_xy])
        path[0] = 0.0
        path = _smooth_path(path)
        path, arclen = _resample(path)
        if len(path) < 2:
            path = np.array([[0.0, 0.0], [1.0, 0.0]])
            arclen = np.array([0.0, 1.0])
```

`plan()`, part 1 — geometry: clears a small region around the ego start (it may start inside an inflated obstacle), computes the goal, moves it to the nearest free cell, runs A*, converts cells to metres, smooths and resamples the path (a straight 1 m path is the fallback).

```
        # Speed profile: best constant acceleration that stays collision free.
        times = np.arange(1, self.horizon + 1) * self.dt
        s_ref = max(speed * times[-1] + 0.5 * np.clip(accel0, -2.0, 1.0) * times[-1] ** 2, 0.0)
        best = None
        for a in self.p.speed.accel_candidates:
            v = np.clip(speed + a * times, 0.0, float(self.p.ego.max_speed))
            # Distance with clamped speed (exact for piecewise-constant acceleration).
            s = np.cumsum(np.concatenate([[0.5 * (speed + v[0]) * self.dt], 0.5 * (v[:-1] + v[1:]) * self.dt]))
            xy, yaw = _pose_at(path, arclen, s)
            hits = self.collision_check(xy, yaw, scene["dynamic_occ"])
            c = (float(self.p.speed.progress_weight) * abs(s[-1] - s_ref) + float(self.p.speed.comfort_weight) *
                 a * a
                 + 1e3 * hits.sum())
            if best is None or c < best[0]:
                best = (c, float(a), s, v)
        _, a_best, s_prof, v_prof = best
```

Part 2 — speed: reference distance `s_ref` from the current speed and (clipped) acceleration; for each candidate constant acceleration the speed profile (clipped to [0, v_max]), the travelled distance, the poses on the path and the collisions against the dynamic occupancy are computed; cost = progress error + comfort (a^2) + 1000 per colliding step; the cheapest candidate wins.

```
        # MPC tracking at 10 Hz.
        n = int(round(times[-1] / float(self.p.mpc.dt)))
        t_fine = np.arange(1, n + 1) * float(self.p.mpc.dt)
        s_fine = np.interp(t_fine, np.concatenate([[0.0], times]), np.concatenate([[0.0], s_prof]))
        v_fine = np.interp(t_fine, np.concatenate([[0.0], times]), np.concatenate([[speed], v_prof]))
```

```
        xy_ref, yaw_ref = _pose_at(path, arclen, s_fine)
        ref = np.concatenate([xy_ref, yaw_ref[:, None], v_fine[:, None], np.zeros((n, 1))], 1)
        steer0 = float(np.clip(np.arctan(yaw_rate * float(self.p.ego.wheelbase) / max(speed, 0.5)), -0.5, 0.5))
        s0 = np.array([0.0, 0.0, 0.0, speed, steer0])
        u0 = np.tile([a_best, 0.0], (n, 1))
        res = self.mpc.solve(s0, ref, u0)
        idx = np.round(times / float(self.p.mpc.dt)).astype(int)
        waypoints, yaws, speeds = res.states[idx, :2], res.states[idx, 2], res.states[idx, 3]
        collision = bool(self.collision_check(waypoints, yaws, scene["dynamic_occ"]).any())
        return PlanResult(waypoints.astype(np.float32), yaws.astype(np.float32), speeds.astype(np.float32), coll
    ↪           ision,
                          self.name, int(command), path.astype(np.float32), res.states.astype(np.float32), goal_
    ↪                          xy,
                          a_best, (time.perf_counter() - t0) * 1000.0,
                          {"astar_reached": reached, "mpc_cost": res.cost, "mpc_iterations": res.iterations, **s
    ↪                          cene})
```

Part 3 — MPC: the chosen profile is interpolated to 10 Hz, the reference states `[x, y, yaw, v, 0]` are built, the initial steering is estimated from the yaw rate, and iLQR tracks the reference. States at t = 0.5 … 3.0 s become the waypoints; the final collision flag is recomputed on them.

```
class LearnedPlanner:
    """Uses the network's planning-head waypoints; reports collisions against predicted occupancy."""

    name = "learned"

    def __init__(self, cfg: Any) -> None:
        self.checker = BaselinePlanner(cfg)
        self.dt = float(cfg.planning.dt)

    def plan(self, ego_state: np.ndarray, command: int, detections: dict[str, np.ndarray],
             learned_waypoints: Optional[np.ndarray] = None, lidar_points: Optional[np.ndarray] = None,
             **_: Any) -> PlanResult:
        if learned_waypoints is None:
            raise ValueError("LearnedPlanner needs the planning-head output (model.trajectory_head.enabled)")
        t0 = time.perf_counter()
        wp = np.asarray(learned_waypoints, dtype=np.float64)
        prev = np.concatenate([np.zeros((1, 2)), wp[:-1]])
        d = wp - prev
        yaws = np.arctan2(d[:, 1], d[:, 0] + 1e-9)
        yaws[np.linalg.norm(d, axis=1) < 0.05] = 0.0
        speeds = np.linalg.norm(d, axis=1) / self.dt
        scene = self.checker.build_scene(detections, lidar_points)
        collision = bool(self.checker.collision_check(wp, yaws, scene["dynamic_occ"]).any())
        return PlanResult(wp.astype(np.float32), yaws.astype(np.float32), speeds.astype(np.float32), collision,
    ↪           self.name,
                          int(command), time_ms=(time.perf_counter() - t0) * 1000.0, debug=scene)
```

`LearnedPlanner.plan()` takes the planning head's waypoints, derives headings and speeds from consecutive differences and reports collisions against the same predicted occupancy (raises `ValueError` if no learned waypoints were given).

```
class HybridPlanner:
    """Learned trajectory with a classical safety fallback."""

    name = "hybrid"

    def __init__(self, cfg: Any) -> None:
        self.learned, self.baseline = LearnedPlanner(cfg), BaselinePlanner(cfg)

    def plan(self, ego_state: np.ndarray, command: int, detections: dict[str, np.ndarray], **kwargs: Any) -> Pla
    ↪       nResult:
        res = self.learned.plan(ego_state, command, detections, **kwargs)
        if res.collision:
            fallback = self.baseline.plan(ego_state, command, detections, **kwargs)
            fallback.planner = "hybrid(baseline fallback)"
            return fallback
        res.planner = "hybrid(learned)"
        return res
```

`HybridPlanner`: uses the learned trajectory unless it collides with the predicted occupancy, in which case the baseline planner's result is returned (planner label `hybrid(baseline fallback)`).

```
def build_planner(cfg: Any, name: Optional[str] = None):
    name = name or str(cfg.planning.planner)
    return {"baseline": BaselinePlanner, "learned": LearnedPlanner, "hybrid": HybridPlanner}[name](cfg)
```

`build_planner(cfg, name)`: factory by name (default `planning.planner`).

training/trainer.py

One `Trainer` instance trains one stage of one experiment. (233 lines)

```
"""Stage-aware trainer for the 7-stage curriculum (and the single-stage ablation runs).

A *stage* (``configs/training.yaml: stages``) defines: config overrides (e.g. which
sensors), checkpoints to initialise from, frozen modules, active losses, epochs and an
LR scale. Checkpoints go to ``checkpoints/<experiment>/<stage>/{last,best}.pt`` (shared
single-sensor stages to ``checkpoints/_shared/<stage>/``) and embed the full config.

Memory / speed features: AMP (auto-disabled where fp16 is broken), gradient
accumulation, gradient checkpointing, gradient clipping, non-finite-loss guard,
persistent DataLoader workers, cuDNN autotuning.
"""
```

Docstring: what a stage is and the memory/speed features.

```
from __future__ import annotations

import json
import logging
import math
import time
from pathlib import Path
from typing import Any, Optional

import torch
from torch.utils.data import DataLoader

from datasets.nuscenes_dataset import NuScenesFusionDataset, collate_batch
from evaluation.evaluator import evaluate_model, summary_line
from models.fusion_model import MultiSensorFusionNet, build_model
from utils.checkpoint import load_checkpoint, load_model_weights, save_checkpoint
from utils.config import Config, deep_merge, load_config, parse_override, resolve_path
from utils.device import amp_enabled, get_device, gpu_memory_report, move_to_device
from utils.logging_utils import setup_logging
from utils.seed import seed_everything, worker_init_fn

logger = logging.getLogger(__name__)
```

Imports.

```
def stage_checkpoint_dir(cfg: Config, experiment: str, stage: str) -> Path:
    spec = cfg.stages[stage]
    owner = "_shared" if spec.get("shared") else experiment
    return resolve_path(cfg.runtime.checkpoint_dir) / owner / stage
```

`stage_checkpoint_dir(cfg, experiment, stage)`: shared single-sensor stages (`shared: true`) are stored under `checkpoints/_shared/<stage>/` so all experiments can reuse them; other stages under `checkpoints/<experiment>/<stage>/`.

```
def build_stage_config(experiment: Optional[str], stage: str, overrides: Optional[list[str]] = None) -> Config:
    cfg = load_config(experiment=experiment, overrides=overrides)
    if stage not in cfg.stages:
        raise KeyError(f"Unknown stage {stage!r}; available: {list(cfg.stages)}")
    merged = cfg.to_dict()
    deep_merge(merged, cfg.stages[stage].get("overrides", {}) or {})
    # CLI overrides win over stage overrides.
    cfg2 = Config(merged)
    for item in overrides or []:
        cfg2.set_path(*parse_override(item))
    cfg2.runtime.stage = stage
    return cfg2
```

`build_stage_config(experiment, stage, overrides)`: loads the experiment config, deep-merges the stage's `overrides` (e.g. camera-only for stage 1) and re-applies the CLI overrides so they still win; records `runtime.stage`.

```
class Trainer:
    def __init__(self, experiment: str, stage: str, overrides: Optional[list[str]] = None, epochs: Optional[int]
        = None,
                 resume: Optional[str] = None) -> None:
        self.experiment, self.stage = experiment, stage
        self.cfg = build_stage_config(experiment, stage, overrides)
        self.spec = self.cfg.stages[stage]
        self.ckpt_dir = stage_checkpoint_dir(self.cfg, experiment, stage)
        self.log_dir = resolve_path(self.cfg.runtime.output_dir) / "logs" / experiment / stage
        setup_logging(self.log_dir)
        seed_everything(int(self.cfg.runtime.seed), bool(self.cfg.runtime.deterministic))
        self.device = get_device(self.cfg.runtime.device)
        self.use_amp = amp_enabled(self.device, bool(self.cfg.train.amp))
```

```
68          self.epochs = int(epochs or self.spec.get("epochs") or self.cfg.train.epochs)
69          self.active_losses = list(self.spec.losses)
70          logger.info("Experiment %s | stage %s: %s", experiment, stage, self.spec.get("description", ""))
71          logger.info("Device %s | AMP %s | epochs %d | losses %s", self.device, self.use_amp, self.epochs, self.ac
    ↪           tive_losses)
```

Constructor, part 1: configuration, checkpoint and log directories, logging to `outputs/logs/<experiment>/<stage>/run.log`, seeding, device and AMP decision (fp16 self-test), epoch count (CLI > stage > `train.epochs`), active losses.

```
73          self.train_set = NuScenesFusionDataset(self.cfg, self.cfg.dataset.train_split, train=True)
74          self.val_set = NuScenesFusionDataset(self.cfg, self.cfg.dataset.val_split, train=False)
75          nw = int(self.cfg.dataset.num_workers)
76          common = dict(collate_fn=collate_batch, num_workers=nw, pin_memory=bool(self.cfg.dataset.pin_memory) and
    ↪           self.device.type == "cuda",
77                        persistent_workers=nw > 0, worker_init_fn=worker_init_fn)
78          self.train_loader = DataLoader(self.train_set, batch_size=int(self.cfg.train.batch_size), shuffle=True,
79                                         drop_last=True, **common)
80          self.val_loader = DataLoader(self.val_set, batch_size=1, shuffle=False, **common)
```

Part 2 — data: train and validation datasets, DataLoaders with `collate_batch`, `num_workers`, pinned memory on CUDA, persistent workers and per-worker seeding; training drops the last incomplete batch; validation uses batch size 1.

```
82          self.model: MultiSensorFusionNet = build_model(self.cfg).to(self.device)
83          self.model.set_grad_checkpoint(bool(self.cfg.train.grad_checkpoint))
84          self._initialise_from_stages()
85          self._freeze()
86          params = [p for p in self.model.parameters() if p.requires_grad]
87          n_train = sum(p.numel() for p in params)
88          n_all = sum(p.numel() for p in self.model.parameters())
89          logger.info("Parameters: %.2fM total, %.2fM trainable", n_all / 1e6, n_train / 1e6)
90          lr = float(self.cfg.train.lr) * float(self.spec.get("lr_scale", 1.0))
91          self.optimizer = torch.optim.AdamW(params, lr=lr, weight_decay=float(self.cfg.train.weight_decay))
92          self.accum = max(int(self.cfg.train.grad_accum), 1)
93          steps_per_epoch = max(len(self.train_loader) // self.accum, 1)
94          total = steps_per_epoch * self.epochs
95          warmup = min(int(self.cfg.train.warmup_iters), max(total // 10, 1))
96          self.scheduler = torch.optim.lr_scheduler.LambdaLR(
97              self.optimizer, lambda s: min(1.0, (s + 1) / warmup) * 0.5 * (1 + math.cos(math.pi * min(s / max(tot
    ↪               al, 1), 1.0))))
98          self.scaler = torch.amp.GradScaler("cuda", enabled=self.use_amp)
99          self.start_epoch, self.step = 0, 0
100         self.best_score = -float("inf")
101         if resume:
102             self._resume(resume)
103
104     # ---------------------------------------------------------------- setup
```

Part 3 — model and optimisation: build the network, enable gradient checkpointing, load initial weights from earlier stages, freeze modules, count parameters, AdamW (lr × stage `lr_scale`, weight decay), gradient accumulation, a LambdaLR schedule with linear warm-up followed by cosine decay over all optimiser steps, the AMP `GradScaler`, and optional resume.

```
105     def _initialise_from_stages(self) -> None:
106         for item in self.spec.get("init_from", []) or []:
107             src = stage_checkpoint_dir(self.cfg, self.experiment, item["stage"])
108             path = src / "best.pt" if (src / "best.pt").exists() else src / "last.pt"
109             if not path.exists():
110                 logger.warning("Init checkpoint for stage %s not found (%s) - training those weights from scratc
    ↪                    h.",
111                                item["stage"], path)
112                 continue
113             state = load_checkpoint(path)["model"]
114             report = load_model_weights(self.model, state, item.get("prefixes"))
115             logger.info("Initialised %d tensors from %s (%d shape-mismatch skipped)", len(report["loaded"]), pat
    ↪                h,
116                         len(report["skipped_shape"]))
```

`_initialise_from_stages()`: for every `init_from` entry of the stage, loads `best.pt` (or `last.pt`) of the source stage and copies only the parameters whose names start with the given prefixes and whose shapes match (`load_model_weights`); a missing source checkpoint is logged and those weights are trained from scratch.

```
118     def _freeze(self) -> None:
119         only = self.spec.get("trainable_only")
120         if only:
121             keep = tuple(p.rstrip(".") + "." for p in only)
122             for name, p in self.model.named_parameters():
```

```
                p.requires_grad_(name.startswith(keep))
            logger.info("Training only %s", only)
        frozen = self.spec.get("frozen") or []
        if frozen:
            names = self.model.freeze(frozen)
            logger.info("Frozen %d tensors under %s", len(names), frozen)
```

`_freeze()`: `trainable_only` freezes everything except the listed prefixes (stage 6 trains only `plan_head`); `frozen` freezes the listed prefixes (e.g. the three encoders in stage 4).

```
    def _resume(self, path: str) -> None:
        ckpt = load_checkpoint(Path(path), map_location=self.device)
        self.model.load_state_dict(ckpt["model"])
        if "optimizer" in ckpt:
            self.optimizer.load_state_dict(ckpt["optimizer"])
        if "scheduler" in ckpt:
            self.scheduler.load_state_dict(ckpt["scheduler"])
        if "scaler" in ckpt:
            self.scaler.load_state_dict(ckpt["scaler"])
        self.start_epoch, self.step = int(ckpt.get("epoch", 0)), int(ckpt.get("step", 0))
        self.best_score = float(ckpt.get("extra", {}).get("best_score", -float("inf")))
        logger.info("Resumed from %s at epoch %d", path, self.start_epoch)

    # ------------------------------------------------------------------ train
```

`_resume(path)`: restores model, optimiser, scheduler, scaler, epoch, step and the best score.

```
    def _score(self, res: dict[str, Any]) -> float:
        if "detection" in self.active_losses:
            return float(res["detection"]["NDS"])
        if "trajectory" in self.active_losses and "planning_learned" in res:
            return -float(res["planning_learned"].get("ade", float("inf")))
        return -float(res.get("val_losses", {}).get("total", float("inf")))
```

`_score(res)`: the model-selection criterion for `best.pt` — NDS for stages with a detection loss, −ADE for the trajectory stage, −validation loss otherwise.

```
    def _set_train_mode(self) -> None:
        """``model.train()``, but BatchNorm layers whose parameters are frozen stay in eval mode,
        so frozen modules do not drift through their running statistics."""
        self.model.train()
        for m in self.model.modules():
            if isinstance(m, (torch.nn.BatchNorm1d, torch.nn.BatchNorm2d)):
                params = list(m.parameters())
                if params and not any(p.requires_grad for p in params):
                    m.eval()
```

`_set_train_mode()`: `model.train()`, then every BatchNorm whose parameters are all frozen is put back into eval mode — otherwise the running statistics of "frozen" modules would keep drifting during later stages.

```
    def train_epoch(self, epoch: int) -> dict[str, float]:
        self._set_train_mode()
        sums: dict[str, float] = {}
        n, skipped = 0, 0
        t_start = time.perf_counter()
        self.optimizer.zero_grad(set_to_none=True)
        for it, batch in enumerate(self.train_loader):
            batch = move_to_device(batch, self.device)
            with torch.autocast(device_type=self.device.type, dtype=torch.float16, enabled=self.use_amp):
                out = self.model(batch)
                losses = self.model.compute_losses(out, batch, self.cfg.loss, self.active_losses)
            loss = losses["total"] / self.accum
            if not torch.isfinite(loss):
                skipped += 1
                logger.warning("Non-finite loss at iteration %d (%s) - batch skipped", it,
                               {k: float(v) for k, v in losses.items()})
                self.optimizer.zero_grad(set_to_none=True)
                continue
            self.scaler.scale(loss).backward()
            if (it + 1) % self.accum == 0:
                self.scaler.unscale_(self.optimizer)
                torch.nn.utils.clip_grad_norm_([p for p in self.model.parameters() if p.requires_grad],
                                               float(self.cfg.train.grad_clip))
                self.scaler.step(self.optimizer)
                self.scaler.update()
                self.scheduler.step()
                self.optimizer.zero_grad(set_to_none=True)
                self.step += 1
            for k, v in losses.items():
                sums[k] = sums.get(k, 0.0) + float(v.detach())
            n += 1
            if (it + 1) % int(self.cfg.train.log_interval) == 0:
                rate = (it + 1) / (time.perf_counter() - t_start)
```

```
                mem = gpu_memory_report(self.device).get("peak_allocated_mb", 0)
                logger.info("ep %d it %d/%d | loss %.3f | %s | lr %.2e | %.2f it/s | peak %.0f MB", epoch + 1, i
    ↪                   t + 1,
                        len(self.train_loader), sums["total"] / n,
                        " ".join(f"{k} {sums[k] / n:.3f}" for k in self.active_losses if k in sums),
                        self.optimizer.param_groups[0]["lr"], rate, mem)
        avg = {k: v / max(n, 1) for k, v in sums.items()}
        avg["skipped_batches"] = skipped
        avg["epoch_time_s"] = time.perf_counter() - t_start
        return avg
```

`train_epoch(epoch)`: for each batch — move to device; forward and `compute_losses(active)` under autocast; divide by `grad_accum`; skip the batch (and log) if the loss is not finite; scaled backward; every `grad_accum` iterations unscale → clip the gradient norm (`train.grad_clip`) → optimiser step → scaler update → scheduler step → zero gradients. Running averages of every loss are logged every `log_interval` iterations with the learning rate, iterations/s and peak GPU memory.

```
    def fit(self) -> dict[str, Any]:
        history_path = self.log_dir / "history.jsonl"
        last_res: dict[str, Any] = {}
        for epoch in range(self.start_epoch, self.epochs):
            train_stats = self.train_epoch(epoch)
            record: dict[str, Any] = {"epoch": epoch + 1, "train": train_stats}
            if (epoch + 1) % int(self.cfg.train.val_interval) == 0 or epoch + 1 == self.epochs:
                res = evaluate_model(self.model, self.val_loader, self.cfg, self.device, self.use_amp)
                logger.info("VAL ep %d | %s", epoch + 1, summary_line(res))
                record["val"] = {"detection": {k: v for k, v in res["detection"].items() if k != "per_class"},
                                 **{k: v for k, v in res.items() if k != "detection"}}
                score = self._score(res)
                last_res = res
                if score > self.best_score:
                    self.best_score = score
                    self._save("best.pt", epoch + 1, res)
                    logger.info("New best (%.4f) -> %s", score, self.ckpt_dir / "best.pt")
            self._save("last.pt", epoch + 1, last_res)
            with open(history_path, "a", encoding="utf-8") as f:
                f.write(json.dumps(record, default=float) + "\n")
        return last_res
```

`fit()`: epoch loop — train, validate every `val_interval` epochs with `evaluate_model`, log a summary line, save `best.pt` when the score improves and `last.pt` every epoch, and append a JSON record to `history.jsonl` (used for the dashboard's training curves).

```
    def _save(self, name: str, epoch: int, res: dict[str, Any]) -> None:
        metrics = {}
        if res:
            metrics = {"detection": {k: v for k, v in res["detection"].items() if k != "per_class"},
                       **{k: v for k, v in res.items() if k in ("planning_learned", "feature_selection", "latenc
    ↪                       y_ms")}}
        save_checkpoint(self.ckpt_dir / name, self.model, self.cfg.to_dict(), optimizer=self.optimizer,
                        scheduler=self.scheduler, scaler=self.scaler, epoch=epoch, step=self.step, metrics=metri
    ↪                       cs,
                        extra={"best_score": self.best_score, "experiment": self.experiment, "stage": self.stage
    ↪                       })
```

`_save(name, epoch, res)`: metrics without the per-class table + `save_checkpoint` with optimiser/scheduler/scaler state, the full configuration and `extra` (best score, experiment, stage).

evaluation/detection_metrics.py

In-memory port of the official nuScenes detection evaluation; cross-checked against the devkit (max difference $3 \cdot 10^{-4}$ on NDS/mAP/TP errors). (205 lines)

```
"""nuScenes detection metrics (mAP, mATE, mASE, mAOE, mAVE, mAAE, NDS) — in-memory port.

This reproduces the official ``nuscenes.eval.detection`` algorithm (center-distance
matching at 0.5/1/2/4 m, 101-point interpolated precision, min recall/precision 0.1, TP
errors at the 2 m threshold with confidence-aligned cumulative means, class-specific
ranges and NaN rules), so numbers are comparable without writing result files. Use
:mod:`evaluation.official_eval` to cross-check with the devkit itself.

Note (official behaviour): a class with **no GT** in the evaluated split gets AP = 0 and
TP errors = 1. On ``mini_val`` (2 scenes) several classes are absent, which caps the
achievable mAP/NDS; ``mAP_present_classes`` is reported additionally for context.

Boxes are ``[x, y, z, l, w, h, yaw, vx, vy]`` in the ego/reference frame (all nuScenes
metrics are invariant to the rigid ego->global transform).
"""
from __future__ import annotations
```

```
17
18 from collections import defaultdict
19 from typing import Optional, Sequence
20
21 import numpy as np
22
23 from utils.geometry import points_in_boxes
```

Docstring: algorithm summary and the official rule that classes without GT score AP = 0 (relevant on mini-val).

```
25 CLASS_RANGE = {"car": 50, "truck": 50, "bus": 50, "trailer": 50, "construction_vehicle": 50, "pedestrian": 40,
26                "motorcycle": 40, "bicycle": 40, "traffic_cone": 30, "barrier": 30}
27 DIST_THS = (0.5, 1.0, 2.0, 4.0)
28 DIST_TH_TP = 2.0
29 MIN_RECALL, MIN_PRECISION = 0.1, 0.1
30 MAX_BOXES_PER_SAMPLE = 500
31 TP_METRICS = ("trans_err", "scale_err", "orient_err", "vel_err", "attr_err")
32 NELEM = 101
```

Official constants: class-specific evaluation ranges (30–50 m), the four matching distances (0.5, 1, 2, 4 m), the 2 m threshold for TP metrics, minimum recall/precision 0.1, 500 boxes per sample, the five TP metrics and the 101-point recall grid.

```
35 def default_attribute(name: str, velocity: np.ndarray) -> str:
36     """Attribute heuristic used by CenterPoint-style detectors (no attribute head)."""
37     moving = float(np.hypot(*velocity[:2])) > 0.2
38     if name in ("car", "truck", "bus", "trailer", "construction_vehicle"):
39         return "vehicle.moving" if moving else "vehicle.parked"
40     if name == "pedestrian":
41         return "pedestrian.moving" if moving else "pedestrian.standing"
42     if name in ("motorcycle", "bicycle"):
43         return "cycle.with_rider" if moving else "cycle.without_rider"
44     return ""
```

`default_attribute(name, velocity)`: attributes are not predicted by the network; this heuristic assigns moving/parked (vehicles), moving/standing (pedestrians), with/without rider (cycles) from the predicted speed (> 0.2 m/s).

```
47 def _angle_diff(x: float, y: float, period: float) -> float:
48     diff = (x - y + period / 2) % period - period / 2
49     if diff > np.pi:
50         diff -= 2 * np.pi
51     return abs(diff)
52
53
54 def _scale_iou(a: np.ndarray, b: np.ndarray) -> float:
55     inter = np.prod(np.minimum(a, b))
56     return float(inter / (np.prod(a) + np.prod(b) - inter))
57
58
59 def _cummean(x: np.ndarray) -> np.ndarray:
60     if np.isnan(x).all():
61         return np.ones(len(x))
62     s = np.nancumsum(x.astype(float))
63     n = np.cumsum(~np.isnan(x))
64     return np.divide(s, n, out=np.zeros_like(s), where=n != 0)
```

Helpers: `_angle_diff` (smallest absolute angle difference with a period — π for barriers, 2π otherwise), `_scale_iou` (IoU of two aligned boxes from their sizes), `_cummean` (NaN-aware cumulative mean; all-NaN → ones).

```
67 class DetectionEvaluator:
68     """Accumulate predictions/GT sample by sample, then call :meth:`evaluate`."""
69
70     def __init__(self, classes: Sequence[str]) -> None:
71         self.classes = list(classes)
72         self.gt: dict[str, list[dict]] = defaultdict(list)     # class -> list of gt dicts
73         self.pred: dict[str, list[dict]] = defaultdict(list)
74         self.samples: list[str] = []
```

`DetectionEvaluator.__init__`: per-class lists of GT and predictions.

```
76     def add_sample(self, token: str, pred_boxes: np.ndarray, pred_scores: np.ndarray, pred_labels: np.ndarray,
77                    gt_boxes: np.ndarray, gt_labels: np.ndarray, gt_attributes: Optional[Sequence[str]] = None,
78                    gt_num_pts: Optional[np.ndarray] = None, gt_velocity_valid: Optional[np.ndarray] = None,
79                    pred_attributes: Optional[Sequence[str]] = None, bike_racks: Optional[np.ndarray] = None) ->
   ↪                    None:
80         self.samples.append(token)
81         racks = bike_racks if bike_racks is not None and len(bike_racks) else None
82
```

```
        def in_rack(name: str, box: np.ndarray) -> bool:
            return racks is not None and name in ("bicycle", "motorcycle") and bool(
                points_in_boxes(box[None, :3].astype(np.float64), racks.astype(np.float64)).any())

        order = np.argsort(-pred_scores)[:MAX_BOXES_PER_SAMPLE]
        for i in order:
            name = self.classes[int(pred_labels[i])]
            box = pred_boxes[i]
            if np.hypot(box[0], box[1]) > CLASS_RANGE[name] or in_rack(name, box):
                continue
            attr = pred_attributes[i] if pred_attributes is not None else default_attribute(name, box[7:9])
            self.pred[name].append({"token": token, "box": box.astype(np.float64), "score": float(pred_scores[i]
                ), "attr": attr})
        for j in range(len(gt_labels)):
            name = self.classes[int(gt_labels[j])]
            box = gt_boxes[j].astype(np.float64).copy()
            if np.hypot(box[0], box[1]) > CLASS_RANGE[name]:
                continue
            if (gt_num_pts is not None and gt_num_pts[j] == 0) or in_rack(name, box):
                continue
            if gt_velocity_valid is not None and not gt_velocity_valid[j]:
                box[7:9] = np.nan
            self.gt[name].append({"token": token, "box": box, "attr": gt_attributes[j] if gt_attributes is not N
                one else ""})

    # ---------------------------------------------------------- accumulate
```

`add_sample(...)`: keeps the 500 highest-scoring predictions, drops boxes beyond the class range, drops bicycles/motorcycles inside bicycle racks (both GT and predictions), drops GT with zero LiDAR+radar points and sets the GT velocity to NaN where it is undefined.

```
    def _accumulate(self, name: str, dist_th: float) -> dict[str, np.ndarray]:
        gts = self.gt[name]
        npos = len(gts)
        empty = {"precision": np.zeros(NELEM), "confidence": np.zeros(NELEM),
                 **{m: np.ones(NELEM) for m in TP_METRICS}}
        if npos == 0:
            return empty
        by_token: dict[str, list[int]] = defaultdict(list)
        for k, g in enumerate(gts):
            by_token[g["token"]].append(k)
        preds = sorted(self.pred[name], key=lambda p: -p["score"])
        taken = np.zeros(npos, dtype=bool)
        tp, fp, conf = [], [], []
        md = {m: [] for m in TP_METRICS}
        md_conf = []
        period = np.pi if name == "barrier" else 2 * np.pi
        for p in preds:
            best, best_d = None, np.inf
            for k in by_token.get(p["token"], ()):
                if taken[k]:
                    continue
                d = float(np.hypot(*(gts[k]["box"][:2] - p["box"][:2])))
                if d < best_d:
                    best, best_d = k, d
            if best is not None and best_d < dist_th:
                taken[best] = True
                tp.append(1)
                fp.append(0)
                g, b = gts[best]["box"], p["box"]
                md["trans_err"].append(best_d)
                md["vel_err"].append(float(np.hypot(*(g[7:9] - b[7:9]))))
                # nuScenes sizes are [w, l, h]; the IoU is symmetric in the order, only alignment matters.
                md["scale_err"].append(1.0 - _scale_iou(g[3:6], b[3:6]))
                md["orient_err"].append(_angle_diff(g[6], b[6], period))
                ga = gts[best]["attr"]
                md["attr_err"].append(np.nan if ga == "" else float(ga != p["attr"]))
                md_conf.append(p["score"])
            else:
                tp.append(0)
                fp.append(1)
            conf.append(p["score"])
        if sum(tp) == 0:
            return empty
        tp_c, fp_c = np.cumsum(tp).astype(float), np.cumsum(fp).astype(float)
        prec = tp_c / (fp_c + tp_c)
        rec = tp_c / float(npos)
        rec_interp = np.linspace(0, 1, NELEM)
        prec_i = np.interp(rec_interp, rec, prec, right=0)
        conf_i = np.interp(rec_interp, rec, np.asarray(conf), right=0)
        out = {"precision": prec_i, "confidence": conf_i}
        md_conf_a = np.asarray(md_conf)
        for m in TP_METRICS:
            tmp = _cummean(np.asarray(md[m], dtype=float))
```

```
160             out[m] = np.interp(conf_i[::-1], md_conf_a[::-1], tmp[::-1])[::-1]
161         return out
```

`_accumulate(name, dist_th)` — the core:

1. Predictions of the class sorted by score.
2. Greedy matching: each prediction takes the nearest unmatched GT of the same sample (2-D centre distance); a match below the threshold is a TP (errors recorded: translation, velocity, scale, orientation, attribute), otherwise an FP.
3. Cumulative TP/FP → precision and recall; precision and confidence are interpolated on the 101-point recall grid.
4. Each TP-error list is turned into a cumulative mean and re-sampled at the interpolated confidences (official alignment).

No GT or no TP → precision 0, errors 1.

```
163     @staticmethod
164     def _calc_ap(md: dict[str, np.ndarray]) -> float:
165         prec = md["precision"][round(100 * MIN_RECALL) + 1:].copy()
166         prec -= MIN_PRECISION
167         prec[prec < 0] = 0
168         return float(np.mean(prec)) / (1.0 - MIN_PRECISION)
169 
170     @staticmethod
171     def _calc_tp(md: dict[str, np.ndarray], metric: str) -> float:
172         first = round(100 * MIN_RECALL) + 1
173         nz = np.nonzero(md["confidence"])[0]
174         last = int(nz[-1]) if len(nz) else 0
175         if last < first:
176             return 1.0
177         return float(np.mean(md[metric][first:last + 1]))
178 
179     # ---------------------------------------------------------------- evaluate
```

`_calc_ap`: mean of `max(precision − 0.1, 0)` above 10 % recall, normalised by 0.9. `_calc_tp`: mean TP error between 10 % recall and the maximum achieved recall (1.0 if that range is empty).

```
180     def evaluate(self) -> dict[str, object]:
181         per_class: dict[str, dict[str, float]] = {}
182         for name in self.classes:
183             aps = {th: self._calc_ap(self._accumulate(name, th)) for th in DIST_THS}
184             md_tp = self._accumulate(name, DIST_TH_TP)
185             res = {"AP": float(np.mean(list(aps.values()))), **{f"AP@{th}": v for th, v in aps.items()},
186                    "num_gt": len(self.gt[name]), "num_pred": len(self.pred[name])}
187             for m in TP_METRICS:
188                 if (name == "traffic_cone" and m in ("attr_err", "vel_err", "orient_err")) or \
189                         (name == "barrier" and m in ("attr_err", "vel_err")):
190                     res[m] = float("nan")
191                 else:
192                     res[m] = self._calc_tp(md_tp, m)
193             per_class[name] = res
194         m_ap = float(np.mean([per_class[c]["AP"] for c in self.classes]))
195         tp_errors = {m: float(np.nanmean([per_class[c][m] for c in self.classes])) for m in TP_METRICS}
196         tp_scores = {m: max(0.0, 1.0 - tp_errors[m]) for m in TP_METRICS}
197         nds = (5 * m_ap + sum(tp_scores.values())) / (5 + len(TP_METRICS))
198         present = [c for c in self.classes if per_class[c]["num_gt"] > 0]
199         return {
200             "NDS": nds, "mAP": m_ap,
201             "mATE": tp_errors["trans_err"], "mASE": tp_errors["scale_err"], "mAOE": tp_errors["orient_err"],
202             "mAVE": tp_errors["vel_err"], "mAAE": tp_errors["attr_err"],
203             "mAP_present_classes": float(np.mean([per_class[c]["AP"] for c in present])) if present else 0.0,
204             "num_samples": len(self.samples), "per_class": per_class,
205         }
```

`evaluate()`: per class AP at the four thresholds (mean = class AP) and TP errors at 2 m (NaN for metrics that do not apply: cone orientation/velocity/attribute, barrier velocity/attribute); mAP = mean AP over the 10 classes; mean TP errors (NaN-mean); `NDS = (5·mAP + Σ_metrics (1 − min(1, error))) / 10`; plus `mAP_present_classes` for context.

evaluation/trajectory_metrics.py

Open-loop planning metrics against the logged human trajectory. (115 lines)

```
"""Planning metrics against the logged (human) ego trajectory.

* **ADE / FDE**: mean / final L2 displacement over valid steps.
* **L2 @ 1 s, 2 s, 3 s**: displacement error at those horizons (ST-P3 / UniAD protocol).
* **Collision rate**: fraction of samples whose ego footprint (placed at the planned
  waypoint, oriented along the trajectory) overlaps a GT box of another agent at the same
  future time (agents taken from the real future annotations, not a forecast).
* **Lateral / longitudinal error**: error decomposed in the frame of the logged trajectory.
* **Smoothness**: mean |jerk| (m/s³) and mean |curvature change|.
"""
from __future__ import annotations

from typing import Any, Iterable

import numpy as np

from utils.geometry import box_corners_bev
```

Docstring listing the metrics.

```
def _ego_box(xy: np.ndarray, yaw: float, ego_cfg: Any) -> np.ndarray:
    c = float(ego_cfg.rear_axle_to_center)
    return np.array([xy[0] + c * np.cos(yaw), xy[1] + c * np.sin(yaw), 0.0, float(ego_cfg.length),
                     float(ego_cfg.width), 1.6, yaw])


def _polygons_intersect(a: np.ndarray, b: np.ndarray) -> bool:
    """Separating-axis test for two convex quadrilaterals ``(4, 2)``."""
    for poly in (a, b):
        for i in range(4):
            edge = poly[(i + 1) % 4] - poly[i]
            axis = np.array([-edge[1], edge[0]])
            pa, pb = a @ axis, b @ axis
            if pa.max() < pb.min() or pb.max() < pa.min():
                return False
    return True
```

`_ego_box(xy, yaw, ego_cfg)`: ego bounding box from a rear-axle pose. `_polygons_intersect(a, b)`: separating-axis test for two convex quadrilaterals.

```
def trajectory_headings(xy: np.ndarray) -> np.ndarray:
    prev = np.concatenate([np.zeros((1, 2)), xy[:-1]])
    d = xy - prev
    yaw = np.arctan2(d[:, 1], d[:, 0])
    yaw[np.linalg.norm(d, axis=1) < 0.05] = 0.0
    return yaw
```

`trajectory_headings(xy)`: heading of each step from the previous point (0 if the ego barely moves).

```
def collision_flags(pred: np.ndarray, agent_future: list[dict], collision_labels: Iterable[int], ego_cfg: Any) ->
    np.ndarray:
    """Per-step collision flags ``(T,)`` against the GT boxes of other agents."""
    labels = set(int(l) for l in collision_labels)
    yaws = trajectory_headings(pred)
    flags = np.zeros(len(pred), dtype=bool)
    for t in range(min(len(pred), len(agent_future))):
        boxes = agent_future[t]["boxes"]
        sel = np.array([int(l) in labels for l in agent_future[t]["labels"]], dtype=bool)
        if not sel.any():
            continue
        ego_poly = box_corners_bev(_ego_box(pred[t], yaws[t], ego_cfg)[None])[0]
        near = np.hypot(boxes[sel, 0] - pred[t, 0], boxes[sel, 1] - pred[t, 1]) < 12.0
        for poly in box_corners_bev(boxes[sel][near].astype(np.float64)):
            if _polygons_intersect(ego_poly, poly):
                flags[t] = True
                break
    return flags
```

`collision_flags(pred, agent_future, collision_labels, ego_cfg)` → `(T,)`: at each step, the ego polygon is tested (SAT) against the **real** future boxes of the collision classes within 12 m. Using the logged agent positions (not a forecast) makes this an objective collision measure; sanity check: the logged human trajectory scores 0 % collisions.

```
def sample_planning_metrics(pred: np.ndarray, gt: np.ndarray, mask: np.ndarray, dt: float,
                            agent_future: list[dict] | None = None, collision_labels: Iterable[int] = (),
                            ego_cfg: Any = None) -> dict[str, float]:
    """Metrics of one sample (only valid future steps are used)."""
    mask = mask.astype(bool)
```

```
    out: dict[str, float] = {}
    if not mask.any():
        return out
    err = np.linalg.norm(pred - gt, axis=1)
    out["ade"] = float(err[mask].mean())
    last = np.nonzero(mask)[0][-1]
    out["fde"] = float(err[last])
    for sec in (1, 2, 3):
        k = int(round(sec / dt)) - 1
        if k < len(mask) and mask[k]:
            out[f"l2_{sec}s"] = float(err[k])
    # Lateral / longitudinal errors in the logged trajectory's frame.
    gt_yaw = trajectory_headings(gt)
    d = pred - gt
    lon = d[:, 0] * np.cos(gt_yaw) + d[:, 1] * np.sin(gt_yaw)
    lat = -d[:, 0] * np.sin(gt_yaw) + d[:, 1] * np.cos(gt_yaw)
    out["lateral_error"] = float(np.abs(lat[mask]).mean())
    out["longitudinal_error"] = float(np.abs(lon[mask]).mean())
    # Smoothness of the predicted trajectory.
    path = np.concatenate([np.zeros((1, 2)), pred], 0)
    vel = np.diff(path, axis=0) / dt
    acc = np.diff(vel, axis=0) / dt
    jerk = np.diff(acc, axis=0) / dt
    out["jerk"] = float(np.linalg.norm(jerk, axis=1).mean()) if len(jerk) else 0.0
    yaws = np.unwrap(trajectory_headings(pred))
    out["curvature_change"] = float(np.abs(np.diff(yaws, 2)).mean()) if len(yaws) > 2 else 0.0
    if agent_future is not None and ego_cfg is not None:
        flags = collision_flags(pred, agent_future, collision_labels, ego_cfg)
        flags &= mask
        out["collision"] = float(flags.any())
        for sec in (1, 2, 3):
            k = int(round(sec / dt))
            out[f"collision_{sec}s"] = float(flags[:k].any())
    return out
```

`sample_planning_metrics(pred, gt, mask, dt, …)`: ADE, FDE, L2 at 1/2/3 s, lateral and longitudinal error (in the GT trajectory's heading frame), mean jerk and curvature change of the prediction, collision flags overall and up to 1/2/3 s.

```
def aggregate(metrics: list[dict[str, float]]) -> dict[str, float]:
    """Mean of each metric over the samples that define it (collision -> collision rate)."""
    keys = sorted(set().union(*metrics)) if metrics else []
    out = {}
    for k in keys:
        vals = [m[k] for m in metrics if k in m]
        name = k.replace("collision", "collision_rate") if k.startswith("collision") else k
        out[name] = float(np.mean(vals)) if vals else float("nan")
    out["num_samples"] = len(metrics)
    return out
```

`aggregate(metrics)`: mean of every metric over the samples that define it; `collision*` keys are renamed `collision_rate*`.

evaluation/evaluator.py

Runs a model over a DataLoader and computes all metric families (used by the trainer, the ablation runner and `scripts/evaluate.py`). (106 lines)

```
"""Run a trained model over a split and compute every metric family.

* detection: nuScenes mAP / mATE / mASE / mAOE / mAVE / mAAE / NDS
* planning: ADE / FDE / L2@1-3 s / collision rate / lateral & longitudinal error / smoothness
  (learned planning head and, optionally, the classical baseline fed with the model's detections)
* feature selection: selected-feature ratios and mean LiDAR / radar importance
* latency: per-stage and total network time (CUDA-synchronised)
"""
from __future__ import annotations

import logging
from collections import defaultdict
from typing import Any, Optional

import numpy as np
import torch
from torch.utils.data import DataLoader

from evaluation.detection_metrics import DetectionEvaluator
from evaluation.trajectory_metrics import aggregate, sample_planning_metrics
from models.fusion_model import ALL_LOSSES, MultiSensorFusionNet
from planning.trajectory_planner import BaselinePlanner
from utils.config import Config
```

```
from utils.device import move_to_device
from utils.timer import StageTimer

logger = logging.getLogger(__name__)
```

Docstring.

```
@torch.no_grad()
def evaluate_model(model: MultiSensorFusionNet, loader: DataLoader, cfg: Config, device: torch.device,
                   use_amp: bool = False, compute_losses: bool = True, run_baseline_planner: bool = False,
                   max_batches: Optional[int] = None) -> dict[str, Any]:
    model.eval()
    infos = loader.dataset.infos
    classes = list(cfg.classes)
    collision_labels = [classes.index(c) for c in cfg.planning.collision_classes]
    dt = float(cfg.planning.dt)
    det_eval = DetectionEvaluator(classes)
    learned_plan, baseline_plan = [], []
    fs_stats: dict[str, list[float]] = defaultdict(list)
    loss_sums: dict[str, float] = defaultdict(float)
    stage_ms: dict[str, list[float]] = defaultdict(list)
    baseline = BaselinePlanner(cfg) if run_baseline_planner else None
    n_batches = 0
    for bi, batch in enumerate(loader):
        if max_batches is not None and bi >= max_batches:
            break
        batch = move_to_device(batch, device)
        timer = StageTimer()
        with torch.autocast(device_type=device.type, dtype=torch.float16, enabled=use_amp):
            out = model(batch, timer=timer)
            if compute_losses and "heatmap" in batch:
                for k, v in model.compute_losses(out, batch, cfg.loss, ALL_LOSSES).items():
                    loss_sums[k] += float(v)
        for k, v in timer.as_dict().items():
            stage_ms[k].append(v)
        stage_ms["total"].append(timer.total_ms)
        dets = model.decode(out, score_threshold=float(cfg.eval.score_threshold))
        if "selection" in out:
            for k, v in model.feature_selection.selection_stats(out["selection"]).items():
                fs_stats[k].append(v)
        for b, meta in enumerate(batch["meta"]):
            info = infos[meta["index"]]
            gt = info["gt"]
            d = {k: v.float().cpu().numpy() if v.is_floating_point() else v.cpu().numpy() for k, v in dets[b].ite
 ↪               ms()}
            det_eval.add_sample(info["token"], d["boxes"], d["scores"], d["labels"], gt["boxes"], gt["labels"],
                                gt["attributes"], gt["num_lidar_pts"] + gt["num_radar_pts"], gt["velocity_valid"]
 ↪                                  ,
                                bike_racks=gt.get("bike_racks"))
            fut = info["ego_future"]
            if not fut["mask"].all():
                continue
            if "plan" in out:
                pred = out["plan"]["waypoints"][b].float().cpu().numpy()
                learned_plan.append(sample_planning_metrics(pred, fut["xy"], fut["mask"], dt, info["agent_future"
 ↪                  ],
                                                            collision_labels, cfg.planning.ego))
            if baseline is not None:
                es = info["ego_state"]
                res = baseline.plan(np.array([es["speed"], es["yaw_rate"], es["accel"]]), info["command"], d)
                baseline_plan.append(sample_planning_metrics(res.waypoints, fut["xy"], fut["mask"], dt, info["age
 ↪                  nt_future"],
                                                             collision_labels, cfg.planning.ego))
        n_batches += 1
    result: dict[str, Any] = {"detection": det_eval.evaluate()}
    if learned_plan:
        result["planning_learned"] = aggregate(learned_plan)
    if baseline_plan:
        result["planning_baseline"] = aggregate(baseline_plan)
    if fs_stats:
        result["feature_selection"] = {k: float(np.mean(v)) for k, v in fs_stats.items()}
    if loss_sums:
        result["val_losses"] = {k: v / max(n_batches, 1) for k, v in loss_sums.items()}
    batch_size = max(int(cfg.train.get("eval_batch_size", 1)), 1)
    result["latency_ms"] = {k: float(np.median(v)) / batch_size for k, v in stage_ms.items()}
    return result
```

`evaluate_model(model, loader, cfg, device, use_amp, compute_losses, run_baseline_planner, max_batches)`: eval mode; for each batch forward (timed per stage), optional validation losses, decoding at `eval.score_threshold`, feature-selection statistics; per sample the detections are added to the `DetectionEvaluator` with the *unfiltered* GT from the infos (the evaluator applies the official filters); for samples with a complete 3 s future the learned planner (and optionally the baseline planner fed with the

detections) are scored. Returns detection metrics, planning metrics, FS stats, mean losses and median latency per stage.

```
def summary_line(res: dict[str, Any]) -> str:
    det = res["detection"]
    parts = [f"NDS {det['NDS']:.4f}", f"mAP {det['mAP']:.4f}", f"mAP(present) {det['mAP_present_classes']:.4f}",
             f"mAVE {det['mAVE']:.3f}"]
    if "planning_learned" in res:
        p = res["planning_learned"]
        parts += [f"ADE {p.get('ade', float('nan')):.3f}", f"FDE {p.get('fde', float('nan')):.3f}",
                  f"col {p.get('collision_rate', float('nan')):.3f}"]
    parts.append(f"{res['latency_ms'].get('total', 0):.1f} ms")
    return " | ".join(parts)
```

`summary_line(res)`: the one-line log summary (NDS, mAP, mAP present, mAVE, ADE, FDE, collision rate, latency).

inference/pipeline.py

Frame-by-frame processing shared by the dashboard and `scripts/inference.py` / `render_bev.py`. (268 lines)

```
"""Frame-by-frame inference pipeline shared by the dashboard and ``scripts/inference.py``.

Three honest modes (no synthetic data anywhere):

* ``dataset``      - raw nuScenes sensors only (works before any training).
* ``ground_truth`` - nuScenes annotations as "perception", logged ego trajectory; the
                     classical planner can be run on the GT objects (clearly labelled).
* ``model``        - the trained network: camera BEV -> LiDAR/radar encoders -> feature
                     selection -> fusion -> detection -> tracking -> planning. Requires a
                     checkpoint; without one the caller must label outputs as UNTRAINED.
"""
```

Docstring: the three honest modes (dataset, ground_truth, model).

```
from __future__ import annotations

import logging
import time
from dataclasses import dataclass, field
from pathlib import Path
from typing import Any, Callable, Optional

import numpy as np
import torch

from datasets.info_builder import COMMANDS, load_infos
from datasets.map_prior import MapPrior
from datasets.nuscenes_dataset import NuScenesFusionDataset, collate_batch
from datasets.splits import MINI_TRAIN, MINI_VAL
from evaluation.detection_metrics import DetectionEvaluator
from evaluation.trajectory_metrics import sample_planning_metrics
from models.fusion_model import MultiSensorFusionNet, build_model
from planning.tracking import MultiObjectTracker
from planning.trajectory_planner import PlanResult, build_planner
from utils.bev_grid import BEVGrid
from utils.checkpoint import load_checkpoint
from utils.config import Config, resolve_path
from utils.device import amp_enabled, get_device, move_to_device
from utils.timer import StageTimer

logger = logging.getLogger(__name__)
MACHINE_KEYS = ("dataset.root", "dataset.version", "dataset.cache_dir", "dataset.num_workers", "runtime.device",
                "runtime.output_dir", "runtime.checkpoint_dir")
```

Imports. `MACHINE_KEYS` lists configuration keys that must come from *this* machine rather than from the checkpoint (dataset location, device, output folders).

```
@dataclass
class FrameResult:
    index: int
    info: dict[str, Any]
    mode: str
    model_trained: bool
    lidar_points: np.ndarray
    radar_points: np.ndarray
    detections: Optional[dict[str, Any]] = None            # boxes (K,9) scores labels names track_ids histories
    plan: Optional[PlanResult] = None
    plan_source: str = ""
    maps: dict[str, np.ndarray] = field(default_factory=dict)
```

```
    channel_gates: dict[str, dict[str, Any]] = field(default_factory=dict)
    selection_stats: dict[str, float] = field(default_factory=dict)
    frame_metrics: dict[str, Any] = field(default_factory=dict)
    timings_ms: dict[str, float] = field(default_factory=dict)
    raw_outputs: dict[str, Any] = field(default_factory=dict)

    @property
    def command_name(self) -> str:
        return COMMANDS[int(self.info["command"])]
```

`FrameResult`: everything a view needs for one frame — info, mode, trained flag, raw LiDAR and radar points, detections (+ track ids and histories), plan, feature maps (energies, gates, attention), channel gates with names, selection statistics, frame metrics, timings and raw outputs (learned waypoints). `command_name` converts the command index to text.

```
def split_of_scene(name: str) -> str:
    if name in MINI_TRAIN:
        return "train"
    if name in MINI_VAL:
        return "val"
    try:
        from nuscenes.utils.splits import create_splits_scenes
        s = create_splits_scenes()
        return "train" if name in s["train"] else "val" if name in s["val"] else "test"
    except ImportError:
        return "unknown"
```

`split_of_scene(name)`: train / val / test membership (mini lists or the devkit lists) — used to warn when a training scene is shown.

```
class FusionPipeline:
    """Loads the data (and optionally a trained checkpoint) and processes frames on demand."""

    def __init__(self, base_cfg: Config, checkpoint: Optional[Path] = None, split: str = "all",
                 allow_untrained_model: bool = False) -> None:
        self.checkpoint = Path(checkpoint) if checkpoint else None
        if self.checkpoint is not None:
            ckpt = load_checkpoint(self.checkpoint)
            cfg = Config(ckpt["config"])
            for key in MACHINE_KEYS:  # data location / device come from *this* machine, not the training one
                cfg.set_path(key, base_cfg.get_path(key))
            self.checkpoint_meta = {"epoch": ckpt.get("epoch"), "metrics": ckpt.get("metrics", {}),
                                    "experiment": ckpt.get("extra", {}).get("experiment"),
                                    "stage": ckpt.get("extra", {}).get("stage")}
        else:
            cfg, ckpt, self.checkpoint_meta = base_cfg, None, {}
        self.cfg = cfg
        self.device = get_device(cfg.runtime.device)
        self.use_amp = amp_enabled(self.device, bool(cfg.train.amp))
        self.grid = BEVGrid.from_config(cfg.bev)
        self.classes = list(cfg.classes)
        self.infos = load_infos(cfg, split)
        self.dataset = NuScenesFusionDataset(cfg, split, train=False, infos=self.infos, load_all_sensors=True,
                                             with_targets=False)
        self.model: Optional[MultiSensorFusionNet] = None
        self.model_trained = False
        if ckpt is not None or allow_untrained_model:
            self.model = build_model(cfg).to(self.device).eval()
            if ckpt is not None:
                self.model.load_state_dict(ckpt["model"])
                self.model_trained = True
        self.tracker = MultiObjectTracker(self.classes)
        self._last_index: Optional[int] = None
        self._planners: dict[str, Any] = {}
        self.map_prior = MapPrior(Path(cfg.dataset.root), resolve_path(cfg.dataset.cache_dir))
        self.collision_labels = [self.classes.index(c) for c in cfg.planning.collision_classes]

    # ------------------------------------------------------------------ meta
```

Constructor: with a checkpoint, its embedded config is used and the machine keys are replaced by the local ones; loads the infos of the split (`'all'` in the dashboard), creates a dataset with `load_all_sensors=True, with_targets=False`, builds the model and loads weights (or a random model only when explicitly allowed), creates the tracker, planner cache and map prior.

```
    def scenes(self) -> list[dict[str, Any]]:
        out: list[dict[str, Any]] = []
        for i, info in enumerate(self.infos):
            if not out or out[-1]["name"] != info["scene_name"]:
                out.append({"name": info["scene_name"], "description": info["scene_description"], "start": i, "c
    ↪                   ount": 0,
                            "location": info["location"], "split": split_of_scene(info["scene_name"])})
```

```
            out[-1]["count"] += 1
        return out

    def planner(self, name: str):
        if name not in self._planners:
            self._planners[name] = build_planner(self.cfg, name)
        return self._planners[name]

    def map_patch(self, info: dict[str, Any], x_range, y_range, ppm: float) -> Optional[np.ndarray]:
        if not info.get("map_filename"):
            return None
        try:
            return self.map_prior.crop_to_ego(info["map_filename"], info["ref_ego2global"], tuple(x_range), tupl
                e(y_range), ppm)
        except (OSError, MemoryError, ValueError) as exc:
            logger.warning("Map prior unavailable: %s", exc)
            return None

    # ------------------------------------------------------------------ run
```

`scenes()` groups the infos into scenes (name, description, first index, frame count, location, split). `planner(name)` caches planner instances; `map_patch()` returns the map prior crop around the ego in display layout (warnings instead of failures).

```
    def run(self, index: int, mode: str = "model", planner: str = "baseline", plan_on_gt: bool = True,
            score_threshold: Optional[float] = None, stage_callback: Optional[Callable[[str, float], None]] = No
                ne) -> FrameResult:
        t_all = time.perf_counter()
        timings: dict[str, float] = {}
        info = self.infos[index]
        t0 = time.perf_counter()
        item = self.dataset[index]
        timings["data_loading"] = (time.perf_counter() - t0) * 1000
        if stage_callback:
            stage_callback("data_loading", timings["data_loading"])
        res = FrameResult(index, info, mode, self.model_trained, item.get("raw_lidar_points", np.zeros((0, 5), n
             p.float32)),
                          item.get("raw_radar_points", np.zeros((0, 10), np.float32)))
        es = info["ego_state"]
        ego_state = np.array([es["speed"], es["yaw_rate"], es["accel"]], dtype=np.float64)
        sequential = self._last_index is not None and index == self._last_index + 1 and \
            self.infos[self._last_index]["scene_token"] == info["scene_token"]
        if not sequential:
            self.tracker.reset()
        self._last_index = index

        if mode == "model" and self.model is not None:
            self._run_model(res, item, timings, score_threshold, stage_callback)
        elif mode == "ground_truth":
            gt = info["gt"]
            keep = (gt["num_lidar_pts"] + gt["num_radar_pts"]) > 0
            res.detections = {"boxes": gt["boxes"][keep].astype(np.float64), "scores": np.ones(int(keep.sum())),
                              "labels": gt["labels"][keep], "names": [n for n, k in zip(gt["names"], keep) if k]
                                  }
```

`run(index, mode, planner, plan_on_gt, score_threshold, stage_callback)`:

1. loads the dataset item (timed);
2. resets the tracker unless this frame directly follows the previous one in the same scene;
3. `model` mode → `_run_model`; `ground_truth` mode → GT boxes with ≥ 1 point become the "detections" (scores 1.0);

```
        if res.detections is not None:
            t0 = time.perf_counter()
            d = res.detections
            # Only confident detections are tracked (the same threshold the planner uses); others get id 0.
            conf = d["scores"] >= float(self.cfg.planning.occupancy.min_score)
            ids = np.zeros(len(d["scores"]), dtype=np.int64)
            ids[conf] = self.tracker.update(d["boxes"][conf], d["labels"][conf], d["scores"][conf], info["ref_eg
                o2global"],
                                            info["timestamp"])
            d["track_ids"] = ids
            d["histories"] = self.tracker.histories_ref(ids, info["ref_ego2global"])
            timings["tracking"] = (time.perf_counter() - t0) * 1000
            if stage_callback:
                stage_callback("tracking", timings["tracking"])

        # Planning.
        if res.detections is not None and (mode == "model" or plan_on_gt):
            t0 = time.perf_counter()
            name = planner
            learned = res.raw_outputs.get("learned_waypoints")
```

```
            if name in ("learned", "hybrid") and learned is None:
                name = "baseline"
            res.plan = self.planner(name).plan(ego_state, int(info["command"]), res.detections,
                                               lidar_points=res.lidar_points, learned_waypoints=learned)
            res.plan_source = res.plan.planner + (" on ground-truth objects" if mode == "ground_truth" else "")
            timings["planning"] = (time.perf_counter() - t0) * 1000
            if stage_callback:
                stage_callback("planning", timings["planning"])
        self._frame_metrics(res)
        timings["total"] = (time.perf_counter() - t_all) * 1000
        res.timings_ms = timings
        return res
```

4. tracking of confident detections (score ≥ `occupancy.min_score`; others get id 0) in the global frame;
5. planning (in model mode, or on GT objects in mode 2 when enabled); learned/hybrid fall back to the baseline when no learned waypoints exist;
6. frame metrics and total time.

```
    @torch.no_grad()
    def _run_model(self, res: FrameResult, item: dict[str, Any], timings: dict[str, float],
                   score_threshold: Optional[float], stage_callback) -> None:
        model = self.model
        batch = move_to_device(collate_batch([{k: v for k, v in item.items() if not k.startswith("raw_")}]), sel
    ↪           f.device)
        timer = StageTimer(callback=stage_callback)
        with torch.autocast(device_type=self.device.type, dtype=torch.float16, enabled=self.use_amp):
            out = model(batch, timer=timer)
        with timer.stage("decode"):
            det = model.decode(out, score_threshold)[0]
        timings.update(timer.as_dict())
        timings["network"] = timer.total_ms
        boxes = det["boxes"].float().cpu().numpy().astype(np.float64)
        labels = det["labels"].cpu().numpy()
        res.detections = {"boxes": boxes, "scores": det["scores"].float().cpu().numpy(), "labels": labels,
                          "names": [self.classes[int(l)] for l in labels]}

        def energy(t: torch.Tensor) -> np.ndarray:
            return t[0].float().pow(2).sum(0).sqrt().cpu().numpy()

        maps = res.maps
        for m, feat in out["raw"].items():
            maps[f"{m}_raw_energy"] = energy(feat)
            maps[f"{m}_selected_energy"] = energy(out["selected"][m])
        maps["fused_energy"] = energy(out["fused"])
        maps["detection_heatmap"] = torch.sigmoid(out["det"]["heatmap"][0].float()).amax(0).cpu().numpy()
        if "modality_weights" in out["fusion_info"]:
            maps["modality_weights"] = out["fusion_info"]["modality_weights"][0].float().cpu().numpy()
            res.raw_outputs["modality_order"] = out["fusion_info"]["modality_order"]
        if "selection" in out:
            sel = out["selection"]
            for m, g in sel["spatial_gates"].items():
                maps[f"{m}_importance"] = g[0, 0].float().cpu().numpy()
            names = {"lidar": getattr(model, "lidar_encoder", None), "radar": getattr(model, "radar_encoder", No
    ↪            ne)}
            for m, g in sel["channel_gates"].items():
                enc = names[m]
                res.channel_gates[m] = {"names": list(enc.channel_names), "gates": g[0].float().cpu().numpy()}
            res.selection_stats = model.feature_selection.selection_stats(sel)
        if "plan" in out:
            res.raw_outputs["learned_waypoints"] = out["plan"]["waypoints"][0].float().cpu().numpy()
            res.maps["planning_attention"] = out["plan"]["attention"][0].float().cpu().numpy()
```

`_run_model()`: collates the single item (without the raw point keys), runs the network under the stage timer, decodes detections, and extracts NumPy maps for the views: per-sensor raw and selected feature energies, the fused energy, the detection heatmap (max over classes), fusion attention weights, spatial gates, named channel gates, selection statistics, learned waypoints and the planning attention map.

```
    def _frame_metrics(self, res: FrameResult) -> None:
        info = res.info
        gt = info["gt"]
        metrics: dict[str, Any] = {}
        if res.detections is not None and res.mode == "model":
            ev = DetectionEvaluator(self.classes)
            d = res.detections
            ev.add_sample(info["token"], d["boxes"], d["scores"], d["labels"], gt["boxes"], gt["labels"], gt["at
    ↪           tributes"],
    ↪                     gt["num_lidar_pts"] + gt["num_radar_pts"], gt["velocity_valid"], bike_racks=gt.get("bi
    ↪                          ke_racks"))
            r = ev.evaluate()
            metrics["detection"] = {k: r[k] for k in ("NDS", "mAP", "mAP_present_classes", "mATE", "mAVE")}
        fut = info["ego_future"]
```

```
        if res.plan is not None and fut["mask"].any():
            metrics["planning"] = sample_planning_metrics(res.plan.waypoints.astype(np.float64), fut["xy"], fut[
    ↪               "mask"],
                                                          float(self.cfg.planning.dt), info["agent_future"],
                                                          self.collision_labels, self.cfg.planning.ego)
        res.frame_metrics = metrics

    def add_to_evaluator(self, evaluator: DetectionEvaluator, res: FrameResult) -> None:
        """Accumulate a processed frame into a running (scene-level) evaluator."""
        if res.detections is None or res.mode != "model":
            return
        gt, d = res.info["gt"], res.detections
        evaluator.add_sample(res.info["token"], d["boxes"], d["scores"], d["labels"], gt["boxes"], gt["labels"],
                             gt["attributes"], gt["num_lidar_pts"] + gt["num_radar_pts"], gt["velocity_valid"],
                             bike_racks=gt.get("bike_racks"))
```

`_frame_metrics()`: single-frame NDS/mAP (model mode only) and planning metrics against the logged trajectory. `add_to_evaluator()` accumulates a frame into a running scene evaluator (dashboard "scene so far").

app/streamlit_app.py

The Streamlit dashboard. Streamlit re-executes this script from top to bottom on every interaction; persistent state lives in `st.session_state` and heavy objects in `st.cache_resource`. (542 lines)

```
"""Multi-Sensor Autonomous Driving System — interactive nuScenes replay & faculty demo.

Launch (either works):

    streamlit run app/streamlit_app.py
    python app/streamlit_app.py

Modes
-----
1. Dataset Viewer            - raw camera / LiDAR / radar of real nuScenes key-frames.
2. Ground Truth Simulation   - nuScenes annotations + logged ego trajectory replayed in BEV
                               (the classical planner can run on the GT objects, labelled as such).
3. AI Autonomous Driving     - the trained network: camera BEV -> LiDAR/radar -> feature selection
                               -> fusion -> detection -> tracking -> trajectory planning.

Nothing is simulated or faked: without a trained checkpoint, mode 3 falls back to
"GROUND TRUTH / DATASET VISUALIZATION" (or, if explicitly allowed, shows the untrained
network with a red UNTRAINED banner).
"""
```

Docstring: launch commands, the three modes and the no-fake-output rule.

```
from __future__ import annotations

import sys
import time
from collections import OrderedDict
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))
```

Standard imports; `ROOT` (project root) is put on `sys.path` so `app/...` imports work when Streamlit runs the file directly.

```
def _inside_streamlit() -> bool:
    try:
        from streamlit.runtime import exists

        return exists()
    except Exception:  # pragma: no cover
        return False


if __name__ == "__main__" and not _inside_streamlit():
    from streamlit.web import cli as stcli

    sys.argv = ["streamlit", "run", str(Path(__file__).resolve()), "--theme.base", "dark", *sys.argv[1:]]
    sys.exit(stcli.main())
```

`_inside_streamlit()` checks whether the Streamlit runtime is active. When the file is started with plain `python app/streamlit_app.py`, the block below re-launches it through `streamlit.web.cli` with the dark theme — so both launch commands work.

```
import altair as alt  # noqa: E402
import numpy as np  # noqa: E402
import pandas as pd  # noqa: E402
import streamlit as st  # noqa: E402

from app.ui import panels  # noqa: E402
from app.ui.resources import (available_versions, base_config, dashboard_methods, experiment_labels, find_checkpo
    int,  # noqa: E402
                              load_ablation_results, load_history)
from app.ui.styles import CSS, FLOW_STAGES, banner_html, chip, flow_html, kpis_html, panel_title, stage_key_for
    # noqa: E402
from evaluation.detection_metrics import DetectionEvaluator  # noqa: E402
from evaluation.trajectory_metrics import aggregate  # noqa: E402
from inference.pipeline import FrameResult, FusionPipeline  # noqa: E402
from utils.config import load_config  # noqa: E402
```

Third-party and project imports (after the re-launch guard so that plain `python` does not import Streamlit components twice).

```
st.set_page_config(page_title="Multi-Sensor Fusion · nuScenes Demo", page_icon="[car]", layout="wide")
st.markdown(CSS, unsafe_allow_html=True)

SPEEDS = {"0.25x": 0.25, "0.5x": 0.5, "1x": 1.0, "2x": 2.0, "5x": 5.0}
MODES = {"1 · Dataset Viewer": "dataset", "2 · Ground Truth Simulation": "ground_truth", "3 · AI Autonomous Drivi
    ng": "model"}
PROPOSED = "F_cam_lidar_radar_fs"
FS_EXPLANATION = ("The feature-selection network learns which LiDAR and Radar features are most informative for "
                  "perception and suppresses redundant or noisy features before fusion.")
SYSTEM_STORY = ("This system takes synchronized Camera, LiDAR, and Radar data from the nuScenes autonomous-drivin
    g dataset. "
                "The camera information is transformed into a Bird's-Eye View representation. LiDAR and Radar are
                     "
                "independently encoded into BEV features. A deep-learning feature-selection network learns which
                    LiDAR "
                "and Radar features are important. These selected features are fused with the BEV representation,
                     after "
                "which the system performs 3D object detection and trajectory planning. The resulting autonomous-
                    driving "
                "scene is replayed and visualized using real nuScenes data.")


# ---------------------------------------------------------------------- resources
```

Page configuration, CSS injection, and constants: playback speeds, the mode labels, the proposed experiment id, and the two explanation texts shown in the faculty demo.

```
@st.cache_resource(show_spinner="Loading nuScenes data and model ...", max_entries=12)
def get_pipeline(version: str, device: str, checkpoint: str | None, experiment: str | None,
                 allow_untrained: bool) -> FusionPipeline:
    overrides = [f"dataset.version={version}", f"runtime.device={device}"]
    if experiment:
        cfg = load_config(experiment=experiment, overrides=overrides)
    else:
        cfg = base_config(version, device)
        if checkpoint is None:  # data-only pipeline: raw sensors are loaded separately, skip network camera prep
            rocessing
            cfg.set_path("fusion.use_camera_bev", False)
    return FusionPipeline(cfg, Path(checkpoint) if checkpoint else None, split="all", allow_untrained_model=allow
        _untrained)
```

`get_pipeline(version, device, checkpoint, experiment, allow_untrained)`: cached `FusionPipeline` per argument combination (≤ 12 entries). A data-only pipeline disables the network's camera preprocessing (images for display are loaded separately).

```
def ss_default(key: str, value) -> None:
    if key not in st.session_state:
        st.session_state[key] = value


for k, v in {"frame_slider": 0, "playing": False, "scene": None, "results": OrderedDict(), "explain": False,
             "scene_eval": None, "scene_eval_key": None, "scene_plan": [], "seen": set(), "compare": None}.items
                 ():
    ss_default(k, v)
ss = st.session_state

# ---------------------------------------------------------------------- sidebar
```

`ss_default` initialises session-state keys once: slider position, playing flag, current scene, the frame-result cache, the explain toggle, scene-level evaluators and the comparison table.

```
st.sidebar.markdown("## [car] Control panel")
faculty = st.sidebar.toggle("[faculty] Faculty Demo", value=True, help="Simplified, presentation-friendly layout
```

```
    ")
probe_cfg = load_config()
versions = available_versions(probe_cfg)
version_opts = [v for v, ok in versions.items() if ok] or ["v1.0-mini"]
version = st.sidebar.selectbox("nuScenes version", version_opts,
                               help="Only versions whose table folder exists under dataset.root are listed.")
missing = [v for v, ok in versions.items() if not ok]
if missing:
    st.sidebar.caption(f"Not found under `{probe_cfg.dataset.root}`: {', '.join(missing)}")
_cli_device = sys.argv[sys.argv.index("--device") + 1] if "--device" in sys.argv[:-1] else "auto"  # streamlit r
    un app.py -- --device cpu
device = st.sidebar.selectbox("Device", ["auto", "cuda", "cpu"], index=["auto", "cuda", "cpu"].index(_cli_device
    ))
mode_label = st.sidebar.radio("Demo mode", list(MODES), index=2)
requested_mode = MODES[mode_label]

methods = dashboard_methods()
labels_by_exp = experiment_labels()
method_status = {m: find_checkpoint(probe_cfg, exp) for m, exp in methods.items()}
method_label = st.sidebar.selectbox("Fusion method", list(methods), index=len(methods) - 1,
                                    format_func=lambda m: f"{'[✓]' if method_status[m] else '[ ]'} {m}")
experiment = methods[method_label]
checkpoint = method_status[method_label]
planner_name = st.sidebar.selectbox("Trajectory planner", ["baseline", "learned", "hybrid"],
                                    help="baseline: A* + speed profile + iLQR MPC · learned: planning head · "
                                         "hybrid: learned with classical safety fallback")
allow_untrained = st.sidebar.checkbox("Allow untrained network (architecture demo only)", value=False,
                                      help="Runs randomly initialised weights; every output is labelled UNTRAINE
                                           D.")
score_min = st.sidebar.slider("Detection confidence threshold", 0.05, 0.9, 0.30, 0.05)
speed_label = st.sidebar.select_slider("Playback speed", list(SPEEDS), value="1x")
with st.sidebar.expander("Display options", expanded=not faculty):
    opts = {
        "show_map": st.checkbox("Road map prior", True),
        "show_lidar": st.checkbox("LiDAR points", True),
        "show_radar": st.checkbox("Radar returns + velocity", True),
        "show_gt": st.checkbox("Ground-truth boxes (comparison)", True),
        "show_tracks": st.checkbox("Track IDs / motion trails", True),
        "show_object_futures": st.checkbox("Logged object futures", False),
        "show_gt_traj": st.checkbox("Logged ego future", True),
    }
    camera_choice = st.selectbox("Main camera", ["CAM_FRONT", "CAM_FRONT_LEFT", "CAM_FRONT_RIGHT", "CAM_BACK",
                                                 "CAM_BACK_LEFT", "CAM_BACK_RIGHT"])
    plan_on_gt = st.checkbox("Run planner on ground-truth objects (mode 2)", True)

# ------------------------------------------------------------- mode handling
```

Sidebar: faculty toggle, nuScenes version (only versions present on disk), device (default from `--device`), demo mode, fusion method ([✓] when a checkpoint exists — `find_checkpoint`), planner, untrained-network opt-in, confidence threshold, playback speed, display options and the main camera.

```
data_pipe = get_pipeline(version, device, None, None, False)
if requested_mode == "model" and checkpoint is not None:
    pipe = get_pipeline(version, device, str(checkpoint), None, False)
    eff_mode, banner = "model", banner_html("model", "MODEL PREDICTION")
elif requested_mode == "model" and allow_untrained:
    pipe = get_pipeline(version, device, None, experiment, True)
    eff_mode, banner = "model", banner_html("untrained", "UNTRAINED NETWORK — OUTPUTS ARE NOT MEANINGFUL")
else:
    pipe = data_pipe
    eff_mode = "ground_truth" if requested_mode in ("model", "ground_truth") else "dataset"
    banner = banner_html("gt", "GROUND TRUTH / DATASET VISUALIZATION")
fallback_note = (requested_mode == "model" and checkpoint is None and not allow_untrained)

scenes = data_pipe.scenes()
scene_names = [s["name"] for s in scenes]
scene_idx = st.sidebar.selectbox("Scene", range(len(scenes)),
                                 format_func=lambda i: f"{scenes[i]['name']} · {scenes[i]['split']} · {scenes[i]
                                      ['description'][:38]}")
scene = scenes[scene_idx]
if ss.scene != scene["name"]:
    ss.scene, ss.frame_slider, ss.playing = scene["name"], 0, False


# ------------------------------------------------------------------ callbacks
```

Mode handling — the honesty rule: mode 3 uses the model only if a checkpoint exists (banner MODEL PREDICTION) or the user explicitly allowed an untrained network (red UNTRAINED banner); otherwise it falls back to ground truth with the banner GROUND TRUTH / DATASET VISUALIZATION and a warning. The scene selector resets the frame when the scene changes.

```
def cb_prev() -> None:
    ss.playing = False
    ss.frame_slider = max(0, ss.frame_slider - 1)


def cb_next() -> None:
    ss.playing = False
    ss.frame_slider = min(scene["count"] - 1, ss.frame_slider + 1)


def cb_play() -> None:
    if ss.frame_slider >= scene["count"] - 1:
        ss.frame_slider = 0
    ss.playing = True


def cb_pause() -> None:
    ss.playing = False


def cb_reset() -> None:
    ss.playing = False
    ss.frame_slider = 0
    ss.scene_eval, ss.scene_eval_key, ss.scene_plan, ss.seen = None, None, [], set()
    pipe.tracker.reset()


def cb_load() -> None:
    cb_reset()


if ss.pop("_advance", False):
    if ss.frame_slider < scene["count"] - 1:
        ss.frame_slider += 1
    else:
        ss.playing = False

# ---------------------------------------------------------------------- header
```

Button callbacks (run *before* the rerun, so they may change the slider's session key): previous/next frame, play (restarts at the end), pause, reset (also clears scene metrics and the tracker), load scene. The `_advance` flag set by the playback loop advances the slider at the top of the next run.

```
exp_label = labels_by_exp.get(experiment, experiment)
sub = f"nuScenes {version} · {scene['name']} ({scene['split']} split, {scene['location']}) · {scene['description
    ']}"
st.markdown(f'<div class="msf-title"><div><h1>MULTI-SENSOR AUTONOMOUS DRIVING SYSTEM</h1>'
            f'<div class="sub">{sub}</div></div><div>{banner}</div></div>', unsafe_allow_html=True)
if fallback_note:
    st.warning(f"No trained checkpoint found for **{method_label}** (experiment `{experiment}`). "
               "Showing ground-truth / dataset visualization instead of pretending to predict. "
               f"Train it with: `python main.py train --stage joint --experiment {experiment}`")
if eff_mode == "model" and scene["split"] == "train" and pipe.model_trained:
    st.info("This scene was part of the **training** split. Use a **val** scene (scene-0103, scene-0916 in nuSce
        nes-mini) "
            "to show performance on data the network has never seen.")

c = st.columns([1.1, 1, 1, 1, 1, 1, 4.4])
c[0].button("[folder] Load Scene", on_click=cb_load, width="stretch")
c[1].button("[prev] Previous", on_click=cb_prev, width="stretch")
c[2].button("▶ Play", on_click=cb_play, width="stretch", type="primary" if not ss.playing else "secondary")
c[3].button("[pause] Pause", on_click=cb_pause, width="stretch")
c[4].button("[next] Next", on_click=cb_next, width="stretch")
c[5].button("[reset] Reset", on_click=cb_reset, width="stretch")
c[6].slider("Keyframe", 0, scene["count"] - 1, key="frame_slider", label_visibility="collapsed",
            help="nuScenes key-frames are 2 Hz (0.5 s apart)")
index = scene["start"] + int(ss.frame_slider)

# ---------------------------------------------------------------------- processing
```

Header with title, scene description and banner; warnings (no checkpoint, training scene); control buttons and the key-frame slider; `index` = global info index of the selected frame.

```
flow_slot = st.empty() if faculty else None
done: dict[str, float] = {}
skipped = []
if eff_mode != "model":
    skipped = ["camera", "sensors", "selection", "fusion"]
else:
    model = pipe.model
    if not model.use_camera:
        skipped.append("camera")
    if not model.use_feature_selection:
```

```
        skipped.append("selection")


def on_stage(name: str, ms: float) -> None:
    key = stage_key_for(name)
    if key is None:
        return
    done[key] = done.get(key, 0.0) + ms
    if flow_slot is not None:
        nxt = next((k for k, _, _ in FLOW_STAGES if k not in done and k not in skipped), None)
        flow_slot.markdown(flow_html(done, nxt, skipped), unsafe_allow_html=True)
```

Processing: in faculty mode an empty placeholder holds the pipeline strip; stages that do not apply are marked skipped. `on_stage` (timer callback) re-renders the strip after every stage, so the audience sees the stages tick live.

```
cache_key = (str(checkpoint) if eff_mode == "model" and checkpoint else f"untrained:{experiment}" if eff_mode ==
    "model" else "data",
            index, eff_mode, planner_name, plan_on_gt)
res: FrameResult
if cache_key in ss.results:
    res = ss.results[cache_key]
    for k, v in res.timings_ms.items():
        key = stage_key_for(k)
        if key:
            done[key] = done.get(key, 0.0) + v
else:
    if flow_slot is not None:
        flow_slot.markdown(flow_html({}, next(k for k, _, _ in FLOW_STAGES if k not in skipped), skipped), unsaf
            e_allow_html=True)
    res = pipe.run(index, eff_mode, planner_name, plan_on_gt, stage_callback=on_stage)
    ss.results[cache_key] = res
    while len(ss.results) > 24:
        ss.results.popitem(last=False)

t_render = time.perf_counter()
images = panels.load_images(Path(pipe.cfg.dataset.root), res)

# Scene-level running metrics (frames visited so far with the current method).
eval_key = (cache_key[0], scene["name"], eff_mode, planner_name)
if ss.scene_eval_key != eval_key:
    ss.scene_eval_key, ss.scene_eval, ss.scene_plan, ss.seen = eval_key, DetectionEvaluator(pipe.classes), [], s
        et()
if index not in ss.seen:
    ss.seen.add(index)
    pipe.add_to_evaluator(ss.scene_eval, res)
    if "planning" in res.frame_metrics:
        ss.scene_plan.append(res.frame_metrics["planning"])
scene_det = ss.scene_eval.evaluate() if eff_mode == "model" and ss.scene_eval.samples else None
scene_plan = aggregate(ss.scene_plan) if ss.scene_plan else None

# ---------------------------------------------------------------------- KPI strip
```

Frame results are cached per (method, index, mode, planner, plan_on_gt) in an LRU of 24 entries; otherwise `pipe.run(...)` is called. Then the scene-level running evaluator accumulates each frame once.

```
dets = res.detections
n_obj = int((dets["scores"] >= score_min).sum()) if dets is not None and eff_mode == "model" else (
    len(dets["boxes"]) if dets is not None else 0)
net_ms = res.timings_ms.get("network")
fps = 1000.0 / max(res.timings_ms.get("total", 1.0) - res.timings_ms.get("data_loading", 0.0), 1e-3)
conf = float(np.mean(dets["scores"][dets["scores"] >= score_min])) if dets is not None and eff_mode == "model" a
    nd n_obj else None
fm = res.frame_metrics
plan_m = fm.get("planning", {})
kpis = [("Detection", f"{n_obj} objects", "model output" if eff_mode == "model" else "ground truth" if dets is n
    ot None else "—"),
        ("FPS", f"{fps:.1f}", f"network {net_ms:.0f} ms" if net_ms else f"pipeline {res.timings_ms.get('total',
            0):.0f} ms")]
if eff_mode == "model":
    kpis.append(("NDS", f"{fm['detection']['NDS']:.3f}" if 'detection' in fm else "—",
                 f"scene so far {scene_det['NDS']:.3f} ({scene_det['num_samples']} frames)" if scene_det else "f
                     rame"))
    kpis.append(("mAP", f"{fm['detection']['mAP']:.3f}" if 'detection' in fm else "—",
                 f"present classes {fm['detection']['mAP_present_classes']:.3f}" if 'detection' in fm else ""))
kpis.append(("ADE", f"{plan_m['ade']:.2f} m" if "ade" in plan_m else "—",
             f"scene {scene_plan['ade']:.2f} m" if scene_plan and "ade" in scene_plan else "vs logged trajectory
                 "))
kpis.append(("Collision", ("YES" if plan_m.get("collision") else "no") if plan_m else "—",
             f"scene rate {scene_plan.get('collision_rate', 0):.0%}" if scene_plan else "vs GT agents"))
kpis.append(("Fusion", experiment_labels(short=True).get(experiment, experiment) if eff_mode == "model" else "—"
    ,
             f"mean confidence {conf:.2f}" if conf else ""))
```

```
st.markdown(kpis_html(kpis), unsafe_allow_html=True)
if flow_slot is not None:
    done["output"] = 0.0
    flow_slot.markdown(flow_html(done, None, skipped), unsafe_allow_html=True)

# ------------------------------------------------------------ row A: cam + fusion
```

KPI strip: number of objects, FPS (excluding data loading), frame and scene NDS / mAP, ADE, collision, fusion method and mean confidence.

```
colA, colB = st.columns([1.05, 1])
with colA:
    st.markdown(panel_title(f"Panel 1 · Camera view ({camera_choice})"), unsafe_allow_html=True)
    cam_img, mosaic = panels.camera_panel(res, images, camera_choice, show_pred=True, show_gt=opts["show_gt"], s
    ↪       core_min=score_min)
    if cam_img is not None:
        st.image(cam_img, width="stretch",
                 caption=("Predicted 3D boxes (class colour, confidence)" + (" + ground truth (green)" if opts["
    ↪                 show_gt"] else ""))
                 if eff_mode == "model" else "Ground-truth 3D boxes projected with the camera calibration")
    with st.expander("All six cameras", expanded=False):
        st.image(mosaic, width="stretch")
with colB:
    st.markdown(panel_title("Panel 3 · Sensor fusion"), unsafe_allow_html=True)
    fs_on = eff_mode == "model" and pipe.model.use_feature_selection
    tiles = panels.fusion_tiles(res, images, fs_on)
    for row in (tiles[:3], tiles[3:]):
        cols = st.columns(3)
        for col, tile in zip(cols, row):
            col.image(tile, width="stretch")
    if faculty:
        st.caption("Top: camera → BEV, LiDAR BEV, radar BEV. Bottom: learned importance of LiDAR and radar featu
    ↪           res, "
                   "and the fused BEV used for detection and planning.")

# ------------------------------------------------------------ row B: BEV + plan
colC, colD = st.columns([1.35, 1])
with colC:
    st.markdown(panel_title("Panel 2 · Autonomous driving BEV"), unsafe_allow_html=True)
    st.image(panels.driving_bev(res, pipe, {**opts, "score_min": score_min}), width="stretch")
with colD:
    st.markdown(panel_title("Panel 5 · Trajectory planning"), unsafe_allow_html=True)
    ego = panels.ego_status(res)
    st.markdown(kpis_html([("Ego X / Y", f"{ego['x']:.1f} / {ego['y']:.1f}", "global frame (m)"),
                           ("Yaw", f"{ego['yaw_deg']:.1f}°", f"yaw rate {ego['yaw_rate']:.2f} rad/s"),
                           ("Velocity", f"{ego['speed']:.2f} m/s", f"accel {ego['accel']:.2f} m/s²"),
                           ("Command", res.command_name, "navigation input")]), unsafe_allow_html=True)
    if res.plan is not None or res.info["ego_future"]["mask"].any():
        st.image(panels.trajectory_closeup(res, pipe), width="stretch")
    if res.plan is not None:
        status = chip("COLLISION PREDICTED", "bad") if res.plan.collision else chip("collision-free (predicted o
    ↪           ccupancy)", "ok")
        st.markdown(f"{chip(res.plan_source)} {status}", unsafe_allow_html=True)
        if plan_m:
            st.markdown(kpis_html([("ADE", f"{plan_m['ade']:.2f} m", "mean displacement"),
                                   ("FDE", f"{plan_m['fde']:.2f} m", "final displacement"),
                                   ("Lateral / long.", f"{plan_m['lateral_error']:.2f} / {plan_m['longitudinal_e
    ↪                                   rror']:.2f} m", "error"),
                                   ("GT collision", "YES" if plan_m.get("collision") else "no", "vs real agent f
    ↪                                   utures")]),
                          unsafe_allow_html=True)
    elif eff_mode == "dataset":
        st.caption("Dataset viewer: the white dashed line is the logged human trajectory. Switch to mode 2 or 3
    ↪           to plan.")

# ------------------------------------------------------------ row C: detection
st.markdown(panel_title("Panel 4 · Object detection"), unsafe_allow_html=True)
colE, colF = st.columns([1.6, 1])
with colE:
    table = panels.detection_table(res, score_min if eff_mode == "model" else 0.0)
    if len(table):
        st.dataframe(table, width="stretch", height=260, hide_index=True,
                     column_config={"Confidence": st.column_config.ProgressColumn("Confidence", min_value=0.0, m
    ↪                       ax_value=1.0, format="%.2f"),
                                    "X (m)": st.column_config.NumberColumn(format="%.1f"), "Y (m)": st.column_co
    ↪                                   nfig.NumberColumn(format="%.1f"),
                                    "Range (m)": st.column_config.NumberColumn(format="%.1f"),
                                    "Velocity (m/s)": st.column_config.NumberColumn(format="%.1f")})
    else:
        st.caption("No objects for this frame / mode.")
with colF:
    cmp = panels.gt_comparison(res, score_min)
    if cmp:
        st.markdown("**Ground truth vs model prediction** (2 m centre distance, same class)")
```

```
        st.markdown(kpis_html([("GT objects", str(cmp["ground_truth"]), ""), ("Predicted", str(cmp["predicted"]), ""),
                               ("True positives", str(cmp["true_positive"]), ""), ("False positives", str(cmp["false_positive"]), ""),
                               ("Missed", str(cmp["missed"]), "")]), unsafe_allow_html=True)
        if scene_det:
            st.caption(f"Scene so far ({scene_det['num_samples']} frames): NDS {scene_det['NDS']:.3f} · mAP {scene_det['mAP']:.3f} · "
                       f"mAP(present classes) {scene_det['mAP_present_classes']:.3f} · mAVE {scene_det['mAVE']:.2f} m/s")
    else:
        st.caption("Ground-truth comparison is available in mode 3 with a trained model.")

# ------------------------------------------------------- explain feature selection
```

Rows A–C: camera panel and the six sensor-fusion tiles; the autonomous-driving BEV and the trajectory panel (ego pose, command, close-up, planner source, collision chip, ADE/FDE/lateral/longitudinal/GT collision); the detection table and the ground-truth comparison (TP/FP/missed at 2 m).

```
st.divider()
if st.button("[magnifier] Explain Feature Selection", type="primary" if not ss.explain else "secondary"):
    ss.explain = not ss.explain
if ss.explain:
    st.markdown(f'<div class="msf-explain">{FS_EXPLANATION}</div>', unsafe_allow_html=True)
    if eff_mode != "model" or not pipe.model.use_feature_selection:
        st.info("Feature-selection maps come from the network's actual gates: choose a trained method with feature "
                "selection (methods 5 or 6) in mode 3.")
    else:
        ex = panels.explain_selection(res, pipe, score_min)
        if not res.model_trained:
            st.error("UNTRAINED weights: these gates are random and carry no meaning.")
        for mod, row in ex["rows"].items():
            st.markdown(f"**{'LiDAR' if mod == 'lidar' else 'Radar'}:** raw features → feature importance → selected features")
            cols = st.columns([1, 0.12, 1, 0.12, 1])
            for i, img in enumerate(row):
                cols[i * 2].image(img, width="stretch")
            cols[1].markdown("<div style='font-size:2rem;text-align:center;margin-top:120px'>→</div>", unsafe_allow_html=True)
            cols[3].markdown("<div style='font-size:2rem;text-align:center;margin-top:120px'>→</div>", unsafe_allow_html=True)
        st.caption("Importance = the network's spatial gate (0 suppressed … 1 kept). A gate only matters where the sensor "
                   "has features, so the 'selected' tile (raw × channel gate × spatial gate) shows what actually reaches fusion.")
        cols = st.columns(3)
        if ex["dominance"] is not None:
            cols[0].image(ex["dominance"], width="stretch")
        if ex["attention"] is not None:
            cols[1].image(ex["attention"], width="stretch")
        if "heatmap" in ex:
            cols[2].image(ex["heatmap"], width="stretch")
        st.markdown("**Which LiDAR and Radar features did the network consider important?**")
        gate_rows = []
        for mod, info in ex["channels"].items():
            for name, g in info["named"]:
                gate_rows.append({"sensor": "LiDAR" if mod == "lidar" else "Radar",
                                  "feature": name.replace("lidar_", "").replace("radar_", "").replace("_", " "),
                                  "gate": g})
        if gate_rows:
            df = pd.DataFrame(gate_rows)
            chart = alt.Chart(df).mark_bar(cornerRadiusEnd=4, height=14).encode(
                x=alt.X("gate:Q", scale=alt.Scale(domain=[0, 1]), title="channel gate (0 = suppressed, 1 = kept)"),
                y=alt.Y("feature:N", sort="-x", title=None),
                color=alt.Color("sensor:N", scale=alt.Scale(domain=["LiDAR", "Radar"], range=["#3987e5", "#d95926"]),
                                legend=alt.Legend(orient="top", title=None)),
                tooltip=["sensor", "feature", alt.Tooltip("gate:Q", format=".3f")],
            ).properties(height=34 * len(df) // 2 + 40)
            gcol1, gcol2 = st.columns([1.2, 1])
            gcol1.altair_chart(chart, width="stretch")
            with gcol2:
                for mod, info in ex["channels"].items():
                    st.markdown(f"{chip('LiDAR' if mod == 'lidar' else 'Radar')} {info['selected_ratio']:.0%} of channels "
                                f"selected · learned channels mean gate {info['learned_mean']:.2f}", unsafe_allow_html=True)
                stats = res.selection_stats
                for mod in ("lidar", "radar"):
                    if f"{mod}_spatial_selected_ratio" in stats:
                        st.markdown(f"{chip('LiDAR' if mod == 'lidar' else 'Radar')} spatial cells selected "
                                    f"{stats[f'{mod}_spatial_selected_ratio']:.0%} · effective selected features
```

```
                                        "
                                    f"{stats[f'{mod}_effective_selected_ratio']:.0%}", unsafe_allow_html=True)
        if ex["class_trust"]:
            st.markdown("**Sensor trust per detected object class** (mean importance inside the detected boxes)"
                )
            st.dataframe(pd.DataFrame(ex["class_trust"]), width="stretch", hide_index=True)

# ---------------------------------------------------------- compare methods
```

"Explain Feature Selection": toggled by a button; available only for trained models with feature selection. Shows raw → importance → selected for LiDAR and radar, the sensor-contribution map, fusion attention, the detection heatmap, an Altair bar chart of the named channel gates, selected ratios and the per-class sensor trust table.

```
with st.expander("[scales] Compare fusion methods on this scene", expanded=False):
    trained = {m: p for m, p in method_status.items() if p is not None}
    st.markdown(" ".join(chip(f"{m.split('. ', 1)[-1]}", "ok" if p else "") for m, p in method_status.items()),
                unsafe_allow_html=True)
    if not trained:
        st.info("No trained checkpoints yet. Run `python main.py ablation --train` to train all methods.")
    else:
        cc = st.columns(2)
        run_frame = cc[0].button("Run all trained methods on this frame")
        run_scene = cc[1].button("Evaluate all trained methods on the whole scene (slower)")
        if run_frame or run_scene:
            rows = []
            prog = st.progress(0.0)
            frames = range(scene["start"], scene["start"] + scene["count"]) if run_scene else [index]
            for mi, (m, path) in enumerate(trained.items()):
                p = get_pipeline(version, device, str(path), None, False)
                ev, plans, lat = DetectionEvaluator(p.classes), [], []
                for fi in frames:
                    r = p.run(fi, "model", planner_name)
                    p.add_to_evaluator(ev, r)
                    if "planning" in r.frame_metrics:
                        plans.append(r.frame_metrics["planning"])
                    lat.append(r.timings_ms.get("network", 0.0))
                det = ev.evaluate()
                pl = aggregate(plans) if plans else {}
                rows.append({"method": m, "experiment": methods[m], "NDS": det["NDS"], "mAP": det["mAP"],
                             "mAP (present classes)": det["mAP_present_classes"], "mAVE": det["mAVE"],
                             "ADE (m)": pl.get("ade"), "FDE (m)": pl.get("fde"), "collision rate": pl.get("colli
                                 sion_rate"),
                             "network ms": float(np.median(lat)), "frames": len(frames)})
                prog.progress((mi + 1) / len(trained))
            ss.compare = pd.DataFrame(rows)
        if ss.compare is not None:
            df = ss.compare
            st.dataframe(df, width="stretch", hide_index=True)
            metric = st.selectbox("Chart metric", ["NDS", "mAP", "mAP (present classes)", "mAVE", "ADE (m)", "FD
                 E (m)",
                                                   "collision rate", "network ms"])
            dfc = df.assign(proposed=df["experiment"].eq(PROPOSED).map({True: "proposed", False: "baseline"}))
            st.altair_chart(alt.Chart(dfc).mark_bar(cornerRadiusEnd=4).encode(
                x=alt.X(f"{metric}:Q"), y=alt.Y("method:N", sort=None, title=None),
                color=alt.Color("proposed:N", scale=alt.Scale(domain=["proposed", "baseline"], range=["#3987e5",
                       "#6b6a64"]),
                                legend=None),
                tooltip=["method", alt.Tooltip(f"{metric}:Q", format=".3f"), "frames"]), width="stretch")

# ---------------------------------------------------------- ablation / curves
```

"Compare fusion methods": runs every trained method on the current frame or the whole scene and charts the chosen metric (proposed method highlighted).

```
with st.expander("[bar chart] Ablation study (offline evaluation on the validation split)", expanded=False):
    results = load_ablation_results(probe_cfg)
    if not results:
        st.info("Run `python main.py ablation --train` to produce outputs/ablation/results.json.")
    else:
        df = pd.DataFrame(results)
        show = ["label", "NDS", "mAP", "mAP_present", "mAVE", "ADE", "FDE", "collision_rate", "latency_ms",
                "lidar_selected_ratio", "radar_selected_ratio"]
        st.dataframe(df[[c for c in show if c in df]], width="stretch", hide_index=True)
        metric = st.selectbox("Metric", [c for c in show[1:] if c in df], key="abl_metric")
        dfa = df.assign(proposed=df["experiment"].eq(PROPOSED).map({True: "proposed", False: "baseline"}))
        st.altair_chart(alt.Chart(dfa).mark_bar(cornerRadiusEnd=4).encode(
            x=alt.X(f"{metric}:Q"), y=alt.Y("label:N", sort=None, title=None),
            color=alt.Color("proposed:N", scale=alt.Scale(domain=["proposed", "baseline"], range=["#3987e5", "#6
                 b6a64"]), legend=None),
            tooltip=["label", alt.Tooltip(f"{metric}:Q", format=".4f")]), width="stretch")
        png = Path(probe_cfg.runtime.output_dir)
        png = (ROOT / png if not png.is_absolute() else png) / "ablation" / "ablation_metrics.png"
```

```
        if png.exists():
            st.image(str(png), width="stretch")

with st.expander("[chart] Training curves", expanded=False):
    hist = load_history(probe_cfg, experiment)
    if not hist:
        st.info(f"No training history for `{experiment}` yet.")
    else:
        rows = [{"epoch": h["epoch"], "train loss": h["train"].get("total"),
                 "val NDS": h.get("val", {}).get("detection", {}).get("NDS"),
                 "val ADE": h.get("val", {}).get("planning_learned", {}).get("ade")} for h in hist]
        dfh = pd.DataFrame(rows)
        cols = st.columns(3)
        for col, m in zip(cols, ["train loss", "val NDS", "val ADE"]):
            col.altair_chart(alt.Chart(dfh.dropna(subset=[m])).mark_line(point=alt.OverlayMarkDef(size=60), stro
    ↪               keWidth=2,
                                                                color="#3987e5").encode(
                x=alt.X("epoch:Q"), y=alt.Y(f"{m}:Q"), tooltip=["epoch", alt.Tooltip(f"{m}:Q", format=".4f")]
            ).properties(title=m, height=220), width="stretch")

with st.expander("(i) About this system", expanded=faculty and not ss.playing):
    st.markdown(f'<div class="msf-explain">{SYSTEM_STORY}</div>', unsafe_allow_html=True)
    if pipe.checkpoint_meta:
        meta = pipe.checkpoint_meta
        st.caption(f"Checkpoint: `{pipe.checkpoint}` · experiment {meta.get('experiment')} · stage {meta.get('st
    ↪            age')} · "
                   f"epoch {meta.get('epoch')}")

# ---------------------------------------------------------------------- playback
```

Ablation results (`outputs/ablation/results.json` + the PNG), training curves from `history.jsonl`, and the "About" text with the checkpoint metadata.

```
if ss.playing:
    interval = 0.5 / SPEEDS[speed_label]
    elapsed = time.perf_counter() - t_render
    time.sleep(max(0.0, interval - elapsed))
    ss["_advance"] = True
    st.rerun()
```

Playback: while playing, sleep for the remainder of `0.5 s / speed` (nuScenes key-frames are 2 Hz), set `_advance` and `st.rerun()`.

`main.py`

The single command-line entry point. (59 lines)

```
"""Single entry point for the whole project.

    python main.py preprocess [--maps]                    build sample-info caches (+ map caches)
    python main.py validate                               Phase 2: sensor sync / calibration report + images
    python main.py smoke [--experiments all]              shapes / losses / memory / latency on real data
    python main.py train --stage joint --experiment F_cam_lidar_radar_fs [--epochs 20]
    python main.py train --stage curriculum               the 7-stage curriculum (stages 1-7 in order)
    python main.py evaluate --checkpoint checkpoints/F_cam_lidar_radar_fs/joint/best.pt [--official]
    python main.py ablation [--train] [--epochs 20]       train/evaluate experiments A-F + plots
    python main.py infer --scene scene-0103 [--video]     render a scene replay to images / MP4
    python main.py bev --scene scene-0103 --frame 12      every BEV of one frame in a single image
    python main.py planner --planner baseline             evaluate planners with GT perception
    python main.py benchmark                              latency / memory per experiment
    python main.py app                                    launch the Streamlit dashboard
    python main.py test                                   run the test-suite
"""
```

Docstring = the usage text printed by `python main.py --help`.

```
from __future__ import annotations

import runpy
import subprocess
import sys
from pathlib import Path

ROOT = Path(__file__).resolve().parent
COMMANDS = {
    "preprocess": "scripts/preprocess.py",
    "validate": "scripts/validate_sensors.py",
    "smoke": "scripts/smoke_test.py",
    "train": "training/cli.py",
    "evaluate": "scripts/evaluate.py",
    "ablation": "evaluation/ablation.py",
    "infer": "scripts/inference.py",
```

```
    "bev": "scripts/render_bev.py",
    "planner": "scripts/eval_planner.py",
    "benchmark": "scripts/benchmark.py",
    "download": "scripts/download_dataset.py",
}


def main() -> None:
    if len(sys.argv) < 2 or sys.argv[1] in ("-h", "--help"):
        print(__doc__)
        sys.exit(0)
    cmd, rest = sys.argv[1], sys.argv[2:]
    if cmd == "app":
        sys.exit(subprocess.call([sys.executable, "-m", "streamlit", "run", str(ROOT / "app" / "streamlit_app.py"
            ), *rest]))
    if cmd == "test":
        sys.exit(subprocess.call([sys.executable, "-m", "pytest", *rest], cwd=ROOT))
    if cmd not in COMMANDS:
        print(f"Unknown command {cmd!r}.\n{__doc__}")
        sys.exit(2)
    script = ROOT / COMMANDS[cmd]
    sys.path.insert(0, str(ROOT))
    sys.argv = [str(script), *rest]
    runpy.run_path(str(script), run_name="__main__")


if __name__ == "__main__":
    main()
```

`COMMANDS` maps sub-commands to scripts. `main()`: `app` and `test` start subprocesses (Streamlit, pytest); every other command puts the project root on `sys.path`, rewrites `sys.argv` and executes the script with `runpy.run_path(..., run_name='__main__')`, so the scripts behave exactly as if they were run directly.

API Reference

Generated from the source code for **every module, class, method and function** of the project. For each function: signature with parameter and return types, purpose (docstring or hand-written note), internal calls to other project functions, callers, raised exceptions and file side effects detected in the code. "–" means none found.

Package app

app/__init__.py

Module: app · 1 lines
Purpose: Package marker.
Imports (project): –
Imports (third-party / stdlib): –
Imported by: –

app/streamlit_app.py

Module: app.streamlit_app · 542 lines · **entry point** (runnable script)
Purpose: Multi-Sensor Autonomous Driving System — interactive nuScenes replay & faculty demo.
Imports (project): app.ui, app.ui.resources, app.ui.styles, evaluation.detection_metrics, evaluation.trajectory_metrics, inference.pipeline, utils.config
Imports (third-party / stdlib): altair, collections, numpy, pandas, pathlib, streamlit, sys, time
Imported by: –

Constant	Value
ROOT	Path(__file__).resolve().parents[1]
SPEEDS	{'0.25x': 0.25, '0.5x': 0.5, '1x': 1.0, '2x': 2.0, '5x': 5.0}
MODES	{'1 · Dataset Viewer': 'dataset', '2 · Ground Truth Simulation': 'ground_truth', '3 · AI Autonomous Driving': 'model'}
PROPOSED	'F_cam_lidar_radar_fs'
FS_EXPLANATION	'The feature-selection network learns which LiDAR and Radar features are most informative for perception and suppresses redundant or noisy features before fu...
SYSTEM_STORY	"This system takes synchronized Camera, LiDAR, and Radar data from the nuScenes autonomous-driving dataset. The camera information is transformed into a Bird...

Functions

_inside_streamlit() -> bool
Purpose: Returns True when executed by the Streamlit runtime (streamlit.runtime.exists()); False under plain python or if Streamlit cannot be imported.
Returns: bool
lines 32–38

get_pipeline(version: str, device: str, checkpoint: str | None, experiment: str | None, allow_untrained: bool) -> FusionPipeline
Purpose: Cached (st.cache_resource, ≤ 12 entries) factory of FusionPipeline for (version, device, checkpoint, experiment, allow_untrained). Data-only pipelines disable the network camera preprocessing.
Parameters: version: str; device: str; checkpoint: str | None; experiment: str | None; allow_untrained: bool
Returns: FusionPipeline
Calls: app.ui.resources.base_config, inference.pipeline.FusionPipeline, utils.config.load_config
Decorators: st.cache_resource(show_spinner='Loading nuScenes data and model ...', max_entries=12)
lines 79–88

ss_default(key: str, value) -> None
Purpose: Initialises a st.session_state key with a default value only if it does not exist yet.
Parameters: key: str; value
Returns: None
lines 91–93

cb_prev() -> None
Purpose: Button callback: stop playback and move the key-frame slider one frame back (not below 0).
Returns: None
lines 169–171

cb_next() -> None
Purpose: Button callback: stop playback and move the slider one frame forward (not past the last frame).
Returns: None

lines 174–176

`cb_play() -> None`
Purpose: Button callback: start playback (restarting at frame 0 if the end was reached).
Returns: None
lines 179–182

`cb_pause() -> None`
Purpose: Button callback: stop playback.
Returns: None
lines 185–186

`cb_reset() -> None`
Purpose: Button callback: stop playback, go to frame 0, clear the scene-level evaluators and reset the tracker.
Returns: None
Called by: app.streamlit_app.cb_load
lines 189–193

`cb_load() -> None`
Purpose: Button callback for 'Load Scene' (same as reset).
Returns: None
Calls: app.streamlit_app.cb_reset
lines 196–197

`on_stage(name: str, ms: float) -> None`
Purpose: Stage-timer callback: accumulates the stage time under its pipeline-strip key and re-renders the strip with the next stage highlighted.
Parameters: `name`: str; `ms`: float
Returns: None
Calls: app.ui.styles.flow_html, app.ui.styles.stage_key_for
lines 244–251

app/ui/__init__.py

Module: `app.ui` · 1 lines
Purpose: Package marker.
Imports (project): –
Imports (third-party / stdlib): –
Imported by: app.streamlit_app, scripts.inference, scripts.render_bev

app/ui/panels.py

Module: `app.ui.panels` · 208 lines
Purpose: Panel renderers for the dashboard. Every image / table comes from a processed :class:`FrameResult`.
Imports (project): inference.pipeline, utils.geometry, visualization.bev_view, visualization.camera_view, visualization.common, visualization.feature_selection, visualization.lidar_view, visualization.radar_view, visualization.trajectory_view
Imports (third-party / stdlib): numpy, pandas, pathlib
Imported by: –

Constant	Value
TILE	300

Functions

`load_images(root: Path, res: FrameResult) -> dict[str, Optional[np.ndarray]]`
Purpose: Loads the six full-resolution camera images of a frame (None for unreadable files).
Parameters: `root`: Path; `res`: FrameResult
Returns: dict[str, Optional[np.ndarray]]
Calls: visualization.camera_view.load_image
Called by: scripts.inference.main, scripts.render_bev.main
lines 24–25

`object_sets(res: FrameResult) -> tuple[Optional[dict], dict]`
Purpose: Returns (model detections or None, GT objects with ≥ 1 LiDAR/radar point incl. their logged futures).
Parameters: `res`: FrameResult
Returns: tuple[Optional[dict], dict]
Called by: app.ui.panels.camera_panel, app.ui.panels.driving_bev, app.ui.panels.gt_comparison, app.ui.panels.trajectory_closeup
lines 28–34

`camera_panel(res: FrameResult, images: dict[str, Optional[np.ndarray]], cam: str, show_pred: bool, show_gt: bool, score_min: float) -> tuple[Optional[np.ndarray], np.ndarray]`
Purpose: Draws GT boxes (and, in model mode, predicted boxes above the confidence threshold with labels) into every camera image; returns the selected camera image and a 2×3 mosaic.
Parameters: `res`: FrameResult; `images`: dict[str, Optional[np.ndarray]]; `cam`: str; `show_pred`: bool; `show_gt`: bool; `score_min`: float

Returns: tuple[Optional[np.ndarray], np.ndarray]
Calls: app.ui.panels.object_sets, visualization.camera_view.camera_mosaic, visualization.camera_view.draw_boxes_on_image
Called by: scripts.inference.compose
lines 37–55

`fusion_tiles(res: FrameResult, images: dict[str, Optional[np.ndarray]], fs_available: bool) -> list[np.ndarray]`
Purpose: Camera BEV | LiDAR BEV | Radar BEV | LiDAR importance | Radar importance | Fused BEV.
Parameters: `res`: FrameResult; `images`: dict[str, Optional[np.ndarray]]; `fs_available`: bool
Returns: list[np.ndarray]
Calls: visualization.camera_view.camera_ipm_bev, visualization.common.put_label, visualization.feature_selection.energy_map, visualization.feature_selection.importance_map, visualization.feature_selection.titled, visualization.lidar_view.render_lidar_bev, visualization.radar_view.render_radar_bev
Called by: scripts.inference.compose
lines 58–85

`driving_bev(res: FrameResult, pipe: FusionPipeline, opts: dict[str, Any]) -> np.ndarray`
Purpose: Builds `SceneLayers` (map patch, points, predicted or GT objects with track ids/trails, logged futures, plan, logged ego future/past) and renders the autonomous-driving BEV; predictions are filtered by `opts['score_min']`.
Parameters: `res`: FrameResult; `pipe`: FusionPipeline; `opts`: dict[str, Any]
Returns: np.ndarray
Calls: app.ui.panels.object_sets, visualization.bev_view.SceneLayers, visualization.bev_view.render_driving_scene
Called by: scripts.inference.compose, scripts.render_bev.main
lines 88–119

`detection_table(res: FrameResult, score_min: float) -> pd.DataFrame`
Purpose: pandas DataFrame of the detections above a threshold: class, track id, confidence, x, y, range, speed — sorted by range.
Parameters: `res`: FrameResult; `score_min`: float
Returns: pd.DataFrame
lines 122–133

`gt_comparison(res: FrameResult, score_min: float, match_dist: float = 2.0) -> dict[str, int]`
Purpose: Greedy centre-distance matching (same rule as the nuScenes TP threshold, 2 m, same class).
Parameters: `res`: FrameResult; `score_min`: float; `match_dist`: float = 2.0
Returns: dict[str, int]
Calls: app.ui.panels.object_sets
lines 136–157

`ego_status(res: FrameResult) -> dict[str, float]`
Purpose: Ego global x/y, yaw (degrees) from the reference pose plus speed, yaw rate and acceleration from the info.
Parameters: `res`: FrameResult
Returns: dict[str, float]
Calls: utils.geometry.yaw_from_rotmat
lines 160–164

`trajectory_closeup(res: FrameResult, pipe: FusionPipeline) -> np.ndarray`
Purpose: Renders the trajectory close-up (planned vs logged) with predicted (score ≥ 0.3) or GT objects.
Parameters: `res`: FrameResult; `pipe`: FusionPipeline
Returns: np.ndarray
Calls: app.ui.panels.object_sets, visualization.trajectory_view.render_trajectory_closeup
Called by: scripts.inference.compose
lines 167–174

`explain_selection(res: FrameResult, pipe: FusionPipeline, score_min: float) -> dict[str, Any]`
Purpose: Raw -> importance -> selected images per modality, channel gates and per-class sensor trust.
Parameters: `res`: FrameResult; `pipe`: FusionPipeline; `score_min`: float
Returns: dict[str, Any]
Calls: visualization.common.heatmap_image, visualization.feature_selection.channel_gate_table, visualization.feature_selection.class_importance_summary, visualization.feature_selection.contribution, visualization.feature_selection.dominance_map, visualization.feature_selection.energy_map, visualization.feature_selection.importance_map, visualization.feature_selection.modality_weight_map, visualization.feature_selection.object_importance, visualization.feature_selection.titled
lines 177–208

app/ui/resources.py

Module: `app.ui.resources` · 68 lines
Purpose: Checkpoint discovery and cached pipeline construction for the dashboard.
Imports (project): utils.config
Imports (third-party / stdlib): json, pathlib
Imported by: app.streamlit_app, scripts.inference, scripts.render_bev

Constant	Value
STAGE_PREFERENCE	`('end_to_end', 'joint', 'detection', 'fusion', 'trajectory')`
SHARED_FALLBACK	`{'A_lidar_only': 'lidar', 'B_radar_only': 'radar'}`

Functions

`dashboard_methods() -> dict[str, str]`
Purpose: Returns the `dashboard_methods` mapping (dropdown label → experiment id) from configs/ablation.yaml.
Returns: dict[str, str]
Calls: utils.config.read_yaml
lines 14–15

`experiment_labels(short: bool = False) -> dict[str, str]`
Purpose: Experiment id → display label (or short label when `short=True`).
Parameters: `short`: bool = False
Returns: dict[str, str]
Calls: utils.config.read_yaml
lines 18–20

`find_checkpoint(cfg: Config, experiment: str) -> Optional[Path]`
Purpose: Newest-stage checkpoint of an experiment: tries stages end_to_end, joint, detection, fusion, trajectory (best.pt, then last.pt); radar/LiDAR-only experiments fall back to the shared stage-2/3 checkpoints. None if nothing exists.
Parameters: `cfg`: Config; `experiment`: str
Returns: Optional[Path]
Calls: utils.config.resolve_path
Called by: scripts.inference.main, scripts.render_bev.main
lines 23–36

`available_versions(cfg: Config) -> dict[str, bool]`
Purpose: Which nuScenes versions (mini / trainval) exist under `dataset.root`.
Parameters: `cfg`: Config
Returns: dict[str, bool]
lines 39–41

`base_config(version: str, device: str) -> Config`
Purpose: Loads the configuration for a version and device (trainval switches the splits to train/val).
Parameters: `version`: str; `device`: str
Returns: Config
Calls: utils.config.load_config
Called by: app.streamlit_app.get_pipeline
lines 44–48

`load_ablation_results(cfg: Config) -> Optional[list[dict[str, Any]]]`
Purpose: Reads `outputs/ablation/results.json` if present.
Parameters: `cfg`: Config
Returns: Optional[list[dict[str, Any]]]
Calls: utils.config.resolve_path
lines 51–55

`load_history(cfg: Config, experiment: str, stage: str = 'joint') -> list[dict[str, Any]]`
Purpose: Reads `outputs/logs/<experiment>/<stage>/history.jsonl` (skips malformed lines).
Parameters: `cfg`: Config; `experiment`: str; `stage`: str = 'joint'
Returns: list[dict[str, Any]]
Calls: utils.config.resolve_path
lines 58–68

app/ui/styles.py

Module: `app.ui.styles` · 103 lines
Purpose: CSS and small HTML components for the dashboard (dark "mission control" theme).
Imports (project): –
Imports (third-party / stdlib): html
Imported by: app.streamlit_app

Constant	Value
CSS	`'\n<style>\n:root {\n --bg: #0d0f14; --panel: #151922; --panel-2: #1b2030; --line: #2a3142;\n --ink: #f2f4f8; --ink-2: #b8bfcc; --muted: #7d8596;\n --accent:...`
FLOW_STAGES	`[('camera', 'CAMERA → BEV', ('camera_bev',)), ('sensors', 'LiDAR + RADAR', ('lidar_encoder', 'radar_encoder')), ('selection', 'FEATURE SELECTION', ('feature_...`

Functions

`flow_html(done: dict[str, float], active: Optional[str] = None, skipped: Iterable[str] = ()) -> str`
Purpose: Pipeline strip; `done` maps stage key -> ms.
Parameters: `done`: dict[str, float]; `active`: Optional[str] = None; `skipped`: Iterable[str] = ()
Returns: str
Called by: app.streamlit_app.on_stage
lines 67–78

`stage_key_for(timer_stage: str) -> Optional[str]`
Purpose: Maps a timer stage name (e.g. 'lidar_encoder') to its pipeline-strip key ('sensors').
Parameters: `timer_stage`: str
Returns: Optional[str]
Called by: app.streamlit_app.on_stage
lines 81–85

`banner_html(kind: str, text: str) -> str`
Purpose: HTML for the mode banner (class gt / model / untrained).
Parameters: `kind`: str; `text`: str
Returns: str
lines 88–89

`kpis_html(items: list[tuple[str, str, str]]) -> str`
Purpose: HTML grid of KPI tiles from (key, value, note) triples (all text HTML-escaped).
Parameters: `items`: list[tuple[str, str, str]]
Returns: str
lines 92–95

`panel_title(text: str) -> str`
Purpose: HTML for an uppercase panel title.
Parameters: `text`: str
Returns: str
lines 98–99

`chip(text: str, kind: str = '') -> str`
Purpose: HTML for a small status chip (ok / bad / warn styles).
Parameters: `text`: str; `kind`: str = ''
Returns: str
lines 102–103

Package `datasets`

`datasets/__init__.py`

Module: `datasets` · 1 lines
Purpose: nuScenes data access: tables, per-sample infos, sensor loaders and the fusion dataset.
Imports (project): –
Imports (third-party / stdlib): –
Imported by: –

`datasets/camera_loader.py`

Module: `datasets.camera_loader` · 171 lines
Purpose: Camera loading, image-space augmentation and calibration matrices.
Imports (project): utils.geometry, utils.logging_utils
Imports (third-party / stdlib): PIL, dataclasses, logging, numpy, pathlib
Imported by: datasets.nuscenes_dataset, scripts.validate_sensors

Constant	Value
IMAGENET_MEAN	`np.array([0.485, 0.456, 0.406], dtype=np.float32)`
IMAGENET_STD	`np.array([0.229, 0.224, 0.225], dtype=np.float32)`

class `ImageAug`

Class attributes / fields: `resize` = float; `resize_dims` = tuple[int, int]; `crop` = tuple[int, int, int, int]; `flip` = bool; `rotate` = float

class `CameraLoader`

Load the surround-view images of one sample with consistent augmentation matrices.
Instance attributes (set in __init__): `root` = Path(root); `cameras` = list(cameras); `final_h` = (int(image_size[0]), int(image_size[1])); `final_w` = (int(image_size[0]), int(image_size[1])); `cfg` = camera_cfg

`__init__(self, root: Path, cameras: list[str], image_size: tuple[int, int], camera_cfg: Any) -> None`
Purpose: Stores dataset root, camera list, final (H, W) and the camera config.
Parameters: `root`: Path; `cameras`: list[str]; `image_size`: tuple[int, int]; `camera_cfg`: Any
Returns: None
Called by: datasets.nuscenes_dataset.NuScenesFusionDataset.__init__, scripts.validate_sensors.augmentation_roundtrip
lines 78–82

`sample_aug(self, orig_w: int, orig_h: int, train: bool, rng: np.random.Generator) -> ImageAug`
Purpose: Samples an `ImageAug` (random in training, deterministic centred/bottom crop at test time).
Parameters: `orig_w`: int; `orig_h`: int; `train`: bool; `rng`: np.random.Generator
Returns: ImageAug

Calls: datasets.camera_loader.ImageAug
Called by: datasets.camera_loader.CameraLoader.load
lines 84–102

`load(self, info: dict[str, Any], train: bool, rng: np.random.Generator) -> dict[str, np.ndarray]`
Purpose: Loads and augments the six images and returns images, intrinsics, post_rots, post_trans, cam2ref and cam_valid (missing/corrupt cameras → zero image, invalid flag).
Parameters: `info`: dict[str, Any]; `train`: bool; `rng`: np.random.Generator
Returns: dict[str, np.ndarray]
Calls: datasets.camera_loader.CameraLoader.sample_aug, datasets.camera_loader.apply_image_aug, datasets.camera_loader.image_aug_matrices
Side effects (file / disk I/O calls): open
lines 104–141

Functions

`_rot2(theta: float) -> np.ndarray`
Purpose: 2×2 image-space rotation matrix matching `PIL.Image.rotate` (counter-clockwise on screen, y down).
Parameters: `theta`: float
Returns: np.ndarray
Called by: datasets.camera_loader.image_aug_matrices
lines 44–46

`image_aug_matrices(aug: ImageAug) -> tuple[np.ndarray, np.ndarray]`
Purpose: 3x3 `post_rot` and 3-vector `post_tran` of an :class:`ImageAug` (LSS convention).
Parameters: `aug`: ImageAug
Returns: tuple[np.ndarray, np.ndarray]
Calls: datasets.camera_loader._rot2
Called by: datasets.camera_loader.CameraLoader.load, scripts.validate_sensors.augmentation_roundtrip
lines 49–63

`apply_image_aug(img: Image.Image, aug: ImageAug) -> Image.Image`
Purpose: Applies resize → crop → optional horizontal flip → optional rotation to a PIL image.
Parameters: `img`: Image.Image; `aug`: ImageAug
Returns: Image.Image
Called by: datasets.camera_loader.CameraLoader.load
lines 66–72

`lidar_depth_maps(points_ref: np.ndarray, cams: dict[str, np.ndarray], image_size: tuple[int, int], stride: int, depth_range: tuple[float, float]) -> np.ndarray`
Purpose: Sparse LiDAR depth per camera at feature resolution `(N, H/stride, W/stride)`.
Details: Each feature cell keeps the *closest* projected LiDAR point (0 = no supervision), exactly the BEVDepth depth-supervision target.
Parameters: `points_ref`: np.ndarray; `cams`: dict[str, np.ndarray]; `image_size`: tuple[int, int]; `stride`: int; `depth_range`: tuple[float, float]
Returns: np.ndarray
Calls: utils.geometry.invert_transform
Called by: datasets.nuscenes_dataset.NuScenesFusionDataset.__getitem__
lines 144–171

datasets/info_builder.py

Module: `datasets.info_builder` · 407 lines
Purpose: Pre-compute per-sample "infos" from the nuScenes tables.
Imports (project): datasets.nuscenes_db, datasets.splits, utils.config, utils.geometry
Imports (third-party / stdlib): hashlib, json, logging, numpy, pathlib, pickle
Imported by: datasets.nuscenes_dataset, inference.pipeline, scripts.check_eval_vs_devkit, scripts.eval_planner, scripts.preprocess, scripts.validate_sensors, tests.conftest

Constant	Value
INFO_FORMAT_VERSION	4
CATEGORY_TO_DETECTION	{'movable_object.barrier': 'barrier', 'vehicle.bicycle': 'bicycle', 'vehicle.bus.bendy': 'bus', 'vehicle.bus.rigid': 'bus', 'vehicle.car': 'car', 'vehicle.co...
COMMANDS	('RIGHT', 'LEFT', 'STRAIGHT')

class CanBus

Optional reader for the nuScenes CAN-bus expansion (`<root>/can_bus/<scene>_pose.json`).
Instance attributes (set in __init__): `dir` = Path(root) / 'can_bus'; `available` = self.dir.is_dir(); `_cache` = {}

`__init__(self, root: Path) -> None`
Purpose: Remembers `<root>/can_bus` and whether it exists; per-scene message cache.
Parameters: `root`: Path
Returns: None

Called by: datasets.info_builder.build_infos
lines 160–163

`pose_messages(self, scene_name: str) -> Optional[list[dict]]`
Purpose: Loads `<scene>_pose.json` once (None if unavailable).
Parameters: `scene_name`: str
Returns: Optional[list[dict]]
Called by: datasets.info_builder.CanBus.ego_state
lines 165–171

`ego_state(self, scene_name: str, timestamp: int) -> Optional[dict[str, float]]`
Purpose: Speed / yaw rate / longitudinal acceleration of the last CAN pose message at or before the timestamp (None without CAN data).
Parameters: `scene_name`: str; `timestamp`: int
Returns: Optional[dict[str, float]]
Calls: datasets.info_builder.CanBus.pose_messages
lines 173–186

Functions

`_sd_matrices(db: NuScenesDB, sd: dict) -> tuple[np.ndarray, np.ndarray, dict]`
Purpose: sensor2ego (calibrated_sensor) and ego2global (ego_pose at the sample_data timestamp) as 4×4 matrices, plus the calibrated_sensor record.
Parameters: `db`: NuScenesDB; `sd`: dict
Returns: tuple[np.ndarray, np.ndarray, dict]
Calls: utils.geometry.transform_matrix
Called by: datasets.info_builder._sensor_record
lines 62–65

`_sensor_record(db: NuScenesDB, sd: dict, ref_global2ego: np.ndarray, ref_timestamp: int) -> dict[str, Any]`
Purpose: Per-sweep record with relative path, timestamp, time-lag, matrices, combined sensor2ref and (cameras) intrinsics/size.
Parameters: `db`: NuScenesDB; `sd`: dict; `ref_global2ego`: np.ndarray; `ref_timestamp`: int
Returns: dict[str, Any]
Calls: datasets.info_builder._sd_matrices
Called by: datasets.info_builder.build_sample_info
lines 68–82

`annotation_velocity(db: NuScenesDB, ann: dict, max_time_diff: float = 1.5) -> np.ndarray`
Purpose: Global-frame velocity of an annotation (identical to `NuScenes.box_velocity`).
Parameters: `db`: NuScenesDB; `ann`: dict; `max_time_diff`: float = 1.5
Returns: np.ndarray
Called by: datasets.info_builder._boxes_for_sample
lines 85–100

`_ann_box_global(ann: dict) -> np.ndarray`
Purpose: nuScenes annotation -> `[x, y, z, l, w, h, yaw]` in the global frame (size is `[w, l, h]`).
Parameters: `ann`: dict
Returns: np.ndarray
Calls: utils.geometry.quat_to_rotmat, utils.geometry.yaw_from_rotmat
Called by: datasets.info_builder._boxes_for_sample, datasets.info_builder.build_sample_info
lines 103–107

`_boxes_for_sample(db: NuScenesDB, sample: dict, ref_global2ego: np.ndarray, classes: list[str], with_velocity: bool = True) -> dict[str, Any]`
Purpose: All annotations of a sample mapped to the configured classes, converted to reference-frame boxes with labels, point counts, attributes, velocity validity, tokens and visibility.
Parameters: `db`: NuScenesDB; `sample`: dict; `ref_global2ego`: np.ndarray; `classes`: list[str]; `with_velocity`: bool = True
Returns: dict[str, Any]
Calls: datasets.info_builder._ann_box_global, datasets.info_builder.annotation_velocity, utils.geometry.transform_boxes
Called by: datasets.info_builder.build_sample_info
lines 110–148

`_lidar_pose(db: NuScenesDB, sample: dict, lidar_channel: str) -> tuple[dict, np.ndarray]`
Purpose: LIDAR_TOP key-frame sample_data and its ego2global matrix (defines the reference frame).
Parameters: `db`: NuScenesDB; `sample`: dict; `lidar_channel`: str
Returns: tuple[dict, np.ndarray]
Calls: utils.geometry.transform_matrix
Called by: datasets.info_builder.build_sample_info
lines 151–154

`_ego_state_from_poses(db: NuScenesDB, lidar_sd: dict, window: float = 0.2, accel_window: float = 0.5) -> dict[str, float]`
Purpose: Causal (backward-difference) ego speed, yaw rate and acceleration from 20 Hz LiDAR poses.
Parameters: `db`: NuScenesDB; `lidar_sd`: dict; `window`: float = 0.2; `accel_window`: float = 0.5
Returns: dict[str, float]

Calls: utils.geometry.quat_to_rotmat, utils.geometry.wrap_angle, utils.geometry.yaw_from_rotmat
Called by: datasets.info_builder.build_sample_info
lines 189–227

`driving_command(future_xy: np.ndarray, mask: np.ndarray, threshold: float) -> int`
Purpose: 0 = RIGHT, 1 = LEFT, 2 = STRAIGHT from the lateral offset of the last valid future point.
Parameters: `future_xy`: np.ndarray; `mask`: np.ndarray; `threshold`: float
Returns: int
Called by: datasets.info_builder.build_sample_info, datasets.nuscenes_dataset.NuScenesFusionDataset.__getitem__
lines 230–239

`build_sample_info(db: NuScenesDB, sample_token: str, cfg: Config, can_bus: Optional[CanBus] = None) -> dict[str, Any]`
Purpose: Build the info dictionary of one key-frame sample.
Parameters: `db`: NuScenesDB; `sample_token`: str; `cfg`: Config; `can_bus`: Optional[CanBus] = None
Returns: dict[str, Any]
Calls: datasets.info_builder._ann_box_global, datasets.info_builder._boxes_for_sample, datasets.info_builder._ego_state_from_poses, datasets.info_builder._lidar_pose, datasets.info_builder._sensor_record, datasets.info_builder.driving_command, utils.geometry.invert_transform, utils.geometry.transform_boxes, utils.geometry.yaw_from_rotmat
Called by: datasets.info_builder.build_infos
lines 242–356

`build_infos(db: NuScenesDB, scene_names: list[str], cfg: Config) -> list[dict[str, Any]]`
Purpose: Builds infos for all key-frames of the given scenes (adds frame_idx / num_frames).
Parameters: `db`: NuScenesDB; `scene_names`: list[str]; `cfg`: Config
Returns: list[dict[str, Any]]
Calls: datasets.info_builder.CanBus, datasets.info_builder.build_sample_info
Called by: datasets.info_builder.load_infos
lines 359–370

`_cache_key(cfg: Config, split: str) -> str`
Purpose: MD5 hash (10 chars) of every setting that influences the infos — part of the cache file name.
Parameters: `cfg`: Config; `split`: str
Returns: str
Called by: datasets.info_builder.load_infos
lines 373–387

`load_infos(cfg: Config, split: str, db: Optional[NuScenesDB] = None, rebuild: bool = False) -> list[dict[str, Any]]`
Purpose: Load cached infos for `split` or build (and cache) them from the tables.
Parameters: `cfg`: Config; `split`: str; `db`: Optional[NuScenesDB] = None; `rebuild`: bool = False
Returns: list[dict[str, Any]]
Calls: datasets.info_builder._cache_key, datasets.info_builder.build_infos, datasets.nuscenes_db.NuScenesDB, datasets.splits.get_split_scenes, utils.config.resolve_path
Called by: datasets.nuscenes_dataset.NuScenesFusionDataset.__init__, inference.pipeline.FusionPipeline.__init__, scripts.check_eval_vs_devkit.main, scripts.eval_planner.main, scripts.preprocess.main, scripts.validate_sensors.main, tests.conftest.val_infos
Side effects (file / disk I/O calls): dump, mkdir, open
lines 390–407

datasets/lidar_loader.py

Module: `datasets.lidar_loader` · 90 lines
Purpose: LiDAR loading: multi-sweep accumulation in the reference frame.
Imports (project): utils.bev_grid, utils.geometry, utils.logging_utils
Imports (third-party / stdlib): logging, numpy, pathlib
Imported by: datasets.nuscenes_dataset, scripts.eval_planner, scripts.validate_sensors, tests.test_dataset

Constant	Value
LIDAR_POINT_FEATURES	('x', 'y', 'z', 'intensity', 'time_lag')

class LidarLoader

Load and accumulate LiDAR sweeps for one sample. Missing or corrupt sweep files are skipped with a warning; if every sweep fails the loader returns an empty `(0, 5)` array so the model can still run on other sensors.
Instance attributes (set in __init__): `root` = Path(root); `grid` = grid; `num_sweeps` = num_sweeps; `max_points` = max_points; `remove_close_radius` = remove_close_radius; `range_margin` = range_margin

`__init__(self, root: Path, grid: BEVGrid, num_sweeps: int = 10, max_points: Optional[int] = None, remove_close_radius: float = 1.5, range_margin: float = 0.0) -> None`
Purpose: Stores root, grid, sweep count, max points and the ego-return removal radius.
Parameters: `root`: Path; `grid`: BEVGrid; `num_sweeps`: int = 10; `max_points`: Optional[int] = None; `remove_close_radius`: float = 1.5; `range_margin`: float = 0.0
Returns: None

Called by: datasets.nuscenes_dataset.NuScenesFusionDataset.__init__, scripts.eval_planner.main, scripts.validate_sensors.augmentation_roundtrip, scripts.validate_sensors.main, scripts.validate_sensors.render_sample, tests.test_dataset.test_lidar_points_inside_gt_boxes_match_annotations
lines 41–48

`load_sweep(self, rec: dict[str, Any]) -> Optional[np.ndarray]`
Purpose: Reads one sweep, removes ego returns, transforms to the reference frame, adds the time-lag column (None on read errors, warned once).
Parameters: `rec`: dict[str, Any]
Returns: Optional[np.ndarray]
Calls: datasets.lidar_loader.read_lidar_bin, utils.geometry.apply_transform
Called by: datasets.lidar_loader.LidarLoader.load, datasets.lidar_loader.LidarLoader.load_keyframe
lines 50–64

`load(self, info: dict[str, Any], num_sweeps: Optional[int] = None, rng: Optional[np.random.Generator] = None, filter_range: bool = True) -> np.ndarray`
Purpose: Accumulates the first n sweeps, filters to the BEV range and subsamples above max_points.
Parameters: `info`: dict[str, Any]; `num_sweeps`: Optional[int] = None; `rng`: Optional[np.random.Generator] = None; `filter_range`: bool = True
Returns: np.ndarray
Calls: datasets.lidar_loader.LidarLoader.load_sweep
lines 66–85

`load_keyframe(self, info: dict[str, Any]) -> np.ndarray`
Purpose: Only the key-frame sweep (used for depth supervision and calibration checks).
Parameters: `info`: dict[str, Any]
Returns: np.ndarray
Calls: datasets.lidar_loader.LidarLoader.load_sweep
lines 87–90

Functions

`read_lidar_bin(path: Path) -> np.ndarray`
Purpose: Read a nuScenes LiDAR file -> `(N, 5) [x, y, z, intensity, ring]` in the sensor frame.
Parameters: `path`: Path
Returns: np.ndarray
Called by: datasets.lidar_loader.LidarLoader.load_sweep
Raises: ValueError: f'Corrupt LiDAR file (size {data.size} not divisible by 5): {path}'
lines 26–31

datasets/map_prior.py

Module: `datasets.map_prior` · 79 lines
Purpose: nuScenes raster map prior (`maps/*.png`, category `semantic_prior`).
Imports (project): –
Imports (third-party / stdlib): PIL, cv2, logging, numpy, pathlib
Imported by: inference.pipeline, scripts.preprocess

Constant	Value
NATIVE_RES	0.1

class MapPrior

Instance attributes (set in __init__): `root` = (Path(root), Path(cache_dir) / 'maps', int(reduce)); `cache_dir` = (Path(root), Path(cache_dir) / 'maps', int(reduce)); `reduce` = (Path(root), Path(cache_dir) / 'maps', int(reduce)); `res` = NATIVE_RES * self.reduce; `_maps` = {}

`__init__(self, root: Path, cache_dir: Path, reduce: int = 5) -> None`
Purpose: Stores root, cache folder, reduction factor (5 → 0.5 m/px) and an in-memory map cache.
Parameters: `root`: Path; `cache_dir`: Path; `reduce`: int = 5
Returns: None
Called by: inference.pipeline.FusionPipeline.__init__, scripts.preprocess.main
lines 23–26

`_cache_path(self, filename: str) -> Path`
Purpose: Cache file path `<cache>/maps/<stem>_r5.npy`.
Parameters: `filename`: str
Returns: Path
Called by: datasets.map_prior.MapPrior.load
lines 28–29

`load(self, filename: str) -> Optional[np.ndarray]`
Purpose: Returns the reduced binary map (from memory, the .npy cache, or by decoding the PNG once with PIL `reduce`).
Parameters: `filename`: str
Returns: Optional[np.ndarray]

Calls: datasets.map_prior.MapPrior._cache_path
Called by: datasets.map_prior.MapPrior.crop_to_ego
Side effects (file / disk I/O calls): mkdir, open, save
lines 31–52

`crop_to_ego(self, filename: str, ego2global: np.ndarray, x_range: tuple[float, float], y_range: tuple[float, float], ppm: float) -> Optional[np.ndarray]`
Purpose: Map patch in *display* layout (forward up, left left) at `ppm` pixels per metre (uint8 0/255).
Parameters: `filename`: str; `ego2global`: np.ndarray; `x_range`: tuple[float, float]; `y_range`: tuple[float, float]; `ppm`: float
Returns: Optional[np.ndarray]
Calls: datasets.map_prior.MapPrior.load
Called by: datasets.map_prior.MapPrior.drivable_on_grid
lines 54–70

`drivable_on_grid(self, filename: str, ego2global: np.ndarray, grid) -> Optional[np.ndarray]`
Purpose: Map prior on a model-layout grid `(ny, nx)` (bool), e.g. for the planner's map cost.
Parameters: `filename`: str; `ego2global`: np.ndarray; `grid`
Returns: Optional[np.ndarray]
Calls: datasets.map_prior.MapPrior.crop_to_ego
lines 72–79

datasets/nuscenes_dataset.py

Module: `datasets.nuscenes_dataset` · 231 lines
Purpose: PyTorch dataset for multi-sensor fusion, detection and planning on nuScenes.
Imports (project): datasets.camera_loader, datasets.info_builder, datasets.lidar_loader, datasets.radar_loader, models.detection.targets, utils.bev_grid, utils.config, utils.geometry, utils.raster
Imports (third-party / stdlib): logging, numpy, pathlib, torch
Imported by: evaluation.ablation, inference.pipeline, scripts.benchmark, scripts.evaluate, scripts.smoke_test, tests.test_dataset, training.trainer

Constant	Value
POINT_KEYS	('lidar_points', 'radar_points')
LIST_KEYS	('gt_boxes', 'gt_labels', 'meta', 'raw_lidar_points', 'raw_radar_points')

class NuScenesFusionDataset(Dataset)

Multi-modal nuScenes key-frame dataset. Args: cfg: full configuration. split: `mini_train` / `mini_val` / `train` / `val`. train: enables augmentation. infos: pre-loaded infos (otherwise loaded from the cache / built). load_all_sensors: always return raw LiDAR and radar points (dashboard visualisation), even for modalities the model does not use. with_targets: build detection / planning targets.
Instance attributes (set in __init__): `cfg` = cfg; `split` = split; `train` = train; `infos` = infos if infos is not None else load_infos(cfg, split); `load_all_sensors` = load_all_sensors; `with_targets` = with_targets; `modalities` = enabled_modalities(cfg); `classes` = list(cfg.classes); `grid` = BEVGrid.from_config(cfg.bev); `head_grid` = self.grid.downsample(int(cfg.model.detection_head.out_stride)); `image_size` = tuple((int(v) for v in dcfg.camera.image_size)); `feature_stride` = int(cfg.model.camera_bev.feature_stride); `camera_loader` = CameraLoader(root, dcfg.cameras, self.image_size, dcfg.camera); `lidar_loader` = LidarLoader(root, self.grid, int(dcfg.lidar.sweeps), dcfg.lidar.max_points, float(dcfg.lidar.remo...; `radar_loader` = RadarLoader(root, self.grid, int(dcfg.radar.sweeps), f.invalid_states, f.dynprop_states, f.ambig_...; `collision_labels` = np.array([self.classes.index(c) for c in cfg.planning.collision_classes if c in self.classes])

`__init__(self, cfg: Config, split: str, train: bool = False, infos: Optional[list[dict]] = None, load_all_sensors: bool = False, with_targets: bool = True) -> None`
Purpose: Loads infos, builds grids and loaders, collision-class labels, optional truncation.
Parameters: `cfg`: Config; `split`: str; `train`: bool = False; `infos`: Optional[list[dict]] = None; `load_all_sensors`: bool = False; `with_targets`: bool = True
Returns: None
Calls: datasets.camera_loader.CameraLoader, datasets.info_builder.load_infos, datasets.lidar_loader.LidarLoader, datasets.nuscenes_dataset.enabled_modalities, datasets.radar_loader.RadarLoader, utils.bev_grid.BEVGrid.from_config
Called by: evaluation.ablation.evaluate_checkpoint, inference.pipeline.FusionPipeline.__init__, scripts.benchmark.main, scripts.evaluate.main, scripts.smoke_test.run, tests.test_dataset.test_dataset_item_shapes, tests.test_dataset.test_lss_geometry_matches_projection, training.trainer.Trainer.__init__
lines 72–98

`__len__(self) -> int`
Purpose: Number of samples.
Returns: int
lines 100–101

`_sample_bda(self, rng: np.random.Generator) -> np.ndarray`
Purpose: Random BEV augmentation matrix in training (identity otherwise).
Parameters: `rng`: np.random.Generator
Returns: np.ndarray
Calls: utils.geometry.bev_rotation_matrix
Called by: datasets.nuscenes_dataset.NuScenesFusionDataset.__getitem__

lines 104–110

`_apply_bda_points(points: np.ndarray, bda: np.ndarray, vel_cols: tuple[tuple[int, int], ...] = ()) -> np.ndarray`
Purpose: Applies bda to xyz and to the given velocity column pairs.
Parameters: `points`: np.ndarray; `bda`: np.ndarray; `vel_cols`: tuple[tuple[int, int], ...] = ()
Returns: np.ndarray
Called by: datasets.nuscenes_dataset.NuScenesFusionDataset.__getitem__
Decorators: staticmethod
lines 113–120

`_gt_mask(self, gt: dict[str, Any]) -> np.ndarray`
Purpose: Mask of GT boxes with enough LiDAR+radar points and centre inside the grid.
Parameters: `gt`: dict[str, Any]
Returns: np.ndarray
Called by: datasets.nuscenes_dataset.NuScenesFusionDataset.__getitem__
lines 122–127

`__getitem__(self, idx: int) -> dict[str, Any]`
Purpose: Builds one item: sensors, augmentation, optional modality dropout, filtered/augmented GT, planning inputs, CenterPoint targets, fg heatmap, future occupancy and meta.
Parameters: `idx`: int
Returns: dict[str, Any]
Calls: datasets.camera_loader.lidar_depth_maps, datasets.info_builder.driving_command, datasets.nuscenes_dataset.NuScenesFusionDataset._apply_bda_points, datasets.nuscenes_dataset.NuScenesFusionDataset._gt_mask, datasets.nuscenes_dataset.NuScenesFusionDataset._sample_bda, models.detection.targets.build_center_targets, models.detection.targets.foreground_heatmap, utils.geometry.augment_boxes_bev, utils.raster.rasterize_boxes
lines 130–210

Functions

`enabled_modalities(cfg: Config) -> dict[str, bool]`
Purpose: `{camera, lidar, radar}` flags from `fusion.use_*`.
Parameters: `cfg`: Config
Returns: dict[str, bool]
Called by: datasets.nuscenes_dataset.NuScenesFusionDataset.__init__
lines 54–56

`collate_batch(samples: list[dict[str, Any]]) -> dict[str, Any]`
Purpose: Stack fixed-size arrays, concatenate point clouds with a leading batch-index column.
Parameters: `samples`: list[dict[str, Any]]
Returns: dict[str, Any]
Called by: inference.pipeline.FusionPipeline._run_model, scripts.benchmark.main, tests.test_dataset.test_dataset_item_shapes, tests.test_dataset.test_lss_geometry_matches_projection
lines 213–231

datasets/nuscenes_db.py

Module: `datasets.nuscenes_db` · 119 lines
Purpose: Lightweight nuScenes table loader.
Imports (project): –
Imports (third-party / stdlib): json, logging, pathlib, time
Imported by: datasets.info_builder, scripts.preprocess

Constant	Value
TABLES	`('attribute', 'calibrated_sensor', 'category', 'ego_pose', 'instance', 'log', 'map', 'sample', 'sample_annotation', 'sample_data', 'scene', 'sensor', 'visibi...`

class NuScenesDB

In-memory nuScenes database with token lookup. Args: root: dataset root containing `samples/`, `sweeps/`, `maps/` and `<version>/`. version: table folder name, e.g. `"v1.0-mini"`.
Instance attributes (set in __init__): `root` = Path(root); `version` = version; `tables` = {}; `_index` = {}

`__init__(self, root: str | Path, version: str = 'v1.0-mini', verbose: bool = True) -> None`
Purpose: Loads all tables of `<root>/<version>` and builds the indexes (FileNotFoundError for a wrong root/version or a missing table).
Parameters: `root`: str | Path; `version`: str = 'v1.0-mini'; `verbose`: bool = True
Returns: None
Calls: datasets.nuscenes_db.NuScenesDB._build_reverse_index
Called by: datasets.info_builder.load_infos, scripts.preprocess.main
Raises: FileNotFoundError: f"nuScenes tables not found at '{table_dir}'. Set dataset.root / dataset.version (confi...; FileNotFoundError: f'Missing nuScenes table {path}'
Side effects (file / disk I/O calls): open
lines 32–58

`_build_reverse_index(self) -> None`
Purpose: Adds sample['data'], sample['anns'], sample_data channel/modality, annotation category_name, scene-by-name and log-to-map lookups.
Returns: None
Calls: datasets.nuscenes_db.NuScenesDB.get
Called by: datasets.nuscenes_db.NuScenesDB.__init__
lines 61–80

`get(self, table: str, token: str) -> dict[str, Any]`
Purpose: Record by table and token (KeyError with table name if unknown).
Parameters: `table`: str; `token`: str
Returns: dict[str, Any]
Called by: datasets.nuscenes_db.NuScenesDB._build_reverse_index, datasets.nuscenes_db.NuScenesDB.iter_sweeps, datasets.nuscenes_db.NuScenesDB.scene_sample_tokens
Raises: KeyError: f'Token {token!r} not found in table {table!r}'
lines 83–87

`has(self, table: str, token: str) -> bool`
Purpose: Whether a token exists in a table.
Parameters: `table`: str; `token`: str
Returns: bool
lines 89–90

`scenes(self) -> list[dict]`
Purpose: List of scene records (property).
Returns: list[dict]
Decorators: property
lines 93–94

`scene_by_name(self, name: str) -> dict`
Purpose: Scene record by name (e.g. 'scene-0103').
Parameters: `name`: str
Returns: dict
lines 96–97

`scene_sample_tokens(self, scene: dict) -> list[str]`
Purpose: Key-frame sample tokens of a scene in temporal order.
Parameters: `scene`: dict
Returns: list[str]
Calls: datasets.nuscenes_db.NuScenesDB.get
lines 99–105

`iter_sweeps(self, sd_token: str, direction: str = 'prev') -> Iterator[dict]`
Purpose: Walk the `prev`/`next` chain of a sample_data record (including itself).
Parameters: `sd_token`: str; `direction`: str = 'prev'
Returns: Iterator[dict]
Calls: datasets.nuscenes_db.NuScenesDB.get
lines 107–113

`map_for_log(self, log_token: str) -> dict | None`
Purpose: Map record of a log (or None).
Parameters: `log_token`: str
Returns: dict | None
lines 115–116

`file_path(self, sample_data: dict) -> Path`
Purpose: Absolute path of a sample_data file.
Parameters: `sample_data`: dict
Returns: Path
lines 118–119

datasets/radar_loader.py

Module: `datasets.radar_loader` · 109 lines
Purpose: Radar loading: binary PCD parsing, filtering, multi-sweep accumulation.
Imports (project): utils.bev_grid, utils.logging_utils
Imports (third-party / stdlib): logging, numpy, pathlib
Imported by: datasets.nuscenes_dataset, scripts.validate_sensors

Constant	Value
RADAR_POINT_FEATURES	`('x', 'y', 'z', 'rcs', 'vx_comp', 'vy_comp', 'vx', 'vy', 'dyn_prop', 'time_lag')`
_PCD_TYPES	`{('F', 4): '<f4', ('F', 8): '<f8', ('I', 1): 'i1', ('I', 2): '<i2', ('I', 4): '<i4', ('U', 1): 'u1', ('U', 2): '<u2', ('U', 4): '<u4'}`

class RadarLoader

Load all radars of a sample (key-frame + previous sweeps) in the reference frame.
Instance attributes (set in __init__): `root` = Path(root); `grid` = grid; `num_sweeps` = num_sweeps; `invalid_states` = None if invalid_states is None else np.asarray(list(invalid_states)); `dynprop_states` = None if dynprop_states is None else np.asarray(list(dynprop_states)); `ambig_states` = None if ambig_states is None else np.asarray(list(ambig_states))

`__init__(self, root: Path, grid: BEVGrid, num_sweeps: int = 6, invalid_states: Optional[Iterable[int]] = (0,), dynprop_states: Optional[Iterable[int]] = range(7), ambig_states: Optional[Iterable[int]] = (3,)) -> None`
Purpose: Stores root, grid, sweep count and the three state filters (None disables a filter).
Parameters: `root`: Path; `grid`: BEVGrid; `num_sweeps`: int = 6; `invalid_states`: Optional[Iterable[int]] = (0,); `dynprop_states`: Optional[Iterable[int]] = range(7); `ambig_states`: Optional[Iterable[int]] = (3,)
Returns: None
Called by: datasets.nuscenes_dataset.NuScenesFusionDataset.__init__, scripts.validate_sensors.radar_box_agreement, scripts.validate_sensors.render_sample
lines 53–62

`_filter(self, pts: np.ndarray) -> np.ndarray`
Purpose: Keeps radar points whose invalid/dyn_prop/ambig states are allowed.
Parameters: `pts`: np.ndarray
Returns: np.ndarray
Called by: datasets.radar_loader.RadarLoader.load_sweep
lines 64–72

`load_sweep(self, rec: dict[str, Any]) -> Optional[np.ndarray]`
Purpose: Parses one radar sweep and returns (N, 10) reference-frame features (None on errors).
Parameters: `rec`: dict[str, Any]
Returns: Optional[np.ndarray]
Calls: datasets.radar_loader.RadarLoader._filter, datasets.radar_loader.read_radar_pcd
Called by: datasets.radar_loader.RadarLoader.load
lines 74–94

`load(self, info: dict[str, Any], num_sweeps: Optional[int] = None, filter_range: bool = True) -> np.ndarray`
Purpose: All radars × n sweeps, concatenated and range-filtered.
Parameters: `info`: dict[str, Any]; `num_sweeps`: Optional[int] = None; `filter_range`: bool = True
Returns: np.ndarray
Calls: datasets.radar_loader.RadarLoader.load_sweep
lines 96–109

Functions

`read_radar_pcd(path: Path) -> np.ndarray`
Purpose: Parse a nuScenes radar `.pcd` (`DATA binary`) into a NumPy structured array.
Parameters: `path`: Path
Returns: np.ndarray
Called by: datasets.radar_loader.RadarLoader.load_sweep
Raises: ValueError: f"Unsupported PCD (expected 'DATA binary'): {path}"; ValueError: f'Truncated radar file {path}'
lines 30–47

datasets/splits.py

Module: `datasets.splits` · 41 lines
Purpose: Official nuScenes scene splits.
Imports (project): –
Imports (third-party / stdlib): logging, nuscenes
Imported by: datasets.info_builder, inference.pipeline

Constant	Value
MINI_TRAIN	['scene-0061', 'scene-0553', 'scene-0655', 'scene-0757', 'scene-0796', 'scene-1077', 'scene-1094', 'scene-1100']
MINI_VAL	['scene-0103', 'scene-0916']

Functions

`get_split_scenes(split: str, available_scenes: list[str]) -> list[str]`
Purpose: Scene names of `split` that exist in the loaded version.
Details: Supported splits: `mini_train`, `mini_val`, `train`, `val`, `test` and `all`.
Parameters: `split`: str; `available_scenes`: list[str]
Returns: list[str]
Called by: datasets.info_builder.load_infos
Raises: ValueError: f"No scene of split '{split}' is present in this nuScenes version."; ImportError: f"Split '{split}' needs the official devkit: pip install nuscenes-devkit --no-deps"
lines 16–41

Package evaluation

evaluation/__init__.py

Module: evaluation · 1 lines
Purpose: Package marker.
Imports (project): –
Imports (third-party / stdlib): –
Imported by: –

evaluation/ablation.py

Module: evaluation.ablation · 241 lines · **entry point** (runnable script)
Purpose: Ablation study runner: train (optional) + evaluate every experiment with one recipe, then plot.
Imports (project): datasets.nuscenes_dataset, evaluation.evaluator, models.fusion_model, training.trainer, utils.checkpoint, utils.config, utils.device, utils.logging_utils
Imports (third-party / stdlib): argparse, csv, json, logging, matplotlib, matplotlib.pyplot, pathlib, sys, time, torch
Imported by: –

Constant	Value
PLOT_METRICS	[('NDS', 'NDS', True), ('mAP', 'mAP', True), ('mAP_present', 'mAP, present classes', True), ('mAVE', 'mAVE, m/s', False), ('ADE', 'Planning ADE, m', False), ...

Functions

checkpoint_for(cfg: Config, experiment: str, stage: str = 'joint', which: str = 'last') -> Optional[Path]
Purpose: which='last' (default): final weights after the full, identical training budget — no model selection on the evaluation split. which='best': the epoch with the best validation NDS (optimistic, because it is selected on the same split it is evaluated on).
Parameters: cfg: Config; experiment: str; stage: str = 'joint'; which: str = 'last'
Returns: Optional[Path]
Calls: utils.config.resolve_path
Called by: evaluation.ablation.main
lines 51–60

evaluate_checkpoint(path: Path, overrides: dict[str, Any] | None = None) -> dict[str, Any]
Purpose: Rebuilds the model from a checkpoint's embedded config, evaluates it on the validation split (incl. baseline planner) and adds peak memory, parameter count and epoch.
Parameters: path: Path; overrides: dict[str, Any] | None = None
Returns: dict[str, Any]
Calls: datasets.nuscenes_dataset.NuScenesFusionDataset, evaluation.evaluator.evaluate_model, models.fusion_model.build_model, utils.checkpoint.load_checkpoint, utils.config.Config, utils.device.amp_enabled, utils.device.get_device, utils.device.gpu_memory_report
Called by: evaluation.ablation.main
lines 63–81

flatten(name: str, label: str, res: dict[str, Any]) -> dict[str, Any]
Purpose: Flattens an evaluation result into one table row (detection, planning, baseline planner, latency, memory, FS ratios).
Parameters: name: str; label: str; res: dict[str, Any]
Returns: dict[str, Any]
Called by: evaluation.ablation.main
lines 84–101

plot(rows: list[dict[str, Any]], out_path: Path, proposed: str) -> None
Purpose: Small-multiple horizontal bar charts (one panel per metric, proposed model highlighted) saved as PNG.
Parameters: rows: list[dict[str, Any]]; out_path: Path; proposed: str
Returns: None
Called by: evaluation.ablation.write_reports
Side effects (file / disk I/O calls): savefig
lines 104–145

hypothesis_table(rows: list[dict[str, Any]]) -> list[str]
Purpose: Markdown table of metric differences for the hypothesis comparisons (E−C, E−D, F−I, F−E, C−A).
Parameters: rows: list[dict[str, Any]]
Returns: list[str]
Called by: evaluation.ablation.write_reports
lines 148–166

main() -> None
Purpose: CLI: optionally trains experiments (joint stage), evaluates last/best checkpoints, writes results.json/csv/md and the plot (or only redraws with --plot-only).
Returns: None
Calls: evaluation.ablation.checkpoint_for, evaluation.ablation.evaluate_checkpoint, evaluation.ablation.flatten, evaluation.ablation.write_reports, evaluation.evaluator.summary_line, training.trainer.Trainer, utils.config.load_config, utils.config.read_yaml, utils.config.resolve_path, utils.logging_utils.setup_logging

Side effects (file / disk I/O calls): mkdir, open, write_text
lines 169–218

`write_reports(rows: list[dict[str, Any]], out_dir: Path, cfg: Config, proposed: str, which: str) -> None`
Purpose: Writes results.md (table + hypothesis deltas) and the plot from result rows.
Parameters: `rows`: list[dict[str, Any]]; `out_dir`: Path; `cfg`: Config; `proposed`: str; `which`: str
Returns: None
Calls: evaluation.ablation.hypothesis_table, evaluation.ablation.plot
Called by: evaluation.ablation.main
Side effects (file / disk I/O calls): write_text
lines 221–237

evaluation/detection_metrics.py

Module: `evaluation.detection_metrics` · 205 lines
Purpose: nuScenes detection metrics (mAP, mATE, mASE, mAOE, mAVE, mAAE, NDS) — in-memory port.
Imports (project): utils.geometry
Imports (third-party / stdlib): collections, numpy
Imported by: app.streamlit_app, evaluation.evaluator, evaluation.official_eval, inference.pipeline, scripts.check_eval_vs_devkit, scripts.inference, tests.test_metrics_planning

Constant	Value
CLASS_RANGE	{'car': 50, 'truck': 50, 'bus': 50, 'trailer': 50, 'construction_vehicle': 50, 'pedestrian': 40, 'motorcycle': 40, 'bicycle': 40, 'traffic_cone': 30, 'barrie...
DIST_THS	(0.5, 1.0, 2.0, 4.0)
DIST_TH_TP	2.0
MAX_BOXES_PER_SAMPLE	500
TP_METRICS	('trans_err', 'scale_err', 'orient_err', 'vel_err', 'attr_err')
NELEM	101

class DetectionEvaluator

Accumulate predictions/GT sample by sample, then call :meth:`evaluate`.
Instance attributes (set in __init__): `classes` = list(classes); `gt` = defaultdict(list); `pred` = defaultdict(list); `samples` = []

`__init__(self, classes: Sequence[str]) -> None`
Purpose: Per-class GT and prediction lists, list of sample tokens.
Parameters: `classes`: Sequence[str]
Returns: None
Called by: evaluation.evaluator.evaluate_model, inference.pipeline.FusionPipeline._frame_metrics, scripts.check_eval_vs_devkit.main, scripts.inference.main, tests.test_metrics_planning.test_perfect_detections_give_full_ap_and_zero_errors, tests.test_metrics_planning.test_shifted_detection_counts_translation_error
lines 70–74

`add_sample(self, token: str, pred_boxes: np.ndarray, pred_scores: np.ndarray, pred_labels: np.ndarray, gt_boxes: np.ndarray, gt_labels: np.ndarray, gt_attributes: Optional[Sequence[str]] = None, gt_num_pts: Optional[np.ndarray] = None, gt_velocity_valid: Optional[np.ndarray] = None, pred_attributes: Optional[Sequence[str]] = None, bike_racks: Optional[np.ndarray] = None) -> None`
Purpose: Adds one sample's predictions and GT with the official filters (range, zero points, bike racks, top-500).
Parameters: `token`: str; `pred_boxes`: np.ndarray; `pred_scores`: np.ndarray; `pred_labels`: np.ndarray; `gt_boxes`: np.ndarray; `gt_labels`: np.ndarray; `gt_attributes`: Optional[Sequence[str]] = None; `gt_num_pts`: Optional[np.ndarray] = None; `gt_velocity_valid`: Optional[np.ndarray] = None; `pred_attributes`: Optional[Sequence[str]] = None; `bike_racks`: Optional[np.ndarray] = None
Returns: None
Calls: evaluation.detection_metrics.default_attribute, utils.geometry.points_in_boxes
lines 76–104

`_accumulate(self, name: str, dist_th: float) -> dict[str, np.ndarray]`
Purpose: Greedy centre-distance matching for one class and threshold; returns interpolated precision, confidence and TP-error curves.
Parameters: `name`: str; `dist_th`: float
Returns: dict[str, np.ndarray]
Calls: evaluation.detection_metrics._angle_diff, evaluation.detection_metrics._cummean, evaluation.detection_metrics._scale_iou
Called by: evaluation.detection_metrics.DetectionEvaluator.evaluate
lines 107–161

`_calc_ap(md: dict[str, np.ndarray]) -> float`
Purpose: Average precision above 10 % recall minus 10 % precision, normalised.
Parameters: `md`: dict[str, np.ndarray]
Returns: float
Called by: evaluation.detection_metrics.DetectionEvaluator.evaluate

Decorators: staticmethod
lines 164–168

`_calc_tp(md: dict[str, np.ndarray], metric: str) -> float`
Purpose: Mean TP error between 10 % recall and the maximum recall.
Parameters: `md`: dict[str, np.ndarray]; `metric`: str
Returns: float
Called by: evaluation.detection_metrics.DetectionEvaluator.evaluate
Decorators: staticmethod
lines 171–177

`evaluate(self) -> dict[str, object]`
Purpose: Per-class AP / TP errors, mAP, mean TP errors, NDS and mAP over present classes.
Returns: dict[str, object]
Calls: evaluation.detection_metrics.DetectionEvaluator._accumulate, evaluation.detection_metrics.DetectionEvaluator._calc_ap, evaluation.detection_metrics.DetectionEvaluator._calc_tp
lines 180–205

Functions

`default_attribute(name: str, velocity: np.ndarray) -> str`
Purpose: Attribute heuristic used by CenterPoint-style detectors (no attribute head).
Parameters: `name`: str; `velocity`: np.ndarray
Returns: str
Called by: evaluation.detection_metrics.DetectionEvaluator.add_sample, evaluation.official_eval.to_submission
lines 35–44

`_angle_diff(x: float, y: float, period: float) -> float`
Purpose: Absolute angle difference with a given period (nuScenes `angle_diff`).
Parameters: `x`: float; `y`: float; `period`: float
Returns: float
Called by: evaluation.detection_metrics.DetectionEvaluator._accumulate
lines 47–51

`_scale_iou(a: np.ndarray, b: np.ndarray) -> float`
Purpose: IoU of two aligned boxes from their sizes (nuScenes `scale_iou`).
Parameters: `a`: np.ndarray; `b`: np.ndarray
Returns: float
Called by: evaluation.detection_metrics.DetectionEvaluator._accumulate
lines 54–56

`_cummean(x: np.ndarray) -> np.ndarray`
Purpose: NaN-aware cumulative mean (all NaN → ones), as in the devkit.
Parameters: `x`: np.ndarray
Returns: np.ndarray
Called by: evaluation.detection_metrics.DetectionEvaluator._accumulate
lines 59–64

evaluation/evaluator.py

Module: `evaluation.evaluator` · 106 lines
Purpose: Run a trained model over a split and compute every metric family.
Imports (project): evaluation.detection_metrics, evaluation.trajectory_metrics, models.fusion_model, planning.trajectory_planner, utils.config, utils.device, utils.timer
Imports (third-party / stdlib): collections, logging, numpy, torch
Imported by: evaluation.ablation, scripts.evaluate, training.trainer

Functions

`evaluate_model(model: MultiSensorFusionNet, loader: DataLoader, cfg: Config, device: torch.device, use_amp: bool = False, compute_losses: bool = True, run_baseline_planner: bool = False, max_batches: Optional[int] = None) -> dict[str, Any]`
Purpose: Runs the model over a loader and returns detection, planning, feature-selection, loss and latency results.
Parameters: `model`: MultiSensorFusionNet; `loader`: DataLoader; `cfg`: Config; `device`: torch.device; `use_amp`: bool = False; `compute_losses`: bool = True; `run_baseline_planner`: bool = False; `max_batches`: Optional[int] = None
Returns: dict[str, Any]
Calls: evaluation.detection_metrics.DetectionEvaluator, evaluation.trajectory_metrics.aggregate, evaluation.trajectory_metrics.sample_planning_metrics, planning.trajectory_planner.BaselinePlanner, utils.device.move_to_device, utils.timer.StageTimer
Called by: evaluation.ablation.evaluate_checkpoint, scripts.evaluate.main, training.trainer.Trainer.fit
Decorators: torch.no_grad()
lines 31–94

`summary_line(res: dict[str, Any]) -> str`
Purpose: One-line text summary of an evaluation result.

Parameters: res: dict[str, Any]
Returns: str
Called by: evaluation.ablation.main, scripts.evaluate.main, training.trainer.Trainer.fit
lines 97–106

evaluation/official_eval.py

Module: evaluation.official_eval · 61 lines
Purpose: Export detections to the official nuScenes submission format and run the devkit evaluation.
Imports (project): evaluation.detection_metrics, utils.geometry
Imports (third-party / stdlib): json, numpy, nuscenes, pathlib
Imported by: scripts.check_eval_vs_devkit, scripts.evaluate, scripts.inference

Functions

box_to_global(box: np.ndarray, ref_ego2global: np.ndarray) -> dict[str, Any]
Purpose: Reference-frame box -> nuScenes global translation / size / rotation / velocity.
Parameters: box: np.ndarray; ref_ego2global: np.ndarray
Returns: dict[str, Any]
Calls: utils.geometry.rotmat_to_quat
Called by: evaluation.official_eval.to_submission, scripts.inference.main
lines 19–27

to_submission(results: dict[str, dict[str, np.ndarray]], infos_by_token: dict[str, dict], classes: list[str], modalities: dict[str, bool]) -> dict[str, Any]
Purpose: Converts reference-frame detections to the nuScenes results JSON (global frame, [w, l, h], quaternion, attribute) within class ranges, ≤ 500 per sample.
Parameters: results: dict[str, dict[str, np.ndarray]]; infos_by_token: dict[str, dict]; classes: list[str]; modalities: dict[str, bool]
Returns: dict[str, Any]
Calls: evaluation.detection_metrics.default_attribute, evaluation.official_eval.box_to_global
Called by: scripts.check_eval_vs_devkit.main, scripts.evaluate.main
lines 30–46

run_official_eval(submission: dict[str, Any], dataroot: str, version: str, eval_set: str, out_dir: Path) -> dict
Purpose: Writes the submission and runs the devkit DetectionEval (requires nuscenes-devkit); returns the serialized metrics.
Parameters: submission: dict[str, Any]; dataroot: str; version: str; eval_set: str; out_dir: Path
Returns: dict
Called by: scripts.check_eval_vs_devkit.main, scripts.evaluate.main
Side effects (file / disk I/O calls): mkdir, write_text
lines 49–61

evaluation/trajectory_metrics.py

Module: evaluation.trajectory_metrics · 115 lines
Purpose: Planning metrics against the logged (human) ego trajectory.
Imports (project): utils.geometry
Imports (third-party / stdlib): numpy
Imported by: app.streamlit_app, evaluation.evaluator, inference.pipeline, scripts.eval_planner, scripts.inference, tests.test_metrics_planning

Functions

_ego_box(xy: np.ndarray, yaw: float, ego_cfg: Any) -> np.ndarray
Purpose: Ego box [x, y, z, l, w, h, yaw] for a rear-axle pose (centre shifted by rear_axle_to_center).
Parameters: xy: np.ndarray; yaw: float; ego_cfg: Any
Returns: np.ndarray
Called by: evaluation.trajectory_metrics.collision_flags
lines 20–23

_polygons_intersect(a: np.ndarray, b: np.ndarray) -> bool
Purpose: Separating-axis test for two convex quadrilaterals (4, 2).
Parameters: a: np.ndarray; b: np.ndarray
Returns: bool
Called by: evaluation.trajectory_metrics.collision_flags
lines 26–35

trajectory_headings(xy: np.ndarray) -> np.ndarray
Purpose: Heading per step from the previous point (0 when nearly stationary).
Parameters: xy: np.ndarray
Returns: np.ndarray
Called by: evaluation.trajectory_metrics.collision_flags, evaluation.trajectory_metrics.sample_planning_metrics
lines 38–43

collision_flags(pred: np.ndarray, agent_future: list[dict], collision_labels: Iterable[int], ego_cfg: Any) -> np.ndarray

Purpose: Per-step collision flags `(T,)` against the GT boxes of other agents.
Parameters: `pred`: np.ndarray; `agent_future`: list[dict]; `collision_labels`: Iterable[int]; `ego_cfg`: Any
Returns: np.ndarray
Calls: evaluation.trajectory_metrics._ego_box, evaluation.trajectory_metrics._polygons_intersect, evaluation.trajectory_metrics.trajectory_headings, utils.geometry.box_corners_bev
Called by: evaluation.trajectory_metrics.sample_planning_metrics
lines 46–62

`sample_planning_metrics(pred: np.ndarray, gt: np.ndarray, mask: np.ndarray, dt: float, agent_future: list[dict] | None = None, collision_labels: Iterable[int] = (), ego_cfg: Any = None) -> dict[str, float]`
Purpose: Metrics of one sample (only valid future steps are used).
Parameters: `pred`: np.ndarray; `gt`: np.ndarray; `mask`: np.ndarray; `dt`: float; `agent_future`: list[dict] | None = None; `collision_labels`: Iterable[int] = (); `ego_cfg`: Any = None
Returns: dict[str, float]
Calls: evaluation.trajectory_metrics.collision_flags, evaluation.trajectory_metrics.trajectory_headings
Called by: evaluation.evaluator.evaluate_model, inference.pipeline.FusionPipeline._frame_metrics, scripts.eval_planner.main, tests.test_metrics_planning.test_trajectory_metrics
lines 65–103

`aggregate(metrics: list[dict[str, float]]) -> dict[str, float]`
Purpose: Mean of each metric over the samples that define it (collision -> collision rate).
Parameters: `metrics`: list[dict[str, float]]
Returns: dict[str, float]
Called by: evaluation.evaluator.evaluate_model, scripts.eval_planner.main, scripts.inference.main
lines 106–115

Package `inference`

`inference/__init__.py`

Module: `inference` · 1 lines
Purpose: Package marker.
Imports (project): –
Imports (third-party / stdlib): –
Imported by: –

`inference/pipeline.py`

Module: `inference.pipeline` · 268 lines
Purpose: Frame-by-frame inference pipeline shared by the dashboard and `scripts/inference.py`.
Imports (project): datasets.info_builder, datasets.map_prior, datasets.nuscenes_dataset, datasets.splits, evaluation.detection_metrics, evaluation.trajectory_metrics, models.fusion_model, planning.tracking, planning.trajectory_planner, utils.bev_grid, utils.checkpoint, utils.config, utils.device, utils.timer
Imports (third-party / stdlib): dataclasses, logging, numpy, nuscenes, pathlib, time, torch
Imported by: app.streamlit_app, app.ui.panels, scripts.inference, scripts.render_bev

Constant	Value
MACHINE_KEYS	('dataset.root', 'dataset.version', 'dataset.cache_dir', 'dataset.num_workers', 'runtime.device', 'runtime.output_dir', 'runtime.checkpoint_dir')

class `FrameResult`

Class attributes / fields: `index` = int; `info` = dict[str, Any]; `mode` = str; `model_trained` = bool; `lidar_points` = np.ndarray; `radar_points` = np.ndarray; `detections` = None; `plan` = None; `plan_source` = ''; `maps` = field(default_factory=dict); `channel_gates` = field(default_factory=dict); `selection_stats` = field(default_factory=dict); `frame_metrics` = field(default_factory=dict); `timings_ms` = field(default_factory=dict); `raw_outputs` = field(default_factory=dict)

`command_name(self) -> str`
Purpose: Text of the navigation command (RIGHT / LEFT / STRAIGHT).
Returns: str
Decorators: property
lines 62–63

class `FusionPipeline`

Loads the data (and optionally a trained checkpoint) and processes frames on demand.
Instance attributes (set in __init__): `checkpoint` = Path(checkpoint) if checkpoint else None; `cfg` = cfg; `device` = get_device(cfg.runtime.device); `use_amp` = amp_enabled(self.device, bool(cfg.train.amp)); `grid` = BEVGrid.from_config(cfg.bev); `classes` = list(cfg.classes); `infos` = load_infos(cfg, split); `dataset` = NuScenesFusionDataset(cfg, split, train=False, infos=self.infos, load_all_sensors=True, with_targ...; `model` = None; `model_trained` = False; `tracker` = MultiObjectTracker(self.classes); `_last_index` = None; `_planners` = {}; `map_prior` = MapPrior(Path(cfg.dataset.root), resolve_path(cfg.dataset.cache_dir)); `collision_labels` = [self.classes.index(c) for c in cfg.planning.collision_classes];

checkpoint_meta = {'epoch': ckpt.get('epoch'), 'metrics': ckpt.get('metrics', {}), 'experiment': ckpt.get('extra', ...

`__init__(self, base_cfg: Config, checkpoint: Optional[Path] = None, split: str = 'all', allow_untrained_model: bool = False) -> None`
Purpose: Loads config (from checkpoint if given, with local machine keys), infos, dataset, model, tracker, map prior.
Parameters: `base_cfg`: Config; `checkpoint`: Optional[Path] = None; `split`: str = 'all'; `allow_untrained_model`: bool = False
Returns: None
Calls: datasets.info_builder.load_infos, datasets.map_prior.MapPrior, datasets.nuscenes_dataset.NuScenesFusionDataset, models.fusion_model.build_model, planning.tracking.MultiObjectTracker, utils.bev_grid.BEVGrid.from_config, utils.checkpoint.load_checkpoint, utils.config.Config, utils.config.resolve_path, utils.device.amp_enabled, utils.device.get_device
Called by: app.streamlit_app.get_pipeline, scripts.inference.main, scripts.render_bev.main
lines 82–114

`scenes(self) -> list[dict[str, Any]]`
Purpose: Scenes of the loaded split with first index, frame count, location and split membership.
Returns: list[dict[str, Any]]
Calls: inference.pipeline.split_of_scene
lines 117–124

`planner(self, name: str)`
Purpose: Cached planner instance by name.
Parameters: `name`: str
Returns: None / not annotated
Calls: planning.trajectory_planner.build_planner
Called by: inference.pipeline.FusionPipeline.run
lines 126–129

`map_patch(self, info: dict[str, Any], x_range, y_range, ppm: float) -> Optional[np.ndarray]`
Purpose: Map-prior patch around the ego in display layout (None if unavailable).
Parameters: `info`: dict[str, Any]; `x_range`; `y_range`; `ppm`: float
Returns: Optional[np.ndarray]
lines 131–138

`run(self, index: int, mode: str = 'model', planner: str = 'baseline', plan_on_gt: bool = True, score_threshold: Optional[float] = None, stage_callback: Optional[Callable[[str, float], None]] = None) -> FrameResult`
Purpose: Processes one frame end-to-end in the requested mode and returns a `FrameResult`.
Parameters: `index`: int; `mode`: str = 'model'; `planner`: str = 'baseline'; `plan_on_gt`: bool = True; `score_threshold`: Optional[float] = None; `stage_callback`: Optional[Callable[[str, float], None]] = None
Returns: FrameResult
Calls: inference.pipeline.FrameResult, inference.pipeline.FusionPipeline._frame_metrics, inference.pipeline.FusionPipeline._run_model, inference.pipeline.FusionPipeline.planner
lines 141–199

`_run_model(self, res: FrameResult, item: dict[str, Any], timings: dict[str, float], score_threshold: Optional[float], stage_callback) -> None`
Purpose: Network forward + decoding and extraction of all maps/gates for the views.
Parameters: `res`: FrameResult; `item`: dict[str, Any]; `timings`: dict[str, float]; `score_threshold`: Optional[float]; `stage_callback`
Returns: None
Calls: datasets.nuscenes_dataset.collate_batch, utils.device.move_to_device, utils.timer.StageTimer
Called by: inference.pipeline.FusionPipeline.run
Decorators: torch.no_grad()
lines 202–241

`_frame_metrics(self, res: FrameResult) -> None`
Purpose: Single-frame detection and planning metrics.
Parameters: `res`: FrameResult
Returns: None
Calls: evaluation.detection_metrics.DetectionEvaluator, evaluation.trajectory_metrics.sample_planning_metrics
Called by: inference.pipeline.FusionPipeline.run
lines 243–259

`add_to_evaluator(self, evaluator: DetectionEvaluator, res: FrameResult) -> None`
Purpose: Accumulate a processed frame into a running (scene-level) evaluator.
Parameters: `evaluator`: DetectionEvaluator; `res`: FrameResult
Returns: None
lines 261–268

Functions

`split_of_scene(name: str) -> str`
Purpose: 'train' / 'val' / 'test' / 'unknown' for a scene name (mini lists or devkit lists).
Parameters: `name`: str
Returns: str
Called by: inference.pipeline.FusionPipeline.scenes
lines 66–76

Package (root)

main.py

Module: main · 59 lines · **entry point** (runnable script)
Purpose: Single entry point for the whole project.
Imports (project): –
Imports (third-party / stdlib): pathlib, runpy, subprocess, sys
Imported by: –

Constant	Value
ROOT	Path(__file__).resolve().parent
COMMANDS	{'preprocess': 'scripts/preprocess.py', 'validate': 'scripts/validate_sensors.py', 'smoke': 'scripts/smoke_test.py', 'train': 'training/cli.py', 'evaluate': ...

Functions

main() -> None
Purpose: Dispatches python main.py <command> ... to the scripts (runpy), Streamlit or pytest.
Returns: None
lines 40–55

Package models

models/__init__.py

Module: models · 1 lines
Purpose: Neural network modules (camera BEV, LiDAR/radar encoders, feature selection, fusion, heads).
Imports (project): –
Imports (third-party / stdlib): –
Imported by: –

models/camera_bev/__init__.py

Module: models.camera_bev · 1 lines
Purpose: Package marker.
Imports (project): –
Imports (third-party / stdlib): –
Imported by: –

models/camera_bev/image_backbone.py

Module: models.camera_bev.image_backbone · 74 lines
Purpose: Image backbone: torchvision ResNet + a light FPN neck at stride 16.
Imports (project): models.common.blocks
Imports (third-party / stdlib): logging, torch, torch.nn.functional, torchvision
Imported by: models.camera_bev.lss

Constant	Value
_WEIGHTS	{'resnet18': 'ResNet18_Weights', 'resnet34': 'ResNet34_Weights', 'resnet50': 'ResNet50_Weights'}

class ImageBackbone(nn.Module)

ResNet trunk (layer1-4) whose stride-32 output is upsampled and merged with stride 16. Input (BN, 3, H, W) -> output (BN, out_channels, H/16, W/16).
Instance attributes (set in __init__): stem = nn.Sequential(net.conv1, net.bn1, net.relu, net.maxpool); layer1 = (net.layer1, net.layer2, net.layer3, net.layer4); layer2 = (net.layer1, net.layer2, net.layer3, net.layer4); layer3 = (net.layer1, net.layer2, net.layer3, net.layer4); layer4 = (net.layer1, net.layer2, net.layer3, net.layer4); neck = nn.Sequential(ConvBNReLU(c3 + c4, out_channels, 1), ConvBNReLU(out_channels, out_channels, 3)); frozen_stages = frozen_stages; norm_eval = norm_eval; grad_checkpoint = False

__init__(self, name: str = 'resnet18', pretrained: bool = True, out_channels: int = 256, frozen_stages: int = 1, norm_eval: bool = True) -> None
Purpose: Builds the ResNet trunk (ImageNet weights if available), the stride-16 neck, and applies freezing.
Parameters: name: str = 'resnet18'; pretrained: bool = True; out_channels: int = 256; frozen_stages: int = 1; norm_eval: bool = True
Returns: None
Calls: models.camera_bev.image_backbone.ImageBackbone._freeze, models.common.blocks.ConvBNReLU
Called by: models.camera_bev.lss.CameraBEVEncoder.__init__
lines 24–45

`_freeze(self) -> None`
Purpose: Freezes stem + the first `frozen_stages` layers (eval mode, no gradients).
Returns: None
Called by: models.camera_bev.image_backbone.ImageBackbone.__init__, models.camera_bev.image_backbone.ImageBackbone.train
lines 47–52

`train(self, mode: bool = True) -> 'ImageBackbone'`
Purpose: train() override that keeps frozen stages frozen and BatchNorm in eval mode when `norm_eval`.
Parameters: `mode`: bool = True
Returns: 'ImageBackbone'
Calls: models.camera_bev.image_backbone.ImageBackbone._freeze, models.camera_bev.image_backbone.ImageBackbone.modules
lines 54–61

`_run(self, layer: nn.Module, x: torch.Tensor) -> torch.Tensor`
Purpose: Runs a stage normally or through gradient checkpointing.
Parameters: `layer`: nn.Module; `x`: torch.Tensor
Returns: torch.Tensor
Called by: models.camera_bev.image_backbone.ImageBackbone.forward
lines 63–66

`forward(self, x: torch.Tensor) -> torch.Tensor`
Purpose: Images (B·N, 3, H, W) → features (B·N, 256, H/16, W/16).
Parameters: `x`: torch.Tensor
Returns: torch.Tensor
Calls: models.camera_bev.image_backbone.ImageBackbone._run, models.camera_bev.image_backbone.ImageBackbone.layer1, models.camera_bev.image_backbone.ImageBackbone.neck, models.camera_bev.image_backbone.ImageBackbone.stem
lines 68–74

models/camera_bev/lss.py

Module: `models.camera_bev.lss` · 170 lines
Purpose: Camera -> BEV: Lift-Splat-Shoot with a BEVDepth-style camera-aware depth network.
Imports (project): models.camera_bev.image_backbone, models.common.blocks, utils.bev_grid
Imports (third-party / stdlib): math, torch, torch.nn.functional
Imported by: models.fusion_model, tests.test_dataset

class `DepthNet`(nn.Module)

Predicts a categorical depth distribution and context features per image cell. `camera_aware`: an MLP over intrinsics/extrinsics produces SE-style gates for the depth and context branches (BEVDepth), so one network serves all six cameras.
Instance attributes (set in __init__): `reduce` = ConvBNReLU(in_channels, mid_channels, 3); `camera_aware` = camera_aware; `context` = nn.Sequential(ResBlock(mid_channels), nn.Conv2d(mid_channels, context_channels, 1)); `depth` = nn.Sequential(ResBlock(mid_channels), ResBlock(mid_channels, dilation=2), nn.Conv2d(mid_channels,...; `cam_bn` = nn.BatchNorm1d(cam_param_dim); `cam_mlp` = nn.Sequential(nn.Linear(cam_param_dim, mid_channels), nn.ReLU(inplace=True), nn.Linear(mid_channe...

`__init__(self, in_channels: int, mid_channels: int, depth_bins: int, context_channels: int, camera_aware: bool = True, cam_param_dim: int = 16) -> None`
Purpose: Builds reduce conv, camera-aware gate MLP, context and depth branches.
Parameters: `in_channels`: int; `mid_channels`: int; `depth_bins`: int; `context_channels`: int; `camera_aware`: bool = True; `cam_param_dim`: int = 16
Returns: None
Calls: models.common.blocks.ConvBNReLU, models.common.blocks.ResBlock
Called by: models.camera_bev.lss.CameraBEVEncoder.__init__
lines 42–52

`forward(self, x: torch.Tensor, cam_params: Optional[torch.Tensor]) -> tuple[torch.Tensor, torch.Tensor]`
Purpose: Returns depth logits (B·N, D, h, w) and context (B·N, C, h, w), gated by the camera descriptor.
Parameters: `x`: torch.Tensor; `cam_params`: Optional[torch.Tensor]
Returns: tuple[torch.Tensor, torch.Tensor]
Calls: models.camera_bev.lss.DepthNet.cam_bn, models.camera_bev.lss.DepthNet.cam_mlp, models.camera_bev.lss.DepthNet.context, models.camera_bev.lss.DepthNet.depth, models.camera_bev.lss.DepthNet.reduce
lines 54–62

class `CameraBEVEncoder`(nn.Module)

Lift-Splat camera BEV encoder producing `(B, C, ny, nx)` on the shared grid.
Instance attributes (set in __init__): `grid` = grid; `image_size` = (int(image_size[0]), int(image_size[1])); `stride` = int(cfg.feature_stride); `dbound` = (dmin, dmax, dstep); `num_bins` = int(round((dmax - dmin) / dstep)); `out_channels` = int(cfg.out_channels); `backbone` = ImageBackbone(cfg.backbone, bool(cfg.pretrained), int(cfg.neck_channels), int(cfg.frozen_stages),...; `depth_net` = DepthNet(int(cfg.neck_channels), int(cfg.neck_channels), self.num_bins, self.out_channels, bool(c...; `bev_refine` = nn.Sequential(ConvBNReLU(self.out_channels, self.out_channels, 3), *[ResBlock(self.out_channels) ...

`__init__(self, cfg: Any, grid: BEVGrid, image_size: tuple[int, int]) -> None`
Purpose: Builds backbone, depth net, BEV refinement and the frustum buffer from the config.
Parameters: `cfg`: Any; `grid`: BEVGrid; `image_size`: tuple[int, int]
Returns: None
Calls: models.camera_bev.image_backbone.ImageBackbone, models.camera_bev.lss.CameraBEVEncoder._create_frustum, models.camera_bev.lss.CameraBEVEncoder.register_buffer, models.camera_bev.lss.DepthNet, models.common.blocks.ConvBNReLU, models.common.blocks.ResBlock
Called by: models.fusion_model.MultiSensorFusionNet.__init__, tests.test_dataset.test_lss_geometry_matches_projection
lines 68–83

`_create_frustum(self) -> torch.Tensor`
Purpose: Frustum (D, h, w, 3) of (u, v, depth) at feature-cell centres and depth-bin centres.
Returns: torch.Tensor
Called by: models.camera_bev.lss.CameraBEVEncoder.__init__
lines 86–96

`get_geometry(self, intrinsics: torch.Tensor, post_rots: torch.Tensor, post_trans: torch.Tensor, cam2ref: torch.Tensor, bda: torch.Tensor) -> torch.Tensor`
Purpose: Reference-frame location of every frustum point: `(B, N, D, h, w, 3)`.
Parameters: `intrinsics`: torch.Tensor; `post_rots`: torch.Tensor; `post_trans`: torch.Tensor; `cam2ref`: torch.Tensor; `bda`: torch.Tensor
Returns: torch.Tensor
Called by: models.camera_bev.lss.CameraBEVEncoder.forward
lines 98–108

`voxel_pool(self, geom: torch.Tensor, feats: torch.Tensor) -> torch.Tensor`
Purpose: Sum-pool lifted features `(B, N, D, h, w, C)` into BEV cells `(B, C, ny, nx)` (z collapsed).
Parameters: `geom`: torch.Tensor; `feats`: torch.Tensor
Returns: torch.Tensor
Called by: models.camera_bev.lss.CameraBEVEncoder.forward
lines 110–124

`camera_params(intrinsics: torch.Tensor, post_rots: torch.Tensor, cam2ref: torch.Tensor, image_size: tuple[int, int]) -> torch.Tensor`
Purpose: 16-d camera descriptor: augmented focal lengths / principal point (normalised) + extrinsics.
Parameters: `intrinsics`: torch.Tensor; `post_rots`: torch.Tensor; `cam2ref`: torch.Tensor; `image_size`: tuple[int, int]
Returns: torch.Tensor
Called by: models.camera_bev.lss.CameraBEVEncoder.forward
Decorators: staticmethod
lines 127–134

`forward(self, images: torch.Tensor, intrinsics: torch.Tensor, post_rots: torch.Tensor, post_trans: torch.Tensor, cam2ref: torch.Tensor, bda: torch.Tensor, cam_valid: Optional[torch.Tensor] = None) -> dict[str, torch.Tensor]`
Purpose: Images + calibration → camera BEV (B, C, ny, nx), depth logits and probabilities.
Parameters: `images`: torch.Tensor; `intrinsics`: torch.Tensor; `post_rots`: torch.Tensor; `post_trans`: torch.Tensor; `cam2ref`: torch.Tensor; `bda`: torch.Tensor; `cam_valid`: Optional[torch.Tensor] = None
Returns: dict[str, torch.Tensor]
Calls: models.camera_bev.lss.CameraBEVEncoder.backbone, models.camera_bev.lss.CameraBEVEncoder.bev_refine, models.camera_bev.lss.CameraBEVEncoder.camera_params, models.camera_bev.lss.CameraBEVEncoder.depth_net, models.camera_bev.lss.CameraBEVEncoder.get_geometry, models.camera_bev.lss.CameraBEVEncoder.voxel_pool
lines 137–151

`depth_loss(self, depth_logits: torch.Tensor, depth_gt: torch.Tensor) -> torch.Tensor`
Purpose: BEVDepth loss: BCE between the predicted distribution and the one-hot LiDAR depth bin.
Parameters: `depth_logits`: torch.Tensor; `depth_gt`: torch.Tensor
Returns: torch.Tensor
lines 153–165

Functions

`depth_bin_centers(dbound: tuple[float, float, float]) -> list[float]`
Purpose: List of depth-bin centres for (min, max, step).
Parameters: `dbound`: tuple[float, float, float]
Returns: list[float]
lines 168–170

models/common/__init__.py

Module: `models.common` · 1 lines
Purpose: Package marker.
Imports (project): –
Imports (third-party / stdlib): –
Imported by: –

models/common/blocks.py

Module: `models.common.blocks` · 71 lines
Purpose: Reusable convolutional / attention building blocks.
Imports (project): –
Imports (third-party / stdlib): torch
Imported by: models.camera_bev.image_backbone, models.camera_bev.lss, models.detection.center_head, models.feature_selection.selection, models.fusion.bev_backbone, models.fusion.fusion_methods, models.lidar_encoder.pillar_encoder, models.radar_encoder.radar_encoder, models.trajectory.planning_head

class ConvBNReLU(nn.Sequential)

`__init__(self, in_ch: int, out_ch: int, kernel: int = 3, stride: int = 1, dilation: int = 1, relu: bool = True) -> None`
Purpose: Conv2d (same padding, no bias) + BatchNorm2d + optional ReLU.
Parameters: `in_ch`: int; `out_ch`: int; `kernel`: int = 3; `stride`: int = 1; `dilation`: int = 1; `relu`: bool = True
Returns: None
Called by: models.camera_bev.image_backbone.ImageBackbone.__init__, models.camera_bev.lss.CameraBEVEncoder.__init__, models.camera_bev.lss.DepthNet.__init__, models.common.blocks.ResBlock.__init__, models.detection.center_head.CenterHead.__init__, models.feature_selection.selection.CrossModalFeatureSelection.__init__, models.fusion.bev_backbone.SECONDFPN.__init__, models.fusion.fusion_methods.AttentionGatedFusion.__init__, models.fusion.fusion_methods.ConcatFusion.__init__, models.lidar_encoder.pillar_encoder.LidarEncoder.__init__, models.radar_encoder.radar_encoder.RadarEncoder.__init__, models.trajectory.planning_head.PlanningHead.__init__
lines 9–16

class ResBlock(nn.Module)

Basic residual block (two 3x3 convs) with optional dilation.
Instance attributes (set in __init__): `conv1` = ConvBNReLU(channels, channels, 3, dilation=dilation); `conv2` = ConvBNReLU(channels, channels, 3, dilation=dilation, relu=False); `act` = nn.ReLU(inplace=True)

`__init__(self, channels: int, dilation: int = 1) -> None`
Purpose: Two 3×3 ConvBNReLU (second without ReLU), optional dilation.
Parameters: `channels`: int; `dilation`: int = 1
Returns: None
Calls: models.common.blocks.ConvBNReLU
Called by: models.camera_bev.lss.CameraBEVEncoder.__init__, models.camera_bev.lss.DepthNet.__init__, models.fusion.fusion_methods.AttentionGatedFusion.__init__, models.fusion.fusion_methods.ConcatFusion.__init__, models.lidar_encoder.pillar_encoder.LidarEncoder.__init__, models.radar_encoder.radar_encoder.RadarEncoder.__init__
lines 22–26

`forward(self, x: torch.Tensor) -> torch.Tensor`
Purpose: ReLU(x + conv2(conv1(x))).
Parameters: x: torch.Tensor
Returns: torch.Tensor
Calls: models.common.blocks.ResBlock.act, models.common.blocks.ResBlock.conv1, models.common.blocks.ResBlock.conv2
lines 28–29

class SqueezeExcitation(nn.Module)

Channel attention (SE): global pooling -> bottleneck MLP -> sigmoid channel weights.
Instance attributes (set in __init__): `mlp` = nn.Sequential(nn.Linear(2 * channels, hidden), nn.ReLU(inplace=True), nn.Linear(hidden, channels))

`__init__(self, channels: int, reduction: int = 4) -> None`
Purpose: Bottleneck MLP 2C → max(C/r, 8) → C.
Parameters: `channels`: int; `reduction`: int = 4
Returns: None
Called by: models.common.blocks.CBAM.__init__
lines 35–38

`weights(self, x: torch.Tensor) -> torch.Tensor`
Purpose: Per-channel sigmoid weights (B, C) from mean+max pooling.
Parameters: x: torch.Tensor
Returns: torch.Tensor
Calls: models.common.blocks.SqueezeExcitation.mlp
Called by: models.common.blocks.SqueezeExcitation.forward
lines 40–42

`forward(self, x: torch.Tensor) -> torch.Tensor`
Purpose: x · channel weights.
Parameters: x: torch.Tensor
Returns: torch.Tensor
Calls: models.common.blocks.SqueezeExcitation.weights
lines 44–45

class SpatialAttention(nn.Module)

CBAM spatial attention: [mean_c, max_c] -> 7x7 conv -> sigmoid map.
Instance attributes (set in __init__): conv = nn.Conv2d(2, 1, kernel, padding=kernel // 2)

__init__(self, kernel: int = 7) -> None
Purpose: One 2→1 channel convolution (kernel 7).
Parameters: kernel: int = 7
Returns: None
Called by: models.common.blocks.CBAM.__init__
lines 51–53

weights(self, x: torch.Tensor) -> torch.Tensor
Purpose: Per-pixel sigmoid weights (B, 1, H, W) from channel mean and max.
Parameters: x: torch.Tensor
Returns: torch.Tensor
Calls: models.common.blocks.SpatialAttention.conv
Called by: models.common.blocks.SpatialAttention.forward
lines 55–56

forward(self, x: torch.Tensor) -> torch.Tensor
Purpose: x · spatial weights.
Parameters: x: torch.Tensor
Returns: torch.Tensor
Calls: models.common.blocks.SpatialAttention.weights
lines 58–59

class CBAM(nn.Module)

Convolutional Block Attention Module (channel attention followed by spatial attention).
Instance attributes (set in __init__): channel = SqueezeExcitation(channels, reduction); spatial = SpatialAttention(kernel)

__init__(self, channels: int, reduction: int = 4, kernel: int = 7) -> None
Purpose: Channel attention followed by spatial attention.
Parameters: channels: int; reduction: int = 4; kernel: int = 7
Returns: None
Calls: models.common.blocks.SpatialAttention, models.common.blocks.SqueezeExcitation
Called by: models.fusion.fusion_methods.AttentionGatedFusion.__init__
lines 65–68

forward(self, x: torch.Tensor) -> torch.Tensor
Purpose: spatial(channel(x)).
Parameters: x: torch.Tensor
Returns: torch.Tensor
Calls: models.common.blocks.CBAM.channel, models.common.blocks.CBAM.spatial
lines 70–71

models/common/pillars.py

Module: models.common.pillars · 122 lines
Purpose: Dynamic pillar feature encoding (pure PyTorch, no custom CUDA ops).
Imports (project): utils.bev_grid
Imports (third-party / stdlib): torch
Imported by: models.lidar_encoder.pillar_encoder, models.radar_encoder.radar_encoder

class PillarIndexer

Computes pillar membership of points on a BEV grid.
Instance attributes (set in __init__): grid = grid

__init__(self, grid: BEVGrid) -> None
Purpose: Stores the grid.
Parameters: grid: BEVGrid
Returns: None
Called by: models.common.pillars.DynamicPillarVFE.__init__, models.common.pillars.bev_raster_stats, models.lidar_encoder.pillar_encoder.LidarEncoder.raster
lines 33–34

__call__(self, points: torch.Tensor, batch_size: int) -> dict[str, torch.Tensor]
Purpose: Pillar indices of batched points: filtered points, ix, iy, unique flat pillar ids and the inverse index.
Parameters: points: torch.Tensor; batch_size: int
Returns: dict[str, torch.Tensor]
lines 36–45

class DynamicPillarVFE(nn.Module)

PointNet over pillars: per-point MLP layers with scatter-max aggregation. Point decoration (in_features + 5 dims): raw features, offset to the pillar's point mean (3), offset to the pillar centre in x/y (2).

Instance attributes (set in __init__): `grid` = grid; `indexer` = PillarIndexer(grid); `layers` = nn.ModuleList(); `out_channels` = channels[-1]

`__init__(self, in_features: int, channels: tuple[int, ...], grid: BEVGrid) -> None`
Purpose: Builds the PointNet layers (Linear-BN-ReLU), halving non-final widths for the max-feature concatenation.
Parameters: `in_features`: int; `channels`: tuple[int, ...]; `grid`: BEVGrid
Returns: None
Calls: models.common.pillars.PillarIndexer
Called by: models.lidar_encoder.pillar_encoder.LidarEncoder.__init__, models.radar_encoder.radar_encoder.RadarEncoder.__init__
lines 55–67

`forward(self, points: torch.Tensor, batch_size: int, normalizer: torch.Tensor | None = None) -> torch.Tensor`
Purpose: Return the BEV canvas `(B, C, ny, nx)`. `normalizer` rescales raw feature columns.
Parameters: `points`: torch.Tensor; `batch_size`: int; `normalizer`: torch.Tensor | None = None
Returns: torch.Tensor
Calls: models.common.pillars.DynamicPillarVFE.indexer, models.common.pillars.scatter_max, models.common.pillars.scatter_mean
lines 69–95

Functions

`scatter_mean(src: torch.Tensor, index: torch.Tensor, size: int) -> torch.Tensor`
Purpose: Per-group mean of rows of `src` grouped by `index` (index_add_ sums / counts).
Parameters: `src`: torch.Tensor; `index`: torch.Tensor; `size`: int
Returns: torch.Tensor
Called by: models.common.pillars.DynamicPillarVFE.forward, models.common.pillars.bev_raster_stats
lines 18–21

`scatter_max(src: torch.Tensor, index: torch.Tensor, size: int) -> torch.Tensor`
Purpose: Per-group max for non-negative `src` (inputs are post-ReLU).
Parameters: `src`: torch.Tensor; `index`: torch.Tensor; `size`: int
Returns: torch.Tensor
Called by: models.common.pillars.DynamicPillarVFE.forward
lines 24–27

`bev_raster_stats(points: torch.Tensor, batch_size: int, grid: BEVGrid, value_cols: list[int], count_scale: float = 3.0) -> torch.Tensor`
Purpose: Hand-crafted BEV raster: `log(1+count)/count_scale` plus the mean of selected point columns.
Details: Returns `(B, 1 + len(value_cols), ny, nx)`. These explicit channels keep physically meaningful quantities (density, height, intensity, radar velocity) visible to the feature-selection gates, which makes the learned importances interpretable.
Parameters: `points`: torch.Tensor; `batch_size`: int; `grid`: BEVGrid; `value_cols`: list[int]; `count_scale`: float = 3.0
Returns: torch.Tensor
Calls: models.common.pillars.PillarIndexer, models.common.pillars.scatter_mean
Called by: models.lidar_encoder.pillar_encoder.LidarEncoder.raster, models.radar_encoder.radar_encoder.RadarEncoder.raster
lines 98–122

models/detection/__init__.py

Module: `models.detection` · 1 lines
Purpose: Package marker.
Imports (project): –
Imports (third-party / stdlib): –
Imported by: –

models/detection/center_head.py

Module: `models.detection.center_head` · 93 lines
Purpose: CenterPoint-style 3D detection head on the BEV feature map.
Imports (project): models.common.blocks
Imports (third-party / stdlib): torch, torch.nn.functional
Imported by: models.fusion_model

Constant	Value
HEAD_BRANCHES	{'reg': 2, 'height': 1, 'dim': 3, 'rot': 2, 'vel': 2}

class CenterHead(nn.Module)

Instance attributes (set in __init__): `num_classes` = num_classes; `shared` = ConvBNReLU(in_channels, head_channels, 3); `heads` = nn.ModuleDict()

`__init__(self, in_channels: int, num_classes: int, head_channels: int = 64) -> None`
Purpose: Shared conv + one branch per output; heatmap bias −2.19.
Parameters: `in_channels`: int; `num_classes`: int; `head_channels`: int = 64
Returns: None
Calls: models.common.blocks.ConvBNReLU

Called by: models.fusion_model.MultiSensorFusionNet.__init__
lines 33–40

`forward(self, x: torch.Tensor) -> dict[str, torch.Tensor]`
Purpose: Dict of head maps (B, ch, H, W).
Parameters: x: torch.Tensor
Returns: dict[str, torch.Tensor]
Calls: models.detection.center_head.CenterHead.shared
lines 42–44

Functions

`gaussian_focal_loss(pred: torch.Tensor, target: torch.Tensor, alpha: float = 2.0, gamma: float = 4.0) -> tuple[torch.Tensor, torch.Tensor]`
Purpose: CornerNet focal loss on probabilities; returns (summed loss, number of positives).
Parameters: `pred`: torch.Tensor; `target`: torch.Tensor; `alpha`: float = 2.0; `gamma`: float = 4.0
Returns: tuple[torch.Tensor, torch.Tensor]
Called by: models.detection.center_head.aux_heatmap_loss, models.detection.center_head.detection_loss
lines 47–55

`_gather(feat: torch.Tensor, ind: torch.Tensor) -> torch.Tensor`
Purpose: `feat (B, C, H, W)`, `ind (B, K)` flat indices -> `(B, K, C)`.
Parameters: `feat`: torch.Tensor; `ind`: torch.Tensor
Returns: torch.Tensor
Called by: models.detection.center_head.detection_loss
lines 58–62

`detection_loss(preds: dict[str, torch.Tensor], batch: dict[str, Any], weights: Any, code_weights: list[float]) -> dict[str, torch.Tensor]`
Purpose: CenterPoint losses. `weights` holds heatmap/center/bbox/orientation/velocity factors.
Parameters: `preds`: dict[str, torch.Tensor]; `batch`: dict[str, Any]; `weights`: Any; `code_weights`: list[float]
Returns: dict[str, torch.Tensor]
Calls: models.detection.center_head._gather, models.detection.center_head.gaussian_focal_loss
Called by: models.fusion_model.MultiSensorFusionNet.compute_losses
lines 65–85

`aux_heatmap_loss(logits: torch.Tensor, target: torch.Tensor) -> torch.Tensor`
Purpose: Class-agnostic deep-supervision heatmap loss (resizes the target if needed).
Parameters: `logits`: torch.Tensor; `target`: torch.Tensor
Returns: torch.Tensor
Calls: models.detection.center_head.gaussian_focal_loss
Called by: models.fusion_model.MultiSensorFusionNet.compute_losses
lines 88–93

models/detection/decoder.py

Module: `models.detection.decoder` · 69 lines
Purpose: Decode CenterHead outputs into 3D boxes `[x, y, z, l, w, h, yaw, vx, vy]` + score + label.
Imports (project): utils.bev_grid
Imports (third-party / stdlib): numpy, torch, torch.nn.functional
Imported by: models.fusion_model

Functions

`circle_nms(centers: torch.Tensor, scores: torch.Tensor, radius: float) -> torch.Tensor`
Purpose: Greedy distance-based NMS (CenterPoint). Returns kept indices (sorted by score).
Parameters: `centers`: torch.Tensor; `scores`: torch.Tensor; `radius`: float
Returns: torch.Tensor
Called by: models.detection.decoder.decode_detections
lines 13–27

`decode_detections(preds: dict[str, torch.Tensor], grid: BEVGrid, classes: list[str], score_threshold: float = 0.1, max_detections: int = 300, nms_radius: Optional[dict[str, float]] = None) -> list[dict[str, torch.Tensor]]`
Purpose: Per-sample detections: `{"boxes": (K, 9), "scores": (K,), "labels": (K,)}` in the reference frame.
Parameters: `preds`: dict[str, torch.Tensor]; `grid`: BEVGrid; `classes`: list[str]; `score_threshold`: float = 0.1; `max_detections`: int = 300; `nms_radius`: Optional[dict[str, float]] = None
Returns: list[dict[str, torch.Tensor]]
Calls: models.detection.decoder.circle_nms
Called by: models.fusion_model.MultiSensorFusionNet.decode
Decorators: torch.no_grad()
lines 31–69

models/detection/targets.py

Module: `models.detection.targets` · 95 lines
Purpose: CenterPoint training targets (NumPy, generated inside the dataset workers).
Imports (project): utils.bev_grid
Imports (third-party / stdlib): numpy
Imported by: datasets.nuscenes_dataset

Functions

`gaussian_radius(height: float, width: float, min_overlap: float = 0.1) -> float`
Purpose: CornerNet/CenterPoint Gaussian radius such that a shifted box keeps `min_overlap` IoU.
Parameters: `height`: float; `width`: float; `min_overlap`: float = 0.1
Returns: float
Called by: models.detection.targets.build_center_targets, models.detection.targets.foreground_heatmap
lines 14–25

`draw_gaussian(heatmap: np.ndarray, center: tuple[int, int], radius: int) -> None`
Purpose: Draw (max-merge) a Gaussian of `radius` cells at `center = (ix, iy)` into `heatmap[iy, ix]`.
Parameters: `heatmap`: np.ndarray; `center`: tuple[int, int]; `radius`: int
Returns: None
Called by: models.detection.targets.build_center_targets, models.detection.targets.foreground_heatmap
lines 28–42

`build_center_targets(boxes: np.ndarray, labels: np.ndarray, velocity_valid: np.ndarray, grid: BEVGrid, num_classes: int, max_objects: int = 500, min_radius: int = 2, gaussian_overlap: float = 0.1) -> dict[str, np.ndarray]`
Purpose: Targets on the detection-head grid.
Details: Args: boxes: `(M, 9)` reference-frame boxes `[x, y, z, l, w, h, yaw, vx, vy]`. labels: `(M,)` class indices. velocity_valid: `(M,)` whether the GT velocity is defined (single-frame tracks are not). grid: the head grid (BEV grid downsampled by the head stride).
Parameters: `boxes`: np.ndarray; `labels`: np.ndarray; `velocity_valid`: np.ndarray; `grid`: BEVGrid; `num_classes`: int; `max_objects`: int = 500; `min_radius`: int = 2; `gaussian_overlap`: float = 0.1
Returns: dict[str, np.ndarray]
Calls: models.detection.targets.draw_gaussian, models.detection.targets.gaussian_radius
Called by: datasets.nuscenes_dataset.NuScenesFusionDataset.__getitem__
lines 45–81

`foreground_heatmap(boxes: np.ndarray, grid: BEVGrid, min_radius: int = 2, gaussian_overlap: float = 0.1) -> np.ndarray`
Purpose: Class-agnostic centre heatmap `(1, ny, nx)` at BEV resolution (feature-selection guidance).
Parameters: `boxes`: np.ndarray; `grid`: BEVGrid; `min_radius`: int = 2; `gaussian_overlap`: float = 0.1
Returns: np.ndarray
Calls: models.detection.targets.draw_gaussian, models.detection.targets.gaussian_radius
Called by: datasets.nuscenes_dataset.NuScenesFusionDataset.__getitem__
lines 84–95

models/feature_selection/__init__.py

Module: `models.feature_selection` · 1 lines
Purpose: Package marker.
Imports (project): –
Imports (third-party / stdlib): –
Imported by: –

models/feature_selection/losses.py

Module: `models.feature_selection.losses` · 39 lines
Purpose: Feature-selection losses.
Imports (project): –
Imports (third-party / stdlib): torch
Imported by: models.fusion_model

Functions

`feature_selection_loss(info: dict[str, Any], fg_heatmap: torch.Tensor, cfg_weights: Any, target_keep: Any) -> dict[str, torch.Tensor]`
Purpose: Coverage, sparsity and binarisation losses of the gates and their weighted sum.
Parameters: `info`: dict[str, Any]; `fg_heatmap`: torch.Tensor; `cfg_weights`: Any; `target_keep`: Any
Returns: dict[str, torch.Tensor]
Called by: models.fusion_model.MultiSensorFusionNet.compute_losses
lines 21–39

models/feature_selection/selection.py

Module: `models.feature_selection.selection` · 142 lines
Purpose: Camera-Guided Cross-Modal Feature Selection (CG-CMFS) — the research component.
Imports (project): models.common.blocks
Imports (third-party / stdlib): torch
Imported by: models.fusion_model, tests.test_models

class `CrossModalFeatureSelection`(nn.Module)

Learned channel + spatial feature selection for LiDAR / radar BEV features. Args: channels: channels per selectable modality, e.g. `{"lidar": 64, "radar": 64}`. guide_channels: channels of the camera BEV guide (`None` = no guide). cfg: `fusion.selection` config.
Instance attributes (set in __init__): `modalities` = list(channels); `channels` = dict(channels); `has_guide` = guide_channels is not None; `gate_mode` = str(cfg.gate_mode); `temperature` = float(cfg.temperature); `threshold` = float(cfg.threshold); `channel_mlp` = nn.Sequential(nn.Linear(stat_dim, hidden), nn.ReLU(inplace=True), nn.Linear(hidden, sum(channels....; `compress` = nn.ModuleDict({m: ConvBNReLU(c, compress, 1) for m, c in all_channels.items()}); `spatial` = nn.Sequential(ConvBNReLU((compress + 2) * len(all_channels), sp_hidden, k), ConvBNReLU(sp_hidden,...

`__init__(self, channels: dict[str, int], guide_channels: Optional[int], cfg: Any) -> None`
Purpose: Builds the cross-modal channel MLP, the per-input compression convs and the spatial gate network; initialises gates open (bias init_open).
Parameters: `channels`: dict[str, int]; `guide_channels`: Optional[int]; `cfg`: Any
Returns: None
Calls: models.common.blocks.ConvBNReLU
Called by: models.fusion_model.MultiSensorFusionNet.__init__, tests.test_models.test_feature_selection_gates_are_bounded_and_hard_mode_is_binary
lines 56–85

`_gate(self, logits: torch.Tensor) -> torch.Tensor`
Purpose: Logits → gates (soft sigmoid or hard-concrete straight-through); ValueError for unknown modes.
Parameters: `logits`: torch.Tensor
Returns: torch.Tensor
Called by: models.feature_selection.selection.CrossModalFeatureSelection.forward
Raises: ValueError: f'Unknown gate_mode {self.gate_mode!r}'
lines 88–98

`forward(self, feats: dict[str, torch.Tensor], guide: Optional[torch.Tensor] = None) -> tuple[dict[str, torch.Tensor], dict[str, Any]]`
Purpose: Computes channel and spatial gates from all inputs and returns the selected features plus gate info (ValueError if the camera guide is missing).
Parameters: `feats`: dict[str, torch.Tensor]; `guide`: Optional[torch.Tensor] = None
Returns: tuple[dict[str, torch.Tensor], dict[str, Any]]
Calls: models.feature_selection.selection.CrossModalFeatureSelection._gate, models.feature_selection.selection.CrossModalFeatureSelection.channel_mlp, models.feature_selection.selection.CrossModalFeatureSelection.spatial
Raises: ValueError: 'Camera-guided selection needs the camera BEV guide'
lines 100–128

`selection_stats(self, info: dict[str, Any]) -> dict[str, float]`
Purpose: Selected-feature ratios (fraction of gates above threshold) and mean importances.
Parameters: `info`: dict[str, Any]
Returns: dict[str, float]
Decorators: torch.no_grad()
lines 131–142

models/fusion/__init__.py

Module: `models.fusion` · 1 lines
Purpose: Package marker.
Imports (project): –
Imports (third-party / stdlib): –
Imported by: –

models/fusion/bev_backbone.py

Module: `models.fusion.bev_backbone` · 47 lines
Purpose: SECOND-FPN BEV backbone on the fused BEV map.
Imports (project): models.common.blocks
Imports (third-party / stdlib): torch
Imported by: models.fusion_model

class `SECONDFPN`(nn.Module)

Instance attributes (set in __init__): `blocks` = nn.ModuleList(); `deblocks` = nn.ModuleList(); `out_channels` = int(sum(cfg.num_upsample_filters)); `out_stride` = int(cfg.layer_strides[0]); `grad_checkpoint` = False

`__init__(self, in_channels: int, cfg: Any) -> None`
Purpose: Builds the down-sampling blocks and the up-sampling deblocks from the config lists.
Parameters: `in_channels`: int; `cfg`: Any
Returns: None
Calls: models.common.blocks.ConvBNReLU
Called by: models.fusion_model.MultiSensorFusionNet.__init__
lines 20–37

`forward(self, x: torch.Tensor) -> torch.Tensor`
Purpose: Runs blocks (optionally checkpointed), deblocks each output, concatenates.
Parameters: x: torch.Tensor
Returns: torch.Tensor
lines 39–47

models/fusion/fusion_methods.py

Module: `models.fusion.fusion_methods` · 56 lines
Purpose: BEV fusion methods.
Imports (project): models.common.blocks
Imports (third-party / stdlib): torch
Imported by: models.fusion.registry

class `ConcatFusion`(nn.Module)

`concat(F_m) -> 3x3 conv -> residual block`.
Instance attributes (set in __init__): `modalities` = list(in_channels); `fuse` = nn.Sequential(ConvBNReLU(sum(in_channels.values()), out_channels, 3), ResBlock(out_channels))

`__init__(self, in_channels: dict[str, int], out_channels: int) -> None`
Purpose: Concat → 3×3 ConvBNReLU → ResBlock.
Parameters: `in_channels`: dict[str, int]; `out_channels`: int
Returns: None
Calls: models.common.blocks.ConvBNReLU, models.common.blocks.ResBlock
lines 24–27

`forward(self, feats: dict[str, torch.Tensor]) -> tuple[torch.Tensor, dict[str, Any]]`
Purpose: Fuses by concatenation; returns (fused, {}).
Parameters: `feats`: dict[str, torch.Tensor]
Returns: tuple[torch.Tensor, dict[str, Any]]
Calls: models.fusion.fusion_methods.ConcatFusion.fuse
lines 29–30

class `AttentionGatedFusion`(nn.Module)

Gated attention fusion. 1. Project each modality to a common space: `P_m = φ_m(F_m)`. 2. Per-cell modality attention `a = softmax_m(g([F_1..F_M])) ∈ Δ^M`. 3. `F_w = Σ_m a_m · P_m` (sensor reliability weighting). 4. `F = CBAM(ResBlock(conv1x1([F_w, P_1..P_M])))` keeps complementary detail.
Instance attributes (set in __init__): `modalities` = list(in_channels); `proj` = nn.ModuleDict({k: ConvBNReLU(c, out_channels, 1) for k, c in in_channels.items()}); `attention` = nn.Sequential(ConvBNReLU(sum(in_channels.values()), 64, 3), nn.Conv2d(64, m, 1)); `mix` = nn.Sequential(ConvBNReLU(out_channels * (m + 1), out_channels, 1), ResBlock(out_channels), CBAM(o...

`__init__(self, in_channels: dict[str, int], out_channels: int) -> None`
Purpose: Per-sensor projections, modality-attention network and the mixer (1×1 + ResBlock + CBAM).
Parameters: `in_channels`: dict[str, int]; `out_channels`: int
Returns: None
Calls: models.common.blocks.CBAM, models.common.blocks.ConvBNReLU, models.common.blocks.ResBlock
lines 42–49

`forward(self, feats: dict[str, torch.Tensor]) -> tuple[torch.Tensor, dict[str, Any]]`
Purpose: Softmax attention over sensors per cell, weighted sum + projected maps → mixer; returns weights in info.
Parameters: `feats`: dict[str, torch.Tensor]
Returns: tuple[torch.Tensor, dict[str, Any]]
Calls: models.fusion.fusion_methods.AttentionGatedFusion.attention, models.fusion.fusion_methods.AttentionGatedFusion.mix
lines 51–56

models/fusion/registry.py

Module: `models.fusion.registry` · 30 lines
Purpose: Fusion-method registry: add a new fusion by decorating a class with `@register_fusion("name")`.
Imports (project): models.fusion.fusion_methods
Imports (third-party / stdlib): torch
Imported by: models.fusion_model

Constant	Value
FUSION_REGISTRY	{'concat': ConcatFusion, 'attention_gated': AttentionGatedFusion}

Functions

`register_fusion(name: str) -> Callable[[type[nn.Module]], type[nn.Module]]`
Purpose: Decorator that registers a fusion class under a name.
Parameters: `name`: str
Returns: Callable[[type[nn.Module]], type[nn.Module]]
lines 20–24

`build_fusion(method: str, in_channels: dict[str, int], out_channels: int) -> nn.Module`
Purpose: Instantiates the registered fusion class (KeyError for unknown names).
Parameters: `method`: str; `in_channels`: dict[str, int]; `out_channels`: int
Returns: nn.Module
Called by: models.fusion_model.MultiSensorFusionNet.__init__
Raises: KeyError: f'Unknown fusion method {method!r}; available: {sorted(FUSION_REGISTRY)}'
lines 27–30

models/fusion_model.py

Module: `models.fusion_model` · 203 lines
Purpose: MultiSensorFusionNet — the complete network.
Imports (project): models.camera_bev.lss, models.detection.center_head, models.detection.decoder, models.feature_selection.losses, models.feature_selection.selection, models.fusion.bev_backbone, models.fusion.registry, models.lidar_encoder.pillar_encoder, models.radar_encoder.radar_encoder, models.trajectory.losses, models.trajectory.planning_head, utils.bev_grid, utils.config, utils.timer
Imports (third-party / stdlib): torch
Imported by: evaluation.ablation, evaluation.evaluator, inference.pipeline, scripts.benchmark, scripts.evaluate, scripts.smoke_test, tests.test_models, training.trainer

Constant	Value
ALL_LOSSES	('detection', 'trajectory', 'feature_selection', 'fusion', 'depth')

class `MultiSensorFusionNet`(nn.Module)

Instance attributes (set in __init__): `cfg` = cfg; `grid` = BEVGrid.from_config(cfg.bev); `classes` = list(cfg.classes); `use_camera` = (bool(f.use_camera_bev), bool(f.use_lidar), bool(f.use_radar)); `use_lidar` = (bool(f.use_camera_bev), bool(f.use_lidar), bool(f.use_radar)); `use_radar` = (bool(f.use_camera_bev), bool(f.use_lidar), bool(f.use_radar)); `modalities` = list(channels); `use_feature_selection` = bool(f.feature_selection) and len(selectable) > 0; `fusion_method` = str(f.method); `fusion` = build_fusion(self.fusion_method, channels, int(f.out_channels)); `deep_supervision` = bool(f.deep_supervision); `bev_backbone` = SECONDFPN(int(f.out_channels), m.bev_backbone); `head_grid` = self.grid.downsample(int(hcfg.out_stride)); `det_head` = CenterHead(self.bev_backbone.out_channels, len(self.classes), int(hcfg.head_channels)); `use_planning_head` = bool(m.trajectory_head.enabled); `camera_bev` = CameraBEVEncoder(m.camera_bev, self.grid, tuple(cfg.dataset.camera.image_size)); `lidar_encoder` = LidarEncoder(m.lidar_encoder, self.grid); `radar_encoder` = RadarEncoder(m.radar_encoder, self.grid); `feature_selection` = CrossModalFeatureSelection(selectable, guide, f.selection); `aux_heads` = nn.ModuleDict(heads); `plan_head` = PlanningHead(self.bev_backbone.out_channels, int(m.trajectory_head.hidden), int(cfg.planning.hori...

`__init__(self, cfg: Config) -> None`
Purpose: Builds the enabled encoders, optional feature selection, fusion, auxiliary heads, backbone, detection head and optional planning head (ValueError if no modality or inconsistent strides).
Parameters: `cfg`: Config
Returns: None
Calls: models.camera_bev.lss.CameraBEVEncoder, models.detection.center_head.CenterHead, models.feature_selection.selection.CrossModalFeatureSelection, models.fusion.bev_backbone.SECONDFPN, models.fusion.registry.build_fusion, models.lidar_encoder.pillar_encoder.LidarEncoder, models.radar_encoder.radar_encoder.RadarEncoder, models.trajectory.planning_head.PlanningHead, utils.bev_grid.BEVGrid.from_config
Called by: models.fusion_model.build_model
Raises: ValueError: 'At least one modality must be enabled in fusion.use_*'; ValueError: 'detection_head.out_stride must equal bev_backbone.layer_strides[0]'
lines 46–93

`set_grad_checkpoint(self, enabled: bool) -> None`
Purpose: Enables/disables gradient checkpointing in the image backbone and the BEV backbone.
Parameters: `enabled`: bool
Returns: None
lines 96–99

`module_groups(self) -> dict[str, nn.Module]`
Purpose: Top-level sub-networks by name (used for staged freezing).
Returns: dict[str, nn.Module]
Calls: models.fusion_model.MultiSensorFusionNet.named_children
lines 101–103

`freeze(self, names: Iterable[str]) -> list[str]`
Purpose: Freeze parameters whose name starts with any of `names` (e.g. `camera_bev.backbone`).
Parameters: `names`: Iterable[str]

Returns: list[str]
Calls: models.fusion_model.MultiSensorFusionNet.named_parameters
lines 105–113

`forward(self, batch: dict[str, Any], timer: Optional[StageTimer] = None) -> dict[str, Any]`
Purpose: Full forward pass (encoders → selection → fusion → backbone → heads) with per-stage timing; returns all intermediates.
Parameters: `batch`: dict[str, Any]; `timer`: Optional[StageTimer] = None
Returns: dict[str, Any]
Calls: models.fusion_model.MultiSensorFusionNet.bev_backbone, models.fusion_model.MultiSensorFusionNet.camera_bev, models.fusion_model.MultiSensorFusionNet.det_head, models.fusion_model.MultiSensorFusionNet.feature_selection, models.fusion_model.MultiSensorFusionNet.fusion, models.fusion_model.MultiSensorFusionNet.lidar_encoder, models.fusion_model.MultiSensorFusionNet.plan_head, models.fusion_model.MultiSensorFusionNet.radar_encoder, utils.timer.NullTimer
lines 116–163

`compute_losses(self, out: dict[str, Any], batch: dict[str, Any], loss_cfg: Config, active: Iterable[str] = ALL_LOSSES) -> dict[str, torch.Tensor]`
Purpose: Configurable weighted total loss over the active loss groups; returns every component.
Parameters: `out`: dict[str, Any]; `batch`: dict[str, Any]; `loss_cfg`: Config; `active`: Iterable[str] = ALL_LOSSES
Returns: dict[str, torch.Tensor]
Calls: models.detection.center_head.aux_heatmap_loss, models.detection.center_head.detection_loss, models.feature_selection.losses.feature_selection_loss, models.trajectory.losses.trajectory_loss
lines 166–193

`decode(self, out: dict[str, Any], score_threshold: Optional[float] = None) -> list[dict[str, torch.Tensor]]`
Purpose: Decodes detections with the configured threshold, max count and NMS radii.
Parameters: `out`: dict[str, Any]; `score_threshold`: Optional[float] = None
Returns: list[dict[str, torch.Tensor]]
Calls: models.detection.decoder.decode_detections
lines 196–199

Functions

`build_model(cfg: Config) -> MultiSensorFusionNet`
Purpose: Factory: `MultiSensorFusionNet(cfg)`.
Parameters: `cfg`: Config
Returns: MultiSensorFusionNet
Calls: models.fusion_model.MultiSensorFusionNet
Called by: evaluation.ablation.evaluate_checkpoint, inference.pipeline.FusionPipeline.__init__, scripts.benchmark.main, scripts.evaluate.main, scripts.smoke_test.run, tests.test_models.test_forward_backward_every_experiment, training.trainer.Trainer.__init__
lines 202–203

models/lidar_encoder/__init__.py

Module: `models.lidar_encoder` · 1 lines
Purpose: Package marker.
Imports (project): –
Imports (third-party / stdlib): –
Imported by: –

models/lidar_encoder/pillar_encoder.py

Module: `models.lidar_encoder.pillar_encoder` · 63 lines
Purpose: LiDAR encoder: dynamic PointPillars -> BEV feature map.
Imports (project): models.common.blocks, models.common.pillars, utils.bev_grid
Imports (third-party / stdlib): torch
Imported by: models.fusion_model

Constant	Value
LIDAR_RASTER_NAMES	('lidar_density', 'lidar_max_height', 'lidar_mean_height', 'lidar_intensity')

class LidarEncoder(nn.Module)

Instance attributes (set in __init__): `grid` = grid; `use_raster` = bool(cfg.raster_channels); `out_channels` = int(cfg.out_channels); `vfe` = DynamicPillarVFE(self.in_features, tuple((int(c) for c in cfg.pillar_channels)), grid); `conv` = nn.Sequential(ConvBNReLU(self.vfe.out_channels + n_raster, learned, 3), *[ResBlock(learned) for _...; `channel_names` = [f'lidar_learned_{i}' for i in range(learned)] + list(LIDAR_RASTER_NAMES[:n_raster])
Class attributes / fields: `in_features` = 5

`__init__(self, cfg: Any, grid: BEVGrid) -> None`
Purpose: Builds the VFE, conv stack, normaliser and channel names.
Parameters: `cfg`: Any; `grid`: BEVGrid

Returns: None
Calls: models.common.blocks.ConvBNReLU, models.common.blocks.ResBlock, models.common.pillars.DynamicPillarVFE, models.lidar_encoder.pillar_encoder.LidarEncoder.register_buffer
Called by: models.fusion_model.MultiSensorFusionNet.__init__
lines 28–40

`raster(self, points: torch.Tensor, batch_size: int) -> torch.Tensor`
Purpose: LiDAR raster (B, 4, ny, nx): density, max height, mean height, intensity (scaled).
Parameters: `points`: torch.Tensor; `batch_size`: int
Returns: torch.Tensor
Calls: models.common.pillars.PillarIndexer, models.common.pillars.bev_raster_stats
Called by: models.lidar_encoder.pillar_encoder.LidarEncoder.forward
lines 42–53

`forward(self, points: torch.Tensor, batch_size: int) -> dict[str, torch.Tensor]`
Purpose: Points → (B, 64, ny, nx) features (+ raster, pillars).
Parameters: `points`: torch.Tensor; `batch_size`: int
Returns: dict[str, torch.Tensor]
Calls: models.lidar_encoder.pillar_encoder.LidarEncoder.conv, models.lidar_encoder.pillar_encoder.LidarEncoder.raster, models.lidar_encoder.pillar_encoder.LidarEncoder.vfe
lines 55–63

models/radar_encoder/__init__.py

Module: `models.radar_encoder` · 1 lines
Purpose: Package marker.
Imports (project): –
Imports (third-party / stdlib): –
Imported by: –

models/radar_encoder/radar_encoder.py

Module: `models.radar_encoder.radar_encoder` · 59 lines
Purpose: Radar encoder: velocity-preserving radar pillars -> BEV feature map.
Imports (project): models.common.blocks, models.common.pillars, utils.bev_grid
Imports (third-party / stdlib): torch
Imported by: models.fusion_model

Constant	Value
RADAR_RASTER_NAMES	`('radar_density', 'radar_vx', 'radar_vy', 'radar_rcs', 'radar_time_lag')`

class RadarEncoder(nn.Module)

Instance attributes (set in __init__): `grid` = grid; `use_raster` = bool(cfg.raster_channels); `out_channels` = int(cfg.out_channels); `vfe` = DynamicPillarVFE(self.in_features, tuple((int(c) for c in cfg.pillar_channels)), grid); `conv` = nn.Sequential(ConvBNReLU(self.vfe.out_channels + n_raster, learned, 3), *[ResBlock(learned, dilat...; `channel_names` = [f'radar_learned_{i}' for i in range(learned)] + list(RADAR_RASTER_NAMES[:n_raster])
Class attributes / fields: `in_features` = 10

`__init__(self, cfg: Any, grid: BEVGrid) -> None`
Purpose: Builds the VFE, dilated conv stack, normalisers and channel names.
Parameters: `cfg`: Any; `grid`: BEVGrid
Returns: None
Calls: models.common.blocks.ConvBNReLU, models.common.blocks.ResBlock, models.common.pillars.DynamicPillarVFE, models.radar_encoder.radar_encoder.RadarEncoder.register_buffer
Called by: models.fusion_model.MultiSensorFusionNet.__init__
lines 31–44

`raster(self, points: torch.Tensor, batch_size: int) -> torch.Tensor`
Purpose: Radar raster (B, 5, ny, nx): density, vx, vy, RCS, time lag (scaled).
Parameters: `points`: torch.Tensor; `batch_size`: int
Returns: torch.Tensor
Calls: models.common.pillars.bev_raster_stats
Called by: models.radar_encoder.radar_encoder.RadarEncoder.forward
lines 46–49

`forward(self, points: torch.Tensor, batch_size: int) -> dict[str, torch.Tensor]`
Purpose: Points → (B, 64, ny, nx) features (+ raster, pillars).
Parameters: `points`: torch.Tensor; `batch_size`: int
Returns: dict[str, torch.Tensor]
Calls: models.radar_encoder.radar_encoder.RadarEncoder.conv, models.radar_encoder.radar_encoder.RadarEncoder.raster, models.radar_encoder.radar_encoder.RadarEncoder.vfe
lines 51–59

models/trajectory/__init__.py

Module: `models.trajectory` · 1 lines
Purpose: Package marker.
Imports (project): –
Imports (third-party / stdlib): –
Imported by: –

models/trajectory/losses.py

Module: `models.trajectory.losses` · 74 lines
Purpose: Trajectory-planning losses.
Imports (project): utils.bev_grid
Imports (third-party / stdlib): torch, torch.nn.functional
Imported by: models.fusion_model

Functions

`ego_footprint_points(waypoints: torch.Tensor, ego_cfg: Any) -> torch.Tensor`
Purpose: Sample points on the ego body `(B, T, 5, 2)` for waypoints `(B, T, 2)` (rear-axle positions).
Parameters: `waypoints`: torch.Tensor; `ego_cfg`: Any
Returns: torch.Tensor
Called by: models.trajectory.losses.trajectory_loss
lines 23–35

`sample_occupancy(occ: torch.Tensor, points: torch.Tensor, grid: BEVGrid) -> torch.Tensor`
Purpose: Bilinear sample `occ (B, T, H, W)` at metric `points (B, T, P, 2)` -> `(B, T, P)`.
Parameters: `occ`: torch.Tensor; `points`: torch.Tensor; `grid`: BEVGrid
Returns: torch.Tensor
Called by: models.trajectory.losses.trajectory_loss
lines 38–45

`trajectory_loss(pred: torch.Tensor, batch: dict[str, Any], grid: BEVGrid, ego_cfg: Any, weights: Any, dt: float) -> dict[str, torch.Tensor]`
Purpose: L1 + collision + smoothness + goal losses of the predicted waypoints.
Parameters: `pred`: torch.Tensor; `batch`: dict[str, Any]; `grid`: BEVGrid; `ego_cfg`: Any; `weights`: Any; `dt`: float
Returns: dict[str, torch.Tensor]
Calls: models.trajectory.losses.ego_footprint_points, models.trajectory.losses.sample_occupancy
Called by: models.fusion_model.MultiSensorFusionNet.compute_losses
lines 48–74

models/trajectory/planning_head.py

Module: `models.trajectory.planning_head` · 59 lines
Purpose: Learned trajectory planner (replaces / complements the classical baseline planner).
Imports (project): models.common.blocks
Imports (third-party / stdlib): torch, torch.nn.functional
Imported by: models.fusion_model

class PlanningHead(nn.Module)

Instance attributes (set in __init__): `horizon` = (horizon, dt, num_commands); `dt` = (horizon, dt, num_commands); `num_commands` = (horizon, dt, num_commands); `reduce` = nn.Sequential(ConvBNReLU(in_channels + 2, hidden, 3, stride=2), ConvBNReLU(hidden, hidden, 3, str...; `query` = nn.Sequential(nn.Linear(ego_state_dim + num_commands, hidden), nn.ReLU(inplace=True), nn.Linear(h...; `attn` = nn.MultiheadAttention(hidden, num_heads, batch_first=True); `gru` = nn.GRUCell(hidden + 2, hidden); `out` = nn.Linear(hidden, 2)

`__init__(self, in_channels: int, hidden: int = 128, horizon: int = 6, dt: float = 0.5, num_heads: int = 4, num_commands: int = 3, ego_state_dim: int = 3) -> None`
Purpose: Builds BEV reduction convs, query MLP, cross-attention, GRU cell and zero-initialised output layer.
Parameters: `in_channels`: int; `hidden`: int = 128; `horizon`: int = 6; `dt`: float = 0.5; `num_heads`: int = 4; `num_commands`: int = 3; `ego_state_dim`: int = 3
Returns: None
Calls: models.common.blocks.ConvBNReLU, models.trajectory.planning_head.PlanningHead.register_buffer
Called by: models.fusion_model.MultiSensorFusionNet.__init__
lines 23–35

`forward(self, bev: torch.Tensor, ego_state: torch.Tensor, command: torch.Tensor) -> dict[str, torch.Tensor]`
Purpose: BEV + ego state + command → waypoints (B, T, 2), the CV prior and attention weights.
Parameters: `bev`: torch.Tensor; `ego_state`: torch.Tensor; `command`: torch.Tensor
Returns: dict[str, torch.Tensor]
Calls: models.trajectory.planning_head.PlanningHead.attn, models.trajectory.planning_head.PlanningHead.gru, models.trajectory.planning_head.PlanningHead.out, models.trajectory.planning_head.PlanningHead.query, models.trajectory.planning_head.PlanningHead.reduce
lines 37–59

Package planning

planning/__init__.py

Module: planning · 1 lines
Purpose: Package marker.
Imports (project): –
Imports (third-party / stdlib): –
Imported by: –

planning/astar.py

Module: planning.astar · 92 lines
Purpose: Grid A* on a BEV cost map (8-connected, Euclidean heuristic).
Imports (project): –
Imports (third-party / stdlib): heapq, math, numpy
Imported by: planning.trajectory_planner, tests.test_metrics_planning

Constant	Value
_MOVES	[(-1, 0, 1.0), (1, 0, 1.0), (0, -1, 1.0), (0, 1, 1.0), (-1, -1, math.sqrt(2)), (-1, 1, math.sqrt(2)), (1, -1, math.sqrt(2)), (1, 1, math.sqrt(2))]

Functions

astar(cost: np.ndarray, start: tuple[int, int], goal: tuple[int, int], max_expansions: int = 60000, heuristic_weight: float = 1.0) -> tuple[list[tuple[int, int]], bool]
Purpose: Shortest path on a cost grid.
Details: Args: cost: (H, W) per-cell traversal cost (>= 0, np.inf = blocked). Moving into a cell costs step_length * (1 + cost[cell]). start, goal: (row, col) cells. Returns: (path, reached) — if the goal is unreachable the path leads to the explored cell closest to the goal (best effort), and reached is False.
Parameters: cost: np.ndarray; start: tuple[int, int]; goal: tuple[int, int]; max_expansions: int = 60000; heuristic_weight: float = 1.0
Returns: tuple[list[tuple[int, int]], bool]
Called by: planning.trajectory_planner.BaselinePlanner.plan, tests.test_metrics_planning.test_astar_routes_around_wall
Raises: ValueError: 'goal outside grid'
lines 14–73

nearest_free(cost: np.ndarray, cell: tuple[int, int], max_radius: int = 20) -> Optional[tuple[int, int]]
Purpose: Closest finite-cost cell to cell within max_radius (square rings).
Parameters: cost: np.ndarray; cell: tuple[int, int]; max_radius: int = 20
Returns: Optional[tuple[int, int]]
Called by: planning.trajectory_planner.BaselinePlanner.plan
lines 76–92

planning/mpc.py

Module: planning.mpc · 135 lines
Purpose: Model Predictive Control: iterative LQR (iLQR) on a kinematic bicycle model.
Imports (project): utils.geometry
Imports (third-party / stdlib): dataclasses, numpy
Imported by: planning.trajectory_planner, tests.test_metrics_planning

class MPCResult

Class attributes / fields: states = np.ndarray; controls = np.ndarray; cost = float; iterations = int

class BicycleILQR

Instance attributes (set in __init__): dt = float(cfg_mpc.dt); iterations = int(cfg_mpc.iterations); L = float(ego_cfg.wheelbase); max_steer = float(ego_cfg.max_steer); max_accel = (float(ego_cfg.max_accel), float(ego_cfg.max_decel)); max_decel = (float(ego_cfg.max_accel), float(ego_cfg.max_decel)); max_speed = float(ego_cfg.max_speed); Q = np.diag([cfg_mpc.w_pos, cfg_mpc.w_pos, cfg_mpc.w_yaw, cfg_mpc.w_speed, cfg_mpc.w_steer]).astype(f...; R = np.diag([cfg_mpc.w_accel, cfg_mpc.w_steer_rate]).astype(float)

__init__(self, cfg_mpc: Any, ego_cfg: Any) -> None
Purpose: Stores time step, iterations, vehicle limits and the Q/R cost matrices.
Parameters: cfg_mpc: Any; ego_cfg: Any
Returns: None
Called by: planning.trajectory_planner.BaselinePlanner.__init__, tests.test_metrics_planning.test_mpc_tracks_straight_reference
lines 35–43

step(self, s: np.ndarray, u: np.ndarray) -> np.ndarray
Purpose: One discrete bicycle-model step with speed and steering clipping.
Parameters: s: np.ndarray; u: np.ndarray

Returns: np.ndarray
Called by: planning.mpc.BicycleILQR.rollout, planning.mpc.BicycleILQR.solve
lines 46–54

`jacobians(self, s: np.ndarray) -> tuple[np.ndarray, np.ndarray]`
Purpose: Analytic A (5×5) and B (5×2) Jacobians at a state.
Parameters: `s`: np.ndarray
Returns: tuple[np.ndarray, np.ndarray]
Called by: planning.mpc.BicycleILQR.solve
lines 56–65

`rollout(self, s0: np.ndarray, controls: np.ndarray) -> np.ndarray`
Purpose: Simulates the states for a control sequence.
Parameters: `s0`: np.ndarray; `controls`: np.ndarray
Returns: np.ndarray
Calls: planning.mpc.BicycleILQR.step
Called by: planning.mpc.BicycleILQR.solve
lines 67–71

`_clip(self, u: np.ndarray) -> np.ndarray`
Purpose: Clips accelerations and steering rates to their limits.
Parameters: `u`: np.ndarray
Returns: np.ndarray
Called by: planning.mpc.BicycleILQR.solve
lines 73–77

`_err(self, s: np.ndarray, ref: np.ndarray) -> np.ndarray`
Purpose: State error with wrapped yaw difference.
Parameters: `s`: np.ndarray; `ref`: np.ndarray
Returns: np.ndarray
Calls: utils.geometry.wrap_angle
Called by: planning.mpc.BicycleILQR.cost, planning.mpc.BicycleILQR.solve
lines 79–82

`cost(self, states: np.ndarray, controls: np.ndarray, ref: np.ndarray) -> float`
Purpose: Quadratic tracking + control cost of a trajectory.
Parameters: `states`: np.ndarray; `controls`: np.ndarray; `ref`: np.ndarray
Returns: float
Calls: planning.mpc.BicycleILQR._err
Called by: planning.mpc.BicycleILQR.solve
lines 84–86

`solve(self, s0: np.ndarray, ref: np.ndarray, u_init: np.ndarray | None = None) -> MPCResult`
Purpose: Track `ref (N, 5)` (`[x, y, yaw, v, 0]` per step) from initial state `s0`.
Parameters: `s0`: np.ndarray; `ref`: np.ndarray; `u_init`: np.ndarray | None = None
Returns: MPCResult
Calls: planning.mpc.BicycleILQR._clip, planning.mpc.BicycleILQR._err, planning.mpc.BicycleILQR.cost, planning.mpc.BicycleILQR.jacobians, planning.mpc.BicycleILQR.rollout, planning.mpc.BicycleILQR.step, planning.mpc.MPCResult
lines 89–135

planning/occupancy_grid.py

Module: `planning.occupancy_grid` · 98 lines
Purpose: Occupancy / cost grids for planning (ego frame, layout `[iy, ix]`).
Imports (project): utils.bev_grid, utils.raster
Imports (third-party / stdlib): cv2, numpy
Imported by: planning.trajectory_planner

class `OccupancyBuilder`

Instance attributes (set in __init__): `cfg` = cfg_planning; `grid` = planning_grid(cfg_planning); `horizon` = int(cfg_planning.horizon); `dt` = float(cfg_planning.dt); `ego_box` = np.array([[float(ego.rear_axle_to_center), 0.0, 0.0, float(ego.length), float(ego.width), 1.6, 0....

`__init__(self, cfg_planning: Any) -> None`
Purpose: Grid, horizon, dt and the ego box from the planning config.
Parameters: `cfg_planning`: Any
Returns: None
Calls: planning.occupancy_grid.planning_grid
Called by: planning.trajectory_planner.BaselinePlanner.__init__
lines 35–41

`dynamic_occupancy(self, boxes: np.ndarray, inflate: float = 0.0, times: Optional[np.ndarray] = None) -> np.ndarray`
Purpose: `(T, ny, nx)` uint8 occupancy at `t = dt, 2dt, ... T·dt` (CV motion forecast).
Parameters: `boxes`: np.ndarray; `inflate`: float = 0.0; `times`: Optional[np.ndarray] = None
Returns: np.ndarray

Calls: planning.occupancy_grid.propagate_boxes, utils.raster.rasterize_boxes
lines 43–50

`static_occupancy(self, boxes: np.ndarray, inflate: float) -> np.ndarray`
Purpose: Rasterised (inflated) boxes at t = 0 → (ny, nx) uint8.
Parameters: `boxes`: np.ndarray; `inflate`: float
Returns: np.ndarray
Calls: utils.raster.rasterize_boxes
lines 52–56

`lidar_obstacles(self, points: Optional[np.ndarray], z_band: tuple[float, float]) -> np.ndarray`
Purpose: Cells with non-ground LiDAR returns (excluding the ego vehicle itself).
Parameters: `points`: Optional[np.ndarray]; `z_band`: tuple[float, float]
Returns: np.ndarray
lines 58–71

`clearance_cost(self, occ: np.ndarray, scale_m: float = 1.5) -> np.ndarray`
Purpose: `exp(-d / scale)` where `d` is the distance (m) to the nearest occupied cell.
Parameters: `occ`: np.ndarray; `scale_m`: float = 1.5
Returns: np.ndarray
lines 73–78

`footprint_points(self, xy: np.ndarray, yaw: np.ndarray, margin: float = 0.0) -> np.ndarray`
Purpose: Points sampled inside the ego footprint `(N, 15, 2)` for rear-axle poses `xy (N, 2)`.
Parameters: `xy`: np.ndarray; `yaw`: np.ndarray; `margin`: float = 0.0
Returns: np.ndarray
lines 80–89

`occupied_at(self, occ: np.ndarray, pts: np.ndarray) -> np.ndarray`
Purpose: Look up `occ (ny, nx)` at metric points `(..., 2)` (outside the grid = free).
Parameters: `occ`: np.ndarray; `pts`: np.ndarray
Returns: np.ndarray
lines 91–98

Functions

`planning_grid(cfg_planning: Any) -> BEVGrid`
Purpose: Planner `BEVGrid` from `planning.grid`.
Parameters: `cfg_planning`: Any
Returns: BEVGrid
Calls: utils.bev_grid.BEVGrid
Called by: planning.occupancy_grid.OccupancyBuilder.__init__
lines 21–23

`propagate_boxes(boxes: np.ndarray, t: float) -> np.ndarray`
Purpose: Constant-velocity forecast of `[x, y, z, l, w, h, yaw, vx, vy]` boxes by `t` seconds.
Parameters: `boxes`: np.ndarray; `t`: float
Returns: np.ndarray
Called by: planning.occupancy_grid.OccupancyBuilder.dynamic_occupancy
lines 26–31

planning/tracking.py

Module: `planning.tracking` · 102 lines
Purpose: Online multi-object tracking for scene replay (CenterPoint-style greedy association).
Imports (project): utils.geometry
Imports (third-party / stdlib): dataclasses, numpy
Imported by: inference.pipeline

Constant	Value
MATCH_DISTANCE	{'car': 4.0, 'truck': 4.0, 'bus': 5.5, 'trailer': 3.0, 'construction_vehicle': 3.0, 'pedestrian': 1.0, 'motorcycle': 13.0, 'bicycle': 3.0, 'barrier': 1.0, 't...

class Track

Class attributes / fields: `track_id` = int; `label` = int; `position` = np.ndarray; `velocity` = np.ndarray; `score` = float; `age` = 1; `misses` = 0; `history` = field(default_factory=list)

class MultiObjectTracker

Instance attributes (set in __init__): `classes` = classes; `max_misses` = max_misses; `history_len` = history

`__init__(self, classes: list[str], max_misses: int = 2, history: int = 8) -> None`
Purpose: Classes, max misses, history length; calls reset().
Parameters: `classes`: list[str]; `max_misses`: int = 2; `history`: int = 8
Returns: None

Calls: planning.tracking.MultiObjectTracker.reset
Called by: inference.pipeline.FusionPipeline.__init__
lines 36–40

`reset(self) -> None`
Purpose: Clears all tracks, restarts ids and time.
Returns: None
Called by: planning.tracking.MultiObjectTracker.__init__, planning.tracking.MultiObjectTracker.update
lines 42–45

`update(self, boxes_ref: np.ndarray, labels: np.ndarray, scores: np.ndarray, ref_ego2global: np.ndarray, timestamp_us: int) -> np.ndarray`
Purpose: Associate detections; returns track ids `(N,)` aligned with the detections.
Parameters: `boxes_ref`: np.ndarray; `labels`: np.ndarray; `scores`: np.ndarray; `ref_ego2global`: np.ndarray; `timestamp_us`: int
Returns: np.ndarray
Calls: planning.tracking.MultiObjectTracker.reset, planning.tracking.Track
lines 47–88

`histories_ref(self, ids: np.ndarray, ref_ego2global: np.ndarray) -> list[np.ndarray]`
Purpose: Motion history of the given track ids in the current reference frame.
Parameters: `ids`: np.ndarray; `ref_ego2global`: np.ndarray
Returns: list[np.ndarray]
Calls: utils.geometry.invert_transform
lines 90–102

planning/trajectory_planner.py

Module: `planning.trajectory_planner` · 250 lines
Purpose: Trajectory planners.
Imports (project): planning.astar, planning.mpc, planning.occupancy_grid
Imports (third-party / stdlib): cv2, dataclasses, numpy, time
Imported by: evaluation.evaluator, inference.pipeline, scripts.eval_planner, tests.test_metrics_planning

Constant	Value
COMMAND_NAMES	('RIGHT', 'LEFT', 'STRAIGHT')

class PlanResult

Class attributes / fields: `waypoints` = np.ndarray; `yaws` = np.ndarray; `speeds` = np.ndarray; `collision` = bool; `planner` = str; `command` = int; `path` = field(default_factory=lambda: np.zeros((0, 2))); `dense` = field(default_factory=lambda: np.zeros((0, 5))); `goal` = None; `accel` = 0.0; `time_ms` = 0.0; `debug` = field(default_factory=dict)

class BaselinePlanner

Instance attributes (set in __init__): `cfg` = cfg; `p` = p; `occ` = OccupancyBuilder(p); `grid` = self.occ.grid; `mpc` = BicycleILQR(p.mpc, p.ego); `horizon` = (int(p.horizon), float(p.dt)); `dt` = (int(p.horizon), float(p.dt)); `classes` = list(cfg.classes)
Class attributes / fields: `name` = 'baseline'

`__init__(self, cfg: Any) -> None`
Purpose: Occupancy builder, grid, iLQR solver and timing settings.
Parameters: `cfg`: Any
Returns: None
Calls: planning.mpc.BicycleILQR, planning.occupancy_grid.OccupancyBuilder
Called by: evaluation.evaluator.evaluate_model, planning.trajectory_planner.HybridPlanner.__init__, planning.trajectory_planner.LearnedPlanner.__init__, tests.test_metrics_planning.test_baseline_planner_brakes_or_avoids_static_obstacle
lines 85–93

`_goal(self, speed: float, command: int) -> np.ndarray`
Purpose: Goal point from speed-dependent lookahead and the command bearing.
Parameters: `speed`: float; `command`: int
Returns: np.ndarray
Called by: planning.trajectory_planner.BaselinePlanner.plan
lines 96–100

`_to_cell(self, xy: np.ndarray) -> tuple[int, int]`
Purpose: Metric point → clamped (row, col) cell.
Parameters: `xy`: np.ndarray
Returns: tuple[int, int]
Called by: planning.trajectory_planner.BaselinePlanner.plan
lines 102–105

`_to_metric(self, cells: list[tuple[int, int]]) -> np.ndarray`
Purpose: Cells → metric cell centres.
Parameters: `cells`: list[tuple[int, int]]
Returns: np.ndarray

Called by: planning.trajectory_planner.BaselinePlanner.plan
lines 107–110

`collision_check(self, waypoints: np.ndarray, yaws: np.ndarray, dyn_occ: np.ndarray) -> np.ndarray`
Purpose: Per-step collision flags `(T,)` of the ego footprint against `dyn_occ (T, ny, nx)`.
Parameters: `waypoints`: np.ndarray; `yaws`: np.ndarray; `dyn_occ`: np.ndarray
Returns: np.ndarray
Called by: planning.trajectory_planner.BaselinePlanner.plan
lines 112–115

`build_scene(self, detections: dict[str, np.ndarray], lidar_points: Optional[np.ndarray] = None, map_drivable: Optional[np.ndarray] = None) -> dict[str, np.ndarray]`
Purpose: Occupancy layers used by planning (also returned for visualisation).
Parameters: `detections`: dict[str, np.ndarray]; `lidar_points`: Optional[np.ndarray] = None; `map_drivable`: Optional[np.ndarray] = None
Returns: dict[str, np.ndarray]
Called by: planning.trajectory_planner.BaselinePlanner.plan
lines 117–139

`plan(self, ego_state: np.ndarray, command: int, detections: dict[str, np.ndarray], lidar_points: Optional[np.ndarray] = None, map_drivable: Optional[np.ndarray] = None, **_: Any) -> PlanResult`
Purpose: Full baseline planning: scene layers → A* path → speed profile → iLQR → PlanResult.
Parameters: `ego_state`: np.ndarray; `command`: int; `detections`: dict[str, np.ndarray]; `lidar_points`: Optional[np.ndarray] = None; `map_drivable`: Optional[np.ndarray] = None; `**_`: Any
Returns: PlanResult
Calls: planning.astar.astar, planning.astar.nearest_free, planning.trajectory_planner.BaselinePlanner._goal, planning.trajectory_planner.BaselinePlanner._to_cell, planning.trajectory_planner.BaselinePlanner._to_metric, planning.trajectory_planner.BaselinePlanner.build_scene, planning.trajectory_planner.BaselinePlanner.collision_check, planning.trajectory_planner.PlanResult, planning.trajectory_planner._pose_at, planning.trajectory_planner._resample, planning.trajectory_planner._smooth_path
lines 142–200

class `LearnedPlanner`

Uses the network's planning-head waypoints; reports collisions against predicted occupancy.
Instance attributes (set in __init__): `checker` = BaselinePlanner(cfg); `dt` = float(cfg.planning.dt)
Class attributes / fields: `name` = 'learned'

`__init__(self, cfg: Any) -> None`
Purpose: Creates a BaselinePlanner used only for the collision check.
Parameters: `cfg`: Any
Returns: None
Calls: planning.trajectory_planner.BaselinePlanner
Called by: planning.trajectory_planner.HybridPlanner.__init__
lines 208–210

`plan(self, ego_state: np.ndarray, command: int, detections: dict[str, np.ndarray], learned_waypoints: Optional[np.ndarray] = None, lidar_points: Optional[np.ndarray] = None, **_: Any) -> PlanResult`
Purpose: Wraps the learned waypoints into a PlanResult with a collision check (ValueError without waypoints).
Parameters: `ego_state`: np.ndarray; `command`: int; `detections`: dict[str, np.ndarray]; `learned_waypoints`: Optional[np.ndarray] = None; `lidar_points`: Optional[np.ndarray] = None; `**_`: Any
Returns: PlanResult
Calls: planning.trajectory_planner.PlanResult
Raises: ValueError: 'LearnedPlanner needs the planning-head output (model.trajectory_head.enabled)'
lines 212–227

class `HybridPlanner`

Learned trajectory with a classical safety fallback.
Instance attributes (set in __init__): `learned` = (LearnedPlanner(cfg), BaselinePlanner(cfg)); `baseline` = (LearnedPlanner(cfg), BaselinePlanner(cfg))
Class attributes / fields: `name` = 'hybrid'

`__init__(self, cfg: Any) -> None`
Purpose: Creates the learned and baseline planners.
Parameters: `cfg`: Any
Returns: None
Calls: planning.trajectory_planner.BaselinePlanner, planning.trajectory_planner.LearnedPlanner
lines 235–236

`plan(self, ego_state: np.ndarray, command: int, detections: dict[str, np.ndarray], **kwargs: Any) -> PlanResult`
Purpose: Learned plan unless it collides; then the baseline plan.
Parameters: `ego_state`: np.ndarray; `command`: int; `detections`: dict[str, np.ndarray]; `**kwargs`: Any
Returns: PlanResult
lines 238–245

Functions

`_smooth_path(path: np.ndarray, window: int = 5) -> np.ndarray`
Purpose: Moving-average smoothing of a polyline (first point kept).
Parameters: `path`: np.ndarray; `window`: int = 5
Returns: np.ndarray
Called by: planning.trajectory_planner.BaselinePlanner.plan
lines 50–58

`_resample(path: np.ndarray, step: float = 0.25) -> tuple[np.ndarray, np.ndarray]`
Purpose: Resamples a polyline at equal arc-length steps; returns points and arc lengths.
Parameters: `path`: np.ndarray; `step`: float = 0.25
Returns: tuple[np.ndarray, np.ndarray]
Called by: planning.trajectory_planner.BaselinePlanner.plan
lines 61–67

`_pose_at(path: np.ndarray, arclen: np.ndarray, s: np.ndarray) -> tuple[np.ndarray, np.ndarray]`
Purpose: Interpolate positions/headings at arc lengths `s` (extrapolates along the last heading).
Parameters: `path`: np.ndarray; `arclen`: np.ndarray; `s`: np.ndarray
Returns: tuple[np.ndarray, np.ndarray]
Called by: planning.trajectory_planner.BaselinePlanner.plan
lines 70–79

`build_planner(cfg: Any, name: Optional[str] = None)`
Purpose: Planner factory by name (baseline / learned / hybrid).
Parameters: `cfg`: Any; `name`: Optional[str] = None
Returns: None / not annotated
Called by: inference.pipeline.FusionPipeline.planner, scripts.eval_planner.main
lines 248–250

Package `scripts`

`scripts/__init__.py`

Module: `scripts` · 1 lines
Purpose: Package marker.
Imports (project): –
Imports (third-party / stdlib): –
Imported by: –

`scripts/benchmark.py`

Module: `scripts.benchmark` · 70 lines · **entry point** (runnable script)
Purpose: Latency / memory benchmark of every experiment configuration (inference, batch 1, real nuScenes frames).
Imports (project): datasets.nuscenes_dataset, models.fusion_model, utils.config, utils.device, utils.logging_utils, utils.timer
Imports (third-party / stdlib): argparse, json, numpy, pathlib, sys, torch
Imported by: –

Functions

`main() -> None`
Purpose: Benchmarks every experiment configuration (median latency, FPS, peak memory, per-stage times).
Returns: None
Calls: datasets.nuscenes_dataset.NuScenesFusionDataset, datasets.nuscenes_dataset.collate_batch, models.fusion_model.build_model, utils.config.list_experiments, utils.config.load_config, utils.device.amp_enabled, utils.device.get_device, utils.device.gpu_memory_report, utils.device.move_to_device, utils.logging_utils.setup_logging, utils.timer.StageTimer
lines 28–66

`scripts/check_eval_vs_devkit.py`

Module: `scripts.check_eval_vs_devkit` · 78 lines · **entry point** (runnable script)
Purpose: Cross-check the in-memory nuScenes metrics against the official devkit.
Imports (project): datasets.info_builder, evaluation.detection_metrics, evaluation.official_eval, utils.config, utils.logging_utils
Imports (third-party / stdlib): json, numpy, pathlib, sys
Imported by: –

Functions

`fake_predictions(info: dict, rng: np.random.Generator, n_classes: int) -> dict[str, np.ndarray]`
Purpose: Perturbed GT (noise, 20 % misses, false positives, random scores) used only to compare two metric implementations.
Parameters: `info`: dict; `rng`: np.random.Generator; `n_classes`: int
Returns: dict[str, np.ndarray]

Called by: scripts.check_eval_vs_devkit.main
lines 25–42

`main() -> None`
Purpose: Evaluates the perturbed predictions with the in-memory port and the official devkit and prints the differences.
Returns: None
Calls: datasets.info_builder.load_infos, evaluation.detection_metrics.DetectionEvaluator, evaluation.official_eval.run_official_eval, evaluation.official_eval.to_submission, scripts.check_eval_vs_devkit.fake_predictions, utils.config.load_config, utils.config.resolve_path, utils.logging_utils.setup_logging
lines 45–74

scripts/download_dataset.py

Module: `scripts.download_dataset` · 70 lines · **entry point** (runnable script)
Purpose: Help obtaining nuScenes.
Imports (project): utils.config
Imports (third-party / stdlib): argparse, pathlib, sys, tarfile, urllib.request
Imported by: –

Constant	Value
MINI_URL	'https://www.nuscenes.org/data/v1.0-mini.tgz'
REQUIRED	('samples', 'sweeps', 'maps')

Functions

`verify(root: Path, version: str) -> bool`
Purpose: Checks the dataset folder structure and prints a ✓/✗ list.
Parameters: `root`: Path; `version`: str
Returns: bool
Called by: scripts.download_dataset.main
lines 27–32

`download_mini(dest: Path) -> None`
Purpose: Downloads and extracts v1.0-mini (only with the explicit terms flag).
Parameters: `dest`: Path
Returns: None
Called by: scripts.download_dataset.main
Side effects (file / disk I/O calls): extractall, mkdir, open, urlretrieve
lines 35–50

`main() -> None`
Purpose: CLI: verify or download.
Returns: None
Calls: scripts.download_dataset.download_mini, scripts.download_dataset.verify, utils.config.load_config
Raises: SystemExit: 'Read https://www.nuscenes.org/terms-of-use and re-run with --i-accept-the-nuscenes-terms'
lines 53–66

scripts/eval_planner.py

Module: `scripts.eval_planner` · 69 lines · **entry point** (runnable script)
Purpose: Evaluate planners on a split with *ground-truth* objects as perception (oracle upper bound), or with the constant-velocity baseline, without needing a trained network.
Imports (project): datasets.info_builder, datasets.lidar_loader, evaluation.trajectory_metrics, planning.trajectory_planner, utils.bev_grid, utils.config, utils.logging_utils
Imports (third-party / stdlib): argparse, json, numpy, pathlib, sys
Imported by: –

Functions

`main() -> None`
Purpose: Evaluates the baseline planner with GT objects, the constant-velocity model, or the logged trajectory (sanity check).
Returns: None
Calls: datasets.info_builder.load_infos, datasets.lidar_loader.LidarLoader, evaluation.trajectory_metrics.aggregate, evaluation.trajectory_metrics.sample_planning_metrics, planning.trajectory_planner.build_planner, utils.bev_grid.BEVGrid.from_config, utils.config.load_config, utils.logging_utils.setup_logging
lines 27–65

scripts/evaluate.py

Module: `scripts.evaluate` · 77 lines · **entry point** (runnable script)
Purpose: Evaluate a checkpoint on the validation split (all metric families) and save a JSON report.
Imports (project): datasets.nuscenes_dataset, evaluation.evaluator, evaluation.official_eval, models.fusion_model, utils.checkpoint, utils.config, utils.device, utils.logging_utils
Imports (third-party / stdlib): argparse, json, numpy, pathlib, sys, torch
Imported by: –

Constant	Value
MACHINE_KEYS	('dataset.root', 'dataset.version', 'dataset.cache_dir', 'runtime.device', 'runtime.output_dir')

Functions

`main() -> None`
Purpose: Evaluates a checkpoint (all metrics, optional official devkit) and writes a JSON report.
Returns: None
Calls: datasets.nuscenes_dataset.NuScenesFusionDataset, evaluation.evaluator.evaluate_model, evaluation.evaluator.summary_line, evaluation.official_eval.run_official_eval, evaluation.official_eval.to_submission, models.fusion_model.build_model, utils.checkpoint.load_checkpoint, utils.config.Config, utils.config.load_config, utils.config.resolve_path, utils.device.amp_enabled, utils.device.get_device, utils.device.move_to_device, utils.logging_utils.setup_logging
Side effects (file / disk I/O calls): mkdir, write_text
lines 31–73

scripts/inference.py

Module: `scripts.inference` · 138 lines · **entry point** (runnable script)
Purpose: Run the full pipeline over a nuScenes scene and export composite frames, an MP4 and results JSON.
Imports (project): app.ui, app.ui.resources, evaluation.detection_metrics, evaluation.official_eval, evaluation.trajectory_metrics, inference.pipeline, utils.config, utils.logging_utils, visualization.common
Imports (third-party / stdlib): argparse, cv2, json, numpy, pathlib, sys
Imported by: –

Functions

`compose(res, pipe: FusionPipeline, images: dict, score_min: float) -> np.ndarray`
Purpose: 1920x1080 frame: camera | fusion tiles (top), driving BEV | trajectory (bottom).
Parameters: `res`; `pipe`: FusionPipeline; `images`: dict; `score_min`: float
Returns: np.ndarray
Calls: app.ui.panels.camera_panel, app.ui.panels.driving_bev, app.ui.panels.fusion_tiles, app.ui.panels.trajectory_closeup, visualization.common.put_label, visualization.common.stack_images
Called by: scripts.inference.main
lines 33–75

`main() -> None`
Purpose: Runs a scene through the pipeline and writes frames, an optional MP4, a summary and detections.
Returns: None
Calls: app.ui.panels.load_images, app.ui.resources.find_checkpoint, evaluation.detection_metrics.DetectionEvaluator, evaluation.official_eval.box_to_global, evaluation.trajectory_metrics.aggregate, inference.pipeline.FusionPipeline, scripts.inference.compose, utils.config.load_config, utils.config.resolve_path, utils.logging_utils.setup_logging
Raises: SystemExit: f"Scene {args.scene} not found; available: {[s['name'] for s in pipe.scenes()]}"
Side effects (file / disk I/O calls): VideoWriter, imwrite, mkdir, write, write_text
lines 78–134

scripts/preprocess.py

Module: `scripts.preprocess` · 46 lines · **entry point** (runnable script)
Purpose: Pre-compute the per-sample info caches (and optionally the reduced map rasters).
Imports (project): datasets.info_builder, datasets.map_prior, datasets.nuscenes_db, utils.config, utils.logging_utils
Imports (third-party / stdlib): argparse, pathlib, sys, time
Imported by: –

Functions

`main() -> None`
Purpose: Builds info caches for train/val/all and optionally the map caches.
Returns: None
Calls: datasets.info_builder.load_infos, datasets.map_prior.MapPrior, datasets.nuscenes_db.NuScenesDB, utils.config.load_config, utils.config.resolve_path, utils.logging_utils.setup_logging
lines 23–42

scripts/render_bev.py

Module: `scripts.render_bev` · 95 lines · **entry point** (runnable script)
Purpose: Render every Bird's-Eye-View representation of one nuScenes frame into a single image.
Imports (project): app.ui, app.ui.resources, inference.pipeline, utils.config, utils.logging_utils, visualization.camera_view, visualization.common, visualization.feature_selection, visualization.lidar_view, visualization.radar_view
Imports (third-party / stdlib): argparse, cv2, numpy, pathlib, sys
Imported by: –

Constant	Value
TILE	360

Functions

`blank(text: str) -> np.ndarray`
Purpose: Placeholder tile with a message.
Parameters: `text`: str
Returns: np.ndarray
Calls: visualization.common.put_label
Called by: scripts.render_bev.main
lines 38–40

`main() -> None`
Purpose: Renders every BEV representation of one frame into `outputs/bev/<scene>_frame<k>_bev.jpg`.
Returns: None
Calls: app.ui.panels.driving_bev, app.ui.panels.load_images, app.ui.resources.find_checkpoint, inference.pipeline.FusionPipeline, scripts.render_bev.blank, utils.config.load_config, utils.config.resolve_path, utils.logging_utils.setup_logging, visualization.camera_view.camera_ipm_bev, visualization.common.put_label, visualization.common.stack_images, visualization.feature_selection.energy_map, visualization.feature_selection.importance_map, visualization.feature_selection.titled …
Side effects (file / disk I/O calls): imwrite, mkdir
lines 43–91

scripts/smoke_test.py

Module: `scripts.smoke_test` · 127 lines · **entry point** (runnable script)
Purpose: End-to-end smoke test on real nuScenes data: shapes, losses, backward, memory, latency.
Imports (project): datasets.nuscenes_dataset, models.fusion_model, utils.config, utils.device, utils.logging_utils, utils.seed, utils.timer
Imports (third-party / stdlib): argparse, json, pathlib, sys, time, torch
Imported by: –

Functions

`describe(obj, prefix: str = '') -> list[str]`
Purpose: Lists tensor names/shapes/dtypes of nested dicts (for the shape report).
Parameters: `obj`; `prefix`: str = ''
Returns: list[str]
Calls: scripts.smoke_test.describe
Called by: scripts.smoke_test.run
lines 32–41

`run(experiment: str, overrides: list[str], verbose: bool) -> dict`
Purpose: One training step + timed inference for an experiment; returns params, memory, latency, losses.
Parameters: `experiment`: str; `overrides`: list[str]; `verbose`: bool
Returns: dict
Calls: datasets.nuscenes_dataset.NuScenesFusionDataset, models.fusion_model.build_model, scripts.smoke_test.describe, utils.config.load_config, utils.device.amp_enabled, utils.device.get_device, utils.device.gpu_memory_report, utils.device.move_to_device, utils.seed.seed_everything, utils.timer.StageTimer
Called by: scripts.smoke_test.main
lines 44–103

`main() -> None`
Purpose: CLI over one/all experiments; exits non-zero if any loss is non-finite.
Returns: None
Calls: scripts.smoke_test.run, utils.config.list_experiments, utils.logging_utils.setup_logging
lines 106–123

scripts/validate_sensors.py

Module: `scripts.validate_sensors` · 194 lines · **entry point** (runnable script)
Purpose: Phase 2 - verify sensor synchronisation and calibration on real nuScenes data.
Imports (project): datasets.camera_loader, datasets.info_builder, datasets.lidar_loader, datasets.radar_loader, utils.bev_grid, utils.config, utils.geometry, utils.logging_utils, visualization.camera_view, visualization.common
Imports (third-party / stdlib): argparse, cv2, json, numpy, pathlib, sys
Imported by: –

Functions

`sync_report(infos: list[dict]) -> dict`
Purpose: Mean / max timestamp offset of every camera and radar vs. the LiDAR key-frame.
Parameters: `infos`: list[dict]
Returns: dict
Called by: scripts.validate_sensors.main
lines 46–54

`lidar_box_agreement(infos: list[dict], lidar: LidarLoader) -> dict`
Purpose: Compares LiDAR points inside GT boxes with num_lidar_pts (exact / within 10 % / correlation).
Parameters: `infos`: list[dict]; `lidar`: LidarLoader
Returns: dict
Calls: utils.geometry.points_in_boxes
Called by: scripts.validate_sensors.main
lines 57–77

`radar_box_agreement(infos: list[dict], root: Path, grid: BEVGrid) -> dict`
Purpose: Correlation of radar points near GT boxes with num_radar_pts.
Parameters: `infos`: list[dict]; `root`: Path; `grid`: BEVGrid
Returns: dict
Calls: datasets.radar_loader.RadarLoader, utils.geometry.points_in_boxes
Called by: scripts.validate_sensors.main
lines 80–96

`augmentation_roundtrip(cfg, info: dict) -> dict`
Purpose: Project LiDAR points into augmented images and lift them back with the encoder's formula.
Parameters: `cfg`; `info`: dict
Returns: dict
Calls: datasets.camera_loader.CameraLoader, datasets.camera_loader.image_aug_matrices, datasets.lidar_loader.LidarLoader, utils.bev_grid.BEVGrid.from_config
Called by: scripts.validate_sensors.main
lines 99–122

`render_sample(cfg, info: dict, out_dir: Path) -> list[Path]`
Purpose: Writes a camera mosaic (LiDAR depth + GT boxes) and a BEV image for a sample.
Parameters: `cfg`; `info`: dict; `out_dir`: Path
Returns: list[Path]
Calls: datasets.lidar_loader.LidarLoader, datasets.radar_loader.RadarLoader, utils.bev_grid.BEVGrid.from_config, visualization.camera_view.camera_mosaic, visualization.camera_view.draw_boxes_on_image, visualization.camera_view.draw_lidar_on_image, visualization.camera_view.load_image, visualization.common.BEVCanvas, visualization.common.class_color, visualization.common.colorize, visualization.common.put_label
Called by: scripts.validate_sensors.main
Side effects (file / disk I/O calls): imwrite
lines 125–161

`main() -> None`
Purpose: Runs all checks and writes report.json and images.
Returns: None
Calls: datasets.info_builder.load_infos, datasets.lidar_loader.LidarLoader, scripts.validate_sensors.augmentation_roundtrip, scripts.validate_sensors.lidar_box_agreement, scripts.validate_sensors.radar_box_agreement, scripts.validate_sensors.render_sample, scripts.validate_sensors.sync_report, utils.bev_grid.BEVGrid.from_config, utils.config.load_config, utils.config.resolve_path, utils.logging_utils.setup_logging
Side effects (file / disk I/O calls): mkdir, write_text
lines 164–190

Package `tests`

tests/__init__.py

Module: `tests` · 1 lines
Purpose: Package marker.
Imports (project): –
Imports (third-party / stdlib): –
Imported by: –

tests/conftest.py

Module: `tests.conftest` · 36 lines
Purpose: Shared pytest fixtures. Tests needing nuScenes are skipped when the dataset is not configured.
Imports (project): datasets.info_builder, utils.config
Imports (third-party / stdlib): pathlib, pytest, sys
Imported by: tests.test_app, tests.test_dataset

Constant	Value
ROOT	`Path(__file__).resolve().parents[1]`

Functions

`_dataset_available() -> bool`
Purpose: True if `<dataset.root>/<version>` exists (otherwise data tests are skipped).
Returns: bool
Calls: utils.config.load_config
lines 16–21

`cfg()`
Purpose: Session fixture: loaded configuration.
Returns: None / not annotated
Calls: utils.config.load_config
Decorators: pytest.fixture(scope='session')
lines 28–29

`val_infos(cfg)`
Purpose: Session fixture: validation-split infos.
Parameters: `cfg`
Returns: None / not annotated
Calls: datasets.info_builder.load_infos
Decorators: pytest.fixture(scope='session')
lines 33–36

tests/test_app.py

Module: `tests.test_app` · 38 lines
Purpose: Headless dashboard test (Streamlit AppTest) on real nuScenes data, all three modes, CPU.
Imports (project): tests.conftest
Imports (third-party / stdlib): pathlib, pytest, streamlit
Imported by: –

Constant	Value
APP	`Path(__file__).resolve().parents[1] / 'app' / 'streamlit_app.py'`

Functions

`_app(mode_index: int, explain: bool = False)`
Purpose: Runs the dashboard headlessly (AppTest) on CPU in the given mode, optionally clicking 'Explain'.
Parameters: `mode_index`: int; `explain`: bool = False
Returns: None / not annotated
Called by: tests.test_app.test_dashboard_modes_render_without_exceptions, tests.test_app.test_dashboard_next_frame
lines 13–22

`test_dashboard_modes_render_without_exceptions(mode_index: int) -> None`
Purpose: Each mode renders without exceptions and shows the title.
Parameters: `mode_index`: int
Returns: None
Calls: tests.test_app._app
Decorators: requires_dataset, pytest.mark.parametrize('mode_index', [0, 1, 2])
lines 27–30

`test_dashboard_next_frame() -> None`
Purpose: 'Next' advances the slider to 1.

Returns: None
Calls: tests.test_app._app
Decorators: requires_dataset
lines 34–38

tests/test_dataset.py

Module: `tests.test_dataset` · 63 lines
Purpose: Real-data checks of the loader, calibration chain and camera geometry (skipped without nuScenes).
Imports (project): datasets.lidar_loader, datasets.nuscenes_dataset, models.camera_bev.lss, tests.conftest, utils.bev_grid, utils.geometry
Imports (third-party / stdlib): numpy, pathlib, torch
Imported by: –

Functions

`test_lidar_points_inside_gt_boxes_match_annotations(cfg, val_infos) -> None`
Purpose: Any error in quaternion order, [w,l,h] size order or frame chaining breaks this agreement.
Parameters: `cfg`; `val_infos`
Returns: None
Calls: datasets.lidar_loader.LidarLoader, utils.bev_grid.BEVGrid.from_config, utils.geometry.points_in_boxes
Decorators: requires_dataset
lines 18–29

`test_dataset_item_shapes(cfg) -> None`
Purpose: Collated batch shapes match the configuration.
Parameters: `cfg`
Returns: None
Calls: datasets.nuscenes_dataset.NuScenesFusionDataset, datasets.nuscenes_dataset.collate_batch
Decorators: requires_dataset
lines 33–41

`test_lss_geometry_matches_projection(cfg, val_infos) -> None`
Purpose: Lift a frustum point with the encoder and re-project it with the dataset matrices.
Parameters: `cfg`; `val_infos`
Returns: None
Calls: datasets.nuscenes_dataset.NuScenesFusionDataset, datasets.nuscenes_dataset.collate_batch, models.camera_bev.lss.CameraBEVEncoder, utils.bev_grid.BEVGrid.from_config
Decorators: requires_dataset
lines 45–63

tests/test_geometry.py

Module: `tests.test_geometry` · 56 lines
Purpose: Geometry conventions (quaternions, transforms, boxes, BEV augmentation, rasterisation).
Imports (project): utils.bev_grid, utils.geometry, utils.raster
Imports (third-party / stdlib): numpy
Imported by: –

Functions

`test_quaternion_roundtrip_and_yaw() -> None`
Purpose: Quaternion ↔ matrix round trip and yaw extraction.
Returns: None
Calls: utils.geometry.quat_to_rotmat, utils.geometry.rotmat_to_quat, utils.geometry.yaw_from_quat
lines 12–18

`test_invert_transform() -> None`
Purpose: T · inv(T) = I.
Returns: None
Calls: utils.geometry.invert_transform, utils.geometry.transform_matrix
lines 21–23

`test_transform_boxes_rotates_yaw_and_velocity() -> None`
Purpose: 90° rotation of a box rotates centre, yaw and velocity.
Returns: None
Calls: utils.geometry.quat_to_rotmat, utils.geometry.transform_boxes, utils.geometry.transform_matrix
lines 26–32

`test_points_in_boxes() -> None`
Purpose: Point-in-rotated-box test.
Returns: None
Calls: utils.geometry.points_in_boxes
lines 35–39

`test_mirror_augmentation_flips_yaw_and_vy() -> None`
Purpose: Mirror augmentation negates y, yaw and vy.
Returns: None
Calls: utils.geometry.augment_boxes_bev, utils.geometry.bev_rotation_matrix
lines 42–48

`test_rasterize_box_area() -> None`
Purpose: Rasterised area ≈ metric area; corner order.
Returns: None
Calls: utils.bev_grid.BEVGrid, utils.geometry.box_corners_bev, utils.raster.rasterize_boxes
lines 51–56

tests/test_metrics_planning.py

Module: `tests.test_metrics_planning` · 81 lines
Purpose: Metric and planner sanity tests.
Imports (project): evaluation.detection_metrics, evaluation.trajectory_metrics, planning.astar, planning.mpc, planning.trajectory_planner, utils.config
Imports (third-party / stdlib): numpy
Imported by: –

Constant	Value
CLASSES	`['car', 'truck', 'construction_vehicle', 'bus', 'trailer', 'barrier', 'motorcycle', 'bicycle', 'pedestrian', 'traffic_cone']`

Functions

`_gt()`
Purpose: Two GT boxes (car, pedestrian) used by the metric tests.
Returns: None / not annotated
Called by: tests.test_metrics_planning.test_perfect_detections_give_full_ap_and_zero_errors, tests.test_metrics_planning.test_shifted_detection_counts_translation_error
lines 17–19

`test_perfect_detections_give_full_ap_and_zero_errors() -> None`
Purpose: Perfect predictions → AP 1, zero errors.
Returns: None
Calls: evaluation.detection_metrics.DetectionEvaluator, tests.test_metrics_planning._gt
lines 22–31

`test_shifted_detection_counts_translation_error() -> None`
Purpose: 0.75 m shift → AP@0.5 = 0, AP@1 = 1, ATE = 0.75.
Returns: None
Calls: evaluation.detection_metrics.DetectionEvaluator, tests.test_metrics_planning._gt
lines 34–43

`test_trajectory_metrics() -> None`
Purpose: Constant lateral offset → ADE = FDE = lateral error = 1.
Returns: None
Calls: evaluation.trajectory_metrics.sample_planning_metrics
lines 46–51

`test_astar_routes_around_wall() -> None`
Purpose: A* goes around a wall.
Returns: None
Calls: planning.astar.astar
lines 54–59

`test_mpc_tracks_straight_reference() -> None`
Purpose: iLQR converges to a straight reference from a 0.5 m offset.
Returns: None
Calls: planning.mpc.BicycleILQR, utils.config.load_config
lines 62–71

`test_baseline_planner_brakes_or_avoids_static_obstacle() -> None`
Purpose: Planner output has 6 waypoints and no collision with a static car ahead.
Returns: None
Calls: planning.trajectory_planner.BaselinePlanner, utils.config.load_config
lines 74–81

tests/test_models.py

Module: `tests.test_models` · 71 lines
Purpose: Network shape / gradient tests for every fusion configuration (random tensors, CPU, tiny grid).
Imports (project): models.feature_selection.selection, models.fusion_model, utils.config
Imports (third-party / stdlib): pytest, torch
Imported by: –

Constant	Value
SMALL	['bev.x_range=[-16,16]', 'bev.y_range=[-16,16]', 'dataset.camera.image_size=[64,176]', 'model.camera_bev.pretrained=false', 'model.camera_bev.depth_bins=[1.0...

Functions

`_batch(cfg: Config, b: int = 2) -> dict`
Purpose: Random batch on a small grid for model tests.
Parameters: `cfg`: Config; `b`: int = 2
Returns: dict
Called by: tests.test_models.test_forward_backward_every_experiment
lines 15–37

`test_forward_backward_every_experiment(experiment: str) -> None`
Purpose: Every experiment config: shapes, finite losses, finite gradients, decoding.
Parameters: `experiment`: str
Returns: None
Calls: models.fusion_model.build_model, tests.test_models._batch, utils.config.list_experiments, utils.config.load_config
Decorators: pytest.mark.parametrize('experiment', list(list_experiments()))
lines 41–56

`test_feature_selection_gates_are_bounded_and_hard_mode_is_binary() -> None`
Purpose: Hard-concrete gates are exactly 0/1 at eval; stats in [0, 1].
Returns: None
Calls: models.feature_selection.selection.CrossModalFeatureSelection, utils.config.Config, utils.config.load_config
lines 59–71

Package `training`

training/__init__.py

Module: `training` · 1 lines
Purpose: Package marker.
Imports (project): –
Imports (third-party / stdlib): –
Imported by: –

training/cli.py

Module: `training.cli` · 38 lines · **entry point** (runnable script)
Purpose: Shared command-line entry point for all training scripts.
Imports (project): training.trainer
Imports (third-party / stdlib): argparse, json, pathlib, sys
Imported by: training.train_bev, training.train_detection, training.train_end_to_end, training.train_fusion, training.train_joint, training.train_lidar, training.train_radar, training.train_trajectory

Constant	Value
CURRICULUM	('camera_bev', 'lidar', 'radar', 'fusion', 'detection', 'trajectory', 'end_to_end')

Functions

`run(default_stage: Optional[str] = None, argv: Optional[list[str]] = None) -> None`
Purpose: Parses CLI arguments and runs one stage or the whole curriculum (`--stage curriculum`).
Parameters: `default_stage`: Optional[str] = None; `argv`: Optional[list[str]] = None
Returns: None
Calls: training.trainer.Trainer
lines 17–34

training/train_bev.py

Module: `training.train_bev` · 13 lines · **entry point** (runnable script)
Purpose: Stage 1 - camera -> BEV encoder (LSS + depth supervision).
Imports (project): training.cli
Imports (third-party / stdlib): pathlib, sys
Imported by: –

training/train_detection.py

Module: `training.train_detection` · 13 lines · **entry point** (runnable script)
Purpose: Stage 5 - detection fine-tuning.
Imports (project): training.cli
Imports (third-party / stdlib): pathlib, sys
Imported by: –

training/train_end_to_end.py

Module: `training.train_end_to_end` · 13 lines · **entry point** (runnable script)
Purpose: Stage 7 - end-to-end fine-tuning.
Imports (project): training.cli
Imports (third-party / stdlib): pathlib, sys
Imported by: –

training/train_fusion.py

Module: `training.train_fusion` · 13 lines · **entry point** (runnable script)
Purpose: Stage 4 - feature selection + fusion.
Imports (project): training.cli
Imports (third-party / stdlib): pathlib, sys
Imported by: –

training/train_joint.py

Module: `training.train_joint` · 13 lines · **entry point** (runnable script)
Purpose: Single-stage joint training (used by the ablation study).
Imports (project): training.cli
Imports (third-party / stdlib): pathlib, sys
Imported by: –

training/train_lidar.py

Module: `training.train_lidar` · 13 lines · **entry point** (runnable script)
Purpose: Stage 2 - LiDAR encoder.
Imports (project): training.cli
Imports (third-party / stdlib): pathlib, sys
Imported by: –

training/train_radar.py

Module: `training.train_radar` · 13 lines · **entry point** (runnable script)
Purpose: Stage 3 - radar encoder.
Imports (project): training.cli
Imports (third-party / stdlib): pathlib, sys
Imported by: –

training/train_trajectory.py

Module: `training.train_trajectory` · 13 lines · **entry point** (runnable script)
Purpose: Stage 6 - learned trajectory planner.
Imports (project): training.cli
Imports (third-party / stdlib): pathlib, sys
Imported by: –

training/trainer.py

Module: `training.trainer` · 233 lines
Purpose: Stage-aware trainer for the 7-stage curriculum (and the single-stage ablation runs).
Imports (project): datasets.nuscenes_dataset, evaluation.evaluator, models.fusion_model, utils.checkpoint, utils.config, utils.device, utils.logging_utils, utils.seed
Imports (third-party / stdlib): json, logging, math, pathlib, time, torch
Imported by: evaluation.ablation, training.cli

class Trainer

Instance attributes (set in __init__): `experiment` = (experiment, stage); `stage` = (experiment, stage); `cfg` = build_stage_config(experiment, stage, overrides); `spec` = self.cfg.stages[stage]; `ckpt_dir` = stage_checkpoint_dir(self.cfg, experiment, stage); `log_dir` = resolve_path(self.cfg.runtime.output_dir) / 'logs' / experiment / stage; `device` = get_device(self.cfg.runtime.device); `use_amp` = amp_enabled(self.device, bool(self.cfg.train.amp)); `epochs` = int(epochs or self.spec.get('epochs') or self.cfg.train.epochs); `active_losses` = list(self.spec.losses); `train_set` = NuScenesFusionDataset(self.cfg, self.cfg.dataset.train_split, train=True); `val_set` = NuScenesFusionDataset(self.cfg, self.cfg.dataset.val_split, train=False); `train_loader` = DataLoader(self.train_set, batch_size=int(self.cfg.train.batch_size), shuffle=True, drop_last=Tru...; `val_loader` = DataLoader(self.val_set, batch_size=1, shuffle=False, **common); `model` = build_model(self.cfg).to(self.device); `optimizer` = torch.optim.AdamW(params, lr=lr, weight_decay=float(self.cfg.train.weight_decay)); `accum` = max(int(self.cfg.train.grad_accum), 1); `scheduler` = torch.optim.lr_scheduler.LambdaLR(self.optimizer, lambda s: min(1.0, (s + 1) / warmup) * 0.5 * (1...; `scaler` = torch.amp.GradScaler('cuda', enabled=self.use_amp); `start_epoch` = (0, 0); `step` = (0, 0); `best_score` = -float('inf')

`__init__(self, experiment: str, stage: str, overrides: Optional[list[str]] = None, epochs: Optional[int] = None, resume: Optional[str] = None) -> None`
Purpose: Sets up config, logging, seed, device/AMP, data, model, weights initialisation, freezing, optimiser, scheduler, scaler, optional resume.
Parameters: `experiment`: str; `stage`: str; `overrides`: Optional[list[str]] = None; `epochs`: Optional[int] = None; `resume`: Optional[str] = None
Returns: None
Calls: datasets.nuscenes_dataset.NuScenesFusionDataset, models.fusion_model.build_model, training.trainer.Trainer._freeze, training.trainer.Trainer._initialise_from_stages, training.trainer.Trainer._resume, training.trainer.build_stage_config, training.trainer.stage_checkpoint_dir, utils.config.resolve_path, utils.device.amp_enabled, utils.device.get_device, utils.logging_utils.setup_logging, utils.seed.seed_everything
Called by: evaluation.ablation.main, training.cli.run
lines 57–102

`_initialise_from_stages(self) -> None`
Purpose: Loads matching weights from earlier stage checkpoints (by name prefix).
Returns: None
Calls: training.trainer.stage_checkpoint_dir, utils.checkpoint.load_checkpoint, utils.checkpoint.load_model_weights
Called by: training.trainer.Trainer.__init__
lines 105–116

`_freeze(self) -> None`
Purpose: Applies `trainable_only` / `frozen` of the stage.
Returns: None
Called by: training.trainer.Trainer.__init__
lines 118–128

`_resume(self, path: str) -> None`
Purpose: Restores training state from a checkpoint.
Parameters: `path`: str
Returns: None
Calls: utils.checkpoint.load_checkpoint
Called by: training.trainer.Trainer.__init__
lines 130–141

`_score(self, res: dict[str, Any]) -> float`
Purpose: Selection score for best.pt (NDS / −ADE / −val loss).
Parameters: `res`: dict[str, Any]
Returns: float
Called by: training.trainer.Trainer.fit
lines 144–149

`_set_train_mode(self) -> None`
Purpose: `model.train()`, but BatchNorm layers whose parameters are frozen stay in eval mode, so frozen modules do not drift through their running statistics.
Returns: None
Called by: training.trainer.Trainer.train_epoch
lines 151–159

`train_epoch(self, epoch: int) -> dict[str, float]`
Purpose: One epoch with AMP, gradient accumulation, clipping, NaN guard and logging.
Parameters: `epoch`: int
Returns: dict[str, float]

Calls: training.trainer.Trainer._set_train_mode, training.trainer.Trainer.model, utils.device.gpu_memory_report, utils.device.move_to_device
Called by: training.trainer.Trainer.fit
lines 161–202

`fit(self) -> dict[str, Any]`
Purpose: Epoch loop with validation, best/last checkpoints and history.jsonl.
Returns: dict[str, Any]
Calls: evaluation.evaluator.evaluate_model, evaluation.evaluator.summary_line, training.trainer.Trainer._save, training.trainer.Trainer._score, training.trainer.Trainer.train_epoch
Side effects (file / disk I/O calls): open, write
lines 204–224

`_save(self, name: str, epoch: int, res: dict[str, Any]) -> None`
Purpose: Writes a checkpoint with metrics and training state.
Parameters: `name`: str; `epoch`: int; `res`: dict[str, Any]
Returns: None
Calls: utils.checkpoint.save_checkpoint
Called by: training.trainer.Trainer.fit
lines 226–233

Functions

`stage_checkpoint_dir(cfg: Config, experiment: str, stage: str) -> Path`
Purpose: Checkpoint folder of a stage (`_shared` for shared stages).
Parameters: `cfg`: Config; `experiment`: str; `stage`: str
Returns: Path
Calls: utils.config.resolve_path
Called by: training.trainer.Trainer.__init__, training.trainer.Trainer._initialise_from_stages
lines 36–39

`build_stage_config(experiment: Optional[str], stage: str, overrides: Optional[list[str]] = None) -> Config`
Purpose: Experiment config + stage overrides + CLI overrides (KeyError for unknown stages).
Parameters: `experiment`: Optional[str]; `stage`: str; `overrides`: Optional[list[str]] = None
Returns: Config
Calls: utils.config.Config, utils.config.deep_merge, utils.config.load_config, utils.config.parse_override
Called by: training.trainer.Trainer.__init__
Raises: KeyError: f'Unknown stage {stage!r}; available: {list(cfg.stages)}'
lines 42–53

Package `utils`

utils/__init__.py

Module: `utils` · 1 lines
Purpose: Shared utilities: configuration, geometry, BEV grids, logging, checkpoints, timing.
Imports (project): –
Imports (third-party / stdlib): –
Imported by: –

utils/bev_grid.py

Module: `utils.bev_grid` · 66 lines
Purpose: The single Bird's-Eye-View grid definition shared by every module.
Imports (project): –
Imports (third-party / stdlib): dataclasses, numpy
Imported by: datasets.lidar_loader, datasets.nuscenes_dataset, datasets.radar_loader, inference.pipeline, models.camera_bev.lss, models.common.pillars, models.detection.decoder, models.detection.targets, models.fusion_model, models.lidar_encoder.pillar_encoder, models.radar_encoder.radar_encoder, models.trajectory.losses, planning.occupancy_grid, scripts.eval_planner, scripts.validate_sensors, tests.test_dataset, tests.test_geometry, utils.raster, visualization.common, visualization.feature_selection

class `BEVGrid`

Class attributes / fields: `x_range` = tuple[float, float]; `y_range` = tuple[float, float]; `z_range` = tuple[float, float]; `resolution` = float

`nx(self) -> int`
Purpose: Number of cells along x.
Returns: int
Decorators: property
lines 28–29

`ny(self) -> int`
Purpose: Number of cells along y.
Returns: int
Decorators: property
lines 32–33

`shape(self) -> tuple[int, int]`
Purpose: `(H, W) = (ny, nx)`.
Returns: tuple[int, int]
Decorators: property
lines 36–38

`downsample(self, stride: int) -> 'BEVGrid'`
Purpose: Same extent with `stride`-times coarser cells (e.g. the detection-head grid).
Parameters: `stride`: int
Returns: 'BEVGrid'
Calls: utils.bev_grid.BEVGrid
lines 40–42

`world_to_grid(self, x: np.ndarray, y: np.ndarray) -> tuple[np.ndarray, np.ndarray]`
Purpose: Continuous grid coordinates (`ix`, `iy`); cell centres are at `k + 0.5`.
Parameters: x: np.ndarray; y: np.ndarray
Returns: tuple[np.ndarray, np.ndarray]
lines 44–46

`grid_to_world(self, ix: np.ndarray, iy: np.ndarray) -> tuple[np.ndarray, np.ndarray]`
Purpose: Continuous grid coordinates → metres.
Parameters: `ix`: np.ndarray; `iy`: np.ndarray
Returns: tuple[np.ndarray, np.ndarray]
lines 48–49

`in_range(self, x: np.ndarray, y: np.ndarray) -> np.ndarray`
Purpose: Boolean mask of points inside the x/y extent.
Parameters: x: np.ndarray; y: np.ndarray
Returns: np.ndarray
lines 51–52

`cell_centers(self) -> tuple[np.ndarray, np.ndarray]`
Purpose: Metric x and y coordinates of the cell centres.
Returns: tuple[np.ndarray, np.ndarray]
lines 54–57

`from_config(cls, bev_cfg: Mapping[str, Any]) -> 'BEVGrid'`
Purpose: Builds a BEVGrid from the `bev` config section.
Parameters: `bev_cfg`: Mapping[str, Any]
Returns: 'BEVGrid'
Called by: datasets.nuscenes_dataset.NuScenesFusionDataset.__init__, inference.pipeline.FusionPipeline.__init__, models.fusion_model.MultiSensorFusionNet.__init__, scripts.eval_planner.main, scripts.validate_sensors.augmentation_roundtrip, scripts.validate_sensors.main, scripts.validate_sensors.render_sample, tests.test_dataset.test_lidar_points_inside_gt_boxes_match_annotations, tests.test_dataset.test_lss_geometry_matches_projection
Decorators: classmethod
lines 60–66

utils/checkpoint.py

Module: `utils.checkpoint` · 81 lines
Purpose: Checkpoint saving/loading with the configuration embedded.
Imports (project): –
Imports (third-party / stdlib): logging, pathlib, torch
Imported by: evaluation.ablation, inference.pipeline, scripts.evaluate, training.trainer

Functions

`save_checkpoint(path: Path, model: nn.Module, config: dict, optimizer: Optional[torch.optim.Optimizer] = None, scheduler: Any = None, scaler: Any = None, epoch: int = 0, step: int = 0, metrics: Optional[dict] = None, extra: Optional[dict] = None) -> None`
Purpose: Atomically write a checkpoint (written to `*.tmp` and renamed).
Parameters: `path`: Path; `model`: nn.Module; `config`: dict; `optimizer`: Optional[torch.optim.Optimizer] = None; `scheduler`: Any = None; `scaler`: Any = None; `epoch`: int = 0; `step`: int = 0; `metrics`: Optional[dict] = None; `extra`: Optional[dict] = None
Returns: None
Called by: training.trainer.Trainer._save
Side effects (file / disk I/O calls): mkdir, replace, save
lines 18–40

`load_checkpoint(path: Path, map_location: str | torch.device = 'cpu') -> dict`
Purpose: `torch.load` of a checkpoint (weights_only=False because the config dict is embedded).

Parameters: `path`: Path; `map_location`: str | torch.device = 'cpu'
Returns: dict
Called by: evaluation.ablation.evaluate_checkpoint, inference.pipeline.FusionPipeline.__init__, scripts.evaluate.main, training.trainer.Trainer._initialise_from_stages, training.trainer.Trainer._resume
lines 43–44

`load_model_weights(model: nn.Module, state_dict: dict, prefixes: Optional[Iterable[str]] = None) -> dict[str, list[str]]`
Purpose: Partially load weights, skipping keys that are absent or have a different shape.
Details: Args: prefixes: only keys starting with one of these prefixes are loaded (`None` = all). Returns: `{"loaded": [...], "skipped_shape": [...], "missing": [...]}` for logging.
Parameters: `model`: nn.Module; `state_dict`: dict; `prefixes`: Optional[Iterable[str]] = None
Returns: dict[str, list[str]]
Called by: training.trainer.Trainer._initialise_from_stages
lines 47–73

`find_checkpoints(root: Path) -> list[Path]`
Purpose: All `*.pt` files under `root`, newest first.
Parameters: `root`: Path
Returns: list[Path]
lines 76–81

utils/config.py

Module: `utils.config` · 203 lines
Purpose: Layered YAML configuration.
Imports (project): –
Imports (third-party / stdlib): copy, os, pathlib, re, yaml
Imported by: app.streamlit_app, app.ui.resources, datasets.info_builder, datasets.nuscenes_dataset, evaluation.ablation, evaluation.evaluator, inference.pipeline, models.fusion_model, scripts.benchmark, scripts.check_eval_vs_devkit, scripts.download_dataset, scripts.eval_planner, scripts.evaluate, scripts.inference, scripts.preprocess, scripts.render_bev, scripts.smoke_test, scripts.validate_sensors, tests.conftest, tests.test_metrics_planning, tests.test_models, training.trainer

Constant	Value
PROJECT_ROOT	`Path(__file__).resolve().parents[1]`
CONFIG_DIR	`PROJECT_ROOT / 'configs'`
BASE_FILES	`('dataset.yaml', 'model.yaml', 'fusion.yaml', 'planning.yaml', 'training.yaml')`
_ENV_PATTERN	`re.compile('\\$\\{env:([A-Za-z_][A-Za-z0-9_]*)(?:,([^}]*))?\\}')`

class `Config`(dict)

A `dict` with attribute access, used for every configuration object.

`__init__(self, *args: Any, **kwargs: Any) -> None`
Purpose: Wraps nested values.
Parameters: `*args`: Any; `**kwargs`: Any
Returns: None
Calls: utils.config.Config.items, utils.config._wrap
Called by: evaluation.ablation.evaluate_checkpoint, inference.pipeline.FusionPipeline.__init__, scripts.evaluate.main, tests.test_models.test_feature_selection_gates_are_bounded_and_hard_mode_is_binary, training.trainer.build_stage_config, utils.config.Config.__deepcopy__, utils.config.Config.set_path, utils.config._wrap, utils.config.load_config
lines 33–36

`__getattr__(self, key: str) -> Any`
Purpose: Attribute access to keys (AttributeError for missing keys).
Parameters: `key`: str
Returns: Any
Raises: AttributeError: key
lines 38–42

`__setattr__(self, key: str, value: Any) -> None`
Purpose: Attribute assignment stores a key.
Parameters: `key`: str; `value`: Any
Returns: None
Calls: utils.config._wrap
lines 44–45

`__deepcopy__(self, memo: dict) -> 'Config'`
Purpose: Deep copy preserving the Config type.
Parameters: `memo`: dict
Returns: 'Config'
Calls: utils.config.Config
lines 47–48

`get_path(self, dotted: str, default: Any = None) -> Any`
Purpose: Return `cfg['a']['b']['c']` for `dotted='a.b.c'` or `default`.
Parameters: `dotted`: str; `default`: Any = None
Returns: Any
lines 50–57

`set_path(self, dotted: str, value: Any) -> None`
Purpose: Set a nested key, creating intermediate dictionaries as needed.
Parameters: `dotted`: str; `value`: Any
Returns: None
Calls: utils.config.Config, utils.config._wrap
lines 59–67

`to_dict(self) -> dict`
Purpose: Plain-`dict` deep copy (safe for YAML/JSON/pickle in checkpoints).
Returns: dict
Calls: utils.config._unwrap
lines 69–71

Functions

`_wrap(value: Any) -> Any`
Purpose: Recursively converts mappings to Config.
Parameters: `value`: Any
Returns: Any
Calls: utils.config.Config, utils.config._wrap
Called by: utils.config.Config.__init__, utils.config.Config.__setattr__, utils.config.Config.set_path
lines 74–81

`_unwrap(value: Any) -> Any`
Purpose: Recursively converts Config back to plain dicts/lists.
Parameters: `value`: Any
Returns: Any
Calls: utils.config._unwrap
Called by: utils.config.Config.to_dict
lines 84–89

`deep_merge(base: MutableMapping, update: Mapping) -> MutableMapping`
Purpose: Recursively merge `update` into `base` (in place) and return `base`.
Parameters: `base`: MutableMapping; `update`: Mapping
Returns: MutableMapping
Calls: utils.config.deep_merge
Called by: training.trainer.build_stage_config, utils.config.load_config
lines 92–99

`_interpolate_env(value: Any) -> Any`
Purpose: Replaces `${env:NAME,default}` in all strings.
Parameters: `value`: Any
Returns: Any
Calls: utils.config._interpolate_env
Called by: utils.config.load_config
lines 102–112

`read_yaml(path: Path) -> dict`
Purpose: Reads a YAML file (empty → {}).
Parameters: `path`: Path
Returns: dict
Called by: app.ui.resources.dashboard_methods, app.ui.resources.experiment_labels, evaluation.ablation.main, utils.config.list_experiments, utils.config.load_config
Side effects (file / disk I/O calls): open
lines 115–118

`parse_override(item: str) -> tuple[str, Any]`
Purpose: Parse `'a.b=3'` into `('a.b', 3)`; the value is parsed as YAML.
Parameters: `item`: str
Returns: tuple[str, Any]
Called by: training.trainer.build_stage_config, utils.config.load_config
Raises: ValueError: f'Override must look like key=value, got {item!r}'
lines 121–126

`list_experiments(config_dir: Path = CONFIG_DIR) -> dict[str, dict]`
Purpose: Return the experiment table from `configs/ablation.yaml`.
Parameters: `config_dir`: Path = CONFIG_DIR
Returns: dict[str, dict]
Calls: utils.config.read_yaml
Called by: scripts.benchmark.main, scripts.smoke_test.main, tests.test_models.test_forward_backward_every_experiment, utils.config.load_config

lines 129–134

`load_config(experiment: Optional[str] = None, hardware: Optional[str] = None, overrides: Optional[Iterable[str]] = None, config_dir: Path = CONFIG_DIR, use_local: bool = True) -> Config`
Purpose: Build the full configuration for a run.
Details: Args: experiment: name of an entry in `configs/ablation.yaml` (e.g. `"F_cam_lidar_radar_fs"`). `None` uses `runtime.default_experiment`. hardware: hardware profile name (`configs/hardware/<name>.yaml`). `None` uses `runtime.hardware_profile`. overrides: `key=value` strings applied last. config_dir: directory containing the YAML files. use_local: merge `configs/local.yaml` when present.
Parameters: `experiment`: Optional[str] = None; `hardware`: Optional[str] = None; `overrides`: Optional[Iterable[str]] = None; `config_dir`: Path = CONFIG_DIR; `use_local`: bool = True
Returns: Config
Calls: utils.config.Config, utils.config._interpolate_env, utils.config.deep_merge, utils.config.list_experiments, utils.config.parse_override, utils.config.read_yaml
Called by: app.streamlit_app.get_pipeline, app.ui.resources.base_config, evaluation.ablation.main, scripts.benchmark.main, scripts.check_eval_vs_devkit.main, scripts.download_dataset.main, scripts.eval_planner.main, scripts.evaluate.main, scripts.inference.main, scripts.preprocess.main, scripts.render_bev.main, scripts.smoke_test.run, scripts.validate_sensors.main, tests.conftest._dataset_available …
Raises: FileNotFoundError: f"Unknown hardware profile '{hw_name}' ({hw_path})"; KeyError: f"Unknown experiment '{exp_name}'. Available: {sorted(experiments)}"
lines 137–197

`resolve_path(path_like: str | os.PathLike, base: Path = PROJECT_ROOT) -> Path`
Purpose: Resolve a config path: absolute paths are kept, relative ones are anchored at the project root.
Parameters: `path_like`: str | os.PathLike; `base`: Path = PROJECT_ROOT
Returns: Path
Called by: app.ui.resources.find_checkpoint, app.ui.resources.load_ablation_results, app.ui.resources.load_history, datasets.info_builder.load_infos, evaluation.ablation.checkpoint_for, evaluation.ablation.main, inference.pipeline.FusionPipeline.__init__, scripts.check_eval_vs_devkit.main, scripts.evaluate.main, scripts.inference.main, scripts.preprocess.main, scripts.render_bev.main, scripts.validate_sensors.main, training.trainer.Trainer.__init__ …
lines 200–203

utils/device.py

Module: `utils.device` · 78 lines
Purpose: Device selection and GPU memory reporting.
Imports (project): –
Imports (third-party / stdlib): logging, torch
Imported by: evaluation.ablation, evaluation.evaluator, inference.pipeline, scripts.benchmark, scripts.evaluate, scripts.smoke_test, training.trainer

Constant	Value
_FP16_OK	{}

Functions

`get_device(preference: str = 'auto') -> torch.device`
Purpose: Use CUDA when available (or requested) and fall back to CPU gracefully.
Parameters: `preference`: str = 'auto'
Returns: torch.device
Called by: evaluation.ablation.evaluate_checkpoint, inference.pipeline.FusionPipeline.__init__, scripts.benchmark.main, scripts.evaluate.main, scripts.smoke_test.run, training.trainer.Trainer.__init__
lines 10–17

`fp16_is_reliable(device: torch.device) -> bool`
Purpose: Numerically test fp16 convolutions on this GPU (cached).
Details: Some GPU / cuDNN combinations return NaN for fp16 convolutions: on a GTX 1650 with cuDNN 9.10 every 3x3 conv with 256 input channels does (measured in this project, see README "Hardware notes"). The test runs exactly that layer type in fp16 and fp32.
Parameters: `device`: torch.device
Returns: bool
Called by: utils.device.amp_enabled
lines 23–49

`amp_enabled(device: torch.device, requested: bool) -> bool`
Purpose: Use fp16 mixed precision only on CUDA devices where fp16 convolutions are verified correct.
Parameters: `device`: torch.device; `requested`: bool
Returns: bool
Calls: utils.device.fp16_is_reliable
Called by: evaluation.ablation.evaluate_checkpoint, inference.pipeline.FusionPipeline.__init__, scripts.benchmark.main, scripts.evaluate.main, scripts.smoke_test.run, training.trainer.Trainer.__init__
lines 52–54

`gpu_memory_report(device: torch.device) -> dict[str, Any]`
Purpose: Current / peak allocated memory in MiB (empty dict on CPU).
Parameters: `device`: torch.device
Returns: dict[str, Any]
Called by: evaluation.ablation.evaluate_checkpoint, scripts.benchmark.main, scripts.smoke_test.run, training.trainer.Trainer.train_epoch
lines 57–67

`move_to_device(obj: Any, device: torch.device, non_blocking: bool = True) -> Any`
Purpose: Recursively move tensors inside dicts/lists/tuples to `device`.
Parameters: `obj`: Any; `device`: torch.device; `non_blocking`: bool = True
Returns: Any
Calls: utils.device.move_to_device
Called by: evaluation.evaluator.evaluate_model, inference.pipeline.FusionPipeline._run_model, scripts.benchmark.main, scripts.evaluate.main, scripts.smoke_test.run, training.trainer.Trainer.train_epoch
lines 70–78

utils/geometry.py

Module: `utils.geometry` · 204 lines
Purpose: Rigid-body geometry helpers (NumPy).
Imports (project): –
Imports (third-party / stdlib): numpy
Imported by: app.ui.panels, datasets.camera_loader, datasets.info_builder, datasets.lidar_loader, datasets.nuscenes_dataset, evaluation.detection_metrics, evaluation.official_eval, evaluation.trajectory_metrics, planning.mpc, planning.tracking, scripts.validate_sensors, tests.test_dataset, tests.test_geometry, utils.raster, visualization.camera_view, visualization.common

Functions

`quat_to_rotmat(q: Sequence[float]) -> np.ndarray`
Purpose: Convert a unit quaternion `[w, x, y, z]` to a 3x3 rotation matrix.
Parameters: `q`: Sequence[float]
Returns: np.ndarray
Called by: datasets.info_builder._ann_box_global, datasets.info_builder._ego_state_from_poses, tests.test_geometry.test_quaternion_roundtrip_and_yaw, tests.test_geometry.test_transform_boxes_rotates_yaw_and_velocity, utils.geometry.transform_matrix, utils.geometry.yaw_from_quat
lines 21–30

`rotmat_to_quat(rot: np.ndarray) -> np.ndarray`
Purpose: Convert a 3x3 rotation matrix to a unit quaternion `[w, x, y, z]` (w >= 0).
Parameters: `rot`: np.ndarray
Returns: np.ndarray
Called by: evaluation.official_eval.box_to_global, tests.test_geometry.test_quaternion_roundtrip_and_yaw
lines 33–51

`transform_matrix(translation: Sequence[float], rotation: Sequence[float] | np.ndarray) -> np.ndarray`
Purpose: Build a 4x4 matrix from a translation and a quaternion (or 3x3 rotation).
Parameters: `translation`: Sequence[float]; `rotation`: Sequence[float] | np.ndarray
Returns: np.ndarray
Calls: utils.geometry.quat_to_rotmat
Called by: datasets.info_builder._lidar_pose, datasets.info_builder._sd_matrices, tests.test_geometry.test_invert_transform, tests.test_geometry.test_transform_boxes_rotates_yaw_and_velocity
lines 54–62

`invert_transform(mat: np.ndarray) -> np.ndarray`
Purpose: Invert a rigid 4x4 transform without a general matrix inverse.
Parameters: `mat`: np.ndarray
Returns: np.ndarray
Called by: datasets.camera_loader.lidar_depth_maps, datasets.info_builder.build_sample_info, planning.tracking.MultiObjectTracker.histories_ref, tests.test_geometry.test_invert_transform, visualization.camera_view.project_points
lines 65–71

`apply_transform(mat: np.ndarray, points: np.ndarray) -> np.ndarray`
Purpose: Apply a 4x4 transform to `(N, 3+)` points; extra columns are kept unchanged.
Parameters: `mat`: np.ndarray; `points`: np.ndarray
Returns: np.ndarray
Called by: datasets.lidar_loader.LidarLoader.load_sweep
lines 74–80

`yaw_from_rotmat(rot: np.ndarray) -> float`
Purpose: Heading of the x-axis of `rot` projected onto the ground plane (nuScenes `quaternion_yaw`).
Parameters: `rot`: np.ndarray
Returns: float
Called by: app.ui.panels.ego_status, datasets.info_builder._ann_box_global, datasets.info_builder._ego_state_from_poses, datasets.info_builder.build_sample_info, utils.geometry.yaw_from_quat

lines 83–85

`yaw_from_quat(q: Sequence[float]) -> float`
Purpose: Yaw of a quaternion.
Parameters: `q`: Sequence[float]
Returns: float
Calls: utils.geometry.quat_to_rotmat, utils.geometry.yaw_from_rotmat
Called by: tests.test_geometry.test_quaternion_roundtrip_and_yaw
lines 88–89

`wrap_angle(angle: np.ndarray | float) -> np.ndarray | float`
Purpose: Wrap an angle (array) to `[-pi, pi)`.
Parameters: `angle`: np.ndarray | float
Returns: np.ndarray | float
Called by: datasets.info_builder._ego_state_from_poses, planning.mpc.BicycleILQR._err
lines 92–94

`rot2d(theta: float) -> np.ndarray`
Purpose: 2×2 rotation matrix.
Parameters: `theta`: float
Returns: np.ndarray
Called by: utils.geometry.bev_rotation_matrix
lines 97–99

`box_corners_bev(boxes: np.ndarray) -> np.ndarray`
Purpose: BEV corners of boxes `(N, >=7) [x, y, z, l, w, h, yaw, ...]` -> `(N, 4, 2)`.
Details: Corner order: front-left, front-right, rear-right, rear-left (counter-clockwise from front).
Parameters: `boxes`: np.ndarray
Returns: np.ndarray
Called by: evaluation.trajectory_metrics.collision_flags, tests.test_geometry.test_rasterize_box_area, utils.geometry.box_corners_3d, utils.raster.rasterize_boxes, visualization.common.BEVCanvas.draw_boxes
lines 102–121

`box_corners_3d(boxes: np.ndarray) -> np.ndarray`
Purpose: 3D corners `(N, 8, 3)`; first four on the top face, last four on the bottom face.
Parameters: `boxes`: np.ndarray
Returns: np.ndarray
Calls: utils.geometry.box_corners_bev
Called by: visualization.camera_view.draw_boxes_on_image
lines 124–132

`points_in_boxes(points: np.ndarray, boxes: np.ndarray) -> np.ndarray`
Purpose: Boolean mask `(N_points, N_boxes)` of points inside 3D yaw-rotated boxes.
Parameters: `points`: np.ndarray; `boxes`: np.ndarray
Returns: np.ndarray
Called by: evaluation.detection_metrics.DetectionEvaluator.add_sample, scripts.validate_sensors.lidar_box_agreement, scripts.validate_sensors.radar_box_agreement, tests.test_dataset.test_lidar_points_inside_gt_boxes_match_annotations, tests.test_geometry.test_points_in_boxes
lines 135–147

`transform_boxes(boxes: np.ndarray, mat: np.ndarray) -> np.ndarray`
Purpose: Transform `[x, y, z, l, w, h, yaw, vx, vy]` boxes with a rigid 4x4 transform.
Details: Velocities are rotated (not translated); yaw is re-extracted from the rotated heading vector so pitch/roll in `mat` cannot corrupt it.
Parameters: `boxes`: np.ndarray; `mat`: np.ndarray
Returns: np.ndarray
Called by: datasets.info_builder._boxes_for_sample, datasets.info_builder.build_sample_info, tests.test_geometry.test_transform_boxes_rotates_yaw_and_velocity
lines 150–165

`project_to_image(points_cam: np.ndarray, intrinsic: np.ndarray) -> tuple[np.ndarray, np.ndarray]`
Purpose: Pinhole projection of camera-frame points `(N, 3)` -> pixels `(N, 2)` and depths `(N,)`.
Parameters: `points_cam`: np.ndarray; `intrinsic`: np.ndarray
Returns: tuple[np.ndarray, np.ndarray]
lines 168–174

`bev_rotation_matrix(rot_angle: float, scale: float, flip_x: bool, flip_y: bool) -> np.ndarray`
Purpose: 4x4 BEV data-augmentation matrix (`bda`): flips, then rotation about z, then isotropic scale.
Details: `flip_y` mirrors the world left/right (y -> -y); `flip_x` mirrors forward/backward.
Parameters: `rot_angle`: float; `scale`: float; `flip_x`: bool; `flip_y`: bool
Returns: np.ndarray
Calls: utils.geometry.rot2d
Called by: datasets.nuscenes_dataset.NuScenesFusionDataset._sample_bda, tests.test_geometry.test_mirror_augmentation_flips_yaw_and_vy
lines 177–187

`augment_boxes_bev(boxes: np.ndarray, bda: np.ndarray) -> np.ndarray`
Purpose: Apply a BEV augmentation matrix (possibly with reflection and scale) to boxes.
Parameters: `boxes`: np.ndarray; `bda`: np.ndarray
Returns: np.ndarray
Called by: datasets.nuscenes_dataset.NuScenesFusionDataset.__getitem__, tests.test_geometry.test_mirror_augmentation_flips_yaw_and_vy
lines 190–204

utils/logging_utils.py

Module: `utils.logging_utils` · 49 lines
Purpose: Logging setup shared by training, evaluation, scripts and the dashboard.
Imports (project): –
Imports (third-party / stdlib): logging, pathlib, sys
Imported by: datasets.camera_loader, datasets.lidar_loader, datasets.radar_loader, evaluation.ablation, scripts.benchmark, scripts.check_eval_vs_devkit, scripts.eval_planner, scripts.evaluate, scripts.inference, scripts.preprocess, scripts.render_bev, scripts.smoke_test, scripts.validate_sensors, training.trainer

Constant	Value
_FORMAT	`'%(asctime)s | %(levelname)-7s | %(name)s | %(message)s'`

class `WarnOnce`

Emit a warning only the first time a key is seen (e.g. once per corrupt file).
Instance attributes (set in __init__): `logger` = logger; `seen` = set(); `limit` = limit

`__init__(self, logger: logging.Logger, limit: int = 50) -> None`
Purpose: Logger, seen-key set and a cap on distinct warnings.
Parameters: `logger`: logging.Logger; `limit`: int = 50
Returns: None
lines 40–43

`__call__(self, key: str, message: str) -> None`
Purpose: Logs a warning the first time a key is seen.
Parameters: `key`: str; `message`: str
Returns: None
lines 45–49

Functions

`setup_logging(log_dir: Optional[Path] = None, level: int = logging.INFO, filename: str = 'run.log') -> None`
Purpose: Configure the root logger once (console) and add a file handler per `log_dir`.
Parameters: `log_dir`: Optional[Path] = None; `level`: int = logging.INFO; `filename`: str = 'run.log'
Returns: None
Called by: evaluation.ablation.main, scripts.benchmark.main, scripts.check_eval_vs_devkit.main, scripts.eval_planner.main, scripts.evaluate.main, scripts.inference.main, scripts.preprocess.main, scripts.render_bev.main, scripts.smoke_test.main, scripts.validate_sensors.main, training.trainer.Trainer.__init__
Side effects (file / disk I/O calls): mkdir
lines 13–30

`get_logger(name: str) -> logging.Logger`
Purpose: `logging.getLogger(name)`.
Parameters: `name`: str
Returns: logging.Logger
lines 33–34

utils/raster.py

Module: `utils.raster` · 45 lines
Purpose: Rasterisation of oriented boxes / points onto a :class:`BEVGrid` (array layout `[iy, ix]`).
Imports (project): utils.bev_grid, utils.geometry
Imports (third-party / stdlib): cv2, numpy
Imported by: datasets.nuscenes_dataset, planning.occupancy_grid, tests.test_geometry, visualization.feature_selection

Constant	Value
_SHIFT	4

Functions

`rasterize_boxes(boxes: np.ndarray, grid: BEVGrid, inflate: float = 0.0, values: np.ndarray | None = None, out: np.ndarray | None = None) -> np.ndarray`
Purpose: Fill oriented BEV boxes `[x, y, z, l, w, h, yaw, ...]` into an `(ny, nx)` array.
Details: `inflate` grows length and width by `2 * inflate` metres (safety margin).
Parameters: `boxes`: np.ndarray; `grid`: BEVGrid; `inflate`: float = 0.0; `values`: np.ndarray | None = None; `out`: np.ndarray | None = None

Returns: np.ndarray
Calls: utils.geometry.box_corners_bev
Called by: datasets.nuscenes_dataset.NuScenesFusionDataset.__getitem__, planning.occupancy_grid.OccupancyBuilder.dynamic_occupancy, planning.occupancy_grid.OccupancyBuilder.static_occupancy, tests.test_geometry.test_rasterize_box_area, visualization.feature_selection.object_importance
lines 13–33

`rasterize_points(points: np.ndarray, grid: BEVGrid) -> np.ndarray`
Purpose: Count of points per cell `(ny, nx)` (float32).
Parameters: `points`: np.ndarray; `grid`: BEVGrid
Returns: np.ndarray
lines 36–45

utils/seed.py

Module: `utils.seed` · 30 lines
Purpose: Reproducibility helpers.
Imports (project): –
Imports (third-party / stdlib): numpy, os, random, torch
Imported by: scripts.smoke_test, training.trainer

Functions

`seed_everything(seed: int, deterministic: bool = False) -> None`
Purpose: Seed Python, NumPy and PyTorch (CPU + CUDA).
Details: `deterministic=True` additionally requests deterministic cuDNN kernels (slower). `index_add_`/`scatter_reduce_` on CUDA use atomics and are not bit-exact even then.
Parameters: `seed`: int; `deterministic`: bool = False
Returns: None
Called by: scripts.smoke_test.run, training.trainer.Trainer.__init__
lines 11–23

`worker_init_fn(worker_id: int) -> None`
Purpose: Give each DataLoader worker its own reproducible NumPy/random stream.
Parameters: `worker_id`: int
Returns: None
lines 26–30

utils/timer.py

Module: `utils.timer` · 54 lines
Purpose: Per-stage latency measurement (CUDA-synchronised so the timings are real).
Imports (project): –
Imports (third-party / stdlib): collections, contextlib, time, torch
Imported by: evaluation.evaluator, inference.pipeline, models.fusion_model, scripts.benchmark, scripts.smoke_test

class StageTimer

Collect wall-clock time per named pipeline stage. `callback(stage_name, ms)` runs after every stage; the dashboard uses it to tick off pipeline stages live while a frame is being processed.
Instance attributes (set in __init__): `enabled` = enabled; `sync_cuda` = sync_cuda and torch.cuda.is_available(); `callback` = callback; `times` = OrderedDict()

`__init__(self, enabled: bool = True, sync_cuda: bool = True, callback: Optional[Callable[[str, float], None]] = None) -> None`
Purpose: Enabled flag, CUDA synchronisation, optional callback, ordered timings.
Parameters: `enabled`: bool = True; `sync_cuda`: bool = True; `callback`: Optional[Callable[[str, float], None]] = None
Returns: None
Called by: evaluation.evaluator.evaluate_model, inference.pipeline.FusionPipeline._run_model, scripts.benchmark.main, scripts.smoke_test.run
lines 19–24

`stage(self, name: str) -> Iterator[None]`
Purpose: Context manager timing a named stage (synchronises CUDA before and after).
Parameters: `name`: str
Returns: Iterator[None]
Calls: utils.timer.StageTimer.callback
Decorators: contextmanager
lines 27–40

`total_ms(self) -> float`
Purpose: Sum of all stage times.
Returns: float
Decorators: property
lines 43–44

`as_dict(self) -> dict[str, float]`
Purpose: Timings as a dict.
Returns: dict[str, float]
lines 46–47

class `NullTimer`(StageTimer)

A timer that records nothing (used in training).

`__init__(self) -> None`
Purpose: Disabled timer (no-op).
Returns: None
Called by: models.fusion_model.MultiSensorFusionNet.forward
lines 53–54

Package `visualization`

`visualization/__init__.py`

Module: `visualization` · 1 lines
Purpose: Package marker.
Imports (project): –
Imports (third-party / stdlib): –
Imported by: –

`visualization/bev_view.py`

Module: `visualization.bev_view` · 120 lines
Purpose: The main "autonomous driving" BEV scene.
Imports (project): visualization.common
Imports (third-party / stdlib): cv2, dataclasses, numpy
Imported by: app.ui.panels

Constant	Value
MAP_COLOR	np.array([38, 44, 60], np.uint8)

class `SceneLayers`

Class attributes / fields: `lidar_points` = None; `radar_points` = None; `map_patch` = None; `det_boxes` = None; `det_names` = field(default_factory=list); `det_scores` = None; `track_ids` = None; `track_histories` = None; `gt_boxes` = None; `gt_names` = field(default_factory=list); `gt_future` = None; `gt_future_mask` = None; `plan` = None; `plan_collision` = False; `ego_future` = None; `ego_future_mask` = None; `ego_past` = None; `astar_path` = None

Functions

`_legend(img: np.ndarray, items: list[tuple[str, tuple[int, int, int], str]]) -> None`
Purpose: Draws the legend box in the bottom-right corner of the BEV image.
Parameters: `img`: np.ndarray; `items`: list[tuple[str, tuple[int, int, int], str]]
Returns: None
Called by: visualization.bev_view.render_driving_scene
lines 44–60

`render_driving_scene(layers: SceneLayers, x_range: Sequence[float] = (-25, 55), y_range: Sequence[float] = (-40, 40), ppm: float = 8.0, show_lidar: bool = True, show_radar: bool = True, show_gt: bool = True, show_pred: bool = True, show_tracks: bool = True, title: str = '', ego_cfg = None) -> np.ndarray`
Purpose: Renders all scene layers on a BEVCanvas (map, rings, LiDAR, radar + arrows, GT, predictions + tracks + velocity, ego past/future, plan, ego, legend, title).
Parameters: `layers`: SceneLayers; `x_range`: Sequence[float] = (-25, 55); `y_range`: Sequence[float] = (-40, 40); `ppm`: float = 8.0; `show_lidar`: bool = True; `show_radar`: bool = True; `show_gt`: bool = True; `show_pred`: bool = True; `show_tracks`: bool = True; `title`: str = ''; `ego_cfg` = None
Returns: np.ndarray
Calls: visualization.bev_view._legend, visualization.common.BEVCanvas, visualization.common.class_color, visualization.common.colorize, visualization.common.put_label
Called by: app.ui.panels.driving_bev
lines 63–120

visualization/camera_view.py

Module: `visualization.camera_view` · 137 lines
Purpose: Camera view: original images with projected 3D boxes, labels and LiDAR points.
Imports (project): utils.geometry, visualization.common
Imports (third-party / stdlib): PIL, cv2, numpy, pathlib
Imported by: app.ui.panels, scripts.render_bev, scripts.validate_sensors

Constant	Value
MOSAIC_ORDER	('CAM_FRONT_LEFT', 'CAM_FRONT', 'CAM_FRONT_RIGHT', 'CAM_BACK_RIGHT', 'CAM_BACK', 'CAM_BACK_LEFT')
_EDGES	[(0, 1), (1, 2), (2, 3), (3, 0), (4, 5), (5, 6), (6, 7), (7, 4), (0, 4), (1, 5), (2, 6), (3, 7)]

Functions

`load_image(root: Path, cam_rec: dict[str, Any]) -> Optional[np.ndarray]`
Purpose: Reads a camera image as RGB (None on error).
Parameters: `root`: Path; `cam_rec`: dict[str, Any]
Returns: Optional[np.ndarray]
Called by: app.ui.panels.load_images, scripts.validate_sensors.render_sample
Side effects (file / disk I/O calls): open
lines 24–29

`project_points(points_ref: np.ndarray, cam_rec: dict[str, Any]) -> tuple[np.ndarray, np.ndarray]`
Purpose: Reference-frame points `(N, 3)` -> pixel coordinates `(N, 2)` and depth `(N,)`.
Parameters: `points_ref`: np.ndarray; `cam_rec`: dict[str, Any]
Returns: tuple[np.ndarray, np.ndarray]
Calls: utils.geometry.invert_transform
Called by: visualization.camera_view.camera_ipm_bev, visualization.camera_view.draw_boxes_on_image, visualization.camera_view.draw_lidar_on_image
lines 32–39

`draw_lidar_on_image(img: np.ndarray, points_ref: np.ndarray, cam_rec: dict[str, Any], max_depth: float = 60.0, radius: int = 2) -> np.ndarray`
Purpose: Projects reference-frame points into an image and draws them coloured by depth.
Parameters: `img`: np.ndarray; `points_ref`: np.ndarray; `cam_rec`: dict[str, Any]; `max_depth`: float = 60.0; `radius`: int = 2
Returns: np.ndarray
Calls: visualization.camera_view.project_points, visualization.common.colorize
Called by: scripts.validate_sensors.render_sample
lines 42–51

`draw_boxes_on_image(img: np.ndarray, boxes: np.ndarray, cam_rec: dict[str, Any], names: Sequence[str], scores: Optional[Sequence[float]] = None, color: Optional[tuple[int, int, int]] = None, thickness: int = 2, show_labels: bool = True) -> np.ndarray`
Purpose: Draw 3D boxes (all 8 corners in front of the camera) with class / confidence labels.
Parameters: `img`: np.ndarray; `boxes`: np.ndarray; `cam_rec`: dict[str, Any]; `names`: Sequence[str]; `scores`: Optional[Sequence[float]] = None; `color`: Optional[tuple[int, int, int]] = None; `thickness`: int = 2; `show_labels`: bool = True
Returns: np.ndarray
Calls: utils.geometry.box_corners_3d, visualization.camera_view.project_points, visualization.common.class_color, visualization.common.put_label
Called by: app.ui.panels.camera_panel, scripts.validate_sensors.render_sample
lines 54–82

`camera_ipm_bev(images: dict[str, np.ndarray], cam_recs: dict[str, dict[str, Any]], x_range: Sequence[float] = (-40, 40), y_range: Sequence[float] = (-40, 40), ppm: float = 4.0, ground_z: float = 0.0) -> np.ndarray`
Purpose: Geometric camera BEV (inverse perspective mapping) using only the calibration.
Details: Every ground-plane cell (`z = ground_z` in the ego frame) is projected into each camera with `K · inv(cam2ref)` and coloured by the pixel it lands on. This is *not* learned and assumes a flat ground — objects above the ground are smeared — which is exactly the limitation the learned Lift-Splat depth distribution removes. Display layout (forward up).
Parameters: `images`: dict[str, np.ndarray]; `cam_recs`: dict[str, dict[str, Any]]; `x_range`: Sequence[float] = (-40, 40); `y_range`: Sequence[float] = (-40, 40); `ppm`: float = 4.0; `ground_z`: float = 0.0
Returns: np.ndarray
Calls: visualization.camera_view.project_points
Called by: app.ui.panels.fusion_tiles, scripts.render_bev.main
lines 85–120

`camera_mosaic(images: dict[str, np.ndarray], width: int = 1200, order: Sequence[str] = MOSAIC_ORDER, label: bool = True) -> np.ndarray`
Purpose: 2x3 surround-view mosaic; each tile is labelled with its camera name.
Parameters: `images`: dict[str, np.ndarray]; `width`: int = 1200; `order`: Sequence[str] = MOSAIC_ORDER; `label`: bool = True
Returns: np.ndarray
Calls: visualization.common.put_label, visualization.common.stack_images
Called by: app.ui.panels.camera_panel, scripts.validate_sensors.render_sample
lines 123–137

visualization/common.py

Module: visualization.common · 251 lines
Purpose: Rendering primitives shared by every view (OpenCV, returns RGB uint8 arrays).
Imports (project): utils.bev_grid, utils.geometry
Imports (third-party / stdlib): cv2, functools, matplotlib, numpy
Imported by: app.ui.panels, scripts.inference, scripts.render_bev, scripts.validate_sensors, visualization.bev_view, visualization.camera_view, visualization.feature_selection, visualization.lidar_view, visualization.radar_view, visualization.trajectory_view

Constant	Value
CLASS_COLORS	{'car': (255, 158, 0), 'truck': (255, 99, 71), 'construction_vehicle': (233, 150, 70), 'bus': (255, 215, 0), 'trailer': (210, 105, 30), 'barrier': (170, 170,...
EGO_COLOR	(0, 230, 118)
PLAN_COLOR	(0, 229, 255)
GT_TRAJ_COLOR	(255, 255, 255)
GT_BOX_COLOR	(120, 255, 120)
LIDAR_COLOR	(150, 150, 165)
RADAR_COLOR	(255, 80, 200)
BG_COLOR	(16, 18, 26)

class BEVCanvas

Top-down drawing surface in metric ego coordinates. Args: x_range, y_range: visible extent in metres (ego frame). ppm: pixels per metre.
Instance attributes (set in __init__): x_range = (tuple(x_range), tuple(y_range), float(ppm)); y_range = (tuple(x_range), tuple(y_range), float(ppm)); ppm = (tuple(x_range), tuple(y_range), float(ppm)); h = int(round((self.x_range[1] - self.x_range[0]) * ppm)); w = int(round((self.y_range[1] - self.y_range[0]) * ppm)); img = np.empty((self.h, self.w, 3), dtype=np.uint8)

__init__(self, x_range: Sequence[float] = (-50, 50), y_range: Sequence[float] = (-50, 50), ppm: float = 6.0, bg: tuple[int, int, int] = BG_COLOR) -> None
Purpose: Creates the canvas for the metric extent at ppm pixels per metre.
Parameters: x_range: Sequence[float] = (-50, 50); y_range: Sequence[float] = (-50, 50); ppm: float = 6.0; bg: tuple[int, int, int] = BG_COLOR
Returns: None
Called by: scripts.validate_sensors.render_sample, visualization.bev_view.render_driving_scene, visualization.lidar_view.render_lidar_bev, visualization.radar_view.render_radar_bev, visualization.trajectory_view.render_trajectory_closeup
lines 92–98

to_px(self, x: np.ndarray, y: np.ndarray) -> tuple[np.ndarray, np.ndarray]
Purpose: Metric ego coordinates -> (column, row) pixel coordinates (float).
Parameters: x: np.ndarray; y: np.ndarray
Returns: tuple[np.ndarray, np.ndarray]
Called by: visualization.common.BEVCanvas._pts, visualization.common.BEVCanvas.crop_to, visualization.common.BEVCanvas.draw_grid_overlay, visualization.common.BEVCanvas.draw_points, visualization.common.BEVCanvas.draw_range_rings
lines 101–103

_pts(self, xy: np.ndarray) -> np.ndarray
Purpose: Metric (..., 2) → pixel (..., 2).
Parameters: xy: np.ndarray
Returns: np.ndarray
Calls: visualization.common.BEVCanvas.to_px
Called by: visualization.common.BEVCanvas.draw_arrows, visualization.common.BEVCanvas.draw_boxes, visualization.common.BEVCanvas.draw_ego, visualization.common.BEVCanvas.draw_polyline
lines 105–107

draw_grid_overlay(self, arr: np.ndarray, grid: BEVGrid, cmap: str = 'inferno', alpha: float = 0.6, vmin: Optional[float] = None, vmax: Optional[float] = None, threshold: Optional[float] = None) -> None
Purpose: Blend a model-layout (ny, nx) map defined on grid under the current drawing.
Parameters: arr: np.ndarray; grid: BEVGrid; cmap: str = 'inferno'; alpha: float = 0.6; vmin: Optional[float] = None; vmax: Optional[float] = None; threshold: Optional[float] = None
Returns: None
Calls: visualization.common.BEVCanvas.to_px, visualization.common.bev_to_display, visualization.common.colorize
lines 110–132

draw_range_rings(self, radii: Iterable[float] = (10, 20, 30, 40, 50), color = (55, 60, 75)) -> None
Purpose: Concentric distance rings with labels.
Parameters: radii: Iterable[float] = (10, 20, 30, 40, 50); color = (55, 60, 75)
Returns: None
Calls: visualization.common.BEVCanvas.to_px, visualization.common.put_label
lines 134–138

`draw_points(self, xy: np.ndarray, color: tuple[int, int, int] | np.ndarray = LIDAR_COLOR, radius: int = 1) -> None`
Purpose: Draws points (per-point or single colour; small circles if radius > 1).
Parameters: `xy`: np.ndarray; `color`: tuple[int, int, int] | np.ndarray = LIDAR_COLOR; `radius`: int = 1
Returns: None
Calls: visualization.common.BEVCanvas.to_px
lines 140–155

`draw_boxes(self, boxes: np.ndarray, colors: Sequence[tuple[int, int, int]] | tuple[int, int, int], thickness: int = 2, fill_alpha: float = 0.0, labels: Optional[Sequence[str]] = None, dashed: bool = False) -> None`
Purpose: Oriented boxes `[x, y, z, l, w, h, yaw, ...]`; a line marks the heading (front).
Parameters: `boxes`: np.ndarray; `colors`: Sequence[tuple[int, int, int]] | tuple[int, int, int]; `thickness`: int = 2; `fill_alpha`: float = 0.0; `labels`: Optional[Sequence[str]] = None; `dashed`: bool = False
Returns: None
Calls: utils.geometry.box_corners_bev, visualization.common.BEVCanvas._dashed_line, visualization.common.BEVCanvas._pts, visualization.common.put_label
Called by: visualization.common.BEVCanvas.draw_ego
lines 157–185

`_dashed_line(self, a: np.ndarray, b: np.ndarray, color, thickness: int, dash: int = 6) -> None`
Purpose: Dashed line segment.
Parameters: `a`: np.ndarray; `b`: np.ndarray; `color`; `thickness`: int; `dash`: int = 6
Returns: None
Called by: visualization.common.BEVCanvas.draw_boxes, visualization.common.BEVCanvas.draw_polyline
lines 187–193

`draw_arrows(self, starts: np.ndarray, vectors: np.ndarray, color = (255, 255, 255), scale: float = 1.0, thickness: int = 1, min_norm: float = 0.2) -> None`
Purpose: Velocity arrows: `starts` (N, 2) metres, `vectors` (N, 2) m/s drawn as `scale` seconds of motion.
Parameters: `starts`: np.ndarray; `vectors`: np.ndarray; `color` = (255, 255, 255); `scale`: float = 1.0; `thickness`: int = 1; `min_norm`: float = 0.2
Returns: None
Calls: visualization.common.BEVCanvas._pts
lines 195–203

`draw_polyline(self, xy: np.ndarray, color = PLAN_COLOR, thickness: int = 2, dashed: bool = False, markers: bool = True, start_at_ego: bool = True) -> None`
Purpose: Polyline (optionally starting at the ego, dashed, with markers).
Parameters: `xy`: np.ndarray; `color` = PLAN_COLOR; `thickness`: int = 2; `dashed`: bool = False; `markers`: bool = True; `start_at_ego`: bool = True
Returns: None
Calls: visualization.common.BEVCanvas._dashed_line, visualization.common.BEVCanvas._pts
lines 205–218

`draw_ego(self, length: float = 4.084, width: float = 1.85, rear_axle_to_center: float = 1.4, color = EGO_COLOR) -> None`
Purpose: Ego vehicle box and heading arrow.
Parameters: `length`: float = 4.084; `width`: float = 1.85; `rear_axle_to_center`: float = 1.4; `color` = EGO_COLOR
Returns: None
Calls: visualization.common.BEVCanvas._pts, visualization.common.BEVCanvas.draw_boxes
lines 220–226

`crop_to(self, x_range: Sequence[float], y_range: Sequence[float]) -> np.ndarray`
Purpose: Crops the canvas to a metric extent.
Parameters: `x_range`: Sequence[float]; `y_range`: Sequence[float]
Returns: np.ndarray
Calls: visualization.common.BEVCanvas.to_px
lines 228–231

Functions

`_lut(cmap: str) -> np.ndarray`
Purpose: Cached 256-entry RGB lookup table of a matplotlib colormap.
Parameters: `cmap`: str
Returns: np.ndarray
Called by: visualization.common.colorize
Decorators: lru_cache(maxsize=16)
lines 40–43

`colorize(values: np.ndarray, cmap: str = 'inferno', vmin: Optional[float] = None, vmax: Optional[float] = None) -> np.ndarray`
Purpose: Map an array of scalars to RGB with a matplotlib colormap.
Parameters: `values`: np.ndarray; `cmap`: str = 'inferno'; `vmin`: Optional[float] = None; `vmax`: Optional[float] = None
Returns: np.ndarray
Calls: visualization.common._lut
Called by: scripts.validate_sensors.render_sample, visualization.bev_view.render_driving_scene, visualization.camera_view.draw_lidar_on_image, visualization.common.BEVCanvas.draw_grid_overlay, visualization.common.heatmap_image, visualization.lidar_view.render_lidar_bev, visualization.radar_view.render_radar_bev

lines 46–52

`bev_to_display(arr: np.ndarray) -> np.ndarray`
Purpose: Convert a model-layout BEV array `[iy, ix, ...]` to display layout (forward up, left left).
Parameters: `arr`: np.ndarray
Returns: np.ndarray
Called by: visualization.common.BEVCanvas.draw_grid_overlay, visualization.common.heatmap_image, visualization.feature_selection.dominance_map, visualization.feature_selection.modality_weight_map
lines 55–58

`heatmap_image(arr: np.ndarray, cmap: str = 'inferno', size: Optional[int] = None, vmin: Optional[float] = None, vmax: Optional[float] = None, percentile: Optional[float] = 99.0) -> np.ndarray`
Purpose: Model-layout `(ny, nx)` map -> display-oriented RGB heatmap.
Parameters: `arr`: np.ndarray; `cmap`: str = 'inferno'; `size`: Optional[int] = None; `vmin`: Optional[float] = None; `vmax`: Optional[float] = None; `percentile`: Optional[float] = 99.0
Returns: np.ndarray
Calls: visualization.common.bev_to_display, visualization.common.colorize
Called by: app.ui.panels.explain_selection, visualization.feature_selection.energy_map, visualization.feature_selection.importance_map
lines 61–70

`put_label(img: np.ndarray, text: str, org: tuple[int, int] = (8, 22), scale: float = 0.55, color: tuple[int, int, int] = (255, 255, 255), bg: Optional[tuple[int, int, int]] = (0, 0, 0)) -> np.ndarray`
Purpose: Draw text with an optional background box (in place) and return the image.
Parameters: `img`: np.ndarray; `text`: str; `org`: tuple[int, int] = (8, 22); `scale`: float = 0.55; `color`: tuple[int, int, int] = (255, 255, 255); `bg`: Optional[tuple[int, int, int]] = (0, 0, 0)
Returns: np.ndarray
Called by: app.ui.panels.fusion_tiles, scripts.inference.compose, scripts.render_bev.blank, scripts.render_bev.main, scripts.validate_sensors.render_sample, visualization.bev_view.render_driving_scene, visualization.camera_view.camera_mosaic, visualization.camera_view.draw_boxes_on_image, visualization.common.BEVCanvas.draw_boxes, visualization.common.BEVCanvas.draw_range_rings, visualization.feature_selection.titled, visualization.lidar_view.render_lidar_bev, visualization.radar_view.render_radar_bev, visualization.trajectory_view.render_trajectory_closeup
lines 73–81

`class_color(name: str) -> tuple[int, int, int]`
Purpose: Display colour of a class.
Parameters: `name`: str
Returns: tuple[int, int, int]
Called by: scripts.validate_sensors.render_sample, visualization.bev_view.render_driving_scene, visualization.camera_view.draw_boxes_on_image, visualization.trajectory_view.render_trajectory_closeup
lines 234–235

`stack_images(images: list[np.ndarray], cols: int, pad: int = 4, bg = (10, 10, 14)) -> np.ndarray`
Purpose: Tile same-height images in a grid (resizing to the first image's size).
Parameters: `images`: list[np.ndarray]; `cols`: int; `pad`: int = 4; `bg` = (10, 10, 14)
Returns: np.ndarray
Called by: scripts.inference.compose, scripts.render_bev.main, visualization.camera_view.camera_mosaic
lines 238–251

visualization/feature_selection.py

Module: `visualization.feature_selection` · 120 lines
Purpose: Explainability of the feature-selection network.
Imports (project): utils.bev_grid, utils.raster, visualization.common
Imports (third-party / stdlib): cv2, numpy
Imported by: app.ui.panels, scripts.render_bev

Constant	Value
LIDAR_RGB	np.array([42, 120, 214], np.float32)
RADAR_RGB	np.array([235, 104, 52], np.float32)
NEUTRAL_RGB	np.array([56, 56, 53], np.float32)

Functions

`energy(feat: np.ndarray) -> np.ndarray`
Purpose: L2 norm over channels of a (C, H, W) map.
Parameters: `feat`: np.ndarray
Returns: np.ndarray
Called by: visualization.feature_selection.energy_map
lines 29–30

`titled(img: np.ndarray, title: str, size: int = 300) -> np.ndarray`
Purpose: Resizes a tile and writes its title.

Parameters: `img`: np.ndarray; `title`: str; `size`: int = 300
Returns: np.ndarray
Calls: visualization.common.put_label
Called by: app.ui.panels.explain_selection, app.ui.panels.fusion_tiles, scripts.render_bev.main, visualization.feature_selection.dominance_map, visualization.feature_selection.energy_map, visualization.feature_selection.importance_map, visualization.feature_selection.modality_weight_map
lines 33–35

`importance_map(gate: np.ndarray, title: str, size: int = 300, cmap: str = 'magma') -> np.ndarray`
Purpose: Spatial gate `(H, W)` in [0, 1] (fixed scale so frames are comparable).
Parameters: `gate`: np.ndarray; `title`: str; `size`: int = 300; `cmap`: str = 'magma'
Returns: np.ndarray
Calls: visualization.common.heatmap_image, visualization.feature_selection.titled
Called by: app.ui.panels.explain_selection, app.ui.panels.fusion_tiles, scripts.render_bev.main
lines 38–40

`energy_map(feat: np.ndarray, title: str, size: int = 300, vmax: Optional[float] = None) -> np.ndarray`
Purpose: Heatmap of a `(C, H, W)` feature map's channel L2 norm, or of a precomputed `(H, W)` energy.
Parameters: `feat`: np.ndarray; `title`: str; `size`: int = 300; `vmax`: Optional[float] = None
Returns: np.ndarray
Calls: visualization.common.heatmap_image, visualization.feature_selection.energy, visualization.feature_selection.titled
Called by: app.ui.panels.explain_selection, app.ui.panels.fusion_tiles, scripts.render_bev.main
lines 43–46

`contribution(selected_energy: np.ndarray, raw_energy: np.ndarray) -> np.ndarray`
Purpose: Selected feature energy normalised by the sensor's own raw-energy scale (99th percentile).
Details: Gates in cells where a sensor has no features change nothing, so "which sensor does the network rely on" must be read from what actually passes the selection, not from the gates alone.
Parameters: `selected_energy`: np.ndarray; `raw_energy`: np.ndarray
Returns: np.ndarray
Called by: app.ui.panels.explain_selection
lines 49–56

`dominance_map(s_lidar: np.ndarray, s_radar: np.ndarray, size: int = 300, min_gate: float = 0.12, title: str = 'Sensor contribution after selection (blue LiDAR / orange radar)') -> np.ndarray`
Purpose: Diverging map: blue = LiDAR dominates, orange = radar dominates, gray = equal; dark = neither.
Parameters: `s_lidar`: np.ndarray; `s_radar`: np.ndarray; `size`: int = 300; `min_gate`: float = 0.12; `title`: str = 'Sensor contribution after selection (blue LiDAR / orange radar)'
Returns: np.ndarray
Calls: visualization.common.bev_to_display, visualization.feature_selection.titled
Called by: app.ui.panels.explain_selection
lines 59–70

`modality_weight_map(weights: np.ndarray, order: list[str], size: int = 300) -> np.ndarray`
Purpose: Fusion attention `(M, H, W)` (softmax over modalities) as an additive RGB composite.
Parameters: `weights`: np.ndarray; `order`: list[str]; `size`: int = 300
Returns: np.ndarray
Calls: visualization.common.bev_to_display, visualization.feature_selection.titled
Called by: app.ui.panels.explain_selection
lines 73–81

`object_importance(boxes: np.ndarray, names: list[str], gates: dict[str, np.ndarray], grid: BEVGrid) -> list[dict[str, Any]]`
Purpose: Mean spatial gate of each modality inside every box (`boxes` in the ego frame).
Parameters: `boxes`: np.ndarray; `names`: list[str]; `gates`: dict[str, np.ndarray]; `grid`: BEVGrid
Returns: list[dict[str, Any]]
Calls: utils.raster.rasterize_boxes
Called by: app.ui.panels.explain_selection
lines 84–95

`class_importance_summary(rows: list[dict[str, Any]], modalities: list[str]) -> list[dict[str, Any]]`
Purpose: Per-class mean importance of each sensor and the more trusted sensor.
Parameters: `rows`: list[dict[str, Any]]; `modalities`: list[str]
Returns: list[dict[str, Any]]
Called by: app.ui.panels.explain_selection
lines 98–111

`channel_gate_table(gates: np.ndarray, names: list[str], top_k: int = 8) -> dict[str, Any]`
Purpose: Named (raster) channels + the top-k learned channels by gate value.
Parameters: `gates`: np.ndarray; `names`: list[str]; `top_k`: int = 8
Returns: dict[str, Any]
Called by: app.ui.panels.explain_selection
lines 114–120

visualization/lidar_view.py

Module: visualization.lidar_view · 26 lines
Purpose: LiDAR views: raw point cloud in BEV (height-coloured) and the encoder's BEV feature energy.
Imports (project): visualization.common
Imports (third-party / stdlib): numpy
Imported by: app.ui.panels, scripts.render_bev

Functions

`render_lidar_bev(points: np.ndarray, x_range: Sequence[float] = (-50, 50), y_range: Sequence[float] = (-50, 50), ppm: float = 4.0, title: str = 'LiDAR BEV', max_points: int = 120000) -> np.ndarray`
Purpose: Points `(N, >=3)` in the ego frame, coloured by height (blue = ground, yellow = high).
Parameters: `points`: np.ndarray; `x_range`: Sequence[float] = (-50, 50); `y_range`: Sequence[float] = (-50, 50); `ppm`: float = 4.0; `title`: str = 'LiDAR BEV'; `max_points`: int = 120000
Returns: np.ndarray
Calls: visualization.common.BEVCanvas, visualization.common.colorize, visualization.common.put_label
Called by: app.ui.panels.fusion_tiles, scripts.render_bev.main
lines 11–21

`feature_energy(feat: np.ndarray) -> np.ndarray`
Purpose: L2 norm over channels of a `(C, H, W)` feature map -> `(H, W)`.
Parameters: `feat`: np.ndarray
Returns: np.ndarray
lines 24–26

visualization/radar_view.py

Module: visualization.radar_view · 25 lines
Purpose: Radar view: detections in BEV with ego-motion-compensated velocity arrows (colour = speed).
Imports (project): visualization.common
Imports (third-party / stdlib): numpy
Imported by: app.ui.panels, scripts.render_bev

Functions

`render_radar_bev(points: np.ndarray, x_range: Sequence[float] = (-50, 50), y_range: Sequence[float] = (-50, 50), ppm: float = 4.0, title: str = 'Radar BEV', arrow_seconds: float = 1.0) -> np.ndarray`
Purpose: `points` columns follow `RADAR_POINT_FEATURES` (x, y, z, rcs, vx_comp, vy_comp, ...).
Parameters: `points`: np.ndarray; `x_range`: Sequence[float] = (-50, 50); `y_range`: Sequence[float] = (-50, 50); `ppm`: float = 4.0; `title`: str = 'Radar BEV'; `arrow_seconds`: float = 1.0
Returns: np.ndarray
Calls: visualization.common.BEVCanvas, visualization.common.colorize, visualization.common.put_label
Called by: app.ui.panels.fusion_tiles, scripts.render_bev.main
lines 11–25

visualization/trajectory_view.py

Module: visualization.trajectory_view · 42 lines
Purpose: Trajectory close-up: planned vs logged (ground-truth) ego trajectory with errors.
Imports (project): visualization.common
Imports (third-party / stdlib): cv2, numpy
Imported by: app.ui.panels

Functions

`render_trajectory_closeup(planned: Optional[np.ndarray], gt: Optional[np.ndarray], gt_mask: Optional[np.ndarray], boxes: Optional[np.ndarray] = None, names: Optional[list[str]] = None, past: Optional[np.ndarray] = None, size: int = 420, collision: bool = False, ego_cfg = None) -> np.ndarray`
Purpose: Zoomed BEV (-8..40 m forward, ±24 m lateral) of the planned and logged ego paths.
Parameters: `planned`: Optional[np.ndarray]; `gt`: Optional[np.ndarray]; `gt_mask`: Optional[np.ndarray]; `boxes`: Optional[np.ndarray] = None; `names`: Optional[list[str]] = None; `past`: Optional[np.ndarray] = None; `size`: int = 420; `collision`: bool = False; `ego_cfg` = None
Returns: np.ndarray
Calls: visualization.common.BEVCanvas, visualization.common.class_color, visualization.common.put_label
Called by: app.ui.panels.trajectory_closeup
lines 12–42

Testing

Run all tests with `python main.py test` (≈ 30 s on the development machine; tests that need nuScenes are skipped when the dataset is not configured). Last run: **29 passed**.

Test	File	What it verifies	Needs nuScenes
`test_dashboard_modes_render_without_exceptions`	tests/test_app.py	Each mode renders without exceptions and shows the title.	yes
`test_dashboard_next_frame`	tests/test_app.py	'Next' advances the slider to 1.	yes
`test_lidar_points_inside_gt_boxes_match_annotations`	tests/test_dataset.py	Any error in quaternion order, [w,l,h] size order or frame chaining breaks this agreement.	yes
`test_dataset_item_shapes`	tests/test_dataset.py	Collated batch shapes match the configuration.	yes
`test_lss_geometry_matches_projection`	tests/test_dataset.py	Lift a frustum point with the encoder and re-project it with the dataset matrices.	yes
`test_quaternion_roundtrip_and_yaw`	tests/test_geometry.py	Quaternion ↔ matrix round trip and yaw extraction.	no
`test_invert_transform`	tests/test_geometry.py	T · inv(T) = I.	no
`test_transform_boxes_rotates_yaw_and_velocity`	tests/test_geometry.py	90° rotation of a box rotates centre, yaw and velocity.	no
`test_points_in_boxes`	tests/test_geometry.py	Point-in-rotated-box test.	no
`test_mirror_augmentation_flips_yaw_and_vy`	tests/test_geometry.py	Mirror augmentation negates y, yaw and vy.	no
`test_rasterize_box_area`	tests/test_geometry.py	Rasterised area ≈ metric area; corner order.	no
`test_perfect_detections_give_full_ap_and_zero_errors`	tests/test_metrics_planning.py	Perfect predictions → AP 1, zero errors.	no
`test_shifted_detection_counts_translation_error`	tests/test_metrics_planning.py	0.75 m shift → AP@0.5 = 0, AP@1 = 1, ATE = 0.75.	no
`test_trajectory_metrics`	tests/test_metrics_planning.py	Constant lateral offset → ADE = FDE = lateral error = 1.	no
`test_astar_routes_around_wall`	tests/test_metrics_planning.py	A* goes around a wall.	no
`test_mpc_tracks_straight_reference`	tests/test_metrics_planning.py	iLQR converges to a straight reference from a 0.5 m offset.	no
`test_baseline_planner_brakes_or_avoids_static_obstacle`	tests/test_metrics_planning.py	Planner output has 6 waypoints and no collision with a static car ahead.	no
`test_forward_backward_every_experiment`	tests/test_models.py	Every experiment config: shapes, finite losses, finite gradients, decoding.	no
`test_feature_selection_gates_are_bounded_and_hard_mode_is_binary`	tests/test_models.py	Hard-concrete gates are exactly 0/1 at eval; stats in [0, 1].	no

Additional validation tools (not part of pytest because they produce reports and images):

Tool	Verifies
`python main.py validate`	sensor time offsets, LiDAR-in-box agreement with `num_lidar_pts`, radar agreement, augmentation round trip, projection images
`python scripts/check_eval_vs_devkit.py`	detection metric port vs official devkit
`python main.py planner --planner logged`	collision metric calibration (expects 0 %)
`python main.py smoke --experiments all --quiet`	every model configuration: shapes, finite losses, memory, latency

Debugging & Troubleshooting Guide

Symptom	Likely cause	Where to look / fix
`FileNotFoundError: nuScenes tables not found at ...`	wrong `dataset.root` / `dataset.version`	`configs/local.yaml`, `NUSCENES_ROOT`, `--set dataset.root=...`
Losses are NaN from the first iteration	fp16 on a GPU with the cuDNN fp16 bug	check the log for "fp16 convolutions are numerically broken"; set `train.amp=false`
`CUDA out of memory`	profile too large for the GPU, or another process on the GPU (e.g. the dashboard during training)	use the `gtx1650` profile, lower `dataset.camera.image_size`, `dataset.lidar.sweeps`, `train.batch_size`; enable `train.grad_checkpoint`; close other GPU programs
Validation NDS stays ≈ 0	too few epochs, or wrong labels/frames	run `python main.py validate` (calibration), `python main.py smoke`; inspect `outputs/bev` images
Boxes drawn in the wrong place	frame/convention error in new code	compare with `utils/geometry.py` conventions; run `tests/test_geometry.py`, `tests/test_dataset.py`
Config change has no effect on data	stale info cache	caches are keyed by the relevant settings; force with `python main.py preprocess --rebuild`
`KeyError: Unknown experiment` / `Unknown stage`	typo	names in `configs/ablation.yaml` / `configs/training.yaml → stages`
Missing weights warning when training a later stage	source stage not trained	train the earlier stage or ignore (those weights start from scratch)
`size mismatch` when loading a checkpoint	model config differs from the checkpoint	load through the checkpoint's embedded config (as `scripts/evaluate.py` and the app do)
DataLoader hangs / slow start on Windows	worker processes are spawned	`--set dataset.num_workers=0` to debug
Dashboard mode 3 shows ground truth	no checkpoint for the selected method	train it; check `find_checkpoint` paths in `checkpoints/<experiment>/<stage>/`
Dashboard is slow	model running on CPU or GPU shared with training	select `cuda` in the sidebar when the GPU is free
`ImportError: nuscenes`	optional devkit missing	`pip install nuscenes-devkit --no-deps` (needed for official eval and trainval splits)
Map prior missing / slow first frame	map PNG decoded on first use (≈ 5 s, ≈ 1.5 GB RAM peak)	`python main.py preprocess --maps`
Radar/LiDAR file errors in the log	corrupt or missing sweep	warned once; the sample still runs (sensor contributes zeros)

Useful inspection points

- `out = model(batch)` returns all intermediates: `out['raw']`, `out['selected']`, `out['selection']`, `out['fused']`, `out['bev_features']`, `out['det']`, `out['plan']` — inspect shapes with `scripts/smoke_test.py`'s `describe()`.
- `FusionPipeline.run(index, 'model')` returns a `FrameResult` with maps, gates, timings and metrics for one frame.
- `python main.py bev --scene <scene> --frame <k>` renders every BEV of a frame into one image.
- Logs: `outputs/logs/<experiment>/<stage>/run.log` (iteration losses, LR, it/s, peak memory) and `history.jsonl`.

Extending the Project

Task	Steps
Add a fusion method	implement a class with the fusion contract, decorate with `@register_fusion("name")`, ensure it is imported, set `fusion.method`, add an experiment to `configs/ablation.yaml` (see *Sensor Fusion*)
Add an experiment	add an entry with `overrides` under `experiments` in `configs/ablation.yaml`; optionally add it to `dashboard_methods`; train with `python main.py ablation --train --experiments <name>`
Use full nuScenes	download trainval into `dataset.root`; `--set dataset.version=v1.0-trainval dataset.train_split=train dataset.val_split=val`; use an RTX profile; requires the devkit for the split lists
Change the BEV grid	edit `bev` in `configs/dataset.yaml`; keep `(x_max − x_min)/resolution` divisible by 4 (backbone strides); planning grid is separate
Change image size / backbone	`dataset.camera.image_size` (divisible by 16), `model.camera_bev.backbone` (resnet18/34/50)
Add a radar/LiDAR point feature	add the column in the loader, update the encoder's `in_features` and `normalizer`; for an explicit raster channel extend `raster()` and the `*_RASTER_NAMES`
Use binary feature masks	`--set fusion.selection.gate_mode=hard_concrete`
Test feature selection without its regulariser	`--set loss.lambda_feature_selection=0.0`
Replace the planner	implement `plan(ego_state, command, detections, **kwargs) -> PlanResult`, add it to `build_planner`, set `planning.planner`
Add a detection class	extend `classes`, `CATEGORY_TO_DETECTION`, evaluation `CLASS_RANGE`, `nms_radius`, colours; rebuild caches
Add a dashboard panel	write a renderer in `app/ui/panels.py` using `FrameResult`, display it in `app/streamlit_app.py`
Use the CAN bus	place the CAN-bus expansion in `<root>/can_bus/`; rebuild the info cache (the ego state then comes from CAN)

Artifacts: Checkpoints & Outputs

Checkpoints

Read from the checkpoint files on disk (epoch, stage and stored validation metrics). Every checkpoint contains `model`, `config` (full merged configuration), `optimizer`, `scheduler`, `scaler`, `epoch`, `step`, `metrics`, `extra`.

Checkpoint	Size (MB)	Experiment / stage	Epoch	Stored val. NDS	Stored val. mAP	Stored val. ADE
checkpoints/A_lidar_only/joint/best.pt	65.1	A_lidar_only / joint	19	0.0749	0.0335	1.2864
checkpoints/A_lidar_only/joint/last.pt	65.1	A_lidar_only / joint	20	0.0601	0.0321	1.3735
checkpoints/B_radar_only/joint/best.pt	65.8	B_radar_only / joint	19	0.0536	0.0059	1.2772
checkpoints/B_radar_only/joint/last.pt	65.8	B_radar_only / joint	20	0.0527	0.0069	1.2588
checkpoints/C_lidar_radar_concat/joint/best.pt	68.6	C_lidar_radar_concat / joint	18	0.1203	0.0347	1.2470
checkpoints/C_lidar_radar_concat/joint/last.pt	68.6	C_lidar_radar_concat / joint	20	0.1160	0.0350	1.3636
checkpoints/D_lidar_radar_attention/joint/best.pt	68.7	D_lidar_radar_attention / joint	10	0.0811	0.0261	1.1868
checkpoints/D_lidar_radar_attention/joint/last.pt	68.7	D_lidar_radar_attention / joint	20	0.0699	0.0208	1.2770
checkpoints/E_lidar_radar_fs/joint/best.pt	69.5	E_lidar_radar_fs / joint	6	0.0838	0.0223	1.5415
checkpoints/E_lidar_radar_fs/joint/last.pt	69.5	E_lidar_radar_fs / joint	20	0.0550	0.0273	1.5142
checkpoints/F_cam_lidar_radar_fs/joint/best.pt	257.9	F_cam_lidar_radar_fs / joint	18	0.0994	0.0433	1.4991
checkpoints/F_cam_lidar_radar_fs/joint/last.pt	257.9	F_cam_lidar_radar_fs / joint	20	0.0943	0.0440	1.5288

`best.pt` = epoch with the best validation NDS during training (selected on the validation split); `last.pt` = final epoch (used for the reported ablation). The stored metrics were computed during training with the in-memory evaluator.

Generated outputs

Folder	Files	Size (MB)	Content
outputs/ablation/	14	0.4	ablation results (results.md/csv/json, per-experiment JSON, plots; *_best_epoch = best-validation protocol)
outputs/bev/	2	0.8	BEV galleries rendered by scripts/render_bev.py
outputs/cache/	9	172.7	info caches (pickles) and reduced map rasters
outputs/curriculum_test/	28	1751.8	mechanical test of the 7-stage curriculum (2 samples per stage, CPU)
outputs/eval_check/	1	1.4	submission written by scripts/check_eval_vs_devkit.py
outputs/evaluation/	2	9.4	reports of scripts/evaluate.py (incl. official devkit run of model F)
outputs/inference/	87	46.1	replay frames and videos written by scripts/inference.py
outputs/logs/	17	2.0	training logs (run.log, history.jsonl) and run logs
outputs/validation/	7	1.0	sensor validation report and images

Known Limitations, Open Issues & Next Steps

Limitations

1. **Data scale.** All training and results use nuScenes-mini (323 training, 81 validation frames, 10 scenes). Absolute metrics are far below published full-nuScenes results and comparisons between configurations are noisy (one seed).
2. **Hypothesis not yet supported.** Learned feature selection did not improve over concatenation on mini.
3. **Curriculum not trained to convergence.** The 7-stage curriculum runs end-to-end but the reported models use the single `joint` stage.
4. **Open-loop planning.** Planning is evaluated against the logged trajectory; there is no closed-loop simulator. The navigation command is derived from the logged future.
5. **No attribute head** — attributes come from a speed heuristic (affects mAAE only).
6. **Map prior** — only the nuScenes semantic raster is used (optional planner cost and display); the vector map expansion is not used.
7. **GTX 1650 constraints** — small images (128 × 352), 5 sweeps, fp32 only.

Recommended next steps

1. Run the ablation on nuScenes trainval with ≥ 3 seeds per configuration on an RTX-class GPU.
2. Isolate the effect of the gates from their regulariser: `--set loss.lambda_feature_selection=0.0`; try `gate_mode=hard_concrete` and other `target_keep_ratio` values.
3. Train the 7-stage curriculum and compare it with `joint`.
4. Add the remaining ablation configurations G, H, I (camera + LiDAR, camera + radar, camera + LiDAR + radar without selection) — already defined in `configs/ablation.yaml`.
5. Add temporal fusion (previous BEV frames) and an attribute head.
6. Closed-loop evaluation (e.g. in a simulator) for the planners.

Appendix A – Glossary

Term	Meaning
BEV	Bird's-Eye View: top-down 2-D grid around the vehicle
LSS	Lift-Splat-Shoot: camera → BEV by predicting a depth distribution per pixel
Frustum	set of 3-D points along each image ray at the depth-bin centres
Pillar	vertical column of a BEV cell containing points
VFE	voxel/pillar feature encoder (PointNet over the points of a pillar)
SE / CBAM	squeeze-and-excitation (channel attention) / convolutional block attention module
Gate	value in [0, 1] multiplying a feature (channel or cell)
bda	BEV data augmentation matrix (flip, rotation, scale)
Key-frame / sweep	annotated 2 Hz sample / intermediate unannotated sensor capture
Reference frame	ego frame at the LIDAR_TOP key-frame time
NDS, mAP, mATE, mASE, mAOE, mAVE, mAAE	nuScenes detection score, mean AP, mean translation/scale/orientation/velocity/attribute errors
ADE / FDE	average / final displacement error of a trajectory
iLQR / MPC	iterative linear-quadratic regulator / model-predictive control
A*	best-first graph search with an admissible heuristic
Infos	cached per-sample dictionaries produced by `datasets/info_builder.py`

Appendix B – Not Identifiable from the Current Codebase

The following information was requested or is commonly part of a handover but cannot be determined from the code or the files of the project. Each item is marked **Not identifiable from the current codebase.**

Item	Status
Licence of the project code	Not identifiable from the current codebase. (No licence file inside `multi_sensor_fusion/`; the nuScenes data itself is CC BY-NC-SA 4.0 under the nuScenes terms.)
Version-control history / authorship per file	Not identifiable from the current codebase. (The project folder is not a git repository.)
Results on full nuScenes (trainval)	Not identifiable from the current codebase. (Supported by the code, never run.)
Performance on RTX 3060/4060/4070 and Jetson Orin Nano	Not identifiable from the current codebase. (Profiles exist; not measured.)
Results of the 7-stage curriculum	Not identifiable from the current codebase. (Implemented and mechanically tested, not trained to convergence.)
Results of experiments G, H, I	Not identifiable from the current codebase. (Defined, not trained.)
CAN-bus based ego state	Not identifiable from the current codebase. (Reader implemented; the CAN-bus expansion is not present in the dataset folder.)
Use of the folders `D:\researcpaper\project`, `D:\researcpaper\dataset`, `D:\researcpaper\nuScenes-panoptic-v1.0-all`	Not identifiable from the current codebase. (Not referenced by any code.)
Seed-to-seed variance of the results	Not identifiable from the current codebase. (Single seed.)

Appendix C – Software Environment

Versions measured in the project's virtual environment on the development machine:

Component	Version
Python	3.11.0
OS	Windows-10-10.0.26200-SP0
PyTorch	2.14.1+cu126
CUDA (PyTorch build)	12.6
cuDNN	91002
torchvision	0.29.1+cu126
NumPy	2.4.6
OpenCV	5.0.0
matplotlib	3.11.2
Streamlit	1.65.0
ReportLab (docs only)	5.0.1
nuscenes-devkit	1.2.0
GPU (nvidia-smi)	NVIDIA GeForce GTX 1650, 4096 MiB, 566.07
Virtual environment	C:\Users\shash\.venvs\msf_fusion\Scripts\python.exe